

Learning a Worker-Selection Policy for Dynamo’s KV Router in Offline Replay

A simulation study on Qwen3-32B (vLLM, TP2, H100), aggregated serving

Rudy Pei

Draft, 2026-10-05. Results are from AIS-timed simulation, with a live GPU check of the ranking.

Abstract

Dynamo’s KV router sends each request to one of N workers by a fixed cost that weighs prefix-cache reuse against load. Published routers weigh the two differently, and which rule serves a workload best shows only under sustained load, which is costly to reproduce on GPUs for every rule and setting. We describe the router’s worker-selection API, in which every routing rule, including Dynamo’s default cost function and the nine other catalog policy types that this study’s router code branch implements, most of them ports of published routers, is a composition of filters, scorers and a picker over declared inputs. Inside that API we learn a policy, **learned-choice**: a conditional-logit choice model whose utility is linear in signals the live router already computes and which contains the default router as one coefficient setting. We tune it with CMA-ES on windowed goodput, defined by inter-token latency and end-to-end slowdown, in offline replay. AISimulate provides both the replay engine (AISim), in which Dynamo’s router drives mock vLLM engines, and the forward-pass performance model (AIS) that times those engines. Every evaluated heuristic receives the same tuning budget, restarts and validation-based selection. The headline test, registered before training, is a one-sided Wilcoxon test over the 12 independent segments of a frozen 60-cell test set that holds out time windows, agent plays, session seeds, a trace family, transform ranges and worker counts. In AIS-timed simulation, the learned router M1-v2 (the conditional logit on an extended 23-feature set) beats the branch’s ported heuristics, each tuned with an equal budget: against the validation-selected best of them, tuned Ramjet (a port of Helix’s Ramjet load balancer), it is ahead on 10 of 12 segments with a segment-mean clipped log-ratio of goodput of +0.0316 (one-sided exact Wilcoxon $p = 0.0017$), and it stays first, ahead of Ramjet, under router-state lag of up to 200 ms and under engine-timing perturbations. Its largest learned term makes uncached prefill more expensive on busier workers. The claim is scoped: only its sign is claimed, because the effect is below the pre-registered minimum detectable effect; on validation, most of the margin is reproduced by the default cost plus a faithful LMetric term, which the learned class contains and the baseline pool lacks; the gain counts requests, and prompts of 32K to 64K tokens reach their first token later than under Ramjet; and a sign constraint on load coefficients removed a degenerate policy that unconstrained search found. The headline is simulated. On six test cells served live on H100 GPUs, the policy ranking agrees with simulation and M1-v2 leads Ramjet in every cell where simulation predicts a lead; absolute goodput and the size of the gaps between policies are not validated.

Contents

1	Introduction	4
2	Background: routing primitives and Dynamo’s worker-selection API	8
2.1	The decision and what the host owns	8
2.2	Filters, scorers and pickers	9
2.3	Dynamo’s default cost function	9
2.4	The ported cost functions as instances of the API	10
2.5	The learned policy as a generalization	11
2.6	Caveats that bound the results	12
3	Problem and objective	12
3.1	The routing decision	12
3.2	Load modes	13
3.3	Objective	14
4	Simulator and workloads	16
4.1	Offline replay with AIS timing	16
4.2	What replay does not model	17
4.3	Determinism and common-random-number replicates	18
4.4	Workloads	18
4.5	SLO and load calibration	20
4.6	Splits and the frozen test set	21
5	The learned policy	22
5.1	Router integration	22
5.2	Features	23
5.3	Utility and decision rule	24
5.4	Context terms and the IIA restriction	25
5.5	Set-size invariance	26
5.6	Parity with the default router	26
5.7	Model ladder	26
6	Training method	27
6.1	Objective	27
6.2	Search spaces	28
6.3	CMA-ES and noise handling	28
6.4	Restarts and initial points	29
6.5	Sign constraints on load coefficients	29
6.6	Selection	30
6.7	Equal budget	30
7	Baselines	31
7.1	Ports, not reproductions	31
7.2	Excluded and secondary baselines	32

8	Evaluation protocol	32
8.1	Splits and hold-out axes	32
8.2	Noise floor	33
8.3	Selection hygiene	33
8.4	Headline test (pre-registered)	33
8.5	Strata and parity	34
8.6	Secondary comparisons	34
8.7	Robustness checks	35
8.8	Audits	36
9	Results	36
9.1	Selection: the finalists	36
9.2	Headline: M1-v2 against the best baseline	37
9.3	Every policy	39
9.4	Strata by load mode	39
9.5	Held-out worker counts	41
9.6	Families and transforms	42
9.7	The model ladder and the ablations	42
9.8	What M1-v2 learned	43
9.9	Robustness	44
9.10	The AIS-informed league	45
9.11	Scope: the in-class heuristic and asymmetric iteration	46
10	Objective gaming, concentration and long prompts	47
10.1	The finding: a sacrificial worker	47
10.2	The remedy: load must not attract	48
10.3	Concentration on test	48
10.4	Where the gain comes from: prompt length	48
10.5	Token-weighted goodput	50
10.6	Other secondary metrics	50
11	Live GPU validation	50
11.1	The live lane	50
11.2	Pre-registered analysis	51
11.3	Results	51
11.4	What the live runs can and cannot show	53
12	Discussion, limitations and threats to validity	54
12.1	Simulator validity	54
12.2	Workload and objective validity	56
12.3	Objective gaming and concentration	57
12.4	Statistical validity	58
12.5	Comparison validity and fairness	58
12.6	Deployment scope	59
13	Related work	60
14	Conclusion	63

A	Notation	69
B	The worker-selection API and the ported policies	71
C	Major caveats	76
D	Simulator, workload and calibration details	78
E	Policy implementation details	85
F	Tuning details	90
G	The pilot (preliminary)	97
H	Evaluation protocol details	100
I	Additional results	104
J	The AIS-informed league	109
K	Full result tables	110
L	Live-run details	116
M	Reproducibility	120
N	Names used in this paper and the campaign record	124

1 Introduction

The decision. A KV-aware router places each request on the worker whose cache already holds the longest prefix of its prompt, unless that worker is busy enough that recomputing the prefix elsewhere is cheaper. On the deployment studied here, Qwen3-32B on vLLM with two H100 GPUs per worker, both sides of this trade are large. AISimulate provides both the replay engine (AISim) and the forward-pass performance model (AIS) that this study uses, and the times below are AIS’s estimates of engine forward passes. Reuse saves prefill: a 120,000-token prompt has a time to first token of 17.8s on an idle worker that lacks its prefix and 56 ms on an idle worker that already holds the first 119,808 tokens. Load costs decode speed: one decode step at 8,192 tokens of context takes 15.3ms at batch size 1 and 109.9ms at batch size 256, so each request placed on a worker slows the token stream of every other request there. Dynamo’s default router makes the trade with a fixed weighted sum: the prefill blocks a worker would still have to compute plus its decode load (Equation (1)). Published routers draw the line elsewhere. LMetric multiplies a worker’s prefill work by its number of running requests instead of adding the two [63]. DualMap hashes a prefix of each prompt to two candidate workers and picks between them by their current state [61]. Lodestar applies a similar two-choice hash filter under high memory utilization [23]. SMetric keeps a session’s follow-up turns on the worker that holds its context while that worker can still meet a latency deadline [54].

These rules disagree, and which one wins depends on the traffic. LMetric’s authors report that the product beats linear scores tuned for each workload, Dynamo’s among them, on their traces [63]. SMetric’s authors put the ideal prefix reuse of those traces at about 50–70%. On an agentic

workload with 82% ideal reuse and no global KV tier (a cluster-wide KV store from which any worker can load a cached prefix), which is the setting of our replay, they measured LMetric 36% below a tuned linear score in throughput within SLO, because its reuse ratio fell to 45% against 74% [54]. Even the best linear weight moves: LMetric’s own sweep put the best weight on the KV-cache term of a linear score at 0.7 on one workload and 0.55 on another with a similar access pattern [63]. An operator has to choose one rule and tune it for a deployment whose traffic may change.

Why the choice is hard to settle. Three properties of the problem make the comparison expensive.

1. *No per-decision label.* A routing decision changes the cache contents and queues that later requests meet, so its quality shows only in aggregate, over minutes of traffic at a realistic load. A policy can be scored only on whole episodes.
2. *Small differences next to workload variation.* On this deployment, the two strongest tuned heuristics finished within 0.002 of each other in mean validation score (a clipped log-ratio of goodput against the default router), and their order flipped between the mean over cells (+0.0018) and the mean over independent workload segments (−0.0087) (Section 9.1). A cell is one workload at one load and worker count; a segment is a disjoint piece of one trace, which several cells can share. Detecting effects of a few percent needs many cells, replicates and independent segments.
3. *Regime dependence.* In classical load balancing, how much waiting a dispatch policy can avoid depends on the number of servers and on the load [53]. In a measured study of learned routing for disaggregated serving, the learned router tied join-the-shortest-queue at decode width 3, led at width 4 and tied round-robin again at width 6 [52]. A policy tuned at one worker count can therefore lose at another.

Settling the choice by measurement means many long runs. LMetric’s authors note that they could not sweep all configurations because replaying each trace on their testbed consumes substantial GPU time [63]. The tuning in this study is larger: 66 CMA-ES restarts in total over all tuned policies, learned and heuristic, each of 400 evaluations, every evaluation replaying all 34 train cells on 2 workload replicates, plus validation. Each train cell simulates 6 to 29 minutes of serving on 4 or 8 workers, that is, 8 to 16 GPUs.

Approach. We first describe the router’s worker-selection API (Section 2). A routing policy is a YAML instance of a catalog type that composes filters, scorers and a picker over inputs it declares: cache overlap, load, capacity and request facts. Dynamo’s default cost function, the heuristics that this study’s router code branch ports from published routers, and the learned policy are all such instances, so all of them are configured, tuned and replayed through the same interface. For development we then use AISim offline replay, timed by AIS (Section 4). It runs Dynamo’s router, including the builtin policy catalog the live router loads, against mock vLLM engines in simulated time, and times every forward pass with AIS for the configured model, GPU and parallelism. A 2,000-request trace replays in about 1.5s of one CPU core, and identical inputs replay identically, so competing policies can be compared on common random numbers. In that environment we tune a worker-selection function, **learned-choice** (Section 5). It is a conditional-logit choice model: it scores every candidate worker with one shared utility, linear in features the live router already computes, and picks the highest. CMA-ES tunes the coefficients on windowed goodput, the good requests per second inside a measurement window, where a request is good if its mean inter-token latency and its end-to-end slowdown over its own uncontended latency both stay within bounds

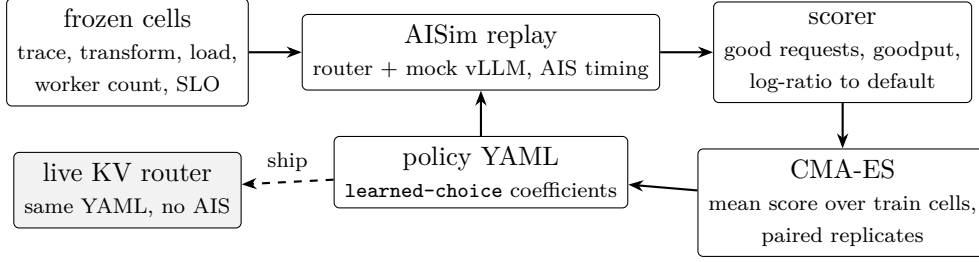


Figure 1: The training loop. A policy is a YAML file read by the router’s builtin policy catalog, so each CMA-ES candidate costs a new YAML and a set of replays, with no rebuild. Every candidate is scored on the same workload replicates, and its score is the clipped log-ratio of its windowed goodput to the default router’s. AIS times the simulated engines and so shapes the score; the headline policy never reads it.

(Sections 3.3 and 6); Figure 1 shows this training loop. The simulator only trains and scores: the headline policy reads no performance-model signal, so the YAML file that training writes runs in the live router unchanged.¹

The choice-model form gives three properties the study relies on. It is defined for any candidate set, so a policy tuned at $N \in \{4, 8\}$ can run at $N = 2$ or 32 and be tested there. It contains the default router: weight -1 on the default-cost feature and 0 on every other feature reproduces the default router’s choice (Section 5.6),² so training starts from the status quo and every learned coefficient reads as a change to it. And it is cheap: a decision costs $O(Nd)$ operations for d features (8 in the base set, 23 in the extended feature set the headline policy uses).

How “better” is decided. The comparison is on equal footing. Every policy, the learned arms and each of 11 tuned heuristic baselines,³ is one configuration tuned on the pooled train cells with the same CMA-ES budget and restarts and selected on validation; the best baseline, and by the same rule the headline learned arm, are chosen on fresh validation replicates (Sections 7 and 8). The cells replay five trace sources that differ in where prefix reuse comes from. Requests in the public Mooncake trace and in two test-only traces from the same project’s FAST’25 release share system prompts [21]; generated multi-turn sessions resend the whole conversation at every turn; and agent runs (plays) recorded in the public AgentX corpus carry long contexts across tool calls [46]. Seeded transforms rescale prompt length, output length and think time, and multiply the number of distinct shared system prompts (Section 4.4). The comparison is made once, on a test set frozen with SHA-256 hashes before any tuning. Its 60 cells come from 12 independent workload segments and are held out along six axes: time windows, agent plays, session seeds, a trace family, transform ranges and worker counts (Section 4.6). The headline is a one-sided exact Wilcoxon signed-rank test over the segments, registered before training. Reported but not tested are robustness to the SLO scale and definition, to stale router state (replay gives the router perfectly fresh cache and load information, a live router does not) and to engine timing, and a check of the ranking on GPUs.

What we found. In AIS-timed simulation, M1-v2, the headline learned router (a **learned-choice** policy on an extended 23-feature set), beats the branch’s ported heuristics, each tuned with an

¹A separately reported set of variants, the AIS-informed league, may also read AIS’s estimates at runtime; it never enters the headline (Appendix J).

²In the notation of Section 5, this is the coefficient vector $\theta = -\mathbf{e}_0$.

³The 11 are Dynamo’s default cost, the eight other catalog types that replay can run (ThunderAgent’s routing needs a request classifier that replay cannot run), with sticky sessions in two variants, and **ramjet** with a jointly tuned router queue threshold (Section 7).

equal budget.⁴ The validation-selected best of those baselines is tuned **ramjet**, a port of Helix’s Ramjet load balancer [14] that scores each worker by its cached-prefix affinity minus a multiple of its load. M1-v2 is ahead of it on 10 of the 12 test segments: all 8 non-session segments and 2 of the 4 session segments. In all four session segments both policies serve nearly every request well, and the differences, of either sign, are below 0.001. The headline statistic, the segment-mean clipped log-ratio, takes the log of M1-v2’s windowed goodput over **ramjet**’s on each cell and replicate, clips it at $\pm \log 3$, and averages it within each segment and then over the 12 segments. It is +0.0316 (a geometric-mean goodput ratio of 1.032 over segments, a point estimate), with one-sided exact Wilcoxon $p = 0.0017$ (Section 9.2). M1-v2’s largest learned term makes uncached prefill more expensive the busier a worker is; zeroing it costs 0.018 of validation score (Section 9.8). Among the 17 policies re-run for robustness (the default router, every tuned baseline and five learned arms), M1-v2 stays first under router-state lag of 10 to 200 ms and under four engine-timing perturbations, and it ranks first under all four alternative definitions of a good request.

Four findings bound this result. First, only the sign is a claim: the effect is below the pre-registered minimum detectable effect (0.038) and below the spread that the timing perturbations induce (0.060), both on the scale of the headline statistic. Second, the result is not “learning beats every heuristic”. The baseline pool holds the branch’s ports as implemented, and its **lmetric** port lacks LMetric’s queued-prefill term; the faithful LMetric rule, with that term, was not among the tested baselines. Faithful LMetric lies inside the learned model’s class. Untuned, it beats every tuned baseline on validation, and the default cost plus faithful LMetric reproduces 53–66% of M1-v2’s validation margin over tuned **ramjet** (Section 9.11). Third, the gain is a request-count property: a token-weighted goodput is mixed, and prompts of 32K to 64K tokens see a higher time to first token than under **ramjet** (Section 10). Fourth, black-box search games a request-count objective. The first tuned model drew very long prompts to the worker with the most outstanding prefill; on the heaviest cell one worker received only such prompts and served none of them within the SLO, while the hot workers served many short cached requests well. Constraining load coefficients so that load cannot attract removed this, and every later learned arm runs under that constraint (Section 10).

A pre-registered live check then served six of the $N = 4$ test cells on H100 GPUs with Dynamo’s router running the same policy files (Section 11). The live ranking of the six policies checked agrees with simulation (pooled Kendall τ_K -b 1.00), and M1-v2 leads **ramjet** in each of the five cells where simulation predicts a lead, each time by more than the registered noise bar. The check validates the ranking and that sign only: with one run per policy on six cells chosen after the test pass, it says nothing about the headline’s magnitude, and live goodput departs from simulation by policy-dependent factors of 0.72 to 2.18, enough that on one cell the simulated gain over the default router is not reproduced beyond live noise.

Contributions.

- A description of Dynamo’s worker-selection API as routing primitives, with the default cost function and nine other catalog policy types written in that vocabulary, and the major caveats the study uncovered in the API, the replay host and the workloads, each with its evidence and status (Section 2.6; full list in Appendix C).
- **learned-choice**, a deployable worker-selection policy for Dynamo’s router plugin catalog: a discrete-choice model over router-observable features that contains the default router as one

⁴After this study froze its results, the **lmetric** port in ai-dynamo/dynamo#15450 was fixed to count the worker’s queued prefill in its prefill factor, as the LMetric paper defines that factor. Every **lmetric** result in this paper, including the **lmetric** member of the baseline pool behind this claim, is from the port before that fix; the fixed port was not evaluated (Section 7.1).

coefficient setting and reads no simulator signal (Section 5).

- A tuning method for routing policies in replay: CMA-ES on an episodic, windowed goodput objective with common-random-number workload replicates, applied with the same budget, restarts and selection rule to the learned model and to every heuristic baseline (Sections 6 and 7).
- An objective and evaluation protocol for routing studies: goodput defined by inter-token latency and end-to-end slowdown over a request’s own uncontended latency, measured in a window that excludes warm-up and drain; held-out worker counts; a pre-registered test over independent workload segments (Sections 3 and 8); and staged adversarial audits (Section 8.8 and Appendix H.9).
- Results with their scope: the headline and its strata, worker-count extrapolation, the model ladder, the learned coefficients, robustness, the objective-gaming finding and its remedy, the secondary metrics that bound the claim, and a pre-registered live GPU check of the ranking (Sections 9 to 11).

Reading guide. Section 2 introduces the routing primitives, the policies and the caveats that bound the results. Section 3 defines the decision and the objective, Section 4 the simulator and workloads, Section 5 the policy and Section 6 its training. Section 7 describes how the baselines are tuned, Section 8 the evaluation protocol, Section 9 the results and Section 10 the objective-gaming and concentration findings. Section 11 reports the live GPU validation. Section 12 discusses limitations and threats to validity, and Section 13 relates the work to prior research. The appendices hold the notation, the API and port details, the full list of caveats, the simulator, workload and calibration details, the policy and tuning details, the smaller pilot tuning run that preceded the main tuning, the evaluation protocol and audits, supplementary results, the AIS-informed league, the full result tables, the live-run details and the reproducibility record. Appendix N maps the names used here to the identifiers of the campaign record.

2 Background: routing primitives and Dynamo’s worker-selection API

Every routing rule in this study, Dynamo’s default, the ported heuristics and the learned policy, runs through one interface: Dynamo’s worker-selection API, a contract between the router *host*, which owns the request path and the shared state, and a *policy*, which turns that state into a choice. This section describes the interface as a small set of routing primitives, writes the default cost function and every other catalog policy in its vocabulary, shows how the learned policy generalizes them, and highlights the caveats that bound the results. The vocabulary is the one the code uses.⁵

2.1 The decision and what the host owns

For each request j the host builds a *candidate table* of eligible workers; eligibility and pins are the host’s and always win over a policy’s preference (Appendix B). The host also owns the state a policy reads. It keeps the KV index that the engines’ cache events feed and computes each candidate’s prefix overlap with the request, per cache tier; it books the load of every request it dispatches (prefill tokens in flight, decode blocks, active requests); and it knows each worker’s configured KV capacity.

⁵Type and method names are those of the router’s plugin API on the study’s branch.

A policy owns only its decision and whatever private state it chooses to keep, such as a random generator, a session map or a sliding window. This split is what makes a policy portable: the same policy file runs wherever a host can fill the inputs it declares, including the offline replay used here (Section 4.1).

2.2 Filters, scorers and pickers

A worker-selection policy is an ordered composition of three kinds of component (Figure 2).

WorkerFilter keeps or drops one eligible candidate. Filters run in declaration order and can only shrink the set.

WorkerScorer writes one finite, lower-is-better cost for every surviving candidate in a single call. The host validates each contribution and adds the contributions of all scorers, so scorers compose additively.

WorkerPicker receives the surviving candidates with their total costs and returns one row. It may ignore the costs entirely.

None of the builtin policies in this study uses a filter; candidate restriction, where a policy needs it, happens inside its picker. Every builtin policy is therefore “scorers plus a picker”, and the interesting differences are in which inputs each reads and whether its choice for one worker depends on the other candidates.

A component declares the optional worker signals it reads, such as cache overlap (**CACHE**), router-tracked load (**LOAD**), a modeled prefill backlog (**PREFILL_TIME**) and a cost multiplier from the request’s preferred routing constraints (**PREFERRED_TAINT**), and the host materializes only those; a policy also reads request facts such as the prompt length, its prefix hashes and the session context (Appendix B).

The policy catalog. A deployment selects policies with a YAML file. Its `worker_selection` block names one instance per worker role (aggregated, prefill, decode, encode) and lists named instances, each a catalog `type` with a `parameters` mapping (Figure 2); Appendix B describes how the host loads and validates the catalog.⁶

2.3 Dynamo’s default cost function

In the API’s terms, the default router (`dynamo-default-cost-fn` in the catalog) is one scorer and one picker. The scorer declares **LOAD**, **PREFERRED_TAINT** and, unless its overlap credit is zero, **CACHE**, and assigns each candidate a cost in KV blocks:

$$c_i^{\text{def}} = \beta_{\text{pf}} \max\left(0, \frac{L_i^{\text{pf}} + \text{ISL}_j}{16} - \beta_{\text{ov}} \gamma_i o_i\right) + L_i^{\text{dec}} + \beta_{\text{req}} L_i^{\text{req}}, \quad (1)$$

where o_i is the candidate’s device-tier overlap in blocks, L_i^{pf} its active prefill tokens, L_i^{dec} its decode cost blocks (including this request’s new blocks), L_i^{req} its active requests, 16 the KV block size, and $\gamma_i = (1 + \beta_{\text{decay}} (L_i^{\text{pf}} - \min_{i' \in S} L_{i'}^{\text{pf}}) / (16 b_j))^{-1}$ discounts the overlap credit of workers with more queued prefill than the least-loaded one. The shipped defaults are an overlap credit $\beta_{\text{ov}} = 1$, a decay

⁶The offline replay host on this study’s branch runs the same catalog; on Dynamo’s main branch at the time of the study, replay rejected catalog policies selected by YAML.

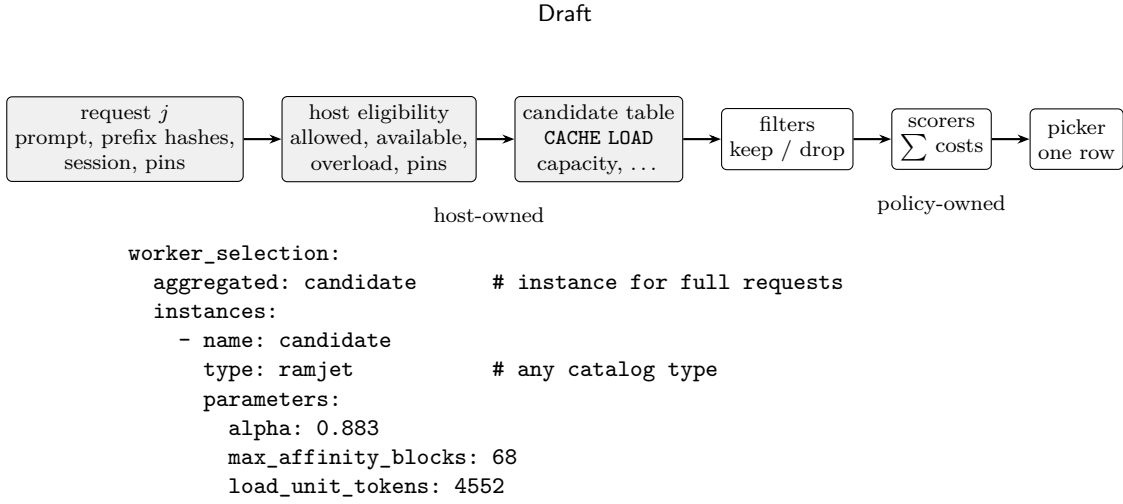


Figure 2: Top: one worker selection. The host fixes the eligible candidates and fills the inputs the policy’s components declared; the policy’s filters, scorers and picker turn them into one row. Bottom: the policy file that selects a catalog type and its parameters; the parameters shown are the tuned **ramjet** configuration that serves as the best baseline in this study.

$\beta_{\text{decay}} = 0$, a prefill load scale $\beta_{\text{pf}} = 1$ and a decode request weight $\beta_{\text{req}} = 0$, so the default router minimizes the prefill blocks the worker would still have to compute plus its decode load.⁷

The picker takes the minimum cost; with a **router_temperature** above zero it samples from a softmax over costs normalized to their range across the candidates, so at positive temperature the default is set-dependent. Appendix B gives its tie-breaking, seeding and session-affinity details.

2.4 The ported cost functions as instances of the API

Table 1 writes every catalog policy type of the study in the API’s vocabulary: its components, the inputs and request facts it reads, whether its choice for one worker depends on the other candidates (set dependence), the state it keeps, and the parameters this study tunes. The pedagogical value of the table is that the policies differ along a few axes only. Most are a single picker over **CACHE** and **LOAD**; they differ in how they combine overlap with load (additive, multiplicative, thresholded or lexicographic), in whether they restrict candidates by a hash of the prompt prefix, and in whether they compare a worker with the rest of the set. The router’s round-robin mode is not a catalog policy: it sends requests to the workers in turn, reads neither cache nor load, has no parameters, and serves as a floor. Appendix B.6 states each rule in full and lists where the ports differ from the published methods; Section 7.1 discusses the difference that bears on the headline.

⁷The full scorer also credits host- and disk-tier and shared-cache hits, has a separate branch for decode-only workers, and multiplies the cost by a preferred-taint factor; none of these enter the aggregated, device-tier-only setting studied here, in which replay requests carry no taint preferences.

Table 1: Catalog policy types as compositions of API components. S: scorer; P: picker (every policy has exactly one). Inputs: declared `WorkerInputs` (C = `CACHE`, L = `LOAD`, PT = `PREFILL_TIME`, T = `PREFERRED_TAINT`). Request facts beyond the prompt length: hashes = prefix hashes, session = session context, capacity = advertised KV capacity. Set-dependent: whether a candidate’s score or eligibility depends on the other candidates at the shipped parameters.

Type (variants)	Components	Inputs, request facts	Set dependence	State	Tuned here
dynamo-default-cost-fn	S default cost; P min cost (softmax if $\tau > 0$)	C, L, T	through γ_i if $\beta_{\text{decay}} > 0$, and the softmax	random generator	5 knobs (M0, Section 5.7)
dynamo-two-tier-cost-fn	P lexicographic (load tier, cache tier)	C, L	yes: request spread and best overlap across the set	none	3 thresholds
lmetric	P min of a product	C, L, hashes	hot-spot classes (holders against non-holders)	class window	2
ramjet	P max of affinity minus load	C, L	yes: affinity relative to the warmest candidate	tie rotation	3
dualmap	P two hashed candidates, warmer unless over budget	C, L, hashes	yes: candidates fixed by hash	prefix window	3
chwb1	P hashed owner within a load bound	L, hashes	yes: bound relative to the set mean	none	2
llm-d-precise-prefix	S prefix, S queue, S KV utilization; P max score	C, L, capacity, policy class	yes: queue score scaled by the set’s range	tie rotation	4
llm-d-optimized-baseline (throughput, modeled)	S token load; P sticky filter and TTFT gate	C, L (PT if modeled)	yes: sticky set and TTFT gate	tie rotation	3
sticky-session (hard, bounded)	S default cost; P session’s worker or min cost	C, L, T, session	bounded: request count against the set mean	session map, generator	5 or 6
thunderagent	S prefill and decode load; P classifier’s target or min cost	L, affinity target	no	in its request classifier	not evaluated
learned-choice (this work)	S default cost (feature 0); P utility argmax	C, L, T, capacity, session, hashes (v2)	v1 at rank 0: no; v2 and context terms: yes	session map, generator	7 to 22 coefficients (headline league)

2.5 The learned policy as a generalization

Each of these rules hand-picks a functional form over a few of the declared inputs. **learned-choice** (Section 5) instead scores every candidate with one utility that is linear in a fixed vector of features computed from the same inputs, and lets the coefficients be tuned. It is built from the same parts as the rules it generalizes: the default cost function’s own scorer supplies its first feature, and a picker computes the rest and takes the argmax. Several of the heuristics are points, or limits, of its parameter space.

- $\theta = -\mathbf{e}_0$ is the default router (Section 5.6).
- A large positive session coefficient is hard stickiness (Appendix E.11).

- With the extended feature set `v2`, $\theta = -(\mathbf{e}_8 + \mathbf{e}_{10})$ takes the argmin of LMetric’s product with the queued-prefill term, and $-(\mathbf{e}_9 + \mathbf{e}_{10})$ reproduces the branch’s `lmetric` port as evaluated, without its hot-spot filter (Section 5.2).
- `v2`’s `hash_home` indicator marks the two rendezvous-hashed candidates of a prompt prefix, the candidate set of DualMap and Lodestar [23, 61].
- Set-relative features and context terms let the comparison between two workers depend on the others (Section 5.4), as the queue score of `llm-d-precise-prefix` or the relative basis of `ramjet` do by construction.

What it cannot express are hard thresholds and lexicographic rules (two-tier’s tiers, DualMap’s budget, CHWBL’s bound, the optimized baseline’s TTFT gate) and history-dependent windows, which a smooth utility can only approximate. The study’s question is whether tuning a generalization of this kind, with the same budget as each hand-picked rule, finds a better trade on one deployment.

2.6 Caveats that bound the results

Building, calibrating and auditing the study surfaced properties of the API, the replay host and the workloads that can change which policy wins, and several of them shape the results. Replay’s router state is perfectly fresh, while a live router sees cache events and load late (caveat 1; Sections 9.9 and 11). Replay fills `expected_output_tokens` with the true output length, a value no live router has; no learned feature reads it (caveat 2). Host knobs change what the inputs mean: without assumed KV reuse every prefix hash is unique to its request, so prefix-keyed policies lose their keys, and the learned policy must run with its training-time knobs (caveat 3; Section 12.6). Logged arrival timestamps are quantized, so the harness spreads arrivals per replicate (caveat 9; Section 4.3). No AgentX play (one recorded agent run; Section 4.4) fits a 32K context, so the deployment runs at 128K with YaRN context extension (a factor of 4 over the model’s original 32,768 tokens), where AIS’s accuracy is least validated, and AIS prices a mixed prefill pass at the batch mean, an artifact a search could exploit (caveats 10 and 13; Sections 10 and 12.1). Finally, the request-count objective can be gamed (caveat 15; Section 10). Appendix C lists all 15 with their evidence and status (fixed, follow-up, inherent).

3 Problem and objective

This section fixes what a routing policy decides, what it may observe, how load is applied, and the score by which every policy, learned or heuristic, is compared.

3.1 The routing decision

Deployment. A deployment runs N identical aggregated workers, each a vLLM engine serving Qwen3-32B with tensor parallelism 2 on H100 SXM GPUs, so prefill and decode share a worker. Each worker holds 18,863 KV blocks of 16 tokens, 301,808 tokens in all. The study covers $N \in \{2, 4, 6, 8, 16, 32\}$; heterogeneous worker sets are out of scope. The router never holds a request back.⁸

⁸`router_queue_threshold` is left unset, which disables router-side queueing; an ablation that tuned it jointly found no measurable gain (Section 9.7).

A sequential discrete choice. When request j arrives at time a_j , the router’s worker-selection policy π picks one worker from the eligible candidate set $S \subseteq \{1, \dots, N\}$. Three properties shape the problem.

- *The choice is sequential.* The chosen worker computes the uncached part of j ’s prompt and keeps the resulting KV blocks, which later requests that share the prefix can reuse and which may evict other prefixes’ blocks. Request j also adds to that worker’s prefill and decode load until it completes. A decision’s cost or benefit therefore lands on other requests, often much later in simulated time, and no per-decision label exists. Policies are scored on whole episodes (Section 3.3).
- *The worker set varies.* N differs across deployments, and S can be smaller than N when the host marks workers ineligible. A policy defined on a fixed-length vector of all workers’ states would need retraining for every N . A policy that scores each candidate with one shared function and picks the best applies to any $|S|$. This is the discrete-choice form of the learned policy (Section 5), and it is what lets the test set hold out worker counts that training never saw.
- *The choice is made online.* It runs once per request on the router’s request path, from information available at a_j , so it must be cheap and must not depend on the future.

What the router observes. At decision time a policy sees, through the router plugin API (Section 2.2):

- the request: its prompt length ISL_j , its prefix block hashes and an optional session identifier;
- per candidate, the router’s cache estimate: blocks of overlap with the request’s prefix and the estimated number of cached tokens;
- per candidate, load the router tracks from its own dispatches: active prefill tokens, decode blocks and active requests;
- per candidate, its configured KV capacity in blocks.

The learned policy reads nothing else. In particular it ignores the request’s expected output length, which offline replay fills with the true output length, and every performance-model estimate; engine-internal state such as waiting-queue length or KV usage is not available to it. Because it uses only these inputs, the artifact trained in replay is the one the live router runs (Section 5.1). Replay applies cache events to the router’s index at once, while a live router sees them late; that difference is a robustness check (Section 8.7), not part of the problem definition.

3.2 Load modes

A cell offers its trace to the workers in one of three ways, which differ in what a policy can change.

Open loop. Requests arrive at the trace’s timestamps, compressed or stretched by a speedup factor that sets the load. A policy cannot change when requests arrive, only how well they are served. In multi-turn session cells the session start times are fixed this way, while each follow-up turn is released a think time after the previous turn completes, as in a real conversation.

Closed loop. A fixed number of sessions is in flight; a session holds its slot through its think times, and a new session starts when one ends. Each Mooncake trace row is its own one-turn session. A policy that serves requests faster also receives more of them, so throughput depends on the policy.

Agentic lanes. A fixed number of lanes each run one recorded agent play at a time. Inside a play, requests follow the recorded dependency graph (sequential turns, subagent spawns and joins, tool delays), and a lane starts its next play when the previous one ends.

Mooncake, FAST25 and session cells run in open and closed loop, and AgentX cells in lanes; calibration sets each cell’s load (Section 4.5). Each family has a primary mode, used to set its service objective: open loop for Mooncake and sessions, lanes for AgentX. Results are reported per mode, because open and closed systems respond to scheduling differently [45]. Session and agentic work is released causally because replaying follow-up turns at their recorded timestamps would fix a release schedule, and cache state, that a different policy is supposed to change [4, 41].

3.3 Objective

Good requests. The service objective in this study concerns the pace of the token stream and the total request latency, not the time to first token. A request is *good* iff it completes and

$$g_j = \mathbf{1}[\overline{\text{ITL}}_j \leq I_{\max}] \cdot \mathbf{1}[\text{E2E}_j \leq s_{\max} E_0(\text{ISL}_j, \text{OSL}_j)], \quad (2)$$

where $\overline{\text{ITL}}_j = (\text{E2E}_j - \text{TTFT}_j) / (\text{OSL}_j - 1)$ is the mean inter-token latency, the ITL clause is skipped when $\text{OSL}_j \leq 1$, and the slowdown clause allows a relative tolerance of 10^{-6} for floating-point rounding. Rejected, errored and incomplete requests are never good. $(I_{\max}, s_{\max})$ is one pair per workload family (Table 5). TTFT_j is reported but is not part of Equation (2); it enters only through E2E_j .

The reference latency. $E_0(\text{ISL}, \text{OSL})$ is the request’s latency alone on an idle worker with no prefix reuse: a full chunked prefill of ISL tokens followed by $\text{OSL} - 1$ decode steps at batch size 1, timed by AIS.⁹ It depends only on (ISL, OSL) and is cached per pair. E_0 is used only for scoring; no policy sees it.

Why a slowdown bound. Request sizes vary widely. Mean prompt lengths are 8,590 tokens on Mooncake and 12,035 to 15,326 on the FAST25 traces, and AgentX plays were admitted as long as every request, prompt plus output, fits the 131,072-token context. In AIS’s timing, prefill alone on an idle worker takes 55.49 ms at 1,024 tokens, 475.97 ms at 8,192, 2,550 ms at 32,768 and 17,815.54 ms at 120,000. A single absolute bound on end-to-end latency would be unattainable for the longest requests even on an idle worker, or slack for the shortest. Fixed first-token deadlines have the same problem; SMetric’s authors note that providers typically set the TTFT deadline linear in the request length [54]. Dividing by E_0 scales the bound with each request’s own unavoidable work: under the Mooncake bound $s_{\max} = 3.46625$, a request may take up to about 3.5 times its uncontended latency, whatever its size.

Two properties of E_0 keep the bound fair across policies. It is policy independent, so every policy is held to the same per-request bound. And it assumes no prefix reuse, so a cache hit lowers E2E_j but not E_0 : a policy that earns more reuse finds the bound easier to meet, never harder.

Why two clauses. The clauses catch different failures. For a long prompt, $s_{\max} E_0$ is dominated by prefill time, so a slow token stream after a fast prefill can pass the slowdown clause; the ITL clause catches it. Conversely, a request that waits a long time and then streams at an ordinary pace

⁹ E_0 agrees with isolated single-request replays to within 2×10^{-8} relative, and an independent audit’s own computation matched it on 880,139 pairs.

passes the ITL clause and fails the slowdown clause. At the calibrated bounds the slowdown clause does most of the work: at the calibration load nearest L2 (the load level at which π_0 's good fraction is 0.85; Section 4.5) on $N = 8$, the ITL clause alone fails 0.6% of π_0 's requests on sessions, 1.8% on AgentX and 4.5% on Mooncake, while 15% of requests are not good at L2 by construction.

Windowed goodput. A cell's score is its windowed goodput $G_{c,k}(\pi)$, good requests per simulated second inside a measurement window $\mathcal{T}_c$ that starts after a warm-up. Its form depends on the load mode. In open-loop cells the window is a fixed interval of arrival times, and

$$G_{c,k}(\pi) = \frac{1}{|\mathcal{T}_c|} \sum_{j: a_j \in \mathcal{T}_c} g_j, \quad (3)$$

so a request that arrives in the window counts even if it finishes after it, and one that never completes counts as not good. In closed-loop and lane cells, G counts the good requests that *complete* inside a half-open window, divided by its length. That window ends at full occupancy, the last instant at which every session or lane slot is busy, so its end, and hence its length, depends on the policy.

The window removes three distortions.

- *Drain.* The replay's native goodput divides good requests by the makespan, which includes the drain after the last arrival, so it mixes SLO attainment with how fast a policy clears the tail. In closed loops and lanes, completions after full occupancy happen with fewer requests in flight, and in lane cells the lanes run out of plays at different times, so counting the tail would credit easy, under-loaded requests.
- *Cold start.* Caches start empty and session populations ramp up, so the window opens after a warm-up long enough for the population to settle (Appendix D.3).
- *Deferral.* An objective that ignored requests finishing after the horizon would reward deferring work past it; Decima's learned scheduler did exactly this under a fixed horizon [30]. Counting in-window arrivals that finish late removes the incentive.

When every arrival is set by the trace, as in the open-loop Mooncake and FAST25 cells, the number of in-window arrivals is the same for every policy on a replicate, so the ratio of two policies' goodputs equals the ratio of their in-window good fractions. In closed-loop and lane cells goodput also rewards throughput, which the policy legitimately controls there. SLO attainment as the primary serving metric follows Zhong et al. [65].

SLO and load calibration. The bounds $(I_{\max}, s_{\max})$ are fixed per family from runs of π_0 on train segments at $N = 8$, and each cell's load is one of three levels L1, L2, L3, at which π_0 's good fraction is 0.95, 0.85 and 0.65. Levels are calibrated separately for every N and transform, on train segments only, and both bounds and levels were frozen before any policy was tuned (Section 4.5). The bounds differ widely by family, from $s_{\max} = 1.16134$ on AgentX to 3.46625 on Mooncake, because each is anchored at its own family's load knee. Every cell therefore sits where the default router is starting to miss its objective, the regime in which routing changes outcomes; that the regime is defined relative to one baseline is a threat discussed in Section 12.

Normalized score. Goodput levels differ by orders of magnitude across cells, so every comparison is a ratio on the same cell and replicate:

$$\rho_{c,k}(\pi, \pi') = \frac{G_{c,k}(\pi) + \varepsilon}{G_{c,k}(\pi') + \varepsilon}, \quad \Delta_{c,k}(\pi, \pi') = \text{clip}(\log \rho_{c,k}(\pi, \pi'), -\log 3, \log 3), \quad (4)$$

with $\varepsilon = 10^{-3}$ good requests per second. Both policies run on the same workload replicate k with the same tie-breaking seed (common random numbers, Section 6.3). Training uses $\pi' = \pi_0$ (Equation (8)); the headline uses $\pi' = \pi_{\text{base}}^*$ (Equation (9)). The log makes the score symmetric, so doubling goodput on one cell and halving it on another nets to zero. The clip limits any one cell to a factor of 3 either way, which matters because a mean of raw ratios is dominated by the few cells where one policy collapses [2], and such cells occur here. On all 9 open-loop session cells of the train and validation splits, round-robin’s goodput is 0.111 to 0.425 times the default router’s, and below 1/3 on 5 of them. A policy that far past its stability limit has no steady state, so the ratio is not a stable quantity.¹⁰ The clip scores every ratio below 1/3 alike, so that unstable detail cannot drive training or the headline. The clipped log-ratio was chosen over the mean of ratios at calibration, before any training.

What “better” means. The headline compares the learned policy with the best baseline *on the same footing*: each policy, learned or heuristic, is one configuration tuned on the pooled train split with the same CMA-ES budget and selected on validation (Sections 6 and 8). The best baseline π_{base}^* is the baseline with the highest mean clipped log-ratio on fresh validation replicates. The learned side is chosen by the same rule among the learned variants that read only router-observable signals, and the number of variants compared is reported. The comparison is made once, on the frozen test set, by the pre-registered test of Section 8.4. Because the objective itself is a modeling choice, the same test records are also re-scored with scaled bounds and with alternative definitions of a good request: ITL alone, slowdown alone, and the ITL bound combined with either a length-scaled TTFT bound or an absolute latency bound. These secondary analyses never change the headline. Appendix A collects the symbols.

4 Simulator and workloads

Every policy comparison in this paper, except the live check of Section 11, comes from offline replay in AISimulate, which provides both the replay engine (AISim) and the forward-pass performance model (AIS). AISim runs Dynamo’s KV router in front of N simulated vLLM engines, in simulated time, on recorded or generated request traces. This section describes that environment and the workloads it replays, so that a reader can judge which results could change on a live deployment. Three properties carry the rest of the paper. Engine timing comes from AIS, not from the mock engine’s fallback model (Section 4.1). A replay is a deterministic function of the policy, the cell and a replicate index, so every comparison between two policies is paired on identical workloads (Section 4.3). And the cells, their service objective and their loads were fixed on train segments with the default router alone, then frozen with SHA-256 hashes before any policy was tuned (Sections 4.5 and 4.6). What replay leaves out is listed in Section 4.2.

4.1 Offline replay with AIS timing

AISim’s offline replay is a discrete-event simulation in virtual time [37, 38]. The event loop, the mock engines and the report collector come from the `aisimulate-core` crate; Dynamo contributes the adapter that places each arriving request through its KV router and feeds the engines’ KV-cache events back into the router’s index. Simulated time jumps from one event to the next (an arrival, an engine step, a completion), so the outcome of a replay does not depend on how fast the host runs it. Each mock engine runs `aisimulate-core`’s model of the vLLM scheduler: continuous batching,

¹⁰On one such cell, doubling the warm-up history halved round robin’s ratio, from 0.118 to 0.055.

Table 2: The simulated deployment: engine configuration of every worker. Software versions and the configuration file’s hash are in Appendix M.

Model, backend	Qwen/Qwen3-32B on vLLM 0.24.0, aggregated (prefill and decode on one worker)
Hardware	H100 SXM (<code>h100_sxm</code>), tensor parallelism 2
Timing	AIS, operator-level estimation; router-side AIS off
KV cache	block size 16; 18,863 blocks (301,808 tokens) per worker, pinned to the AIS estimate so every replay uses the same capacity
Context	<code>max_model_len</code> 131,072 (YaRN)
Batching	<code>max_num_seqs</code> 1,024; <code>max_num_batched_tokens</code> 8,192; chunked prefill and prefix caching on
Workers	$N \in \{2, 4, 6, 8, 16, 32\}$ identical workers

chunked prefill under a per-step token budget, and prefix caching over a paged KV cache. The duration of each forward pass comes from AIS, an operator-level performance model of the configured model, GPU, backend and parallelism. Batch-level simulators of this kind are an established tool for sizing LLM serving [3].

Worker selection in replay goes through the same Rust policy catalog and the same policy YAML as the live router (Section 2.2).

AIS times the engines only. The router-side AIS estimator, which some router modes use to model prefill load, is off: the policies under study see the same router state a live router without a performance model would see (Section 3.1).¹¹

Engine facts. Table 2 lists the deployment. Setup confirmed that replay uses AIS timing: a 1,024-token request replayed alone matches the AIS estimate to within 1 ns. The mock engine’s polynomial fallback underestimates a 120,000-token prefill 7.2 times (Table 22 in Appendix D), so replay without AIS would mislead about long-context traffic. Appendix D gives further engine facts.

4.2 What replay does not model

Each item below is a difference between replay and a live deployment that could change which policy wins; Table 21 in Appendix C gives their status. Section 12 weighs them, and the robustness and live checks of Sections 8.7 and 11 probe the first and the last directly.

- **Router state is perfectly fresh.** Replay applies the engines’ KV events to the router’s index synchronously; a live router sees them late. Load counters come from the router’s own request tracking in both settings, but whether their update timing matches is an assumption. A replay patch that delays these updates is used only as a test-time robustness check (Section 8.7).
- **One cache tier.** Replay models only the GPU (device) KV cache. A live router with host or disk tiers blends them into its overlap estimate, a signal no policy saw in training.
- **Fields a live router would not have.** Replay fills the request’s expected output length with the true output length, and in closed-loop mode it passes the router a single-use session identifier for every flat trace row, which open-loop mode and a live client do not. No headline policy reads the output length (Section 5.1). A single-use session identifier never has a previous request, so session affinity is unaffected, and the `v2` home indicator keys on the prompt prefix only (Appendix E).

¹¹A separate, secondary AIS-informed league of policies, which may read AIS-derived estimates at runtime, runs on its own build with the router-side estimator on (Section 7.2 and Appendix J).

- **AIS fidelity** [hypothesis about the size of the effect]. AIS times a prefill pass that mixes requests from the mean new and mean cached length of the batch, so adding a short prompt to a long-context chunk can shorten the predicted pass (from 1,744 ms to 1,150 ms in one probe). Its attention table for this configuration covers prompts up to 16,384 tokens with no cached prefix, so longer prefills are extrapolated. The likely effect is a small optimism when short prompts share a worker with long-context prefills; a search procedure could exploit it, which matters most for AgentX.
- **Errors near capacity.** Calibration places every cell near the default router’s throughput knee (Section 4.5), which is where routing matters and also where small timing errors are most amplified [3]. This study does not validate AIS’s absolute accuracy; it relies on replay to rank policies, which the live check tests.
- **No network, no failures, identical workers,** and, in open-loop cells, arrivals that do not react to latency (Section 3.2).

4.3 Determinism and common-random-number replicates

Determinism. For a fixed policy, workload and policy seed, a replay reproduces every per-request row exactly, across repeated runs in one process and across fresh processes. The only run-to-run variation found at setup was the default picker’s unseeded tie-break, which this study removes: the branch adds a `seed` parameter to `dynamo-default-cost-fn`, and the harness always passes one. Bit-exact results also need the same operating-system image (Appendices D and M.6).

Where noise comes from. Because a replay is deterministic, noise has to come from the workload: what is arbitrary in a recorded trace is the order of requests logged at the same instant. Replicate k of a cell re-draws only that arbitrary part (each session’s first arrival inside its logging slot, or the order of tied sessions or of recycled AgentX copies) together with the policy seed $k + 1$, and the same replicate is used for every policy: common random numbers (Appendix D).

Spreading arrivals inside their slot is a modeling choice, not only a noise source. Replay delivers a logged burst at once whatever the arrival speedup, and with load matched per worker a worker’s share of each burst grows as N shrinks. With bursts kept, the default router’s 85th percentile slowdown at $N = 2$ and 4,000 input tokens per second per worker was 8.32; with arrivals spread it was 1.46. Without the spread, small- N cells would measure burst queueing that the logging grid created.

Replicate noise is comparable to the effects of interest but smaller than segment heterogeneity (Section 8.2 and Appendix D). Replicates re-draw one segment; they do not create new independent workloads (Section 8).

Cost. Replay is cheap enough for black-box search. A 2,000-request Mooncake replay takes about 1.5 s of one CPU core, nearly flat in N ; an AgentX cell at $N = 32$ takes 2–4 minutes. Appendix F.9 gives this study’s compute.

4.4 Workloads

The workloads differ in where prefix reuse comes from, because that is what a cache-aware router exploits: shared system prompts across users (Mooncake, FAST25), a conversation’s own growing history (sessions), or long per-agent contexts carried across tool calls (AgentX). Table 3 summarizes the sources and Table 4 how they are split. A *segment* is a disjoint piece of one source, such as

Table 3: Source traces as recorded or generated, before windows, transforms or recycling. Reuse is the mean fraction of a request’s leading blocks already seen earlier in the trace, an infinite-cache hit rate; for AgentX it is computed within each play, since plays never share prefixes. Sessions are pooled over the ten 720 s generator seeds.

Source	Requests	Span	Prompt tokens			Output tokens		Reuse
			mean	p50	p90	mean	p50	
Mooncake	23,608	60 min	8,590	6,345	16,787	182	30	0.63
FAST25 conversation	12,031	59 min	12,035	6,909	27,367	343	350	0.38
FAST25 synthetic	3,993	17 min	15,325	11,587	38,615	149	69	0.42
Sessions (10 seeds)	36,170	12 min each	6,542	5,539	11,932	341	249	0.83
AgentX (82 plays)	3,132	—	66,313	66,944	103,488	749	314	0.87

Table 4: Segments and cells per split. Open: open-loop arrivals; closed: fixed concurrency; lanes: fixed number of concurrently running agent plays. FAST25 conversation shares Mooncake’s arrival and length skeleton, so its windows are aligned to Mooncake’s w3 and w5 and counted as those segments.

Family	Segments	Load mode	Cells		
	train / val / test		Train	Val	Test
Mooncake	w0, w2, w4 / w1 / w3, w5	open	12	3	18
		closed	6	2	7
Sessions	s0–s3 / s4, s5 / s6–s9	open	6	3	9
		closed	2	2	6
AgentX	A1–A3 / V1–V3 / T1–T4	lanes	8	4	14
FAST25 conversation	– / – / (w3, w5)	open	–	–	2
		closed	–	–	1
FAST25 synthetic	– / – / x0, x1	open	–	–	2
		closed	–	–	1
All	10 / 6 / 12		34	14	60

a time window, a generator seed or a subset of agent plays. Segments are the unit of statistical independence: cells cut from one segment at different N , loads or transforms count as one workload (Section 8).

Mooncake. The public Mooncake trace [21] holds 23,608 single-turn requests over one hour, with prefixes given as 512-token hash blocks under four distinct first blocks. It is cut into six disjoint 10-minute slices, each a 4-minute warm-up followed by a 6-minute measurement window: train w0, w2 and w4, validation w1, test w3 and w5.

FAST25 conversation and synthetic. Two traces from the Mooncake project’s FAST’25 release [21] are test-only families. The conversation trace shares Mooncake’s arrival grid and length skeleton but has a different prefix structure; its windows are aligned to Mooncake’s w3 and w5 and are counted as those segments. The synthetic trace is independent of both, and its longest prompts exceed the context limit and are dropped by the context-cap transform described below. A third trace from that release is excluded as a relabeled copy of Mooncake (Appendix D.2).

Synthetic sessions. Replay’s built-in synthetic-session source cannot express a conversation: a later turn does not extend the earlier turns’ context, which leaves session affinity nothing to exploit.

We therefore generate multi-turn sessions in which every turn resends the whole conversation so far plus a new user message, so its hash blocks extend the previous turn’s. Each later turn is released a think time after the previous turn *completes*, so release is causal: a slow policy delays the session’s next request, as it would with a real user. One generator seed defines one segment (s0–s3 train, s4 and s5 validation, s6–s9 test); Appendix D.2 gives the generator’s parameters.

AgentX. AgentX plays come from a public corpus of recorded agent sessions [46] (393 plays, 98,827 requests; Appendix D.2 gives the pinned revision). A play is one agent run: a dependency graph of requests, including subagents that run in parallel and join, with recorded tool and think gaps between them. We keep the 82 plays in which every request satisfies prompt plus $\max(\text{output}, 1) \leq 131,072$, whole and unchanged: 3,132 requests with 39 explicit subagent groups and a mean prompt of 66,313 tokens. The plays were recorded with several Claude models and are replayed as Qwen3-32B traffic, so they supply realistic structure (lengths, reuse, dependencies, gaps), not Qwen3-32B’s behavior. Idle gaps over 300s are capped by a time warp that keeps every busy period and ordering (Appendix D.2). The plays are dealt, by a seeded round robin that places plays with subagents first, into ten disjoint segments: A1–A3 (11 plays each) for training, V1–V3 and T1–T4 (7 each) for validation and test. To keep 16 and 32 workers loaded, each play is converted once into AISim’s agentic trace format and recycled, with disjoint hash ranges so copies never share KV prefixes (Appendix D.2.1).

Transforms. Seeded, deterministic transforms stretch prompt length (shared and unique parts separately), output length, prefix-root count and think time, and cap context, so that the test can ask whether a policy extrapolates (Appendix D.2). Train cells stay within $[1, 1.5]$ for both prompt multipliers, $[0.5, 2]$ for output and think time, and $[1, 2]$ for prefix roots. Ten test cells go beyond these ranges: prompt multipliers of 2.5, output multipliers of 0.25, 2.5 and 4, four prefix roots, and think-time multipliers of 0.25 and 4.

4.5 SLO and load calibration

Before any policy was tuned, calibration fixed each family’s SLO pair $(I_{\max}, s_{\max})$ and each cell’s load, using only π_0 (`default@defaults`) and round robin, and only train segments. Calibration has to solve two problems at once. Routing only matters in a band of load: far below capacity every policy meets the SLO, far above it every policy fails. And the SLO and the band depend on each other, because whether a load is “heavy” depends on the SLO it is scored against. The rules below fix both, and Figure 5 in Appendix D.3 shows the result at $N = 8$.

Load levels. L1, L2 and L3 are the per-worker loads at which π_0 ’s windowed good fraction, averaged over the train segments and two replicates, falls to 0.95, 0.85 and 0.65 under the family’s SLO. Loads are expressed per worker (offered input tokens per second for Mooncake-format open loop, sessions per second for session open loop, concurrent sessions for closed loop, lanes for AgentX) and multiplied by N , rounding to an integer for closed loop and lanes. Each cell’s measurement window starts after a warm-up set per load mode (Appendix D.3).

Matching load per worker does not hold the routing regime fixed across worker counts. Under join-the-shortest-queue, waiting vanishes as the number of servers grows at a fixed per-server load [53], and one measured learned router tied a simple baseline, led it and tied it again as the pool widened [52]. Levels are therefore calibrated separately at every $N \in \{2, 4, 6, 8, 16, 32\}$. They differ: Mooncake’s open-loop L2, for example, is 7,477 input tokens per second per worker at $N = 2$ and

Table 5: Calibrated service objective per family and per-worker loads at $N = 8$ (rounded). The SLO applies to every split, load mode and N of its family; the FAST25 families inherit Mooncake’s. The last column is round robin’s good fraction at π_0 ’s L2 in the primary mode, a measure of how much routing matters at that load.

Family	$I_{\max}$ (ms)	$s_{\max}$	Mode	Unit	Per-worker load at $N = 8$			RR good frac. at L2
					L1	L2	L3	
Mooncake	252.117	3.46625	open	input tok/s	5,903	8,798	11,142	0.711
			closed	sessions	3.84	7.67	14.15	
Sessions	46.0478	2.11784	open	sessions/s	0.350	0.380	0.409	0.198
			closed	sessions	29.0	33.1	37.5	
AgentX	38.6205	1.16134	lanes	lanes	2.57	3.39	4.74	0.572

8,993 at $N = 16$.¹² Calibration therefore ran π_0 and round robin at the held-out worker counts, but only on train segments, and it replayed no test trace. Transformed cells get their own levels, calibrated with the transform applied to the train segments, because a transform changes how much load a worker can carry; the held-out FAST25 families inherit Mooncake’s per-worker levels and SLO.

The SLO as a fixed point. Each family’s bounds are set at $N = 8$ in its primary mode: $s_{\max}$ so that π_0 ’s good fraction is 0.65 at 0.9 times the family’s load knee (where π_0 starts to saturate), which makes that load L3, and $I_{\max}$ at 1.2 times the 95th percentile of π_0 ’s per-request mean ITL at L2, iterated to a fixed point (Appendix D.3). The resulting values are in Table 5.

What calibration does not fix. Calibration fixes where the cells sit; it does not guarantee that every cell separates policies (Figure 5). One intended spread was not achieved. Measured as the distinct prompt tokens touched in 100s of replay divided by the workers’ total KV capacity, every one of the 55 open-loop cells has a cache pressure between 0.62 and 2.75, so no split contains a cell with little cache pressure.

4.6 Splits and the frozen test set

The splits follow the segments of Table 4. Training uses $N \in \{4, 8\}$, validation adds $N = 6$, and the test set covers all six worker counts. Every test cell lies on a test segment, so it is held out at least by workload; each also has a primary hold-out axis:

- **time window:** Mooncake w3 and w5 at $N \in \{4, 8\}$;
- **workload draw:** session seeds s6–s9;
- **agent plays:** AgentX segments T1–T4;
- **family:** FAST25 conversation and synthetic, never trained on;
- **transform extrapolation:** the ten cells beyond the train ranges (Section 4.4);
- **worker count:** $N \in \{2, 6, 16, 32\}$, where $N = 2, 16$ and 32 are extrapolation and $N = 6$, which validation also uses, is labeled selection-exposed.

¹²At $N = 16$ and 32 the AgentX grid starts at 3 lanes per worker, where π_0 ’s good fraction is already 0.916 and 0.934, so AgentX has no L1 there and no cell uses one.

Table 30 in Appendix H gives the number of test cells per axis and per N . Calibration added four AgentX test cells at $N = 16$ and 32, which recycling made possible (Appendix D.2.1).

Freeze. After calibration the cells were frozen. The freeze recorded SHA-256 hashes of the test cells and their contents, the traces they replay, the engine configuration and the split manifest. The freeze rule is that no test cell is evaluated before the single test pass, which verified all 149 hashes before running. The test set was frozen twice, once before and once after a calibration fix, with no test record in between (Appendix D.4).

5 The learned policy

learned-choice takes over the final step of Dynamo’s worker selection. Given the eligible candidates for one request and the signals the router already tracks for them, it scores each candidate with a linear utility over a handful of features and routes to the highest score. Three constraints shape the design. It reads only inputs the live router has, so the coefficients tuned in replay can run unchanged in a live router (Section 5.1). Its coefficients are shared by all candidates and its features do not change when the candidate set is duplicated, so one coefficient vector applies at every worker count, including counts that training never sees (Section 5.5). And one coefficient vector, $\theta = -\mathbf{e}_0$, reproduces the default router, so tuning starts from the status quo and every learned coefficient reads as a deviation from it (Section 5.6).¹³

5.1 Router integration

In the API of Section 2, **learned-choice** registers under the type string **learned-choice** as one scorer and one picker. The scorer is the default router’s own cost function (Equation (1)) evaluated at default weights; its output becomes feature 0 (Section 5.2). The picker (a **WorkerPicker**) computes the other features, the utilities and the choice. It declares the plugin inputs **CACHE** and **LOAD** and reads exactly the inputs of Section 3.1, the prefix hashes only for feature set v2.

The coefficients are ordinary parameters in the policy YAML, so a new coefficient vector is a new file rather than a rebuild. Each CMA-ES candidate in training is one such file, and the file that training selects is the file a live router would load. Appendix E shows such a file (Figure 6) and gives the provider that instantiates the policy and its strict parsing rules.

Cost per decision. A decision costs $O(Nd + rd)$ arithmetic plus an $O(N \log N)$ sort; its latency against **dynamo-default-cost-fn** was not measured (Appendix E).

Excluded inputs. The policy never reads the request’s expected output length, which offline replay fills with the true output length, so reading it would give the policy future information that no live router has. It never reads the modeled prefill backlog (the router’s **PREFILL_TIME** input) or any other estimate from AIS, the performance model that times the simulated engines, so it stays independent of the simulator it is trained in and needs nothing at runtime that a live router lacks. Two independent audits checked these exclusions (Appendix E). Section 12.6 lists the host settings a deployment must match.

Table 6: Feature set v1, in order. $T_{\text{ch}} = 8192$ tokens, the engine’s chunked-prefill batch; $b_j = \max(\lceil \text{ISL}_j / 16 \rceil, 1)$. Feature 0 is computed by the default scorer itself at default weights, so $\theta = -\mathbf{e}_0$ reproduces the default router.

f	Name	$x_{i,f}$
0	default_logit_scaled	c_i^{def}/b_j at default weights
1	overlap_frac	o_i/b_j
2	new_prefill_tokens_k	$\max(\text{ISL}_j - h_i, 0)/T_{\text{ch}}$
3	active_prefill_tokens_k	$L_i^{\text{pf}}/T_{\text{ch}}$
4	kv_load_frac	$L_i^{\text{dec}}/\text{cap}_i$
5	active_requests_s	$L_i^{\text{req}}/32$
6	session_affinity	1 if i served the previous request of j ’s session, else 0
7	isl_x_prefill_load	$(\text{ISL}_j/T_{\text{ch}})x_{i,3}$

5.2 Features

Table 6 defines the eight v1 features, which have been frozen since calibration began. They are built only from the inputs listed in Section 3.1.

- **The anchor.** Feature 0 is the default cost divided by the request’s block count. The division does not change the default’s argmin, because b_j is the same for every candidate. Because the default scorer is composed into the policy, the cases it distinguishes (missing cache tiers, unavailable load, untracked prefill) are handled exactly as the default handles them, and the host’s own score-weight settings do not change feature 0.
- **Cache, load and session.** Features 1–5 come from the router’s cache estimate and its own request tracking; feature 6 needs a bounded session map, because replay never sets the router’s affinity target (Appendix E).
- **Prompt length.** A request quantity alone, such as ISL_j , is the same for every candidate and cannot change an argmax. Prompt length matters only through features that combine it with a worker quantity (features 0, 1, 2 and 7; feature 7 is the explicit product of prompt length and queued prefill) or through the context term (Section 5.4).

The features’ within-set spreads differ by two orders of magnitude, from 0.060 to 6.6 on candidate tables reconstructed from the default router’s replays in the pilot, a smaller tuning run that preceded the main one (Appendix G; *preliminary (pilot, train subset)*). Training therefore bounds each coefficient relative to its feature’s spread (Section 6.2).

Reading the coefficients. With $\theta_0 = -1$, as in every learned arm of the model ladder (M1, M2 and their variants; Section 5.7), $u_i = -c_i^{\text{def}}/b_j + \sum_{f \geq 1} \theta_f x_{i,f}$, so every term is measured in default-cost blocks per request block. Two consequences help when reading tuned values. When the clip in Equation (1) is inactive, that is, when the overlap credit does not exceed the prompt’s blocks plus the queued prefill blocks, c_i^{def} contains $-o_i$, so $\theta_1 x_{i,1} = \theta_1 o_i/b_j$ adds θ_1 blocks of credit per overlapping block to the default’s one: $\theta_1 > 0$ is more cache-seeking than the default. And features 0 and 1 are per request block while features 2–7 are not, so at fixed θ the anchor’s differences between candidates shrink as $1/b_j$ relative to the load features: the balance between the default cost and the learned load terms depends on prompt length by construction.

¹³The normative specification, `FEATURES.md`, sits next to the policy’s Rust code; this section follows it.

Table 7: Features that v2 appends to v1. $(y)^+ = \max(y, 0)$; $(\bar{x}_S)_f$ is the candidate mean of feature f . Indices 12–20 apply one transform to each of features 4, 3 and 5, in that order; the ratio floors are 0.01, $512/T_{\text{ch}}$ and $1/32$, about one unit of each load.

f	Name	$x_{i,f}$
8	log_ptok	$\ln(\max(L_i^{\text{pf}} + (\text{ISL}_j - h_i)^+, 1)/1024)$
9	log_new_prefill	$\ln(\max((\text{ISL}_j - h_i)^+, 1)/1024)$
10	log_bs	$\ln(1 + L_i^{\text{req}})$
11	hash_home	1 if i has one of the two highest rendezvous weights for the request’s prefix key (the hash of the prompt’s first 256 tokens); 1 for all candidates when $ S \leq 2$; 0 for all without prefix hashes
12–14	*_ratio	$x_{i,f}/(\text{floor}_f + (\bar{x}_S)_f)$
15–17	*_below_frac	$ \{i' \in S : x_{i',f} < x_{i,f}\} / S $
18–20	*_excess	$(x_{i,f} - (\bar{x}_S)_f)^+$
21	new_prefill_x_requests	$x_{i,2} x_{i,5}$
22	prefill_attn	$x_{i,2} (\text{ISL}_j/T_{\text{ch}} - x_{i,2}/2)$, the attention work of prefilling the uncached suffix, in units of T_{ch}^2

Feature set v2. Table 7 lists 15 features that v2 appends to the eight of v1, which keep their indices. They are used by the M1-v2 arm (Section 5.7). They add three things v1 cannot express. Log-scale load terms make products of loads linear in the utility, so LMetric’s multiplicative rule [63] lies inside the model class: $\theta = -(\mathbf{e}_8 + \mathbf{e}_{10})$ routes to the minimum of $(L_i^{\text{pf}} + (\text{ISL}_j - h_i)^+)(L_i^{\text{req}} + 1)$, LMetric’s product with queued prefill, and $\theta = -(\mathbf{e}_9 + \mathbf{e}_{10})$ reproduces the branch’s `lmetric` port as evaluated (Section 7.1) without its hot-spot filter when cached tokens are whole overlapping blocks, as in replay.¹⁴ A prefix-hash home indicator marks two hash-chosen candidates per prompt prefix, the consistent-hash candidate membership of DualMap and Lodestar [23, 61]; Appendix E discusses its key. Set-relative load terms make a worker’s load count relative to the other candidates; they enter nonlinearly, because a linear set-relative term would be a no-op (Section 5.4). The ratio form follows divisive normalization [56]. The product of new prefill and active requests stands for the prefill–decode interference that two routers model [17, 48], and the attention term is SMetric’s prefill cost [54].

A third feature set, v3, adds AIS estimates for the secondary AIS-informed league, which never enters the headline (Appendix J).

5.3 Utility and decision rule

For request j with candidates S and features x_i , the policy scores

$$u_i = \theta^\top x_i + \sum_{\ell=1}^r z_{S,\ell} p_\ell^\top x_i = w_S^\top x_i, \quad w_S = \theta + \sum_{\ell=1}^r z_{S,\ell} p_\ell, \quad (5)$$

and decides

$$\tau = 0 : \quad i^* \in \arg \max_{i \in S} u_i \text{ (seeded tie-break);} \quad \tau > 0 : \quad \Pr[i^* = i] = \frac{\exp(u_i/\tau)}{\sum_{i' \in S} \exp(u_{i'}/\tau)}. \quad (6)$$

With $r = 0$ this is a conditional logit, the simplest member of the generalized extreme-value family of choice models [51]: one coefficient vector shared by every candidate, so the same parameters apply

¹⁴A unit test checks both representations on 1,000 random tables.

at any N . The context values $z_{S,\ell}$ are scalars computed once per decision (Section 5.4); folding them into w_S is what keeps a decision at $O(Nd)$. A non-finite utility fails the selection rather than routing on it.

M1, M2 and their variants are trained and evaluated at $\tau = 0$, and a $\tau > 0$ variant is only a conditional robustness extra.¹⁵ At $\tau = 0$, scaling θ and the p_ℓ together never changes a decision, which is why training pins $\theta_0 = -1$ (Section 6.2). Algorithm 1 in Appendix E gives one decision in full.

Determinism and tie-breaking. Candidates are visited in worker-ID order, and under the default `one_draw` discipline every decision consumes exactly one random draw, so a seeded policy decides identically across processes and common random numbers stay aligned across the CMA-ES candidates of a generation (Appendix E).

5.4 Context terms and the IIA restriction

With $r = 0$, the policy inherits the conditional logit’s independence of irrelevant alternatives (IIA). At $\tau > 0$,

$$\frac{\Pr[i^* = i]}{\Pr[i^* = i']} = \exp(\theta^\top (x_i - x_{i'})/\tau), \quad (7)$$

which depends only on the two workers’ own features, and at $\tau = 0$ the same holds for which of the two ranks higher. The exchange rate between features, for example how many queued prefill tokens one cached block is worth (θ_1 against θ_3), is then the same whether the other workers are idle or saturated. A router plausibly wants it to change: when every candidate is busy, waiting behind the worker that holds the prefix may beat recomputing the prefix elsewhere, and when idle workers exist it may not. This is a hypothesis about the best policy, not an established property; the drift check described below measures it.

A set statistic added directly to the utility cannot express this. A term that is the same for every candidate, or a linear set-relative feature such as $x_{i,f} - (\bar{x}_S)_f$, shifts every utility in S by the same amount and leaves the argmax unchanged. In the linear context logit, choice likewise depends only on each item’s deviation from the set mean [50, Lemma 4.3]. Set information has to change the coefficients instead, and the context term in Equation (5) does exactly that: $w_S = \theta + \sum_\ell z_{S,\ell} p_\ell$, so the exchange rate between two features becomes a linear function of the context values. This is the linear context logit (LCL) of Tomlinson and Benson [50], $u_i = (\theta + A z_S)^\top x_i$, with two changes: the mean feature vector is replaced by a short vector of named set and request statistics (Table 23 in Appendix E), and A is restricted to one free column p_ℓ per statistic. It is not their decomposed variant (DLCL), which is a mixture of logits.

When context should pay. Context helps only where the best coefficients drift with the set statistics, and a larger nested model can lose on a fixed budget [50]. A drift check before M2 was inconclusive (Appendix F.8), so it did not choose M2’s sources; they were fixed before any M2 evaluation: `isl_k`, because prompt length is M1’s documented failure axis (Section 10), and `mean_active_prefill_tokens_k`, the candidate set’s prefill pressure.

¹⁵The temperature is not range-normalized, unlike the default’s `router_temperature`, which divides costs by their range before scaling; at a fixed $\tau > 0$ the probability mass on workers other than the best grows with N , so a sampled policy does not mean the same thing at every worker count.

5.5 Set-size invariance

Training sees $N \in \{4, 8\}$; the test also scores $N = 2, 16$ and 32 (Section 8.1). Two properties make one coefficient vector meaningful at any worker count. First, the coefficients are shared by all candidates and no feature is a worker’s identity, so the policy is defined for any number of candidates; Lodestar’s router uses the same principle [23]. Second, every feature and context source is designed to be invariant under duplicating the candidate set: if every worker is replaced by two identical copies ($N \rightarrow 2N$), each candidate’s features, every context value and the chosen worker (up to which copy) stay the same. Means and maxima pass this test; a mean of a one-hot feature, which is $1/N$, does not. A unit test duplicates random `v2` candidate tables and checks that features, sources and the chosen worker are unchanged (Appendix E). Only `hash_home` and `session_affinity` are exempt, because they are relations to particular workers.

Duplication invariance is necessary for transfer across worker counts, not sufficient. At a different N , the same load level produces different per-worker states, so the held-out worker counts in the test measure transfer directly (Section 8).

5.6 Parity with the default router

With $\theta = -\mathbf{e}_0$, $r = 0$ and $\tau = 0$, Equations (5) and (6) pick $\arg \max_i (-c_i^{\text{def}}/b_j) = \arg \min_i c_i^{\text{def}}$, the default router’s choice. Unit tests check through the deployment YAML path that `learned-choice` at $-\mathbf{e}_0$ picks the default’s unique minimum on 2,000 random tables (Appendix E). In replay, the `reservoir` tie-break, which replays the seeded default picker’s random draws, reproduces the default’s per-request rows on 88 of 88 (cell, replicate) pairs; the `one_draw` tie-break used in training breaks ties differently but changes goodput by no more than replicate noise (mainly on Mooncake cells; the session and AgentX cells sat at ceiling under the provisional SLO used at integration; Appendix E).

This parity is what makes $-\mathbf{e}_0$ a meaningful start for training (restart s1 in Section 6.4) and lets every learned coefficient be read as a deviation from the default router.

5.7 Model ladder

Each rung adds capacity only where the previous one leaves a measured gap, and a rung counts only if it improves held-out results. Table 8 lists the arms that were trained. Every arm received three restarts of the same budget (Section 6.7). Training and initialization are in Section 6; the rule that picks the headline learned model among the router-observable arms, the same rule that picks the best baseline, is in Section 6.6.

The later router-observable arms test specific questions. M1-noaff pins the session coefficient at zero: the hypothesis was that session affinity carries little beyond cache overlap in replay, where the router’s cache index is perfectly fresh, but may matter under router-state lag or live, so the arm also ran in the lag robustness check. M1-v2 targets the pilot’s weakest case, the agent workloads on which M1 trailed cache-aware heuristics, with `v2`’s log, set-relative and prefill-attention terms; one of its starts is LMetric’s representable point. Because θ_0 stays pinned at -1 , that point is approached as the log-term coefficients grow relative to the anchor rather than reached exactly (our inference from Table 7).

Two ablations were planned, and neither ships. The queue-threshold ablation tuned the router’s dispatch-or-hold threshold `router_queue_threshold` jointly with the selected learned model and with the best baseline; it ran, and found nothing measurable (Section 9.7).¹⁶

¹⁶The other ablation, adding simulator-only signals such as engine waiting counts and KV usage, was not run because

Table 8: The learned arms and the tuned default. “Free” counts searched coefficients. Every learned arm runs at $\tau = 0$ with θ_0 pinned at -1 and, after the gaming finding, with load coefficients clamped so that load cannot attract (Section 6.5). Only router-observable arms can be the headline learned model; results are in Section 9.7.

Arm	Features	Free	Role and outcome
M0	<code>dynamo-default-cost-fn</code>	5	tuned default: β_{ov} , β_{decay} , β_{pf} , β_{req} and the router temperature
M1	<code>v1</code> , $r = 0$	7	headline candidate; sign-constrained re-run
M1 unconstrained	<code>v1</code> , $r = 0$	7	the first tuned M1; flagged for gaming, reported as a secondary row
M1 continued	<code>v1</code> , $r = 0$	7	400 more evaluations from the selected M1; the comparator for M2
M2 rank 2	<code>v1</code> , two named sources	20	headline candidate; sources <code>isl_k</code> and <code>mean_active_prefill_tokens_k</code> ; ranks 3–4 not run (their pre-registered gate failed)
M1-noaff	<code>v1</code> , $\theta_6 = 0$	6	headline candidate; session coefficient pinned at 0
M1 + queue threshold	<code>v1</code> and <code>router_queue_threshold</code>	8	headline candidate (queue-threshold ablation)
M1-v2	<code>v2</code> , $r = 0$	22	headline candidate; extended features; selected
M3	nested logit	–	not run: no nontrivial nests (Appendix E.10)
M1-ais, M2-ais	<code>v3</code> (and two named sources)	11, 33	secondary AIS-informed league

6 Training method

Training searches a policy’s parameters directly against the replayed objective. No part of the procedure scores a single routing decision: a decision changes the queues and caches that later requests see, so its effect is visible only in aggregate over a replay. Each policy configuration is therefore a black box that maps to one number, the objective below, and the search is derivative-free. The spaces are small (7 free coefficients for M1 and 22 for M1-v2, two to six parameters per baseline), and one tuning harness tunes every policy, learned or heuristic, with the same settings (Algorithm 2 in Appendix F). A smaller pilot with the same method preceded the main tuning that this section describes (Appendix G).

6.1 Objective

A configuration π is scored by replaying it on every train cell and averaging the clipped log-ratio of windowed goodput (Equation (4)) against the default router on the same workload replicates:

$$J(\pi) = \frac{1}{|\mathcal{C}_{\text{train}}|} \sum_{c \in \mathcal{C}_{\text{train}}} \frac{1}{K} \sum_{k \in \mathcal{K}_t} \Delta_{c,k}(\pi, \pi_0), \quad \mathcal{K}_t = \{(tK + i) \bmod K_{\text{pool}} : 0 \leq i < K\}, \quad (8)$$

with $|\mathcal{C}_{\text{train}}| = 34$, $K = 2$ and $K_{\text{pool}} = 8$. Each term is a paired comparison: the candidate and π_0 replay the same cell on the same common-random-number (CRN) replicate with the same policy of its implementation cost (Appendix F); the AIS-informed league measures part of the same question (Appendix J).

seed $k + 1$ (Section 4.1).

The objective is episodic: one value per configuration and generation, aggregated over minutes of simulated traffic per cell, with no per-decision credit assignment. Every cell weighs the same whatever its absolute goodput. The clipped log-ratio was fixed before training and is the same per-cell statistic the headline test uses (Section 8.4); the arithmetic mean of ratios is reported but not optimized, because a few cells in which π_0 nearly collapses dominate it (Section 3.3). Appendix F describes how the harness checks the pairing, handles failed replays and what a generation costs.

6.2 Search spaces

Every parameter is bounded and searched in internal coordinates in $[0, 1]$, linear or logarithmic (Appendix F.2).

M1. At $\tau = 0$ the decision (Equation (6)) is unchanged when θ is multiplied by a positive constant, a direction along which the objective is exactly flat and along which CMA-ES’s step size tends to grow [12]. M1 therefore pins $\theta_0 = -1$, which fixes the scale and the default router’s sign and makes the start $\theta = -\mathbf{e}_0$ the default router (Section 5.6). The temperature is fixed at $\tau = 0$ rather than searched: with the scale pinned, $\tau > 0$ is a different, stochastic policy and is a separate variant (Section 8.7). The tie-break mode is `one_draw`. Seven coefficients remain free.

Raw coefficients have no common scale, because the features’ spreads within a candidate set differ by two orders of magnitude. Each free coefficient is therefore bounded at about four times its *scale ratio* $\text{sd}(x_{\cdot,0})/\text{sd}(x_{\cdot,f})$, measured within candidate sets on tables reconstructed from π_0 ’s pilot replays, so the initial search distribution is isotropic in units of decision-relevant spread: the per-variable scaling Hansen [12] recommends, and the analogue of the state normalization of Mania et al. [27] (Table 24 in Appendix F.2).

Baselines. Each tuned baseline searches the 2 to 6 numeric parameters its catalog entry exposes, starting from its shipped defaults, with parameters that only rescale others and structural choices pinned; Appendix F.2 lists every space (Table 25).

Initial step size. Because parameter units are not comparable across policies, each space’s initial step size σ_0 is the step at which one CMA-ES step changes 17.5% of routing decisions; five baselines cannot reach that rate by the step-size cap and start at the cap (Appendix F.2).

6.3 CMA-ES and noise handling

We use CMA-ES [12] from the `cma` package, version 4.5.0, with population size $\lambda = 16$, the package’s default of 8 parents, and a budget of $B = 400$ objective evaluations per run, which is 25 generations.¹⁷ Its function-value stopping rules are switched off, so a run normally ends when the budget is spent; the equal-budget check (Section 6.7) flags any finished run that did not spend exactly B .

The objective is noisy in a specific way. Replay is deterministic, so a configuration scored on a given set of cells and replicates always gets the same value. What varies is which replicates a generation draws (Section 4.1) and, behind that, which workloads the train split happens to contain. Three measures keep this noise from steering the search.

1. **Common random numbers within a generation.** All 16 candidates of generation t and π_0 replay the same cells and replicates $\mathcal{K}_t$ with the same policy seeds, so CMA-ES ranks

¹⁷For the 2 to 7 free coordinates of these spaces the package’s default population would be 6 to 9.

paired differences rather than independent draws, the fixed-scenario device of PEGASUS [34]. Conditioning on the exogenous input sequence lowered policy-gradient variance 50-fold in a two-server load-balancing example [31]; for rank-based CMA-ES the analogue is paired scoring on shared replicates. The `one_draw` tie-break keeps the random stream aligned across candidates whose decisions differ (Section 5.3).

2. **Rotating replicates.** $\mathcal{K}_t = \{2t, 2t + 1\} \bmod 8$, so consecutive generations see different replicates, each pair recurs every fourth generation, and the CMA-ES distribution, which accumulates information over generations, is not fit to one draw.
3. **Population size.** $\lambda = 16$ is about twice the default. In the analysis behind UH-CMA-ES, raising the population size together with the number of parents, as the package does, is preferable to re-evaluating each candidate several times when step sizes adapt properly [13].

A rank-stability diagnostic is logged but steers nothing (Appendix F).

Every 5 generations, the tuning harness scores two candidates on all 14 validation cells with replicates $k = 0, 1, 2$ ($K_{\text{val}} = 3$): the best train sample so far and the mean of the CMA-ES distribution. The mean is included because it is not itself chosen by a noisy train score. The run keeps the higher-scoring validated candidate as its selection (Section 6.6). Algorithm 2 in Appendix F summarizes one run.

6.4 Restarts and initial points

Each tuned policy runs $R = 3$ restarts with CMA-ES seeds 1, 2 and 3. Baselines start every restart at their shipped defaults; only the seed differs. M1’s three restarts start from three different points, all computed from train-split data only (Table 24; Appendix F gives the fits and their rationale).

s1 $\theta = -\mathbf{e}_0$, the default router.

s2 A behavior clone of `llm-d-precise-prefix`: the conditional-logit maximum-likelihood fit to that heuristic’s choices on train decisions.

s3 A cache-heavy point aimed at the agent workloads (AgentX), on which the pilot’s M1 trailed the cache-aware heuristic.

The initial points are not objective evaluations. The clone uses one replicate of the heuristic’s own train replays (22 fresh and 12 cached), not evaluations of M1, so every M1 restart still spends exactly B . Bootstrapping a policy search from existing policies is a known lever [29], as is initializing a richer choice model at a simpler model’s maximum-likelihood fit [47]. The starts still give M1 information that a baseline’s restarts do not have, namely another policy’s decisions. We disclose this and also report M1 selected over its default-router start alone (Section 8.6).

Other learned arms. Table 8’s other arms are trained like M1, with the same budget, restarts, cells and selection rule; they differ in their spaces and starting points, and all of them ran to completion (Appendix F). M1-v2’s three starts are the default, a behavior clone refit on v2, and `lmetric`’s representable point scaled to 90% of the bound, which agrees with the pure LMetric rule on 0.924 of reconstructed decisions.

6.5 Sign constraints on load coefficients

An audit midway through tuning found that the first tuned M1 gamed the objective by attracting long requests to a busy worker (Section 10), which triggered a remedy registered in advance: re-run

the affected restarts with load coefficients constrained so that load cannot attract, at the same budget. With θ_0 pinned at -1 , the minimal such set for **v1** is $\theta_3, \theta_4, \theta_5, \theta_7 \leq 0$ (active prefill, KV load, active requests and the prompt-length-by-prefill interaction), so that no load signal adds utility for any prompt length. Every later learned arm uses the same rule, extended to its own load features and context directions, and the constraint is a clamp on the decoded value rather than a box bound (Appendix F). The re-runs used the same seeds, budget, cells, replicates, validation schedule and objective; the unconstrained runs are kept as a disclosed secondary row, flagged for gaming.

6.6 Selection

Selection happens at two levels, and neither uses the best train sample, whose score is biased upward by the search itself by an amount comparable to the differences being measured [7].

1. **Within a policy.** A policy’s configuration is the validated candidate with the highest validation objective ($k = 0, 1, 2$) over its three restarts: two candidates at each of five checkpoints per restart, up to 30 per policy. The best train sample is kept only for reference (Algorithm 2).
2. **Across policies.** The best baseline π_{base}^* is the tuned baseline whose selected configuration has the highest mean clipped log-ratio against π_0 on *fresh* validation replicates $k = 3, \dots, 10$. The headline learned policy π_θ is chosen by the same rule among the router-observable learned arms (M1, M2, M1-noaff, M1-v2 and any later learned arm). No within-policy selection reads replicates 3 to 10, so the cross-policy choice is not made on the scores that picked each configuration.

Every learned variant scored on validation is counted, and the number of learned arms compared is reported with the headline.

6.7 Equal budget

Every tuned policy, learned or baseline, gets the same resources: $B = 400$ objective evaluations per restart on all 34 train cells with $K = 2$ replicates, $\lambda = 16$, three restarts, five validation checkpoints of two candidates on 14 cells with $K_{\text{val}} = 3$, the same reference and objective, and the same selection rule. Every finished run was checked to have spent exactly 400 evaluations and five validation checkpoints; the rank-stability diagnostic is not counted (Appendix F.3). No run starts from the pilot’s tuned configurations, which would give M0 and M1 the pilot’s 480 evaluations on top of B .

Equal budget fixes what each policy may spend, not how far it can get. The spaces differ in dimension (2 to 22) and in how strongly decisions respond to the parameters (Table 25), and only the learned arms start from points derived from another policy (Section 6.4). Which method wins can depend on the budget [11], so two checks ask whether any policy was still improving at B . In the first round of tuning, four baselines, **llm-d-precise-prefix**, **ramjet** and both **sticky-session** variants, were selected at their last validated generation, having gained 0.0001 to 0.0030 over the last validation interval, at most 0.078 of the minimum detectable effect (MDE, Section 8.4). No baseline tuned after the first round (the queue-threshold ablation’s baseline arm and the two baselines of the AIS-informed league) was selected at its last validated generation; the later learned arms selected there had gained 0.0001 to 0.0033 over the last validation interval, at most 0.09 of the MDE. And an auditor’s validation-oracle probes, which deliberately favor the baselines, found no headroom: 38 **ramjet** configurations around all three restarts’ bases reached at most 0.1421 against the tuned 0.1420, and widened probes of M0 and **llm-d-precise-prefix** stayed below their tuned values (0.1299 against 0.1334 and 0.1202 against 0.1414), all far below M1-v2’s 0.1767.

The drift check that preceded M2, and the compute and numerics behind these runs, are in Appendices F.8 and F.9.

7 Baselines

The headline compares the learned policy with the best heuristic a deployment could use after tuning it on the same data. That heuristic is chosen from eleven tuned configurations, each tuned exactly like M1 (Algorithm 2 and Section 6.7): three restarts of $B = 400$ evaluations, the same objective, train cells, replicates and validation rule.¹⁸ Ten are the catalog policies of Table 20 in Appendix B.6; the eleventh is **ramjet** with a jointly tuned router queue threshold, the baseline arm of the queue-threshold ablation (Section 5.7). Fresh validation replicates then pick π_{base}^* among the eleven selected configurations (Section 6.6); it was tuned **ramjet**. Every heuristic is also evaluated untuned, at its shipped defaults, and round-robin is evaluated as a floor; these rows are reported for context.

All of them are catalog policies, selected by the same kind of policy YAML as **learned-choice** (Figure 2), and in the headline league every one reads only inputs the live router has. They are evaluated as implemented on the study’s branch: the default cost function and the two-tier policy as on Dynamo’s main branch, the six ported policies as in an unmerged pull-request stack (Appendix B.6; its **lmetric** has changed since, Section 7.1), and **sticky-session** as written for this study. Table 1 summarizes their composition and Appendix B.6 their rules.

Each baseline searches its 2 to 6 numeric parameters, starting from their shipped defaults, with parameters that only rescale others, categorical choices, the session-map capacity and inputs replay does not supply pinned (Appendix F.2).

7.1 Ports, not reproductions

By design, the ports are evaluated as implemented: we built no paper-faithful variants and did not try to reproduce published numbers. Implementations of the same baseline can differ materially [15], so beating **lmetric** or **dualmap** here means beating these ports in this simulator, at tuned and at shipped parameters, not the published systems. A brief sanity check (a setup audit and the pilot’s screen of the untuned ports) found every port running without error and none routing against its documented direction in aggregate (Appendix B.6).

The ports also differ from the published methods in known ways (Appendix B.6), and one of these differences bears on the headline’s scope (Section 9.11). **lmetric**, as evaluated, omits the prefill already queued on a worker from its prefill factor, the term the paper credits for its advantage over a cache-hit-ratio indicator [63]; the port also counts its hot-spot window in decisions rather than time. After this study froze its results, the port in ai-dynamo/dynamo#15450 was fixed to count the worker’s queued prefill in the prefill factor, so it now matches the paper’s definition of that factor. The study’s branch keeps the port as evaluated: the **lmetric** baseline, its tuning and every **lmetric** number in this paper are from the port before the fix, and this study did not evaluate the fixed port.

Tuning absorbs two of the other differences: **dualmap**’s prefill budget, a fixed default that its paper derives from a TTFT SLO our objective lacks, is searched directly, and a miscalibrated prefill rate in the optimized baseline, which uses llm-d’s hardware calibration, is equivalent to a

¹⁸Only **dynamo-default-cost-fn** and **sticky-session** draw random numbers, for ties and softmax samples; the harness seeds them with $k + 1$ on replicate k , as it seeds **learned-choice**. The other policies break ties by worker ID or by a rotating counter, so every baseline replays identically on a given replicate.

different TTFT penalty, which is searched. The missing queued-prefill term in `lmetric` as evaluated is structural, and no parameter setting adds it. The learned family is not limited the same way. Feature set `v2` can express the paper’s product with the queued term: $\theta = -(\mathbf{e}_8 + \mathbf{e}_{10})$, on `log_ptok` and `log_bs` (Table 7), takes its argmin. The arm M1-v2, one of the candidates for the headline learned model, uses `v2`, and one of its restarts starts near that point (Section 5.7). M1-v2 was selected as the headline learned arm, so the headline policy can represent a variant of LMetric that the `lmetric` baseline cannot. This asymmetry bounds the headline’s scope (Section 9.11).

7.2 Excluded and secondary baselines

ThunderAgent’s worker selection [19] follows the preference of a paired request classifier, and replay runs worker-selection policies but not classifiers, so it is not evaluated and lies outside every claim. SMetric [54] is not in the router catalog, and this study builds no new implementations; its published advantage also assumes a global KV tier, which replay does not have.

Baselines that read AIS estimates (M0 with the AIS prefill-load model and the optimized baseline with modeled TTFT) are tuned with the same budget but compete only in the secondary AIS-informed league (Appendix J).

Fairness. Equal budget is not equal search difficulty. The baseline spaces have 2 to 6 free parameters against M1’s 7 and M1-v2’s 22. In five of them a CMA-ES step changes few decisions even at the largest allowed step, and the hashed key depths of `chwb1` and `dualmap` make their objectives jump. `lmetric`’s two parameters act only through its hot-spot detector, which on these workloads rarely alarms, which we attribute to every worker holding the few hot class prefixes, so tuned `lmetric` should stay close to its defaults. In the other direction, the learned arms restart from informed points: the default router, a behavior clone of the strongest untuned heuristic in the pilot (`llm-d-precise-prefix`) and a third informed point, for M1-v2 the faithful LMetric point (Section 6.4). These points cost no objective evaluations but carry information from heuristic decisions, while every baseline restarts from its shipped defaults. Three checks bound these effects: the test also reports M1 restricted to its default-router start, Section 9.11 shows how much of the learned margin the faithful LMetric point carries by itself, and no baseline gained more than 0.078 of the MDE over its last validation interval (Section 6.7). Section 12.5 weighs these asymmetries.

8 Evaluation protocol

One comparison decides the headline: a pre-registered test over independent workload segments, run once on the frozen test set; every other analysis is secondary or descriptive. The primary test was registered at calibration on 2026-10-02, before any training run (Appendix M). Later additions, the secondary comparisons and the rule that picks the headline learned arm, were registered before any test cell was evaluated, and none changed the primary test.

8.1 Splits and hold-out axes

The test set has 60 cells on 12 segments; Tables 29 and 30 in Appendix H give the three splits and the test cells by worker count and hold-out axis. Each test cell is held out along one primary axis (Section 4.6); cells at $N = 6$ are labeled *selection-exposed* because validation also used that count [7].

Unit of independence. Cells are not independent: all cells cut from one segment share its arrival and length structure, whatever their worker count, load level or transform. Every statistic below

is therefore computed over segments. The test has 12, holding between 1 and 15 cells each. The FAST25 conversation cells share the arrival and length skeleton of Mooncake windows w3 and w5 and are averaged into those two segments. CRN replicates and duplicated trace copies re-draw or lengthen a segment but add no independent units. One train cell draws from the plays of two AgentX train segments (Table 29 in Appendix H); train segments enter no test statistic.

8.2 Noise floor

Replay is deterministic for a given policy, workload and seed, so a repeated run reproduces every per-request row and measures nothing. Every policy therefore runs seeded, and noise is measured over workload replicates, each of which re-draws the arbitrary parts of the workload and the policy seed (Section 4.1). Replicate noise is small: over 8 replicates per cell at calibrated loads, π_0 's windowed goodput has a median coefficient of variation of 1.4% across the 48 train and validation cells. Policies differ by segment far more than replicates differ: in the pilot's comparison of M1 with `llm-d-precise-prefix`, the standard error of the mean over 6 validation segments was 0.019, against a pooled replicate standard deviation of the paired ratio of 0.015, and M1's lead came almost entirely from one Mooncake segment (*preliminary (pilot, validation split)*). Segment heterogeneity, not replicate noise, therefore limits what the study can resolve; hence the segment-level test and minimum detectable effect below. Appendix H gives the per-cell ranges, the per-segment pilot numbers and the noise rule the pilot used.

8.3 Selection hygiene

The data are split into four disjoint uses, so that selection cannot leak into the result.

1. Train cells, replicates $k = 0, \dots, 7$: the CMA-ES objective (Section 6.1).
2. Validation cells, $k = 0, 1, 2$: selection within a policy (Section 6.6).
3. Validation cells, $k = 3, \dots, 10$: selection across policies, of π_{base}^* and of the headline learned arm π_θ ; the pilot's review also used them. No within-policy selection reads them.
4. Test cells, $k = 0, 1, 2$: one pass, for a fixed list of finalists.

The test split was frozen at calibration (Section 4.6), and a completeness audit confirmed that the test cells were evaluated once (Appendix H).

8.4 Headline test (pre-registered)

The headline compares the selected learned policy π_θ with the best baseline π_{base}^* (Section 6.6) on the test set. For each test segment ω ,

$$D_\omega = \frac{1}{|\omega|} \sum_{c \in \omega} \frac{1}{3} \sum_{k=0}^2 \Delta_{c,k}(\pi_\theta, \pi_{\text{base}}^*), \quad (9)$$

where $|\omega|$ is the number of test cells in the segment, so each segment weighs the same however many cells it holds. The primary test is a one-sided exact Wilcoxon signed-rank test over the 12 segment statistics, against the alternative that their median is positive, at level 0.05 [10].

It compares one learned policy with one baseline, each chosen on validation before the test, so no multiplicity correction applies. It does not test the stronger claim that the learned policy beats *every* baseline; that is a secondary intersection-union comparison (Section 8.6).

What 12 segments allow. With 12 segments the smallest attainable one-sided p is $1/4096$. The test rejects at 0.05 exactly when the segments on which the learned policy loses have ranks, by $|D_\omega|$, summing to at most 17; it can therefore tolerate a few losses if they are small. With 4 segments the exact test cannot reach 0.05 at all (its minimum p is $1/16$), and with 5 it needs every segment to favor the same policy. Per-family claims, which rest on 2 to 4 segments, are therefore descriptive only.

Effect size. We report $\Pr(\pi_\theta \succ \pi_{\text{base}}^*)$, the share of segments with $D_\omega > 0$, with a 95% bootstrap interval that resamples whole segments together with all their cells. The effect is called significant if the interval’s lower end exceeds 0.5 and meaningful if its upper end exceeds 0.75, following Bouthillier et al. [6].¹⁹

Minimum detectable effect. The MDE is $\max(1\%, 2\text{SE})$, where SE is the standard error of the paired segment mean measured on validation at the pilot. Against `llm-d-precise-prefix` at shipped defaults, the best untuned heuristic on validation then, it is 0.038 in clipped log-ratio (SE 0.019 over 6 segments); against π_0 the same rule gives 0.087. Both are *preliminary (pilot, validation split)*. The MDE is not a threshold of the headline test. It is used where a decision needs a size: a difference within it counts as parity at $N = 2$ (Section 8.5), it set the drift-check rule (Appendix F.8), and M2 at ranks 3 and 4 trains only if rank 2 beats M1 on validation by at least the MDE.

8.5 Strata and parity

Strata. Every table and the bootstrap are also given per load mode (8 open-loop, 6 closed-loop and 4 agentic-lane segments), per worker count, for the 10 transform-extrapolation cells (7 segments) as their own stratum, and per family, descriptively. A significance claim for an individual worker count uses Holm’s correction across the six counts. Under it, no unseen worker count, each with 6 test segments, can produce a significant per-count result whatever the data, so results at unseen counts are descriptive or take the parity and no-regression form below (our arithmetic, not part of the pre-registration; Appendix H).

Parity and no regression. Two kinds of claim say “no worse”. For `learned-choice` at $\theta = -e_0$ against π_0 , and for any claim of no regression at an unseen worker count, the protocol uses two one-sided tests with a margin of $\pm 0.5\%$ normalized goodput. At $N = 2$ a difference within the MDE counts as parity rather than failure, because informed routing tends to tie simpler policies when there are few workers to choose from [52].²⁰ In the event, the check held at every unseen worker count against both π_0 and `ramjet`: the lower end of the interval exceeded $-\ln 1.005$ (Section 9.5).

8.6 Secondary comparisons

These were registered before the test pass and are reported next to the headline. Only the headline is a confirmatory test; the rest are descriptive or secondary.

- **Learned-arm variants:** M1 restricted to its default-router start, that is, selected over that restart alone (Section 6.4), and M1-noaff, with the session coefficient pinned at 0, as test finalists.

¹⁹Next to it we report the interquartile mean and the geometric mean of segment ratios with intervals [2], the arithmetic mean of per-cell ratios, a per-cell map of which policy wins, and the gap to the per-cell best tuned baseline.

²⁰The $\pm 0.5\%$ margin is small next to the pilot’s segment standard error of 0.019 (*preliminary (pilot, validation split)*), so before the test we expected most no-regression claims to remain unestablished.

- **Reference points:** M0, round-robin, and every heuristic at its shipped defaults.
- **Oracles:** the best baseline per family and the best tuned baseline per cell, which no single policy can match. The gap to the per-cell oracle shows how much a per-workload choice of heuristic would add, a comparison Xia et al. [58] suggest for learned network policies.
- **The secondary AIS-informed league:** arms that read AIS-derived estimates at runtime, compared with the best AIS-informed baseline and with the best baseline overall. Every arm is normalized by the same π_0 , which does not use AIS, so the two leagues share one scale and we can state how much the AIS estimates add. These arms cannot become π_θ (Appendix J).
- **Every tuned baseline and the runner-up.** The learned arm against each of the 11 tuned baselines, as an intersection-union test, and against the best baseline of another type, `llm-d-precise-prefix`, registered because the validation choice between it and `ramjet` was a near tie.
- **Token-weighted goodput,** registered after the concentration finding: the headline pipeline with each good in-window request weighted by its prompt plus output tokens, a check that the request-count objective does not hide a loss in served work.

8.7 Robustness checks

These checks are reported, not tested, for the finalists on the frozen test cells and replicates. Each asks whether the headline ranking survives a change that training never saw. Agreement between two rankings of the finalists is summarized by Kendall’s rank correlation τ_K . As a reading rule taken from the literature review rather than from the pre-registration, a headline gap smaller than the spread these perturbations induce is reported as fragile.

SLO re-scoring. Every evaluation keeps its per-request rows, so the good-request indicator (Equation (2)) can be recomputed under another SLO without new replays or retraining. The scale sweep multiplies $(I_{\max}, s_{\max})$ by 0.5, 0.75, 1, 1.5, 2 and 3 and states where the headline ranking flips. Four alternative definitions of a good request, registered before the test pass:

- the mean-ITL bound alone;
- the end-to-end slowdown bound alone;
- the mean-ITL bound plus a TTFT bound that grows with prompt length, $b + a \text{ISL}_j$, where a is three times AIS’s uncontended per-token prefill time and b is π_0 ’s median TTFT at L2 on train cells;
- the mean-ITL bound plus an absolute end-to-end bound, π_0 ’s 90th-percentile end-to-end latency at L2 on train cells.

For each definition we report every finalist’s goodput ratio against π_0 , the paired segment delta against π_{base}^* , and τ_K against the ranking under the trained SLO. None of them changes selection or the headline test. Their constants were computed per family from train records and written down before any test record was scored (Appendix H).

Router-state lag. Replay hands the router perfectly fresh state, and load balancing on stale information can behave very differently [33]. A replay patch delays three kinds of worker-originated updates by δ milliseconds of simulated time: KV cache events, prefill completions and request completions. A delayed update becomes visible at the first router callback at or after its due time;

the router’s own bookkeeping of what it has dispatched stays immediate, as in a live router. At $\delta = 0$ the patched replay reproduces the unpatched one, and at 50 and 200 ms it changes routing (Appendix H). The robustness runs re-evaluated 17 finalists on all test cells at $\delta \in \{10, 50, 200\}$ ms, and we report whether the headline ranking holds; M1-noaff ran alongside M1 to test the hypothesis that session affinity matters more under lag. Training never sees lag. Because a deterministic argmax over stale state can herd requests onto one worker, two guards against herding, a positive temperature and a hash-home restriction, ran as variants of M1 (Appendices H and I).

Timing perturbation. Two timing scale factors of the mock engines are set to 0.8 and 1.2. The first, `speedup_ratio`, scales every forward pass; the second, `decode_speedup_ratio`, scales decode only, so the effective decode speedup is their product. This tests sensitivity of the ranking to a global error in AIS’s timing, not to errors confined to particular batch sizes or context lengths. A timing perturbation also changes what an uncontended request can achieve, so each perturbed record is scored twice: against the nominal E_0 and against the perturbation-consistent $E'_0 = \text{prefill}/s + \text{decode}/(s d)$ for speedup s and decode speedup d . Under the nominal E_0 at $s = 0.8$, no zero-reuse AgentX request can be good, a degenerate scoring that is flagged where it appears. The spread rule and τ_K use E'_0 .

Live validation. A check on GPUs of the relative result, with its own pre-registered analysis, is described in Section 11; it validates the ranking only and never feeds selection.

8.8 Audits

Each stage of the study ended at a checkpoint that independent auditors reviewed before the next stage started (Appendix H.9 describes the process and lists every round in Table 31). Auditors who recomputed results from the raw records with their own code reproduced every pilot number to four decimals and the headline bit for bit. Every major finding was judged valid and resolved, by a fix or, once, by excluding a trace; among them were the objective gaming of Section 10 and the narrowing of the headline’s wording (Section 9.11).

Every evaluation record carries guard metrics for worker concentration, session splitting, starvation and length segregation; after the gaming audit, window-basis versions computed over exactly the scored requests were added (Section 10 and Appendix H).

9 Results

Apart from the live-run numbers that limit 5 below quotes from Section 11, every number in this section comes from AIS-timed offline replay. Intervals are 95% percentile intervals from a bootstrap that resamples whole test segments with all their cells; where a stratum has fewer than three segments, cells are resampled within each segment instead, marked †, and those intervals are optimistic because cells of one segment are not independent. Only the pre-registered headline is a test; every other p -value is descriptive.

9.1 Selection: the finalists

Each policy’s configuration was chosen on validation replicates $k = 0, 1, 2$ over its three restarts and their checkpoints. The cross-policy choices were then made on fresh replicates $k = 3, \dots, 10$, by the same rule for baselines and learned arms, and the list of finalists was frozen before the first test record was written.

- **Best baseline.** Tuned `ramjet`, 0.1411, ahead of `ramjet` with a tuned queue threshold (0.1405) and `llm-d-precise-prefix` (0.1393). The choice between `ramjet` and `llm-d-precise-prefix` was a near tie: `ramjet` led by 0.0018 in the mean over cells but trailed by 0.0087 in the mean over segments, so the comparison with `llm-d-precise-prefix` was registered as a secondary before the test pass.
- **Headline learned arm.** M1-v2, 0.1783, ahead of M1-noaff (0.1692), M1 with a tuned queue threshold (0.1684), M2 rank 2 (0.1677) and M1 (0.1656). Its configuration is the best-so-far sample at generation 20 of the restart that started from the default router ($\theta = -\mathbf{e}_0$). On the fresh replicates it led `ramjet` by +0.0372 per cell and +0.0290 per segment, on 5 of 6 validation segments.
- **Multiplicity.** 314 learned variants were validated over the study (every validation checkpoint of every learned run, the pilot and the post-hoc variants); 5 learned arms competed for the headline.

9.2 Headline: M1-v2 against the best baseline

Headline. In AIS-timed simulation, the learned router M1-v2 beats the branch’s ported heuristics, each tuned with an equal budget. Against the validation-selected best baseline, tuned `ramjet`, its windowed goodput on the 60 frozen test cells is higher by a segment-mean clipped log-ratio of +0.0316 (95% segment-bootstrap interval [+0.0150, +0.0501]; geometric-mean ratio 1.032, interval [1.015, 1.051]). It is ahead on 10 of 12 independent test segments: all 8 non-session segments and 2 of the 4 session segments. All four session segments tie at ceiling, with differences below 0.001 of either sign. The pre-registered one-sided exact Wilcoxon test gives $p = 0.0017$, and $\Pr(\text{M1-v2} \succ \text{ramjet}) = 0.833$ with interval [0.583, 1.0], significant and meaningful by the registered rule.

Table 9 shows the twelve units the test counts (Figure 7 in Appendix I plots them). The claim was pre-registered, evaluated once, reproduced bit for bit from the raw per-request rows by an independent auditor, and not refuted by the robustness and simulator-dependence audit. It survives leaving out any one segment (largest p 0.0034), any one family (largest p 0.027), the $N = 6$ cells and the FAST25 conversation cells; with the latter removed the segment mean is +0.026 and p is unchanged.

Limits that travel with the headline. Five limits bound the headline.

1. **Only the sign is a claim.** The effect is below the pre-registered minimum detectable effect (0.038) and below the spread that timing perturbations induce (0.060). Its magnitude depends on the perturbation (Section 9.9).
2. **It is not “learning beats every heuristic”.** The baseline pool lacks faithful LMetric, which lies inside M1-v2’s class; added to the default cost, it carries most of M1-v2’s validation margin (Section 9.11). “The branch’s ported heuristics” means the ports as evaluated: the `lmetric` port has since been fixed to count queued prefill, and that fixed port is not in this comparison (Section 7.1).
3. **It holds at the calibrated SLO and looser.** At SLO scales 1 to 3 it is significant; at 0.75 and 0.5 the sign holds but significance does not, and at 0.75 all four session segments turn against M1-v2 (Section 9.9).

Table 9: The headline per test segment: D_ω (Equation (9)), the mean over the segment’s cells and replicates $k = 0, 1, 2$ of the clipped log-ratio of M1-v2’s windowed goodput against tuned **ramjet**’s.

Segment	Family	Cells	Load modes (cells)	D_ω
w3	Mooncake 11, FAST25 conv. 2	13	open 11, closed 2	+0.0668
w5	Mooncake 14, FAST25 conv. 1	15	open 9, closed 6	+0.1009
s6	sessions	6	open 6	−0.0000
s7	sessions	2	open 1, closed 1	−0.0001
s8	sessions	2	open 1, closed 1	+0.0000
s9	sessions	5	open 1, closed 4	+0.0006
T1	AgentX	4	lanes 4	+0.0366
T2	AgentX	2	lanes 2	+0.0309
T3	AgentX	5	lanes 5	+0.0397
T4	AgentX	3	lanes 3	+0.0426
x0	FAST25 synthetic	2	open 1, closed 1	+0.0556
x1	FAST25 synthetic	1	open 1	+0.0051
Segment mean (SE)				+0.0316 (0.0093)
One-sided exact Wilcoxon signed-rank p , 12 segments				0.0017
Pr(M1-v2 $\succ$ ramjet) over segments, 95% interval				0.833 [0.583, 1.0]
Geometric mean of segment ratios, 95% interval				1.032 [1.015, 1.051]
Interquartile mean of segment ratios, 95% interval				1.026 [1.005, 1.050]
Mean over cells (SE); cells ahead				+0.0506 (0.0117); 50 of 60

4. **It is a request-count property.** The token-weighted goodput is mixed; prompts of 32K to 64K tokens wait longer for their first token than under **ramjet**, and prompts of 32K tokens and more lose about 4 points of good fraction on average (Section 10).
5. **The headline is simulated.** The live finalist runs (Section 11; validation only, on six $N = 4$ test cells chosen after the test pass, not at random, with one run per policy) agree with the simulated policy ranking (pooled Kendall τ_K -b 1.00, per cell 0.87 to 1.00) and with the sign of M1-v2 minus **ramjet** on hardware. They test neither the magnitude nor the significance of the headline, and they validate neither absolute goodput (live over simulated 0.72 to 2.18, depending on the policy) nor the size of the gaps between policies: on the sessions cell, the simulated gain of every tuned policy over π_0 is not reproduced beyond live noise.

Secondary statements that hold.

- **Each tuned baseline.** M1-v2 is Wilcoxon-significant against each of the 11 tuned baselines (intersection-union test, largest p 0.0105). The probability-of-improvement criterion fails for **lmetric** and **sticky-session** bounded, which M1-v2 trails by up to 0.5% on the at-ceiling session segments. It also beats every heuristic at its shipped defaults (largest p 0.0105).
- **The best baseline on test.** On test the best baseline is **llm-d-precise-prefix** (tuned), not **ramjet**. M1-v2 beats it by +0.0293, ahead on 12 of 12 segments ($p = 0.00024$); this comparison was registered before the test pass.
- **Per cell.** M1-v2 is the single best of the 29 router-observable test policies on 31 of 60 cells, at or above the per-cell best tuned baseline (a virtual best no single policy can match) on 42 of 60 cells with a mean gap of +0.037, and above π_0 on all 60.

Table 10: The headline by stratum, M1-v2 minus tuned **ramjet**. One-sided exact Wilcoxon p over the stratum’s segments; only the first row is a test. Rows marked $\dagger$ use the within-segment cell bootstrap.

Stratum	Cells	Seg.	M1-v2 – ramjet [95% CI]	Ahead	One-sided p
all (pre-registered test)	60	12	+0.0316 [+0.0150, +0.0501]	10/12	0.0017
load mode: open loop	31	8	+0.0281 [+0.0032, +0.0608]	5/8	0.0742
load mode: closed loop	15	6	+0.0354 [+0.0106, +0.0633]	4/6	0.0781
load mode: agentic lanes	14	4	+0.0374 [+0.0331, +0.0411]	4/4	0.0625
$N = 2$ (unseen)	8	6	+0.0382 [+0.0138, +0.0660]	6/6	0.0156
$N = 4$ (train count)	13	7	+0.0548 [+0.0110, +0.1085]	5/7	0.0391
$N = 6$ (selection-exposed)	7	6	+0.0296 [−0.0004, +0.0655]	3/6	0.2188
$N = 8$ (train count)	17	8	+0.0427 [+0.0140, +0.0788]	7/8	0.0117
$N = 16$ (unseen)	7	6	+0.0288 [+0.0096, +0.0484]	5/6	0.0312
$N = 32$ (unseen)	8	6	+0.0240 [+0.0079, +0.0416]	6/6	0.0156
unseen N (2, 16, 32)	23	7	+0.0342 [+0.0159, +0.0515]	7/7	0.0078
seen N (4, 8)	30	12	+0.0346 [+0.0123, +0.0610]	9/12	0.0081
transforms beyond train ranges	10	7	+0.0250 [+0.0071, +0.0443]	4/7	0.1094
family: Mooncake	25	2	+0.0505 [+0.0368, +0.0645] †	2/2	0.2500
family: FAST25 conversation	3	2	+0.3868 [+0.2857, +0.4879] †	2/2	0.2500
family: FAST25 synthetic	3	2	+0.0304 [+0.0164, +0.0443] †	2/2	0.2500
family: sessions	15	4	+0.0001 [−0.0001, +0.0004]	2/4	0.4375
family: AgentX	14	4	+0.0374 [+0.0331, +0.0411]	4/4	0.0625

9.3 Every policy

Figure 3 places every test policy against π_0 ; Table 35 in Appendix K gives the numbers, and Figure 14 there gives, for four leading policies, the share of cells on which each beats π_0 by more than a given margin.²¹ Four observations frame the headline.

- **Cache-aware routing matters a lot here.** The learned arms and the strong heuristics beat π_0 by 0.13 to 0.18 in segment-mean clipped log-ratio, while round-robin, which ignores the cache, loses 0.56.
- **Tuning the default’s knobs is not enough.** M0, the default cost function tuned with the same budget, gains +0.116 over π_0 , less than untuned **llm-d-precise-prefix** (+0.144).
- **Hash-pinned routing hurts on these workloads.** Tuned **chwb1** and **dualmap** are near π_0 (−0.030 and −0.004), and below it at their shipped defaults.
- **Validation transferred to test.** The ranking on fresh validation predicts the test ranking (Figure 8 in Appendix I); test numbers are higher than validation numbers mainly because test holds more cells at $N = 16$ and 32, where every cache-aware policy gains more.

9.4 Strata by load mode

Table 10 breaks the headline down by load mode, worker count, family and transform; the next two subsections read its worker-count, family and transform rows. Only the first row is a test; the rest are descriptive, and per-worker-count claims would need Holm’s correction, under which every per-count p is at least 0.070.

M1-v2 leads in all three load modes, by +0.028 in open loop, +0.035 in closed loop and +0.037 in agentic lanes, none significant on its own (p 0.06 to 0.08); Table 40 in Appendix K gives selected policies by load mode. AgentX ran only in agentic lanes.

²¹Without clipping or logarithms, the mean over test cells and replicates of the plain goodput ratio against π_0 , with a two-stage bootstrap over segments and then replicates, is 1.222 [1.154, 1.286] for M1-v2 and 1.156 [1.108, 1.216] for **ramjet**.

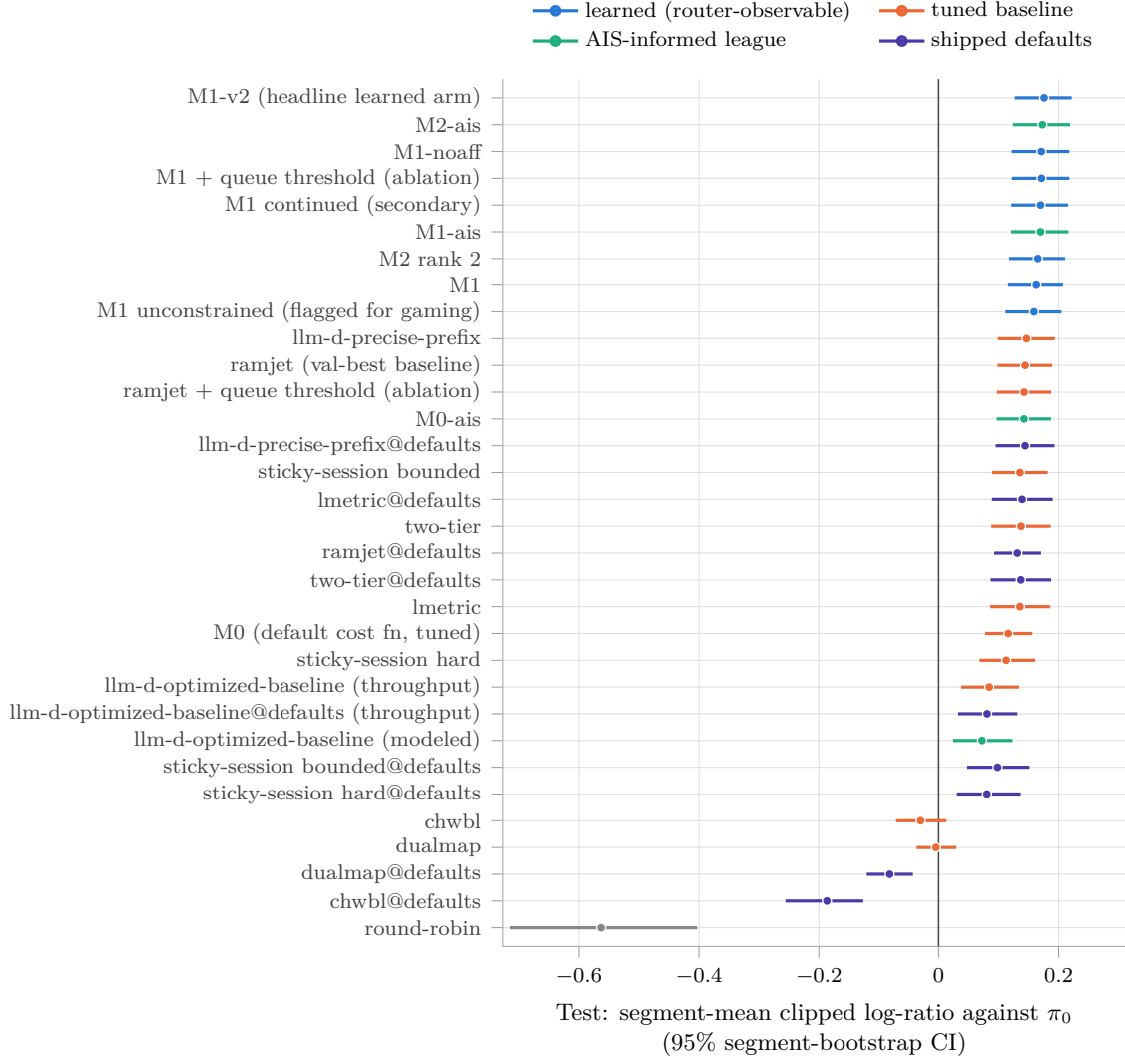


Figure 3: Every test policy against π_0 (60 cells, 12 segments; segment means with 95% segment-bootstrap intervals). Blue: router-observable learned arms; orange: tuned baselines; green: the AIS-informed league; purple: heuristics at their shipped parameters (suffix @defaults); grey: round-robin. The M1 row is also M1 restricted to its default-router start (Section 9.7).

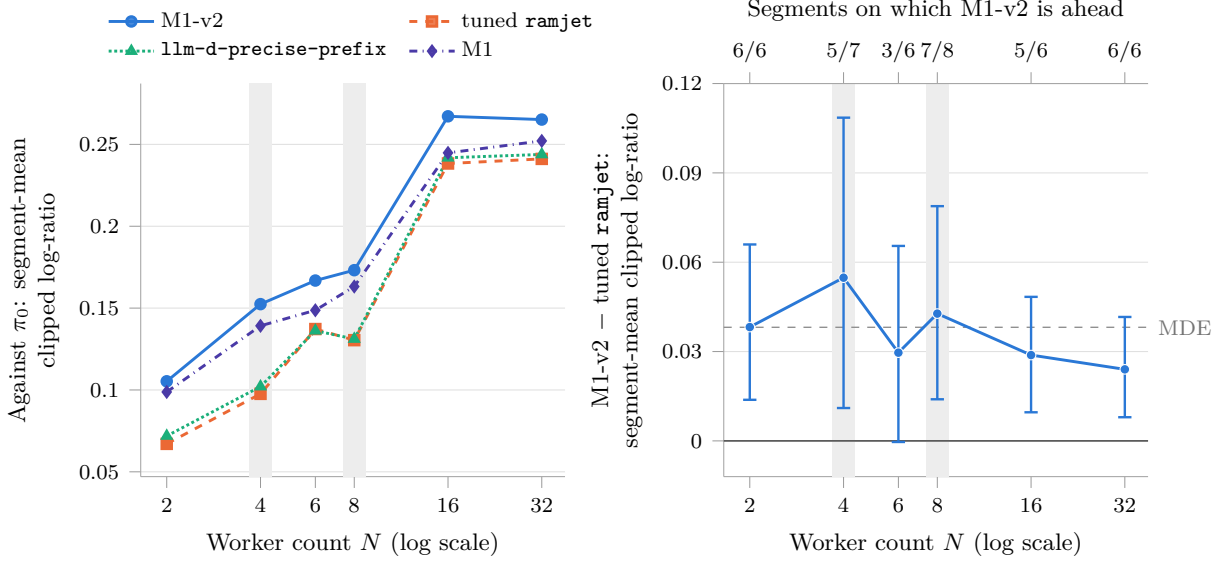


Figure 4: Worker-count extrapolation on test. Left: segment means against π_0 by N (intervals in Table 37 in Appendix K). Right: M1-v2 minus tuned **ramjet** by N , with 95% segment-bootstrap intervals and, on the top axis, the segments on which M1-v2 is ahead; dashed: the MDE. Shaded: the train counts 4 and 8. Each N holds different cells, and loads are matched to π_0 ’s knee at each N , not to a fixed per-worker rate.

9.5 Held-out worker counts

Every learned arm was trained on $N \in \{4, 8\}$ only and runs unchanged at any N , because its utility is per worker and its features are set-size invariant (Section 5.5). Figure 4 plots the test results by N .

- **The margin persists at unseen worker counts.** Over **ramjet** it is positive at every N : +0.038 at $N = 2$, +0.029 at 16 and +0.024 at 32 (Table 36 in Appendix K). Over the 7 segments with cells at unseen counts it is ahead on all 7 (+0.034, uncorrected p 0.008); this stratum was not pre-registered as a test.
- **No regression.** The pre-registered no-regression check holds at $N = 2, 16$ and 32 against both **ramjet** and π_0 : the lower end of the interval exceeds $-\ln 1.005$.
- **Gains over the default grow with N** for every cache-aware policy, M1-v2’s from +0.105 at $N = 2$ to +0.27 at 16 and 32, but the learned margin over **ramjet** does not grow at the unseen large counts.
- **$N = 6$ is selection-exposed** and the weakest stratum: 3 of 6 segments ahead, p 0.219.
- **The curve is not a pure N effect.** Each N holds different cells and segments, and with loads matched to π_0 ’s knee per N the per-worker load varies 1.2 to 1.9 times across N . A per-worker-matched extrapolation was not run.

M1-v2’s gain rises with cache pressure on the 22 open-loop Mooncake and FAST25 test cells (Spearman ρ 0.79 against π_0 , 0.71 against **ramjet**), a descriptive trend confounded with load level (Appendix I).

Table 11: The model ladder. Val $k_{0..2}$ and Val $k_{3..10}$: mean clipped log-ratio against π_0 on the selection replicates and on the fresh validation replicates. Test against π_0 is a mean over cells; test against tuned **ramjet** is a segment mean with its 95% interval and one-sided p . Table 35 in Appendix K gives the restart and generation at which each configuration was selected.

Rung	Val $k_{0..2}$	Val $k_{3..10}$	Test vs π_0	Test vs ramjet [95% CI]	p
M0	0.1334	0.1326	+0.1102	−0.0281 [−0.0439, −0.0137]	0.9993
M1	0.1660	0.1656	+0.1741	+0.0186 [+0.0013, +0.0378]	0.0549
M1 continued (+400 evals)	0.1704	0.1672	+0.1811	+0.0256 [+0.0090, +0.0437]	0.0212
M2 rank 2	0.1688	0.1677	+0.1774	+0.0211 [+0.0043, +0.0406]	0.0549
M1-noaff	0.1682	0.1692	+0.1821	+0.0270 [+0.0109, +0.0443]	0.0134
M1 + queue threshold (ablation)	0.1690	0.1684	+0.1819	+0.0272 [+0.0115, +0.0444]	0.0171
M1-v2	0.1767	0.1783	+0.1877	+0.0316 [+0.0150, +0.0501]	0.0017
M1 unconstrained (flagged for gaming)	0.1636	n/a	+0.1687	+0.0147 [+0.0032, +0.0284]	0.0386
M0-ais	0.1410	0.1397	+0.1313	−0.0019 [−0.0086, +0.0048]	0.6890
M1-ais	0.1731	0.1718	+0.1810	+0.0256 [+0.0113, +0.0423]	0.0012
M2-ais	0.1751	0.1735	+0.1837	+0.0286 [+0.0138, +0.0458]	0.0005

9.6 Families and transforms

By family (Table 38 in Appendix K; descriptive, with 2 to 4 segments each), the margin is largest on the FAST25 conversation cells (+0.387), a family no policy was trained on; those three cells carry about 35% of M1-v2’s cell-mean gain, and their request-count gains (+0.48, +0.49, +0.08) contrast with token-weighted deltas of +0.02, −0.15 and +0.02 (Section 10). On Mooncake the lead is +0.050, on AgentX +0.037 with all four segments ahead, and on FAST25 synthetic +0.030. Sessions sit at ceiling at the calibrated SLO: every strong policy serves nearly every in-window session request well, so the four session segments tie. The ten cells with transforms beyond the train ranges show +0.025 with 4 of 7 segments ahead (p 0.109), so generalization to held-out transform values is not established. Table 41 in Appendix K gives selected policies at unseen and seen worker counts and on the transform cells.

9.7 The model ladder and the ablations

Table 11 and the rung-to-rung comparisons of Table 32 (Appendix I, which also plots the ladder in Figure 9) read as follows.

- **M0 → M1 is the one clear step.** Learning over the default cost’s features beats tuning the default cost’s knobs: +0.047 per segment, 11 of 12 segments ahead.
- **M1 is the sign-constrained re-run.** The first, unconstrained M1 gamed the objective (Section 10); it is reported only as a flagged secondary row, and it is weaker on test.
- **M1 → M2 rank 2: no gain.** The context term adds +0.0025 on test (6 of 12 segments) and does not beat M1 continued for the same added 400 evaluations (−0.0045). The pre-registered gate for ranks 3 and 4 failed: M2 rank 2 minus the better of M1 and M1 continued was −0.0016 on validation, against an MDE of 0.038, so ranks 3 and 4 were not run. M3 (nested logit) was never built, since this deployment has no nests.
- **M1 → M1-v2 is the step that passes.** The extended feature set, with log, set-relative, interaction and prefill-attention terms, beats M1 by +0.013 on 11 of 12 segments (p 0.0034). The originally planned M1 and M2 alone do *not* pass against **ramjet** on test (p 0.055 each); the passing arm is a redesign made after the pilot, on validation evidence (Section 9.11).

Table 12: M1-v2’s coefficients. sd_f : pooled within-candidate-set standard deviation of feature f on train; constraint: the sign rule of Section 6.5.

f	Feature	θ_f	sd_f	$\theta_f sd_f$	Constraint
0	default_logit_scaled	-1	6.51	-6.510	pinned -1
1	overlap_frac	+5.034	0.2561	+1.289	free
2	new_prefill_tokens_k	+1.275	0.3519	+0.449	free
3	active_prefill_tokens_k	-0.2811	1.285	-0.361	≤ 0
4	kv_load_frac	+0	0.08438	+0.000	≤ 0
5	active_requests_s	-2.146	0.06	-0.129	≤ 0
6	session_affinity	+1.391	0.3244	+0.451	free
7	isl_x_prefill_load	+0	2.13	+0.000	≤ 0
8	log_ptok	+0	1.156	+0.000	≤ 0
9	log_new_prefill	-0.9276	0.8219	-0.762	free
10	log_bs	+0	0.2708	+0.000	≤ 0
11	hash_home	-1.582	0.4668	-0.738	free
12	kv_load_ratio	+0	0.2377	+0.000	≤ 0
13	active_prefill_ratio	+0	0.9491	+0.000	≤ 0
14	active_requests_ratio	-0.348	0.2576	-0.090	≤ 0
15	kv_load_below_frac	+0	0.2817	+0.000	≤ 0
16	active_prefill_below_frac	-1.326	0.2398	-0.318	≤ 0
17	active_requests_below_frac	-2.617	0.267	-0.699	≤ 0
18	kv_load_excess	+0	0.05047	+0.000	≤ 0
19	active_prefill_excess	+0	0.9154	+0.000	≤ 0
20	active_requests_excess	-5.845	0.03463	-0.202	≤ 0
21	new_prefill_x_requests	-16.79	0.1268	-2.128	≤ 0
22	prefill_attn	-0.2967	0.2846	-0.084	free

- **The queue-threshold ablation: nothing measurable.** It tunes the router’s queue threshold jointly with a policy (Section 5.7). Against their counterparts without it, the learned arm with the threshold is +0.0015 (p 0.055, against M1 continued) and **ramjet** with it is -0.0016, and removing the tuned threshold after the fact changes their validation scores by +0.0001 and -0.0006.
- **Session affinity.** M1-noaff, retrained without θ_6 , is not worse than M1 on test (+0.0084, p 0.21), and zeroing θ_6 in the selected M1 changes its validation score by -0.0010. Under router-state lag, M1 minus M1-noaff stays between -0.008 and -0.005, so the hypothesis that affinity gains value as the router’s state goes stale is not supported. For M1-v2, zeroing θ_6 changes the validation score by +0.0005.
- **Initialization.** The selected M1 came from the s1 restart, the default router, so the registered secondary finalist, M1 restricted to its default start, is the same policy as M1. M1-v2 also came from s1; its informed restarts s2 (the behavior clone) and s3 (the faithful LMetric point) reached 0.1609 and 0.1724 on the selection replicates, against s1’s 0.1767.

9.8 What M1-v2 learned

The shipped artifact is one policy YAML: **learned-choice** with **feature_set:** v2, $\tau = 0$, **tie_break:** one_draw and the 23 coefficients of Table 12. It reads no AIS signal; a decision costs $O(Nd)$ with $d = 23$. We read the coefficients through standardized magnitudes and ablations, not raw values: Table 12 scales each θ_f by the feature’s within-candidate-set standard deviation on train, so $\theta_f sd_f$ is the utility change for a typical difference between two candidates. The anchor, the default cost per request block, has $\theta_0 sd_0 = -6.51$ (Figure 10 in Appendix I plots the nonzero terms).

1. **The default router plus corrections.** $\theta_0 = -1$ dominates; every learned term is a fraction

of the anchor’s spread.

2. **Uncached prefill costs more the busier the worker.** The largest learned term is `new_prefill_x_requests` ($\theta_{21} \text{sd}_{21} = -2.13$). With θ_2 , the linear weight on uncached prefill is $+1.275 - 0.525 \times (\text{active requests})$ per 8K tokens, negative on any worker with three or more active requests: uncached work is steered to near-idle workers, while cache affinity is kept on lightly loaded ones. Zeroing θ_{21} costs 0.018 on validation (13 of 14 cells worse), and the fairness audit attributes M1-v2’s increment over the in-class LMetric heuristic to this term.
3. **Cache terms reward hits beyond the default’s credit.** `overlap_frac` (+1.29 per sd) and the log of uncached prefill (−0.76 per sd) both favor the worker holding more of the prompt; the log term rewards near-complete hits most. Zeroing θ_1 alone costs only 0.002 on validation: overlap is largely redundant with the anchor and the prefill terms.
4. **Rank-like load terms do the balancing.** The set-relative terms `active_requests_below_frac` (−0.70) and `active_prefill_below_frac` (−0.32), with request excess and ratio, outweigh the absolute load terms (−0.36 and −0.13). LMetric’s own product terms (`log_ptok`, `log_bs`) are 0 in the selected policy.
5. **Session and prefix-home terms are unneeded.** `session_affinity` (+0.45) and `hash_home` (−0.74, a mild avoidance of the prefix’s rendezvous pair) can each be zeroed with no validation loss (+0.0005 and +0.0010), so the policy depends on neither session IDs nor rendezvous homes.

The v1 arms agree on a simpler structure (Table 42 in Appendix K): a strong extra overlap credit ($\theta_1 \text{sd}_1$ of +4.2 to +4.9) and a strong penalty on active requests (−2.1 to −2.9) on top of the default. The unconstrained M1 used overlap and request weights about 3.5 to 3.7 times larger and a positive θ_7 , the gaming pattern of Section 10.

9.9 Robustness

Table 13 answers whether the ranking survives changes training never saw (Figure 13 in Appendix I plots it).²²

- **Router-state lag.** At 10, 50 and 200 ms the lead is +0.031, +0.029 and +0.030 ($p \leq 0.0081$), M1-v2 stays first among the 17 robustness finalists, and the ranking’s τ_K -b is 0.92 to 0.95. At lag 0 the patched replay reproduces the unpatched one per request (Appendix I). Beyond the registered range, on validation only, 1,000 ms gives +0.033 against +0.029 at 0: the baselines lose more to staleness than M1-v2 does.
- **Engine timing.** The sign holds under speedup and decode speedup of 0.8 and 1.2, on both references, and with the ITL bound also rescaled the lead is +0.022 to +0.080 ($p \leq 0.0081$). Speedup 0.8 on the nominal E_0 is not significant (p 0.117), a degenerate scoring in which no zero-reuse AgentX request can be good. The spread across conditions (0.060, set by speedup 0.8) exceeds the nominal effect, so by the reading rule of Section 8.7 the direction is robust and the magnitude is perturbation-dependent.
- **SLO scale.** The sign is positive at every scale; significance holds at 1 to 3 and is lost at 0.75 and 0.5. At 0.75 the four session segments reverse (−0.014 to −0.042), as they do at speedup 0.8 on the nominal reference, so “sessions at ceiling” holds only where the SLO does not bind.

²²The measurement window matters little: starting the window 25% or 50% later gives +0.0345 and +0.0297; halving or doubling the warm-up, or a common completion window, leaves the result unchanged on the same cells.

Table 13: The headline under perturbations and alternative SLOs. “Nominal E_0 ” scores timing-perturbed replays against the unperturbed E_0 ; “ E'_0 ” uses the perturbation-consistent reference (Section 8.7). s : speedup; d : decode speedup. τ_K -b compares the condition’s ranking with the nominal ranking, over the 17 robustness finalists for lag and timing and over all 32 non-default policies for SLO variants. The last row is the token-weighted secondary metric.

Condition	M1-v2 – ramjet [95% CI]	Ahead	p	τ_K -b	Rank M1-v2 / ramjet
nominal	+0.0316 [+0.0150, +0.0501]	10/12	0.0017	1.000	1 / 7 of 17
lag 10 ms	+0.0312 [+0.0139, +0.0508]	9/12	0.0081	0.950	1 / 7 of 17
lag 50 ms	+0.0288 [+0.0125, +0.0477]	10/12	0.0024	0.950	1 / 6 of 17
lag 200 ms	+0.0304 [+0.0135, +0.0497]	9/12	0.0061	0.917	1 / 7 of 17
s0.8 nominal E0	+0.0790 [+0.0042, +0.1666]	8/12	0.1167	0.783	1 / 10 of 17
s0.8 E0'	+0.0814 [+0.0229, +0.1511]	9/12	0.0105	0.800	1 / 10 of 17
s1.2 nominal E0	+0.0151 [+0.0064, +0.0247]	10/12	0.0081	0.950	1 / 6 of 17
s1.2 E0'	+0.0219 [+0.0105, +0.0333]	10/12	0.0046	0.950	1 / 6 of 17
d0.8 nominal E0	+0.0473 [+0.0131, +0.0872]	9/12	0.0261	0.817	1 / 7 of 17
d0.8 E0'	+0.0391 [+0.0145, +0.0691]	10/12	0.0024	0.917	1 / 8 of 17
d1.2 nominal E0	+0.0224 [+0.0099, +0.0360]	8/12	0.0134	0.950	1 / 6 of 17
d1.2 E0'	+0.0304 [+0.0140, +0.0477]	9/12	0.0105	0.950	1 / 6 of 17
SLO scale 0.5	+0.0716 [−0.0289, +0.1734]	10/12	0.0647	0.544	1 / 13 of 32
SLO scale 0.75	+0.0353 [−0.0015, +0.0776]	8/12	0.1167	0.907	1 / 10 of 32
SLO scale 1.5	+0.0156 [+0.0064, +0.0256]	12/12	0.0002	0.956	1 / 10 of 32
SLO scale 2.0	+0.0113 [+0.0052, +0.0178]	12/12	0.0002	0.931	2 / 10 of 32
SLO scale 3.0	+0.0120 [+0.0048, +0.0207]	12/12	0.0002	0.919	2 / 10 of 32
SLO: ITL bound only	+0.0089 [+0.0022, +0.0161]	10/12	0.0134	0.879	1 / 16 of 32
SLO: slowdown bound only	+0.0351 [+0.0152, +0.0574]	11/12	0.0012	0.988	1 / 11 of 32
SLO: ITL + length-scaled TTFT	+0.0134 [+0.0046, +0.0227]	10/12	0.0105	0.847	1 / 10 of 32
SLO: ITL + absolute E2E bound	+0.0189 [+0.0082, +0.0303]	10/12	0.0061	0.891	1 / 15 of 32
token-weighted goodput (secondary)	−0.0092 [−0.0645, +0.0244]	10/12	0.0320	n/a	n/a

At 0.5 the scaled slowdown bound falls below 1 for AgentX, so only cache hits can be good there.

- **SLO definition (registered before the test pass).** M1-v2 ranks first under all four alternative definitions of a good request, and the ranking’s τ_K -b against the trained SLO is 0.85 to 0.99: ITL only +0.0089 (p 0.013), slowdown only +0.035 (p 0.0012), length-scaled TTFT plus ITL +0.013 (p 0.0105), absolute end-to-end plus ITL +0.019 (p 0.0061).
- **Simulator model.** On validation, four audit-only changes to AIS’s pricing of mixed prefill and decode passes, which act on M1-v2’s concentration mechanism, keep the lead between +0.0268 and +0.0291 against +0.0290; they bracket fidelity errors, are not measurements of reality and do not cover FAST25, a test-only family (Appendix I).
- **Herding guards.** Run as variants of M1 (they were registered before M1-v2 won), neither a positive temperature nor restricting candidates to the prefix’s hash-home pair helps, and no herding failure of the argmax appeared under the tested lags in replay (Appendix I).

9.10 The AIS-informed league

The AIS-informed league, whose policies may read AIS estimates at runtime and so can never become the headline, adds nothing at the learned level: M2-ais trails M1-v2 by 0.0029 (3 of 12 segments ahead), although the AIS load model helps the default cost (+0.026). The league has no lag or timing robustness evidence and is not portable as is (Appendix J).

Table 14: The in-class heuristic. *Auditor diagnostic, not pre-registered, never used for selection; train and validation only, the test split was not evaluated.* Mean clipped log-ratio against π_0 ; the last column is the margin over **ramjet** on the fresh validation replicates, per cell and per segment.

Policy (tuning)	Train $k_{0..1}$	Val $k_{0..2}$	Val $k_{3..10}$	vs ramjet on $k_{3..10}$: cell / segment
M1-v2 (3×400 evaluations)	0.1851	0.1767	0.1783	+0.0372 / +0.0290 (5 of 6)
default + $10 \times$ faithful LMetric (8 train evaluations)	0.1566	0.1609	0.1609	+0.0197 / +0.0179 (4 of 6)
M1-v2’s s3 start, $c = 20.7$ (untuned)	0.1566	0.1596	0.1612	+0.0200 / +0.0190 (6 of 6)
faithful LMetric alone (untuned)	0.1466	0.1493	0.1490	+0.0079 / +0.0110 (6 of 6)
ramjet (best of 11 tuned baselines, 3×400)	0.1435	0.1420	0.1411	—

9.11 Scope: the in-class heuristic and asymmetric iteration

Equal footing was verified from the raw tuning histories by two auditors: every tuned policy, learned or baseline, ran three restarts of 400 evaluations with the same cells, replicates, validation schedule, objective and reference, and the splits share no trace files. The fairness audit then found a gap that equal budgets do not close. By design the baseline pool holds the branch’s ports as implemented, and the **lmetric** port as evaluated lacks LMetric’s queued-prefill term (Section 7.1). Faithful LMetric is inside M1-v2’s hypothesis class, and M1-v2’s restart s3 started from default plus 20.7 times the faithful LMetric point. Table 14 shows what that heuristic does on its own.

- **Faithful LMetric beats every tuned baseline untuned**, on the selection replicates (0.1493 against at most 0.1423) and on the fresh ones (0.1490 against 0.1411).
- **Most of the margin is the heuristic.** Default plus faithful LMetric reproduces 53 to 66% of M1-v2’s fresh-validation margin over **ramjet**.
- **The learned increment** over it is about +0.011 per segment (standard error 0.0065, 5 of 6 segments; +0.017 per cell), about 0.29 of the MDE, and it *was not measured on test*; it sits mostly on one Mooncake validation segment.
- **Follow-up.** A pre-registered comparison with faithful LMetric and an SMetric-style baseline, each tuned with the equal budget and evaluated on fresh segments or on live runs over cells outside the frozen test set, has not been run and is not part of this paper. (The live runs of Section 11 used frozen test cells and so cannot serve it.)
- **The port has changed since.** After this study froze its results, the **lmetric** port in ai-dynamo/dynamo#15450 was fixed to count the worker’s queued prefill, so it now scores the same product as the faithful LMetric rows above, plus its hot-spot filter. The fixed port was not evaluated, and the diagnostic is unchanged: its faithful rows are points of M1-v2’s class, not the port, the tuned baselines and every **lmetric** number in this paper are from the port before the fix, and the follow-up above has still not been run.

Asymmetric iteration. The learned side was redesigned on validation evidence: multi-start initialization, the sign constraints, and M1-v2 itself, which was written to target the pilot’s weakest family, plus M2, M1-noaff and the queue-threshold arm. The baseline side gained one arm and no new heuristic types. Selection among the five learned arms used the same fresh-validation rule as the baselines, and the test pass was single, so the headline p -value is valid for the selected arm; but the originally planned M1 and M2 do not pass on test. Two registration choices (excluding M1

continued from the headline candidates and including the queue-threshold **ramjet** in the baseline pool) cannot be shown to predate the fresh-validation results; both were outcome-neutral on those values, and we do not claim strict pre-registration for them.

10 Objective gaming, concentration and long prompts

The objective counts good requests. A request is good or not, whatever its size, so a policy can raise the count by serving many short requests well at the expense of a few long ones. This section reports how the search exploited that, how the exploit was removed, and what remains in the headline policy.

10.1 The finding: a sacrificial worker

Midway through tuning, an adversarial audit replayed the first tuned M1, before any sign constraint, and every contender on the 14 validation cells with its own scorer, and examined each request.

- **Concentration.** M1 sent more than the cap $\min(2/N, 1/N + 0.25)$ of a cell’s requests to one worker on 4 of the 5 Mooncake validation cells (up to 0.454 against a cap of 0.333), while every contender baseline stayed within the cap. The concentration was in requests, not work: the hottest worker’s share of prefill tokens stayed near $1/N$. It carried one shared-prefix group, and the extra concentration bought almost no cache: on the violating cells, M1’s prefix reuse was at most 0.0065 above the best within-cap baseline and below it on two.
- **A worker of lost requests.** On the heaviest Mooncake cell, in every replicate, M1 left one worker receiving only 14 to 16 in-window requests, all very long (median prompt 98K to 100K tokens), and none of them good, while the hot workers served 519 to 1,014 mostly short, cached requests at 0.90 to 0.99 good. These long requests share only one 512-token block, so stacking them has no cache rationale, and they were not doomed: π_0 , **ramjet** and M0 served every request of 60K tokens or more well on that cell, M1 only 42.5%.
- **Mechanism: load attracted long requests.** When a request of 60K tokens or more arrived, M1 sent it to the worker with the most outstanding prefill 58% of the time, against a chance rate of 12.5% and 0 to 2% for the baselines. The selected coefficients predict this: with $\theta_7 > 0$ (the product of prompt length and active prefill), the net coefficient on a worker’s active prefill turns positive for prompts above about 77K tokens, so a long request is drawn to a worker that is already busy. Setting $\theta_7 = 0$ alone removed the lost-requests worker on all three restarts and brought the long requests back to 0.94 to 1.00 good.
- **Not starvation in the long-wait sense.** M1’s share of requests slower than three times the slowdown bound was below every baseline’s; the sacrificed requests missed narrowly (median slowdown 1.13 times the bound).

The audit’s verdict was objective gaming, which triggered the remedy registered in advance. Black-box search exploiting a gap in its objective is a known pattern [22], and a binary per-request goodput invites exactly this trade [55]; a router tuned by an evolution strategy on tail TTFT alone sent all traffic to one replica [42]. A companion audit of simulator exploitation found the long-prompt sacrifice stable across 26 perturbations of timing, lag, window and SLO, so it is a property of the policy, not of a fragile corner of the simulator.

10.2 The remedy: load must not attract

The remedy re-ran all three M1 restarts with $\theta_3, \theta_4, \theta_5, \theta_7 \leq 0$, at the same budget, seeds and starting points (Section 6.5), and every later learned arm used the same rule. A re-audit with the audit’s own scripts, unchanged, then compared the constrained M1 with the unconstrained one.

- **Removed.** No Mooncake validation cell has a lost-requests worker (the unconstrained M1 had one per replicate on two cells); requests of 60K tokens or more are 0.92 to 1.00 good (0.425 to 0.966 before); the share of them sent to the worker with the most outstanding prefill fell to 0.07 to 0.20, near or below chance (0.37 to 0.58 before).
- **Remaining.** The constrained M1 still exceeds the share cap on 4 of 5 Mooncake cells, at higher shares than before (0.56 against a cap of 0.25 on the two cells at $N = 8$), and still segregates by prompt length: the normalized mutual information between worker and prompt-length quartile is 0.21 to 0.23 at $N = 8$, against at most 0.095 for the contender baselines. Long prompts still lose: on the heaviest cell 0.826 of prompts of 32K tokens or more are good, against 0.995 under π_0 and 0.963 under `ramjet`, and 65 to 69% of the requests that turn bad against π_0 fall in the top prompt-length quartile.

The sign constraint removed the pathological mechanism, a worker that collects long requests it cannot serve, but not the underlying trade: short cached requests are packed onto a hot worker and long prompts compete for the rest. The flags are reported on every learned row and, by a rule registered with the remedy, never trigger retuning or reselection; the request-count objective, the headline and selection were kept, and one token-weighted secondary metric was added (Section 10.5). The same re-audit on the later learned arms (M2, M1-v2, M1-noaff and M1 continued) flagged each on 3 or 4 Mooncake validation cells, with no lost-requests worker for any.

10.3 Concentration on test

Table 15 applies the window-basis guards to every test policy. Every learned arm concentrates and segregates by prompt length far more than the strong baselines: M1-v2 exceeds the share cap on 24 of 60 test cells, against 5 for `ramjet` and 0 for π_0 , and its worker assignment carries information about prompt length on 19 cells, against 5. A worker with no good request appears on 5 of M1-v2’s cells and 3 of `ramjet`’s; the unconstrained M1 has 9, and more long-request flags than M1-v2 (20, 20 and 15 cells against 11, 10 and 10).

Despite the concentration, the engine stays inside the simulator’s measured range: on Mooncake, M1-v2’s hottest worker carries 1.85 times the per-worker mean of in-flight requests, and peak in-flight requests of 59 and 65 lie inside AIS’s measured vLLM decode grid, so the concentration does not push the engine into the performance model’s extrapolation.²³ M1-v2 also shows no preference for pairing short prompts with long prefill chunks, the mixed-batch pricing artifact a search could exploit (caveat 13 of Table 21 in Appendix C).

10.4 Where the gain comes from: prompt length

Table 16 shows the trade the request-count objective rewards; Figure 11 in Appendix I plots it.

- **The gain sits in short and mid prompts.** M1-v2 gains 4.3 points of good fraction below 8K tokens and gives up 1.7 to 3.8 points at 8K and above.

²³The hottest worker carries 1.04 times the mean under `ramjet` and 1.18 times under π_0 on Mooncake, and 2.21 and 1.67 times under M1-v2 on FAST25 conversation and synthetic. Both peaks are far below `max_num_seqs`, and M1-v2 has fewer requests without prefix reuse that finish below their uncontended latency, the signature of the mixed-batch pricing discount, than `ramjet` or `llm-d-precise-prefix`.

Table 15: Concentration and long-request flags on test, counts of the 60 test cells. Share > cap: the window-basis largest worker request share exceeds $\min(2/N, 1/N + 0.25)$. NMI: normalized mutual information between worker and prompt-length quartile above 0.10. $\geq 32K$, $\geq 64K$, top decile: the good fraction of those prompts is more than 0.05 below π_0 's. Worker with no good request: a worker with 0% good in-window requests. Last column: mean over cells of the smallest per-worker good fraction.

Policy	Share > cap	NMI > 0.10	$\geq 32K$	$\geq 64K$	Top decile	Worker with no good request	Min worker good
M1-v2 (headline learned arm)	24	19	11	10	10	5	0.841
M2-ais	22	20	13	7	10	3	0.844
M1-noaff	23	19	11	6	11	2	0.836
M1 + queue threshold (ablation)	24	19	12	7	11	3	0.840
M1 continued (secondary)	23	18	14	8	11	2	0.830
M1-ais	25	22	11	6	10	5	0.837
M2 rank 2	23	20	11	6	10	3	0.832
M1	21	20	11	7	10	1	0.834
M1 unconstrained (flagged for gaming)	20	20	20	20	15	9	0.753
llm-d-precise-prefix	5	6	6	4	5	2	0.845
ramjet (val-best baseline)	5	5	1	0	3	3	0.849
ramjet + queue threshold (ablation)	5	5	0	0	1	2	0.852
M0-ais	5	5	3	1	2	2	0.850
llm-d-precise-prefix@defaults	6	8	4	2	4	3	0.840
sticky-session bounded	3	3	2	0	4	2	0.850
lmetric@defaults	2	2	5	3	5	4	0.835
two-tier	3	1	5	1	4	3	0.832
ramjet@defaults	3	2	2	1	3	2	0.854
two-tier@defaults	4	5	3	2	3	3	0.827
lmetric	2	0	5	3	5	3	0.837
M0 (default cost fn, tuned)	3	3	2	1	3	1	0.848
sticky-session hard	0	0	3	6	11	1	0.843
llm-d-optimized-baseline (throughput)	3	4	2	4	9	3	0.799
llm-d-optimized-baseline@defaults (throughput)	5	5	1	3	11	2	0.793
llm-d-optimized-baseline (modeled)	5	5	15	8	19	2	0.792
sticky-session bounded@defaults	0	0	1	0	2	1	0.800
sticky-session hard@defaults	0	0	1	6	10	1	0.785
chwbl	14	7	14	18	27	0	0.724
dualmap	14	16	11	11	31	3	0.646
dualmap@defaults	13	22	18	19	33	4	0.542
chwbl@defaults	14	16	25	22	44	1	0.555
round-robin	0	0	39	30	57	0	0.419
default@defaults (reference)	0	0	0	0	0	0	0.755

Table 16: Good fraction and TTFT by prompt length on test (descriptive, computed after the test pass from its records). Good-fraction differences in percentage points, with the number of segments on which M1-v2 is higher; TTFT p90 as a segment-mean log-ratio.

Prompt tokens	Good frac. M1-v2 – ramjet (pp)	Seg. higher	Good frac. ramjet – π_0 (pp)	TTFT p90 log-ratio, M1-v2 vs ramjet	Seg. higher
0-2048	+4.3	8/12	+19.4	+0.027	6/12
2048-8192	+4.3	8/12	+16.6	−0.012	5/12
8192-32768	−1.7	7/12	+7.2	+0.029	10/12
32768-65536	−3.8	5/10	+4.5	+0.135	11/12
65536-	−3.7	4/8	+1.1	+0.033	4/8

- **Long prompts wait longer than under `ramjet`.** Its TTFT p90 for 32K to 64K prompts is higher on 11 of 12 segments (log-ratio +0.135, a factor of 1.145); counting only records with at least 20 such prompts in both arms it is +0.169 on 8 of 8. That is about the default router’s own long-prompt TTFT (-0.011 against π_0), while `ramjet` is -0.180 below π_0 there. Overall TTFT is on par with `ramjet`: p50 +0.006, p90 -0.002 , p99 -0.082 .

10.5 Token-weighted goodput

A token-weighted secondary metric, registered before it was computed, re-runs the headline pipeline with each good in-window request weighted by its prompt plus output tokens, on exactly the requests and good rule that produce the headline’s goodput. It never selects a policy or decides the headline. Against `ramjet`, M1-v2’s token-weighted goodput is mixed: segment mean -0.0092 , interval $[-0.0645, +0.0244]$, ahead on 10 of 12 segments (p 0.032). One segment drives the mean: FAST25 synthetic x0, at -0.29 . The three FAST25 conversation cells, which carry about 35% of the headline’s cell-mean gain, gain $+0.48$, $+0.49$ and $+0.08$ in request count but $+0.02$, -0.15 and $+0.02$ in tokens. The headline’s magnitude is therefore partly a request-count effect.

10.6 Other secondary metrics

Against `ramjet`, in segment-mean log-ratios with the number of segments on which M1-v2’s value is higher (Tables 43 and 44 in Appendix K), M1-v2’s end-to-end p90 over the scored in-window requests is better (-0.048 , higher on 3 of 12 segments), its prefix reuse lower (-0.024 , higher on 1 of 12) and its per-request ITL p90 on par ($+0.006$); AgentX trajectory latency is slightly better (-0.012). Against π_0 it cuts TTFT p50 by 44% (-0.573 log-ratio) and raises prefix reuse by 27% ($+0.238$).

11 Live GPU validation

The headline is simulated, and a simulator that matches aggregate measurements can still rank policies differently from real hardware [52, 59]. The live check asks only whether the *relative* result holds on GPUs: whether M1-v2 still beats tuned `ramjet`, and whether the finalists keep their simulated order. It validates; it never feeds selection and never changes a finalist, the headline or any pre-registered number.

11.1 The live lane

The lane deploys Dynamo’s frontend with its KV router in front of vLLM 0.24.0 workers serving Qwen3-32B at tensor parallelism 2, four workers per $8\times H100$ node, with KV events from every worker, no session affinity and no busy thresholds. The router loads the same policy YAML files the replays used, byte for byte. Load comes from AIPerf 0.13.0 [36], driven from inputs generated from the study’s cells. A scoring adapter turns AIPerf’s per-request export into the records the replay scorer reads, so live and replay are scored by the same code with the same window and SLO.

Load-generator equivalence. Before any comparison, a CPU-only audit checked, request by request, that AIPerf replays every Mooncake, FAST25 and session cell as the offline replay does; replaying replay’s own latencies through it moved goodput by at most 0.06%. AgentX has no live load generator, and live latency includes client-side overhead that biases every policy slightly toward “not good” at the slowdown boundary (Appendix L).

The smoke test. A single-cell smoke test, one GPU run of π_0 on one Mooncake train cell, matched replay’s windowed goodput to within 0.25% (ratio 0.9976), with live prefill slower and decode faster than AIS’s, two errors that offset there (Appendix L). One cell, one policy and one run validate the lane, not any policy ranking.

11.2 Pre-registered analysis

The analysis was registered in the study’s live plan before any finalist live job was submitted. It is summarized here; the plan file is the binding version, and Appendix L gives the parts omitted here.

- **Deployment.** One 8×H100 SXM node per job, four vLLM workers at tensor parallelism 2.
- **Cells.** Six frozen test cells at $N = 4$: Mooncake w3 open loop at L2, Mooncake w5 open loop at L3 and closed loop at L3, FAST25 conversation (w3) open loop at L2, FAST25 synthetic x0 open loop at L2, and sessions s6 open loop at L2. There is no AgentX cell, since AgentX has no live load generator. The cells were chosen at registration, after the frozen test pass, for family and load-mode coverage and run time, so their simulated outcomes were known when they were chosen (Section 11.4).
- **Policies,** each exactly as frozen, with policy files byte-identical to those the test pass replayed: π_0 , run twice per cell to estimate live run-to-run noise; tuned **ramjet**; M1-v2; M1; M0; and round-robin. Every run uses replicate $k = 0$ with policy seed 1.
- **Scoring.** The replay scorer, with the AIS reference latency E_0 as the primary metric.
- **Readouts and wording rules.** Paired deltas against π_0 (the mean of the default runs in the same job) and against **ramjet**, per cell, without significance claims. The Kendall τ_K -b between simulated and live deltas against π_0 , per cell and pooled over the six policies; the live ranking “agrees” with simulation if the pooled τ_K -b is at least 0.6. The sign of M1-v2 minus **ramjet** per cell; “M1-v2 > **ramjet** holds live” only if it is positive on at least two thirds of the five informative cells and positive on average over them. A cell is informative if its simulated M1-v2 minus **ramjet** exceeds 0.01 in size; the sessions cell is not, because the four tuned policies tie at ceiling there in simulation. Live noise from the repeated π_0 runs is reported next to every delta.

11.3 Results

In brief. Both registered verdicts are positive. All 42 planned cell runs are valid, two of them after a registered re-run (Appendix L). The live ranking of the six policies agrees with simulation (pooled Kendall τ_K -b 1.00), and M1-v2 > **ramjet** holds live: M1-v2 leads in all five informative cells, by +0.219 on average against +0.162 in simulation. That is all the live runs validate. Absolute goodput departs from simulation by factors of 0.72 to 2.18, differently for different policies in the same cell, so the size of a gap between two policies is not validated either; on the sessions cell, the simulated gain of every tuned policy over π_0 is not reproduced beyond live noise. On the token-weighted secondary, M1-v2 > **ramjet** does not hold live, as it does not in simulation.

Live noise. The two runs of π_0 in each cell, r apart in clipped log-ratio, give a per-run noise $\sigma_{\text{live}} = \sqrt{r^2/2} = 0.0090$. A live delta against π_0 then has a standard deviation of about 0.011 where the reference is the mean of two same-job default runs and 0.013 where it is one, and a delta against **ramjet** about 0.013. Of the 30 live deltas against π_0 , 24 exceed twice their standard deviation (Appendix L lists the six that do not). The estimate rests on six pairs, so it has six

Table 17: M1-v2 minus tuned **ramjet** per live cell: simulated at $k = 0$ and live. The registered sign test counts the five informative cells.

Cell	Sim ($k = 0$)	Live	Same sign	$ \text{live} > 2\sqrt{2}\sigma_{\text{live}}$
Mooncake open L2	+0.069	+0.077	yes	yes
Mooncake open L3	+0.122	+0.190	yes	yes
Mooncake closed L3	+0.141	+0.133	yes	yes
FAST25 conv. open L2	+0.470	+0.663	yes	yes
FAST25 synth. open L2	+0.011	+0.035	yes	yes
Sessions open L2 (not informative: $ \text{sim} \leq 0.01$)	-0.0004	+0.0002	no	no

degrees of freedom: its 95% interval is about 0.006 to 0.020, and it is assumed to hold for every policy, including the concentrating learned ones whose run-to-run noise was not measured.

Ranking agreement. The registered primary, τ_K -b between the six policies’ mean gains over π_0 across cells, is 1.00: the policies have the same order live as in simulation, M1-v2 (+0.273 live, +0.219 simulated), M1 (+0.256, +0.218), tuned **ramjet** (+0.090, +0.084), M0 (+0.059, +0.066), π_0 (0) and round-robin (-0.295, -0.332). By the registered rule ($\tau_K\text{-b} \geq 0.6$) the live ranking agrees with simulation. The exact one-sided permutation p over the 720 orderings is 0.0014; it is descriptive, and its null of a random order of all six policies is one that round-robin and π_0 nearly reject by themselves. Over all 30 (cell, policy) points τ_K -b is 0.74, and per cell it is 0.87 to 1.00 (mean 0.89). The registered sensitivities, with the simulated side taken as the mean over replicates $k = 0..2$ or from replay with router-state lag of 10 or 50 ms, all give a pooled τ_K -b of 1.00 (round-robin keeps its $k = 0$ value under lag). Tables 45 and 46 in Appendix L give every paired delta and the per-cell values. Two qualifications apply. The pooled 1.00 at $k = 0$ rests on a simulated gap of 0.0009 between the mean gains of M1-v2 and M1; had the two swapped, it would read 0.87. Over replicates $k = 0..2$ the gap is 0.0074 and the pooled value is still 1.00. And all five per-cell discordances are pairs that simulation separates by at most 0.011.

M1-v2 against ramjet. On each of the five informative cells the live delta is positive, has the sign simulation predicts, and exceeds the registered noise bar $2\sqrt{2}\sigma_{\text{live}} = 0.0256$ (Table 17). It averages +0.219 live against +0.162 in simulation. By the registered rule (positive on at least two thirds of the informative cells, and positive on average), M1-v2 > **ramjet** holds live. On the sessions cell, where the four tuned policies tie at ceiling, both values are ties (-0.0004 simulated, +0.0002 live). The bar inherits the uncertainty in σ_{live} : at its one-sided 95% upper bound (0.017) the bar would be 0.049, which FAST25 synthetic’s +0.035 does not clear; the other four cells clear it.

Where live and simulation differ. Absolute goodput agrees closely on some cells and not on others (Table 47 in Appendix L). Live goodput is 0.72 to 2.18 times the simulated value; on the three Mooncake cells every policy but round-robin is within 0.94 to 1.01 of it. The ratio differs between policies in the same cell, so the departures also move the gaps between policies, in both directions. We flag a live delta as departing from simulation when the two differ by more than twice the standard deviation of the live delta; the simulated counterpart replays the identical requests, so live noise is the only sampling term. The rule is descriptive and was added after the registered analysis, at the request of the study’s audit of the live results. Under it, 18 of the 30 deltas against π_0 depart (9 higher live, 9 lower), and 13 of the 30 against **ramjet** (Table 48 in Appendix L). On the sessions cell every tuned policy’s simulated gain over π_0 (+0.084 to +0.086) is only +0.011 to +0.016 live, inside live noise, so it is not reproduced. On FAST25 conversation every gain over π_0 is

larger live, which widens M1-v2’s lead over `ramjet` to +0.663 from a simulated +0.470. Appendix L gives the per-cell detail.

A plausible reading, which these runs do not test, is engine timing rather than cache reuse, since the live prefix-cache hit rate matches replay’s (below): live time to first token is longer than AIS’s and decoding is faster (Table 49 in Appendix L), so the same cell runs at a different effective load or SLO point on hardware. FAST25 conversation would then run closer to prefill saturation live, where the policies that leave more prefill work queued lose more goodput. On sessions, live mean-ITL p50 is 0.84 to 0.85 of simulation for every policy, which loosens the SLO in effect for the policies not already at ceiling (π_0 and round-robin).

Registered secondaries.

- **Token-weighted goodput.** The pooled τ_K -b is 0.60, exactly at the registered threshold (mean per cell 0.58; permutation p 0.068), and M1-v2 minus `ramjet` is positive live on three of the five informative cells, with mean -0.035 , so M1-v2 $>$ `ramjet` does not hold live on this metric. By the same rule it does not hold in simulation on these cells either (two of five positive, mean -0.044 ; the signs agree on four of five), in line with the request-count limit of Section 10.5.
- **Live reference latency.** Re-scoring against an E_0 fitted on live idle runs keeps the pooled τ_K -b at 1.00 and M1-v2 ahead on all five informative cells (Appendix L).
- **Prefix-cache reuse.** Live prefix-cache reuse matched replay’s to within 0.015 in all 42 runs (Appendix L).
- **Concentration.** The worker-share cap (0.5 at $N = 4$) is exceeded live by M1-v2 on Mooncake open and closed L3 and by M1 on Mooncake closed L3 and FAST25 conversation; in simulation the same holds, and M1-v2 also exceeds it on FAST25 conversation. The concentration of Section 10 therefore also happens on hardware. That section’s long-prompt guards were not part of the registered live analysis, whose size-aware check is the token-weighted goodput above.

11.4 What the live runs can and cannot show

They show that on six $N = 4$ test cells, on real H100 GPUs serving the same requests, the live ranking of these six policies agrees with the simulator’s, and that M1-v2 leads `ramjet` live in every cell where simulation predicts a lead of more than 0.01, each time by more than the registered noise bar. They cannot show the following.

- **The headline’s magnitude or significance.** There is one run per cell and policy, at one replicate, on six cells, and no test was registered.
- **A representative effect.** The cells are not a random sample. They were chosen after the frozen test pass, with its per-cell results in the registration, and they include the three largest of the 13 simulated $N = 4$ gaps between M1-v2 and `ramjet` (FAST25 conversation +0.470, Mooncake closed L3 +0.141, Mooncake open L3 +0.122). Their simulated gap averages +0.135 at $k = 0$, against +0.071 over all 13 $N = 4$ test cells at $k = 0$ and the $N = 4$ stratum’s segment mean of +0.055 over $k = 0..2$ (Table 36 in Appendix K). FAST25 conversation alone contributes 58% of the six-cell sum; the other five average +0.068. Live deltas exceed the simulated ones on four of the five informative cells, but with single runs that is not evidence that the effect is larger on hardware.

- **The size of the gaps between policies.** 18 of the 30 live deltas against π_0 and 13 of the 30 against `ramjet` depart from simulation, in both directions. On sessions, the simulated gain over π_0 is not reproduced beyond live noise, so the live runs do not support the size of the simulated sessions-family gains over π_0 (Table 39 in Appendix K); on FAST25 conversation every gain over π_0 is larger live.
- **Close pairs.** M1-v2 minus M1 is within live noise on four of the six cells, and so is M0 minus `ramjet`; the live order inside those pairs is mostly unresolved.
- **Absolute goodput.** Live over simulated goodput ranges from 0.72 to 2.18 and differs between policies in a cell; AIS’s absolute accuracy is not validated here.
- **Session context.** The session header reaches the frontend exactly when replay sends a session context (no mismatch in 42 runs), and the code path to the policy exists, but no run shows `learned-choice`’s session feature receiving a session live. Only the sessions cell could exercise it, and that cell cannot inform M1-v2 minus `ramjet`.
- **Scope.** One model, one engine version, one GPU type and $N = 4$ only; no AgentX; one closed-loop cell; FAST25 and sessions in open loop only. Nothing here tests worker-count extrapolation, the AIS-informed league, controlled router-state lag on hardware, or faithful LMetric, which was not in the live matrix.
- **A token-weighted gain.** M1-v2 $>$ `ramjet` does not hold live on the token-weighted goodput, as in simulation.

12 Discussion, limitations and threats to validity

Apart from the live check of Section 11, every claim this study makes is a claim about AIS-timed offline replay of one deployment. This section lists what could make the headline wrong inside that setting, or keep it from transferring to a live router, with what the study did about each and what stays open. Table 18 summarizes; the subsections give the evidence.

What the results say. Within replay, the picture is consistent. Cache-aware routing matters a great deal on this deployment (round-robin trails the default router by a segment-mean clipped log-ratio of -0.56), and the strong cache-aware heuristics, once tuned, are close to each other. A conditional-logit policy over the router’s own signals, started from the default router, beats the branch’s ported heuristics, each tuned with an equal budget; the step that passes is the extended feature set, whose largest learned term makes uncached prefill more expensive the busier a worker is. The margin is small, below the pre-registered minimum detectable effect, and stable in sign under lag, timing, SLO-definition and window perturbations. Three findings bound it. Most of the validation margin is reproduced by the default cost plus a faithful LMetric term, which the baseline pool did not contain. The gain is concentrated on short and mid prompts, paid for by long prompts. And an unconstrained search found a degenerate policy that only a sign constraint removed. On GPUs, six test cells served live agree with the simulated ranking and with the sign of M1-v2 minus `ramjet`, but not with absolute goodput or with the size of the gaps between policies (Section 11).

12.1 Simulator validity

Simulated timing only. Every latency outside Section 11 comes from AIS’s forward-pass model, not from GPU measurement, and the study does not validate AIS’s absolute accuracy. What matters

Table 18: Threats to validity. “Direction” is the likely effect on the learned policy’s measured advantage, where one can be argued.

Threat	Likely direction	Mitigation	Status
AIS timing differs from GPUs	unknown	live ranking check; timing $\times 0.8$, $\times 1.2$	timing: sign holds; live: ranking and sign agree, absolute goodput and gap sizes do not
Live check on few, chosen cells	inflates live gaps	registered analysis; validation only	six cells, chosen after the test pass; one run each
Router state perfectly fresh in replay	favors policies that lean on exact state	lag 10–200 ms on test	ranking holds
No host, disk or global KV tier	excludes tier-dependent wins	scope statement	inherent
AgentX recorded with other models, agentic lanes only	unknown	family reported separately	inherent
SLOs and loads set with π_0	unknown	SLO scale and definition re-scoring	sign holds; significance only at scales 1–3
Objective gaming by concentration	inflates	guards; audit; sign constraint	lost-requests worker removed; segregation reported
12 independent test segments	low power at small gaps	pre-registered test; MDE	effect below MDE: sign only
Selection and redesign on validation	inflates validation numbers	single frozen test pass; symmetric selection	disclosed
Ports differ from papers	inflates	stated per port; in-class diagnostic	headline scoped to the ports as evaluated; lmetric since fixed, not re-run
Equal budget, unequal search	unknown	equal B and restarts; disclosed starts	no baseline gained more than 0.078 of the MDE over its last validation interval

here is narrower: whether replay ranks policies as GPUs would. That is not implied by aggregate accuracy. In a study of learned routing for disaggregated serving, a simulator re-fitted to match measured goodput to a mean absolute residual of 0.068 still did not reproduce the measured winner on any workload [52]; that simulator was a hand-built discrete-event model, so how far the lesson carries to AIS-timed engine replay is an open question. Controllers trained in emulation have also lost to simple ones in the real world [59]. Two features of this study raise the stakes. The calibrated loads sit in the default router’s transition region, where performance-model errors grow fastest [3]. And AgentX requests reach 128K tokens, where AIS’s accuracy has not been checked against hardware; the setup only showed that replay’s 120K-token prefill time equals an independent sum of AIS’s chunk estimates (17,815.54 ms, about 64% model FLOPs utilization), an internal-consistency check. No AgentX play fits a 32K context (caveat 10 of Table 21 in Appendix C), so this family cannot be moved to a better-validated range. What exists: engine speed scaled by 0.8 and 1.2 on test, four simulator-model counterfactuals that act on M1-v2’s concentration mechanism (Section 9.9), and one live smoke run (Section 11.1). The finalist comparison on GPUs compares rankings rather than absolute goodput (Section 11). On six $N = 4$ test cells the live ranking agrees with simulation and M1-v2 leads **ramjet** wherever simulation predicts it, but live goodput ranges from 0.72 to 2.18 times the simulated value, differently for different policies in a cell, and most live deltas against π_0 depart from simulation beyond live noise, in both directions (Section 11.3). So replay’s ranking held

on hardware where it was checked, and its gap sizes did not.

Perfectly fresh router state. Replay applies KV events to the router’s index synchronously, so a policy always sees exactly which blocks each worker holds; a live router sees those events late. Load counters (active prefill tokens, decode blocks, active requests) come from the router’s own request tracking in both settings, but that their update timing matches is an assumption. Stale state matters most for a deterministic argmax: routing to the least-loaded server on old information can perform much worse than a randomized choice [33], and greedy dispatch on delayed state herds [49]. The expected direction was that a policy leaning harder on exact overlap and on small load differences, as the learned arms do, would lose more of its replay advantage under lag than the baselines. Training never sees lag, by design; the robustness runs on the test cells re-ran 17 finalists at $\delta \in \{10, 50, 200\}$ ms on a separate build that delays the router’s view of worker state (Section 8.7). The expectation was not borne out: M1-v2 kept first place at every lag with its lead inside the nominal interval, and on validation at 1,000 ms the baselines lost more to staleness than M1-v2 did (Section 9.9). Two limits remain. The lag model is uniform and deterministic, while live staleness varies with load and with router replicas; and a concentrated policy is the kind most exposed to herding, which the live runs did not isolate: they ran one router at its natural staleness and applied no controlled lag (Section 11.4).

One cache tier, and no global KV store. Replay models only the GPU tier. A live Dynamo router’s accounting estimate blends host and disk tiers into the effective overlap, so feature 1 (`overlap_frac`) is the device overlap in replay but a weighted blend live, a distribution the policy never saw. Replay also has no global KV store, which changes which published routers can win. SMetric’s advantage over the best baseline holds “except for setups without any global KV\$ store”, and in SMetric’s own measurements on an agentic workload without a global tier, LMetric’s product score served 36% less traffic within SLO than a tuned linear cache-plus-load score [54, 63]. Session migration is cheap only when a global tier holds the context, so gains of that kind cannot appear here, and SMetric was not ported. The results speak to deployments whose KV cache lives on the serving GPUs.

Replay-specific inputs. Two replay artifacts could leak into a policy, and both were checked: replay fills a request’s expected output length with the true one, which `learned-choice` never reads, as a leakage audit confirmed (Section 5.1); and it gives every request of a closed-loop flat trace a synthetic single-use session identifier that a live client would not send, which is harmless for feature 6, since a single-use session never has a previous worker, and the `v2` home indicator keys on the prompt prefix, not on the session identifier (Section 4.2; Appendix E).

12.2 Workload and objective validity

AgentX is other models’ traffic. The 82 AgentX plays were recorded with several Claude models, not Qwen3-32B, and are replayed as Qwen3-32B traffic at up to 128K tokens. Request lengths, tool-call gaps and the dependency structure between calls are the recorded ones, so the family has realistic agent structure but not Qwen3-32B’s own output lengths or stopping behavior. Idle gaps longer than 300s are capped (Appendix D.2). Plays are recycled with disjoint hash ranges so that worker counts up to 32 stay loaded, which by construction rules out KV reuse across copies. AgentX runs only in agentic lanes (caveat 7 of Table 21 in Appendix C) and has no live load generator, so it is the one family the live check cannot cover. It is also the family on which the pilot

M1 trailed and on which M1-v2 now leads on all four segments, so these choices bear directly on the headline.

Generated and closely related traces. Session traffic is generated: seeded multi-turn conversations whose turns extend the whole context, with lognormal lengths and exponential think times measured from the previous turn’s completion. Replay’s native session source could not express a growing conversation. The flat traces, Mooncake and the two FAST25 traces, come from one provider, and a third public trace was excluded as a relabeled copy of Mooncake. The FAST25 conversation trace shares the arrival and length skeleton of two Mooncake windows, so it adds cells, not independent workloads.

Arrival burstiness is a modeling choice. Spreading logged arrivals inside their slot is a modeling choice that moves absolute goodput a lot (Section 4.3); paired comparisons on one replicate are less exposed.

Open-loop arrivals do not react. In open-loop cells requests arrive on the trace’s schedule whatever the latency, while a real client slows down when served slowly. Trace-driven simulation is biased when the policy changes the traffic it replays [4]. Closed-loop and agentic-lane cells release work causally and are reported separately.

SLOs and loads are defined by the default router. Each family’s $(I_{\max}, s_{\max})$ was fixed from π_0 ’s runs at $N = 8$, and each cell’s load is where π_0 ’s good fraction falls to 0.95, 0.85 or 0.65. That puts every cell in the default router’s transition region, which is where routing matters, but the region is defined relative to one baseline, and every ratio is normalized by the same baseline. The SLOs also differ widely across families ($s_{\max}$ from 1.16 for AgentX to 3.47 for Mooncake), and the end-to-end slowdown clause does most of the work (Section 3.3). The objective is therefore close to “finish near the uncontended latency”, and a different SLO could rank policies differently. The re-scoring of the test records under four other definitions of a good request kept M1-v2 first under each of them; under scaled SLOs it stayed first among the router-observable policies at every scale, though at scales 2 and 3 M2-ais of the AIS-informed league edged it overall. Significance holds only at the calibrated scale and looser, and at a tighter SLO the session segments reverse (Section 9.9).

Worker counts are compared at matched stress. Loads were matched to π_0 ’s knee at each N , not to a fixed per-worker rate; across worker counts the per-worker load differs by $1.2\text{--}1.9\times$ ($2.4\times$ in AgentX prefill). The held-out- N result therefore compares policies at equally stressed operating points of the default, which is the intended question, but it does not show behavior at a matched per-worker rate. At large N capacity pools and simple policies catch up, and at small N scarcity can favor randomization, so per- N differences need that context [52, 53].

12.3 Objective gaming and concentration

Black-box search exploits whatever its objective leaves out [22]. Three failure modes are documented for policies like this one. A binary per-request goodput rewards giving up on requests that will miss their SLO anyway [55]. An evolution strategy that tuned a routing cost for p95 TTFT alone set the queue weight to zero and sent all traffic to one replica [42]. And learned load balancers can drift into reserving or segregating servers in ways that break under workload shift [29]. Here, the ITL clause bounds the second failure, but only weakly, because a mean ITL averages out a long stall [55].

The audit found the first failure, sacrifice, for the first tuned M1, and the sign constraint removed its mechanism (Section 10). What remains is segregation by request size, which the request-count objective rewards: M1-v2 packs short cached requests onto a hot worker and gives up some long-prompt goodput. Whether that is acceptable depends on what the deployment values. Under the study’s request-count objective it is a legitimate win; under a size-aware objective it is partly a transfer, as the mixed token-weighted goodput shows (Section 10.5). Concentration also interacts with stale state: a policy that funnels traffic to a hot worker is the kind most exposed to herding; replay tested only uniform, deterministic lag, and the live runs did not isolate herding (Section 12.1).

12.4 Statistical validity

Few independent units. The headline rests on 12 test segments, the unit of independence, and per-family claims rest on 2–4 and are descriptive only [10]. The segment structure is uneven: Mooncake and FAST25 conversation contribute 28 of the 60 test cells but 2 of the 12 segments, AgentX 14 cells and 4 segments (Table 9). Workload replicates and duplicated trace copies lengthen and re-draw a segment; they do not add independent segments.

Effects near the detection limit. The headline effect, +0.0316 per segment, is below the minimum detectable effect of 0.038 that the pilot fixed, and below the 0.060 spread across timing perturbations. The test still rejects, because 10 of 12 segments favor M1-v2 and the two that do not are ties at ceiling; but a significant small effect is not a measured size, so only the sign is claimed.²⁴

Selection on validation. Many configurations are compared on validation, and selection overfits what it selects on [7]. Disjoint uses of data contain this (Section 8.3), and the test set is evaluated once. The cross-policy step reuses the same fresh validation replicates for both choices, of the best baseline and of the headline learned arm, which keeps the comparison symmetric but makes both selected validation scores optimistic; only the test numbers are clean. Every learned variant scored on validation is counted and reported [11], and the 7 test cells at $N = 6$, a worker count validation also used, are labeled selection-exposed. With 6 validation segments, selection itself is noisy: the choice of `ramjet` over `llm-d-precise-prefix` flipped between the cell and the segment mean, and on test `llm-d-precise-prefix` was the stronger baseline. The learned side was also redesigned on validation evidence while the baseline side was not (Section 9.11); the single test pass keeps the headline’s p -value valid for the selected arm, but the originally planned M1 and M2 would not have passed.

12.5 Comparison validity and fairness

Ports, not papers. Baselines run as implemented on the study’s branch; by design there are no paper-faithful variants, although different implementations of one method can perform materially differently [15]. The one structural gap tuning cannot close is the `lmetric` port’s missing queued-prefill term, which the learned feature set can represent, and this asymmetry is why the headline is scoped to the branch’s ports (Sections 7.1 and 9.11).

²⁴The MDE rule’s 1% floor is stated in ratio units; the binding 2SE term, 0.038, is on the clipped-log-ratio scale of the headline statistic. At these sizes the two scales agree to within a few percent of the effect ($\log 1.038 = 0.0373$).

Equal budget, unequal search. Every tuned policy gets $B = 400$ evaluations per restart and $R = 3$ restarts, but the policies differ in what that budget can buy. The search spaces range from 2 free parameters (`lmetric`, `chwbl`) to 7 (M1) and 22 (M1-v2). Each space’s initial step size was set so that one step changes about 15–20% of routing decisions; five baselines cannot reach that and change at most 13.3% of decisions even at the largest allowed step (Table 25 in Appendix F.2). For those policies tuning cannot move far from the shipped behavior; that is a property of their parameterization, and a learned model with a richer one is expected to gain more from the same budget. The equal-budget rule fixes what each policy may spend, not how far each can get. No baseline gained more than 0.078 of the MDE over its last validation interval, and the fairness audit’s validation-oracle probes of `ramjet`, M0 and `llm-d-precise-prefix` found no headroom (Section 6.7).

Initialization. Baselines restart from their shipped defaults. The learned arms’ second and third restarts start from a behavior clone of `llm-d-precise-prefix` and from a third informed point, for M1-v2 the faithful LMetric point. These starts use train-split decisions, not extra objective evaluations, so the budget is unchanged, but they hand the learned arms information about strong heuristics. Both selected configurations, M1’s and M1-v2’s, came from the default-router start `s1`, and M1 restricted to its default start is the same policy as M1 (Section 9.7). The faithful LMetric start is nevertheless strong on its own, which is the scope issue of Section 9.11.

Excluded and separated baselines. ThunderAgent is not evaluated, because its routing half needs a request classifier that replay cannot run [19]; policies that read AIS estimates at decision time compete in a separate, secondary league, the AIS-informed league (Section 7.2 and Appendix J); and SMetric and faithful LMetric were not tuned as baselines, so an equal-budget comparison with them on fresh data is the open follow-up of Section 9.11.

12.6 Deployment scope

The study covers one model (Qwen3-32B), one engine (vLLM 0.24.0), one GPU type (H100 SXM), tensor parallelism 2, aggregated serving, identical workers, and 2 to 32 workers. Disaggregated prefill and decode, other models and hardware, and heterogeneous workers are out of scope. The coefficients are specific to this deployment; moving to another means re-tuning, and the cost of doing so (Table 34 in Appendix I) is part of what a team would weigh.

The headline policy reads only inputs the live router already computes, so its YAML runs in the live router unchanged; whether those inputs carry the same values and timing live is what the lag check and the live run test. It decides by argmax ($\tau = 0$) and leaves the router’s queue threshold unset, so it never holds a request back; a randomized variant and a jointly tuned queue threshold were tested and added nothing. Shipping it needs four settings to match replay: the router’s structural knobs (`router_track_prefill_tokens`, `router_assume_kv_reuse`, `router_track_active_blocks`) at their defaults, since they change what the features mean (caveat 3 of Table 21 in Appendix C); host session affinity off, or accepted as a replacement for θ_6 , since the policy asks the host to treat an affinity target as exclusive; no `seed`, so that router replicas do not break ties in lockstep; and monitoring of the concentration guards of Section 10, since the policy’s expected behavior includes a hot worker at about twice the mean in-flight load. A live router’s overlap estimate also blends host and disk tiers, which replay does not have. The AIS-informed league would need AIS estimates, or a closed form calibrated offline, inside the live router.

13 Related work

Five lines of work meet in this study. For each we say what we take from it and where we differ.

KV-cache-aware routing. Most of our baselines come from this line (Section 7), and the published rules weigh prefix reuse against load in different ways. Preble sends a request to the GPU holding its prefix when the shared part outweighs the rest of the prompt, and otherwise to the GPU with the lowest cost, which counts recent load, the eviction the request would cause and the prefill of its missed tokens [48]. LMetric replaces weighted sums with the product of a worker’s new prefill tokens and its batch size, arguing that the best weight of a linear score changes with the workload: 0.7 on one of its traces and 0.55 on another [63]. DualMap restricts each prefix to two hashed candidates; it argues that picking the best of all workers amounts to a power-of- N -choices rule that scatters shared prefixes, and that a per-request minimum-TTFT rule oscillates between cache-aware and load-aware decisions [61]. Lodestar applies a similar two-choice consistent-hash filter, but only while cluster memory utilization is above a threshold, 80% in its evaluation [23]. CacheRoute periodically plans which replicas serve the most popular prefixes, and gates each plan on a shadow replay because a higher cache-hit rate alone did not guarantee lower tail latency or higher capacity [8]. llm-d scores workers with a prefix scorer fed by an index of the engines’ KV events [25]. For agent traffic, SMetric sends a session’s follow-up turns to the worker with the most cached context unless that worker’s estimated load would break a slack multiple of the TTFT SLO [54]; Autellix keeps an agent program’s long calls on the program’s engine and sends short calls to the least-used engine [26]; and ThunderAgent schedules whole programs, observing that a router-side prefix tree that approximates worker caches goes stale under eviction thrashing [19]. A workload study simulated cache-first, load-first and sticky routing at several instance counts and tracked hit ratio, per-session replication and load imbalance [35].

These rules disagree about which signal should dominate. On an agentic workload without a global KV tier, which is the setting of our replay, SMetric measured LMetric serving 36% less SLO-meeting throughput than a tuned linear cache-plus-load score, with prefix reuse of 45% against 74% [54]. We do not propose another heuristic. We tune these rules on equal terms, and the learned policy’s features draw on their signals: uncached prefill, active prefill, batch size, KV utilization and session affinity, and, in feature set v2, LMetric’s log-product and a hashed home-worker indicator in the spirit of DualMap (Tables 6 and 7).

Learned routers for LLM serving. Learned routers differ in what they learn and in where they learn it. Jain et al. train a reinforcement-learning router with a heuristic-guided reward; its action picks one of m instances or none, so the network’s shape is tied to the instance count [17]. Lodestar learns online an MLP that predicts each instance’s reward from per-instance features, with parameters shared across instances and no instance identity, so it does not depend on the instance count. It also shows that a latency predictor trained offline on logs of other routers loses its ranking signal once it routes, because the routing policy changes the states the predictor is evaluated on [23]. GORGO tunes the weights of an additive routing cost with an evolution strategy; optimizing p95 TTFT alone drove the queue weight to zero and sent every request to the closest replica [42]. Wu et al. analyze KV eviction and learning-based load balancing together [57]. Routers that choose among models or service-quality levels, rather than among replicas of one model, address a different decision [9, 18]. The closest study in method is Calibrate-then-Route [52], a router with hand-set weights and fitted constants that was developed in a simulator and then measured on GPUs in disaggregated serving. With simulator-derived constants it lost 4.5 goodput points to the same router with measured constants (0.819 against 0.864). After re-fitting, the simulator matched measured goodput

to a mean absolute residual of 0.068 but did not reproduce the measured winner on any workload. In a starved pool, greedy cost minimization collapsed to 0.292 goodput while round-robin held 0.680. AgentServeSim releases agent turns causally in simulation and searches policies with an LLM-driven evolutionary loop; its best evolved policies improved mean job completion time by 0.5% for KV retention and 2.8% for scheduling over hand-written starting policies [41]. Our policy differs from these in three ways. It is trained on-policy, since every candidate is scored by replaying it, which avoids the logged-policy mismatch Lodestar describes, and it is frozen afterwards. Its objective counts a request as good only if both its inter-token latency and its end-to-end slowdown are within bounds, where GORGO’s objective was p95 TTFT alone. And Calibrate-then-Route’s findings are why ranking stability under router-state lag and on GPUs is part of the protocol (Sections 8.7 and 11). Among the learned routers we read, none trains at some worker counts and tests at unseen ones; Lodestar’s shared predictor is independent of the instance count by construction.

Learned systems policies and black-box search. Decima learns cluster scheduling with reinforcement learning in a simulator, and it fixes the job-arrival sequence across episodes to cut the variance that stochastic arrivals add [30, 31]. It shows both sides of transfer: an agent trained on a cluster ten times smaller was 3% worse in average job completion time (630 against 610 s), while one trained on a mismatched load did worse than the tuned heuristic (104.8 against 91.2 s) [30]. Pensieve trains a bitrate controller in a chunk-level simulator and disabled TCP slow-start restart on its video server so that deployment matched a simulator assumption [28]. Park catalogs the difficulties of learning systems policies; one of its load balancers learned to keep a server free for small jobs and was not robust when the job mix changed, and it suggests bootstrapping search from existing policies [29], as M1’s heuristic-derived starting points do (Section 6.4). The cautions are empirical. Puffer’s randomized trial found it difficult for sophisticated or learned bitrate controllers to outperform a simple buffer-based scheme despite good results in emulation, and a controller whose predictor was trained in emulation performed poorly in deployment [59]. Trace-driven simulation is biased when the policy changes the traces it replays [4]. Learned policies that win on average can still lose to the baselines in a substantial fraction of environments [58]. Randomizing latency and observation noise was necessary for simulation-trained robot control to transfer [39]. And search exploits simulator bugs [22]. These findings motivate the closed-loop and agent-lane cells, the per-cell winner map, the timing and lag perturbations, the concentration audit and the live check (Section 8).

On method, static linear policies trained by random search matched reinforcement learning’s sample efficiency on the MuJoCo benchmarks, given normalized inputs [27]; evolution strategies’ gradient estimates do not depend on episode length, which suits long horizons with delayed reward [44]. We use CMA-ES [12], score every candidate of a generation on the same random scenarios, as PEGASUS fixes scenarios to make policy evaluation deterministic [34], and borrow UH-CMA-ES’s rank-change measurement as a diagnostic, without its treatment of the measured uncertainty [13]. Load-balancing theory frames two risks of our setting: sending work to the least-loaded server on old load information can significantly hurt performance [33], and delayed information makes join-the-shortest-queue dispatch herd [49]; how balancing rules compare also changes with the number of servers and the load regime [53].

Discrete choice with context effects. With rank 0, `learned-choice` is a conditional logit, the base case of the generalized extreme-value family [51]: one coefficient vector scores every candidate, so the same parameters serve any worker count, and at $\tau = 0$ the policy takes the logit’s most likely choice. Halo models add pairwise terms between specific items [60]; `learned-choice` has no

worker-identity terms, so its parameters apply unchanged as workers come and go. A plain logit satisfies the independence of irrelevant alternatives: the preference between two candidates never depends on which others are present, however rich the item score [43]. M2’s context terms and v2’s set-relative features add that dependence. In the linear context logit (LCL) of Tomlinson and Benson [50] the coefficient vector shifts linearly with the mean features of the choice set, $\theta + A\bar{x}_S$. M2’s pooled form is an LCL with a low-rank A ; its named-source form is an LCL over a vector of named set statistics, with one free column of A per statistic. Neither is their DLCL, a mixture of logits with separate intercepts. Three of their results shaped our design. Choice depends only on deviations from the set mean, so terms that are constant across the set cancel. A is identifiable only from at least $d + 1$ choice sets with affinely independent mean features. And LCL did not beat the plain logit within a fixed training budget although it nests it, one reason the ladder adds context in a small step that starts from the selected M1 and is compared with M1 trained for the same added evaluations (Section 5.7).

Other set-dependent models sum pairwise context vectors over the set [47] or use set-dependent aggregation [43], pooled set embeddings [40], mixtures of random-utility networks [5], self-attention [20] or stacked interaction layers [64]; divisive normalization, which fits human choices across set sizes from 2 to 12 [56], inspired v2’s ratio features. Low-rank choice parameters are unique only up to a multiplicative factor unless a penalty fixes the scale [20]; M2’s pooled form shares that freedom (scaling p_ℓ up and q_ℓ down by one factor changes nothing), while its named-source form, linear in p_ℓ , does not. A nested logit groups alternatives whose unobserved utilities are correlated [51], but under a deterministic argmax its nest parameters have no effect (our inference), so node structure would enter as nest-level features rather than as a GEV model (Appendix E.10). In reinforcement learning, Jain et al. [16] use a shared utility followed by a softmax over the available actions, a conditional logit over a variable action set; generalizing it to unseen action sets took subsampled action sets, entropy regularization and model selection on held-out action sets. Our analogue of the last is validation at $N = 6$, a worker count training never sees. The named context sources are request-level quantities or means and maxima over candidates, a permutation-invariant pooling [62], chosen so that duplicating every worker leaves them unchanged.

Goodput, SLOs and evaluation methodology. DistServe defines goodput as the highest request rate that meets an SLO-attainment target, and sweeps a single scale factor over its SLOs [65]. We measure good requests per second at a fixed load inside a window, and judge a request by its mean inter-token latency and its end-to-end slowdown against an uncontended latency (Section 3.3); as our analogue of DistServe’s sweep, the finalists’ test runs are re-scored at several SLO scales. Binary SLO goodput rewards giving up on requests that will miss anyway, and a TPOT-style bound averages out stalls in the middle of a request [55]; both are why the audits look for sacrificed requests and concentrated load (Section 8.8). Open and closed load generators behave differently: scheduling helps a closed system only at moderate load, and, as a rule of thumb, sessions of more than ten requests make a partly open system behave as closed [45]; we therefore stratify by load mode. Simulator error grows near capacity: in Vidur, small prediction errors near the capacity point blow up into large latency errors [3]. A load-testing study reports that handling warm-up properly improved the accuracy of capacity estimates by 22.2% across 14 industrial cases [1]. Our statistics follow the guidance for comparisons over few independent tasks: a Wilcoxon signed-rank test over independent units [10], bootstrap intervals and interquartile means over tasks [2], a probability-of-improvement effect size with thresholds for significant and meaningful [6], several seeds and restarts [15], budget-aware reporting [11], and selection on data the final test never sees [7].

Position of this work. The study combines four choices that we did not find together in the work above. The policy is a deployable choice rule, linear in inputs the live router already has, rather than a latency predictor. Every baseline is tuned with the same budget and selection rule as the learned model, and the headline comparison is against the validation-selected best of them. Worker counts are held out: training uses $N \in \{4, 8\}$, and the test adds 2, 6, 16 and 32. And the headline test is pre-registered over independent trace segments. Two of our findings echo this literature. The learned policy’s advantage over the ported heuristics is largely an LMetric-style interaction between uncached prefill and load (Sections 9.8 and 9.11), which supports LMetric’s point that the two should interact, though here as a correction added to the default’s additive cost rather than in place of it; and the first unconstrained search found the kind of objective gaming GORGO and Lehman et al. document, which a sign constraint on load coefficients removed (Section 10). The price is that every result outside the live check is simulated; as Calibrate-then-Route and Puffer show, a simulated ranking need not survive deployment. Our live check on six test cells found the simulated ranking intact on GPUs, but not the simulated gap sizes (Section 11).

14 Conclusion

Dynamo’s worker-selection API makes every routing rule, from the default cost function to the ported heuristics, a small composition of scorers and a picker over a few declared inputs. That uniformity made it possible to tune every rule with the same budget in AIS-timed replay, and to learn a generalization of them, a conditional-logit policy over the router’s own signals that starts from the default router. In simulation, the learned policy M1-v2 beats the branch’s ported heuristics, each tuned with an equal budget, on a test set that holds out time windows, agent plays, session seeds, a trace family, transform ranges and worker counts, and it stays first, ahead of tuned **ramjet**, under router-state lag, timing perturbations and alternative definitions of a good request. The effect is small, and only its sign is claimed; the per-axis results, such as those at unseen worker counts, are descriptive. A pre-registered check on H100 GPUs, on six of the test cells, found the same policy ranking and the same sign of M1-v2 against **ramjet** as replay, while absolute goodput and the size of the gaps between policies departed from replay.

For a deployment, the evidence supports three practical statements and one caution.

- **Prefer a cache-aware rule over the shipped default, and tune it.** Every strong heuristic beats the default router by a wide margin here, and tuning the default’s own knobs closes only part of the gap.
- **Make uncached prefill more expensive on busy workers.** The learned policy’s largest term is an interaction between uncached prefill and a worker’s active requests; zeroing it costs about half of M1-v2’s validation margin over the best tuned baseline. Faithful LMetric multiplies the same two quantities, and the default cost plus faithful LMetric reproduces 53 to 66% of that margin on validation, so it is the simpler candidate to validate next to M1-v2.
- **Constrain what search may learn.** Tuning a request-count objective without constraints produced a policy that parked long requests on a busy worker. A sign constraint on load coefficients removed it with no loss (the constrained M1 is +0.004 per test segment against the unconstrained one), and should be the default for any learned router of this kind; the remaining size segregation should be monitored with the window-basis guards.
- **Caution.** Long prompts fare worse under M1-v2 than under **ramjet**: those of 32K to 64K tokens wait longer for their first token, and those of 32K or more lose about 4 points of good

fraction. A deployment that values served tokens, or long-context latency, may prefer the heuristic.

Next steps.

- A larger live check: repeated runs per policy, randomly drawn cells, other worker counts and an AgentX load generator, which could test the size of the effect on hardware and not only its sign (Section 11.4).
- An equal-budget comparison with faithful LMetric and an SMetric-style baseline on fresh segments or on live runs over non-test cells, which would measure the learned increment the test set could not.
- A size-aware objective or constraint, if a deployment values tokens rather than requests.
- Disaggregated prefill and decode, other models, heterogeneous workers, and a live load generator for agentic traffic.

References

- [1] Abdelfattah et al. Load testing for machine learning model serving systems at scale. arXiv:2606.22013, 2026. URL <https://arxiv.org/abs/2606.22013>.
- [2] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL <https://arxiv.org/abs/2108.13264>.
- [3] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee, and Alexey Tumanov. Vidur: A large-scale simulation framework for LLM inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. URL <https://arxiv.org/abs/2405.05465>.
- [4] Abdullah Alomar, Pouya Hamadani, Arash Nasr-Esfahany, Anish Agarwal, Mohammad Alizadeh, and Devavrat Shah. CausalSim: A causal framework for unbiased trace-driven simulation. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023. URL <https://arxiv.org/abs/2201.01811>.
- [5] Ali Aouad and Antoine Désir. Representing random utility choice models with neural networks. *Management Science*, 2023. URL <https://arxiv.org/abs/2207.12877>.
- [6] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Naz Sepah, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Dmitriy Serdyuk, Tal Arbel, Chris Pal, Gaël Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021. URL <https://arxiv.org/abs/2103.03098>.
- [7] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 2010. URL <https://www.jmlr.org/papers/v11/cawley10a.html>.
- [8] Huang Cheng. CacheRoute: Planned prefix-affinity routing for large-scale LLM serving. arXiv:2608.19677, 2026. URL <https://arxiv.org/abs/2608.19677>.

- [9] Da and Kalyvianaki. RouteBalance: Fused model routing and load balancing for heterogeneous LLM serving. arXiv:2606.17949, 2026. URL <https://arxiv.org/abs/2606.17949>.
- [10] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006. URL <https://www.jmlr.org/papers/v7/demsar06a.html>.
- [11] Jesse Dodge, Suchin Gururangan, D. Card, Roy Schwartz, and Noah A. Smith. Show your work: Improved reporting of experimental results. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019. URL <https://arxiv.org/abs/1909.03004>.
- [12] Nikolaus Hansen. The CMA evolution strategy: A tutorial. arXiv:1604.00772 (v2, 2023), 2016. URL <https://arxiv.org/abs/1604.00772>.
- [13] Nikolaus Hansen, André S. P. Niederberger, Lino Guzzella, and Petros Koumoutsakos. A method for handling uncertainty in evolutionary optimization with an application to feedback control of combustion. *IEEE Transactions on Evolutionary Computation*, 13(1), 2009. URL <http://www.cmap.polytechnique.fr/~nikolaus.hansen/TEC2009.pdf>.
- [14] Helix. Ramjet (helix load balancer). Source repository. URL <https://github.com/helixml/ramjet>.
- [15] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *AAAI Conference on Artificial Intelligence*, 2018. URL <https://arxiv.org/abs/1709.06560>.
- [16] Ayush Jain, Andrew Szot, and Joseph J. Lim. Generalization to new actions in reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, 2020. URL <https://arxiv.org/abs/2011.01928>.
- [17] Kunal Jain, Anjaly Parayil, Ankur Mallick, Esha Choukse, Xiaoting Qin, Jue Zhang, Íñigo Goiri, Rujia Wang, Chetan Bansal, Victor Rühle, Anoop Kulkarni, Steve Kofsky, and Saravan Rajmohan. Intelligent router for LLM workloads: Improving performance through workload-aware load balancing. arXiv:2408.13510, 2024. URL <https://arxiv.org/abs/2408.13510>.
- [18] Jha et al. Learned best-effort LLM serving. arXiv:2401.07886, 2024. URL <https://arxiv.org/abs/2401.07886>.
- [19] Hao Kang, Ziyang Li, Weili Xu, Xinyu Yang, Yinfang Chen, Junxiong Wang, Beidi Chen, Tushar Krishna, Chenfeng Xu, and Simran Arora. ThunderAgent: A simple, fast and program-aware agentic inference system. In *International Conference on Machine Learning (ICML)*, 2026. URL <https://arxiv.org/abs/2602.13692>. arXiv:2602.13692.
- [20] Joohwan Ko and Andrew A. Li. Modeling choice via self-attention. arXiv:2311.07607, 2024. URL <https://arxiv.org/abs/2311.07607>.
- [21] kvcache-ai. Mooncake: trace release, including the FAST’25 traces. Source repository, FAST25-release/traces. URL <https://github.com/kvcache-ai/Mooncake>.
- [22] Joel Lehman, Jeff Clune, Dusan Misevic, et al. The surprising creativity of digital evolution: A collection of anecdotes from the evolutionary computation and artificial life research communities. *Artificial Life*, 26(2), 2020. URL <https://arxiv.org/abs/1803.03453>.

- [23] Gangmuk Lim, Wanyu Zhao, Brighten Godfrey, Jiaxin Shan, Le Xu, and Liguang Xie. Lodestar: An online-learning LLM inference router. arXiv:2606.00946, 2026. URL <https://arxiv.org/abs/2606.00946>.
- [24] llm-d project. Sticky until saturated: token-aware routing. Blog post, optimized-baseline profile. URL <https://llm-d.ai/blog/sticky-until-saturated-token-aware-routing>.
- [25] llm-d project. KV-cache wins you can see. Blog post, precise prefix-cache-aware scheduling, 2025. URL <https://llm-d.ai/blog/kvcache-wins-you-can-see>.
- [26] Luo et al. Autellix: An efficient serving engine for LLM agents as general programs. arXiv:2502.13965, 2025. URL <https://arxiv.org/abs/2502.13965>.
- [27] Horia Mania, Aurelia Guy, and Benjamin Recht. Simple random search provides a competitive approach to reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. URL <https://arxiv.org/abs/1803.07055>.
- [28] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. Neural adaptive video streaming with pensieve. In *Proceedings of ACM SIGCOMM*, 2017. URL <https://web.mit.edu/pensieve/>.
- [29] Hongzi Mao, Parimarjan Negi, Akshay Narayan, Hanrui Wang, Jiacheng Yang, Haonan Wang, Ryan Marcus, Ravichandra Addanki, Mehrdad Khani, Songtao He, Vikram Nathan, Frank Cangialosi, Shaileshh Bojja Venkatakrishnan, Wei-Hung Weng, Song Han, Tim Kraska, and Mohammad Alizadeh. Park: An open platform for learning-augmented computer systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/f69e505b08403ad2298b9f262659929a-Abstract.html>.
- [30] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. Learning scheduling algorithms for data processing clusters. In *Proceedings of ACM SIGCOMM*, 2019. URL <https://arxiv.org/abs/1810.01963>.
- [31] Hongzi Mao, Shaileshh Bojja Venkatakrishnan, Malte Schwarzkopf, and Mohammad Alizadeh. Variance reduction for reinforcement learning in input-driven environments. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://arxiv.org/abs/1807.02264>.
- [32] Vahab Mirrokni, Mikkel Thorup, and Morteza Zadimoghaddam. Consistent hashing with bounded loads. In *Proceedings of the ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2018.
- [33] Michael Mitzenmacher. How useful is old information? SRC Technical Note 1998-002, Digital Equipment Corporation Systems Research Center, 1998. URL <https://www.eecs.harvard.edu/~michaelm/abstracts/tpds2000.html>. Journal version in IEEE Transactions on Parallel and Distributed Systems, 2000.
- [34] Andrew Y. Ng and Michael Jordan. PEGASUS: A policy search method for large MDPs and POMDPs. In *Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI)*, 2000. URL <https://arxiv.org/abs/1301.3878>.
- [35] Nixon et al. A year in LLM serving: Workload evolution, caching and load-balancing. arXiv:2608.13573, 2026. URL <https://arxiv.org/abs/2608.13573>.

- [36] NVIDIA. AIPerf: benchmarking and load generation for generative AI serving. Source repository, version 0.13.0, 2026. URL <https://github.com/ai-dynamo/aiperf>.
- [37] NVIDIA. AISimulate: predicting serving behavior and searching deployment configurations offline. Python package, version 0.13.0.dev2026093000000000061, 2026. URL <https://pypi.org/project/aisimulate/>.
- [38] NVIDIA. NVIDIA Dynamo: a datacenter-scale distributed inference serving framework. Source repository, 2026. URL <https://github.com/ai-dynamo/dynamo>.
- [39] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2018. URL <https://arxiv.org/abs/1710.06537>.
- [40] Karlson Pfannschmidt, Pritha Gupta, Björn Haddenhorst, and Eyke Hüllermeier. Learning context-dependent choice functions. *International Journal of Approximate Reasoning*, 2022. URL <https://arxiv.org/abs/1901.10860>.
- [41] Rakibul Hasan Rajib, Mengxin Zheng, and Qian Lou. AgentServeSim: Serving-system simulation and policy search for LLM agent programs. arXiv:2606.09613, 2026. URL <https://arxiv.org/abs/2606.09613>.
- [42] Alessio Ricci Toniolo, Rome Thorstenson, and Abinaya Dinesh. GORGO: Online tuning for cross-region network-aware LLM serving. arXiv:2602.11688, 2026. URL <https://arxiv.org/abs/2602.11688>.
- [43] Nir Rosenfeld, Kojin Oshiba, and Yaron Singer. Predicting choice with set-dependent aggregation. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020. URL <https://arxiv.org/abs/1906.06365>.
- [44] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. arXiv:1703.03864, 2017. URL <https://arxiv.org/abs/1703.03864>.
- [45] Bianca Schroeder, Adam Wierman, and Mor Harchol-Balter. Open versus closed: A cautionary tale. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2006. URL https://www.cs.utoronto.ca/~bianca/papers/nsdi_camera.pdf.
- [46] semianalysisai. cc-traces-weka-062126: recorded agent sessions. Dataset on Hugging Face, revision 23f152f6. URL <https://huggingface.co/datasets/semianalysisai/cc-traces-weka-062126>.
- [47] Arjun Seshadri, Alexander Peysakhovich, and Johan Ugander. Discovering context effects from raw choice data. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *PMLR*, 2019. URL <https://arxiv.org/abs/1902.03266>.
- [48] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiyang Zhang. Preble: Efficient distributed prompt scheduling for LLM serving. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2407.00023>. arXiv:2407.00023.
- [49] Anam Tahir, Kai Cui, and Heinz Koepl. Learning mean-field control for delayed information load balancing in large queuing systems. In *Proceedings of the 51st International Conference on Parallel Processing (ICPP)*, 2022. URL <https://arxiv.org/abs/2208.04777>.

- [50] Kiran Tomlinson and Austin R. Benson. Learning interpretable feature context effects in discrete choice. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2021. URL <https://arxiv.org/abs/2009.03417>. arXiv:2009.03417.
- [51] Train. GEV. In *Discrete Choice Methods with Simulation*, chapter 4. Cambridge University Press, 2nd edition, 2009. URL https://eml.berkeley.edu/books/choice2nd/Ch04_p76-96.pdf.
- [52] Srikanta Datta Tumkur, Jay Iyer, Mehar Simhadri, Sai Pavan Kumar, Sai Kapil Kumar, and Ramesh Nampelly. Calibrate, then route: A measured study of learned request routing for disaggregated LLM serving. arXiv:2609.16206, 2026. URL <https://arxiv.org/abs/2609.16206>.
- [53] Mark van der Boor, Sem C. Borst, Johan S. H. van Leeuwen, and Debankur Mukherjee. Scalable load balancing in networked systems: A survey of recent advances. *Statistical Science*, 2022. URL <https://arxiv.org/abs/1806.05444>.
- [54] Jiahao Wang, Kaizhan Lin, Kaixi Zhang, Jinbo Han, Xingda Wei, Sijie Shen, Chenguang Fang, Wenyuan Yu, Rong Chen, and Haibo Chen. SMetric: Rethink LLM scheduling for serving agents with balanced session-centric scheduling. arXiv:2607.08565, 2026. URL <https://arxiv.org/abs/2607.08565>.
- [55] Zhibin Wang, Shipeng Li, Yuhang Zhou, Xue Li, Zhonghui Zhang, Nguyen Cam-Tu, Rong Gu, Chen Tian, Guihai Chen, and Sheng Zhong. Revisiting service level objectives and system level metrics in large language model serving. arXiv:2410.14257 (v2, 2025), 2024. URL <https://arxiv.org/abs/2410.14257>.
- [56] Ryan Webb, Paul W. Glimcher, and Kenway Louie. The normalization of consumer valuations: Context-dependent preferences from neurobiological constraints. *Management Science*, 2021. URL <https://www.neuroeconomicslab.org/s/Normalization-of-Consumer-Valuations.pdf>.
- [57] Fangzhou Wu, Sandeep Silwal, and Qiuyi Zhang. Randomization boosts KV caching, learning balances query load: A joint perspective. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://arxiv.org/abs/2601.18999>. arXiv:2601.18999.
- [58] Zhengxu Xia, Yajie Zhou, Francis Y. Yan, and Junchen Jiang. Genet: Automatic curriculum generation for learning adaptation in networking. In *Proceedings of ACM SIGCOMM*, 2022. URL <https://arxiv.org/abs/2202.05940>.
- [59] Francis Y. Yan, Hudson Ayers, Chenzhi Zhu, Sadjad Fouladi, James Hong, Keyi Zhang, Philip Levis, and Keith Winstein. Learning in situ: a randomized experiment in video streaming. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020. URL <https://arxiv.org/abs/1906.01113>.
- [60] Reza Yousefi Maragheh, Alexandra Chronopoulou, and James Mario Davis. A customer choice model with HALO effect. arXiv:1805.01603, 2018. URL <https://arxiv.org/abs/1805.01603>.
- [61] Ying Yuan, Pengfei Zuo, Bo Wang, Zhangyu Chen, Zhipeng Tan, and Zhou Yu. DualMap: Enabling both cache affinity and load balancing for distributed LLM serving. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://arxiv.org/abs/2602.06502>. arXiv:2602.06502.

- [62] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabás Póczos, Ruslan Salakhutdinov, and Alexander J. Smola. Deep sets. In *Advances in Neural Information Processing Systems (NIPS)*, 2017. URL <https://arxiv.org/abs/1703.06114>.
- [63] Dingyan Zhang, Jinbo Han, Kaixi Zhang, Xingda Wei, Sijie Shen, Chenguang Fang, Wenyuan Yu, Jingren Zhou, and Rong Chen. Simple is better: Multiplication may be all you need for LLM request scheduling. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2026. URL <https://arxiv.org/abs/2603.15202>. arXiv:2603.15202.
- [64] Shuhan Zhang, Zhi Wang, Rui Gao, and Shuang Li. DeepHalo: A neural choice model with controllable context effects. arXiv:2601.04616, 2026. URL <https://arxiv.org/abs/2601.04616>.
- [65] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024. URL <https://arxiv.org/abs/2401.09670>.

A Notation

Table 19 fixes the symbols used throughout. A few are easily confused. S is the set of eligible candidate workers for one request, while $s_{\max}$ is the end-to-end slowdown bound (and $I_{\max}$ the mean-ITL bound). k indexes common-random-number (CRN) workload replicates, while $\ell = 1, \dots, r$ indexes context terms. B is the CMA-ES evaluation budget per run, while b_j is a request’s block count. $T_{\text{ch}} = 8192$ is the token normalizer of the prefill features. Bold $\mathbf{e}_f$ is a standard basis vector, so the parity anchor is $\theta = -\mathbf{e}_0$, while E_0 is a latency. τ is the policy temperature, and τ_K is Kendall’s rank correlation. σ belongs to CMA-ES (step size σ , initial σ_0), while workload segments are $\omega \in \Omega$, ratios ρ and restarts R .

Table 19: Notation. “Value” is the setting in this study; code and file names are in typewriter. Pilot values are *preliminary (pilot, validation split)*.

Symbol	Meaning	Value / code name
<i>Deployment and requests</i>		
N	number of identical workers (one Qwen3-32B TP2 vLLM engine each)	$\{2, 4, 6, 8, 16, 32\}$; train $\{4, 8\}$, val $\{4, 6, 8\}$, test all; <code>num_workers</code>
j	request index	
a_j	arrival time of request j (simulated)	
$\text{ISL}_j, \text{OSL}_j$	prompt and output length (tokens)	<code>input_length</code> , <code>output_length</code>
b_j	request blocks, $\max(\lceil \text{ISL}_j / 16 \rceil, 1)$	KV block size 16; <code>request_blocks</code>
$\text{TTFT}_j, \text{E2E}_j$	time to first token; end-to-end latency (simulated)	<code>per_request</code> rows
$\overline{\text{ITL}}_j$	mean inter-token latency, $(\text{E2E}_j - \text{TTFT}_j) / (\text{OSL}_j - 1)$, for $\text{OSL}_j > 1$	
$E_0(\text{ISL}, \text{OSL})$	uncontended no-reuse latency: the request alone on an idle worker, full prefill, then $\text{OSL} - 1$ decode steps at batch 1, timed by AIS. Evaluation only; no policy sees it	<code>ais-chunked-estimator-v2</code> ; <code>learned_routing/e0.py</code>

Symbol	Meaning	Value / code name
$I_{\max}$	mean-ITL bound of the SLO	Mooncake and FAST25 252.117 ms; sessions 46.0478 ms; AgentX 38.6205 ms
$s_{\max}$	end-to-end slowdown bound	Mooncake and FAST25 3.46625; sessions 2.11784; AgentX 1.16134
g_j	good-request indicator, Equation (2)	<code>learned_routing/goodput.py</code>
<i>Policy</i>		
S	eligible candidate set of one request; $ S \leq N$	host eligibility
i	a candidate worker, $i \in S$	
$x_i \in \mathbb{R}^d$	feature vector of candidate i , fixed order (Table 6)	$d = 8$ (v1) or 23 (v2); <code>feature_set</code>
$\bar{x}_S$	candidate mean of x over S	
$\mathbf{e}_f$	standard basis vector; $\theta = -\mathbf{e}_0$ reproduces the default router	
T_{ch}	token normalizer of the prefill features	8192
$\theta \in \mathbb{R}^d$	linear coefficients	<code>parameters.theta</code> ; $\theta_0 = -1$ pinned in training
r, ℓ	context rank; context-term index $\ell = 1, \dots, r$	$r = 0$ (M1), $r = 2$ (M2); <code>len(context.p)</code>
$p_\ell \in \mathbb{R}^d$	context direction of term ℓ	<code>context.p</code>
$z_{S,\ell}$	context value of term ℓ : a named source (Table 23) or, in the pooled form, $q_\ell^\top \bar{x}_S$	<code>context.sources</code> or <code>context.q</code>
w_S	effective weights, $\theta + \sum_\ell z_{S,\ell} p_\ell$	<code>features.rs</code>
u_i	utility of candidate i , $w_S^\top x_i$	
τ	temperature: $\tau = 0$ is argmax with a seeded tie-break, $\tau > 0$ samples softmax(u/τ)	<code>temperature</code> ; fixed at 0 for M1 and M2
π, π_θ	a routing policy; the learned policy	one policy YAML each
π_0	<code>default@defaults: dynamo-default-cost-fn</code> at shipped parameters; the normalizer of every ratio	harness name <code>default</code>
π_{base}^*	the validation-selected best baseline (Section 8)	tuned <code>ramjet</code>
<i>Workloads, metric and tests</i>		
c	cell: one trace segment, transform, load mode and value, N and SLO	<code>cell_id</code>
$\mathcal{C}_{\text{train}}, \mathcal{C}_{\text{val}}, \mathcal{C}_{\text{test}}$	cells of each split	34, 14, 60
$\omega, \Omega_{\text{split}}$	segment (unit of statistical independence); segments of a split	10, 6, 12
$\mathcal{T}_c$	measurement window of cell c (Section 3.3)	<code>measure</code>
L1, L2, L3	load levels at which π_0 's good fraction is 0.95, 0.85, 0.65	<code>load.level</code>
k	CRN replicate index; replicate k also sets policy seed $k + 1$	<code>repeat</code>
$G_{c,k}(\pi)$	windowed goodput (good requests per simulated second)	<code>goodput_rps_window</code>
ε	offset in goodput ratios	10^{-3} ; <code>--eps</code>
$\rho_{c,k}(\pi, \pi')$	$(G_{c,k}(\pi) + \varepsilon)/(G_{c,k}(\pi') + \varepsilon)$; $\pi' = \pi_0$ unless stated	<code>pair_score</code>
$\Delta_{c,k}(\pi, \pi')$	clipped log-ratio, $\text{clip}(\log \rho, -\log 3, \log 3)$	<code>clipped_log_ratio</code>
D_ω	per-segment headline statistic, Equation (9)	<code>facts/HEADLINE_TEST.json</code>
MDE	minimum detectable effect, $\max(1\%, 2 \times$ validation segment SE)	0.038, preliminary (pilot, validation split)

Symbol	Meaning	Value / code name
δ	router-state lag, robustness check only	$\{10, 50, 200\}$ ms on test; <code>router_state_lag_ms</code>
τ_K	Kendall rank correlation of two policy rankings (a perturbed against the nominal condition; simulation against live GPUs)	τ -b
<i>Training</i>		
$J(\pi)$	training objective, Equation (8)	<code>--objective clipped_log_ratio</code>
ϕ	CMA-ES internal coordinates of the free parameters (bounded ones in $[0, 1]$)	<code>space.yaml</code>
t	CMA-ES generation	$0, \dots, 24$
λ	population size	16; <code>--popsize</code>
B	objective evaluations per run	$400 = 25 \times 16$; <code>--budget-evals</code>
K, K_{pool}	replicates per objective evaluation; the pool they rotate through	2 and 8; <code>--replicates, --replicate-pool</code>
K_t	replicates of generation t , $\{(tK + i) \bmod K_{\text{pool}} : i < K\}$	<code>Trainer.eval_set</code>
K_{val}	replicates per validation score inside the tuning harness	3 ($k = 0, 1, 2$)
σ_0	initial CMA-ES step size, in internal units	per policy (Section 6)
R	restarts per policy	3

B The worker-selection API and the ported policies

This appendix completes Sections 2 and 7: how the host builds and fills the candidate table, how it loads a policy catalog, the default picker’s remaining details, and every ported rule in full.

B.1 Eligibility and pins

For each request j the host builds a *candidate table*: one row per eligible worker (or worker and data-parallel rank). Eligibility is the host’s: a worker must be in the request’s allowed set, available, not marked overloaded by the host’s busy thresholds, routable, and must satisfy the request’s routing constraints. If the request is pinned to a worker, that worker is the only candidate, and a policy may also ask the host to treat a session-affinity target as exclusive, in which case an eligible target is pinned the same way. Pins and eligibility always win over a policy’s preference.

B.2 Declared inputs

A component declares the optional worker signals it needs, as a set of `WorkerInputs`, and the host materializes only the union of what the policy’s components declared. Each component then sees only its own declarations, even when another component of the same policy asked for more.

CACHE the accounting cache estimate (effective overlap blocks and estimated cached tokens), the per-tier overlaps (device, host, disk) and shared-cache hits.

LOAD active prefill tokens, decode cost blocks and active requests, with a flag that separates an unobserved worker from an idle one.

PREFILL_TIME the host’s modeled prefill backlog in milliseconds, available only when the router runs a prefill-load model (`router_prefill_load_model: ais`).

OCCUPANCY host-owned active-request counts.

PREFERRED_TAINT a cost multiplier from the request’s preferred routing constraints.

No policy in this study declares **OCCUPANCY**, and only the AIS-informed variants read **PREFILL_TIME** (Section 7.2).

B.3 Request facts

Request-level values arrive through **WorkerSelectionContext**: the prompt length in tokens and in KV blocks and the block size; the *prefix hashes*, chained hashes of the prompt’s complete blocks, so that two requests share their first $i + 1$ blocks exactly when their entry i is equal; whether the request is tracked for prefill load; the session context; the session-affinity target the host resolved; the expected output length, if the request carries one; each worker’s advertised KV capacity; the policy class; the request’s priorities; and a per-request router temperature override. These are the only request facts a policy can use.

B.4 Catalog loading

A deployment’s policy file names catalog types and their parameters (Section 2.2). A registered provider for each type validates the parameters once, at startup, and returns a factory that builds one policy instance per routing partition and worker role, so every instance owns its state. Unknown types and malformed parameters fail at startup with a message naming the problem. The name **default** is reserved for the builtin selector.

B.5 Default picker details

The default picker (Section 2.3) breaks ties at random. On Dynamo’s main branch the generator is unseeded, so identical replays of one cell differ, and this study added a **seed** parameter (caveat 6, Appendix C). The picker deliberately ignores the per-request temperature override, and the default policy asks the host to treat an eligible session-affinity target as exclusive.

B.6 The ported policies

We describe each rule in the router’s terms. For request j with candidates S , worker i has overlap o_i blocks, L_i^{pf} active prefill tokens, L_i^{dec} decode cost blocks, L_i^{req} active requests and a configured capacity of cap_i blocks; a KV block holds 16 tokens. The ports read the device-tier overlap, which in replay equals o_i , so worker i ’s uncached prompt is $\max(\text{ISL}_j - 16 o_i, 0)$ tokens.

The six ported policies are those of the pull-request stack ai-dynamo/dynamo#15450 and #15453 at the commit the study’s branch starts from; the stack’s **lmetric** has changed since (Section 7.1).

Table 20 summarizes every baseline’s rule at its shipped defaults, and Table 1 gives each policy’s composition; the default cost is Equation (1), and the paragraphs below state each ported rule in full.

B.6.1 The rules in full

Two-tier. **dynamo-two-tier-cost-fn** ports the thresholds and selection order of the experimental SGLang router’s **cache_aware_zmq** policy. Instead of one additive cost it applies the first of three rules that holds. (1) If the spread of active requests across candidates exceeds **balance_abs_threshold** (default 32) and the largest count exceeds **balance_rel_threshold** (1.1) times the smallest, it takes the worker with the fewest active requests. (2) Otherwise, if the largest overlap

Table 20: Baselines at their shipped defaults, as implemented on the study’s branch. Every policy except round-robin is also tuned (Table 25). o_i : overlap blocks of worker i ; L_i^{req} : its active requests; one KV block is 16 tokens. “Tie rotation” is a counter that rotates among tied workers.

Policy	Rule at shipped defaults	State kept by the policy
<code>round_robin</code>	requests go to the workers in turn; reads neither cache nor load	position in the cycle
<code>dynamo-default-cost-fn</code>	$\arg \min_i c_i^{\text{def}}$ (Equation (1)): prefill blocks left after the overlap credit, plus decode blocks	none; random tie-breaks
<code>dynamo-two-tier-cost-fn</code>	fewest active requests; but while load is not badly imbalanced (spread ≤ 32 or largest $\leq 1.1 \times$ smallest) and some worker caches more than half the prompt, the least loaded of the best-cached workers	none
<code>lmetric</code>	$\arg \min_i \max(\text{ISL}_j - 16 o_i, 1) (L_i^{\text{req}} + 1)$, with a hot-spot filter for popular prefixes	class shares over the last 1000 requests
<code>ramjet</code>	$\arg \max_i$ of capped relative prefix affinity minus 4 times the worker’s load units	tie rotation
<code>dualmap</code>	two hashed candidates per prefix; the warmer one unless its prefill backlog would exceed 65,536 tokens	prefix shares over the last 1000 requests
<code>chwb1</code>	hashed owner of the prompt’s first 256 tokens while its active requests stay within 1.25 times the mean (counting this request); reads no cache	none
<code>llm-d-precise-prefix</code>	$\arg \max_i$ of prefix, queue and KV-utilization scores weighted 2:1:1	tie rotation
<code>llm-d-optimized-baseline</code>	lowest prefill token load among workers caching at least 80% of the prompt, unless their TTFT estimate trails by more than 18s	tie rotation
<code>sticky-session, hard</code>	the session’s previous worker when eligible, otherwise the default cost’s choice	session map (65,536)
<code>sticky-session, bounded</code>	as hard, but leaves that worker when its active requests exceed 1.25 times the candidate mean	session map (65,536)

$\max_i o_i$ exceeds `cache_threshold` (0.5) times the request’s b_j blocks, strictly, it takes the least-loaded worker among those holding that overlap. (3) Otherwise it takes the least-loaded worker. Load means L_i^{req} alone, and ties go to the lowest worker ID. The port reads Dynamo’s KV-event overlap where the SGLang router keeps its own approximate cache history.

LMetric. Following Zhang et al. [63], the `lmetric` port routes to

$$\arg \min_{i \in S} \max(\text{ISL}_j - 16 o_i, 1) (L_i^{\text{req}} + 1), \quad (10)$$

the product of the worker’s uncached prompt tokens and its batch size counting this request, with ties to the lowest worker ID. A product needs no weight between its factors: a weight on either one scales every candidate’s score alike. A hot-spot detector handles a popular prefix that few workers hold. Requests sharing their first `class_prefix_blocks` (4) blocks form a class, and the workers whose overlap covers those blocks hold it. A class is alarmed when the odds of its share of the last `window_requests` (1000) decisions exceed the ratio of holders to non-holders. While it is alarmed, once twice as many consecutive class requests as there are holders would have gone to a holder, later ones go to the best non-holder. The paper measures the window in time; the port counts decisions

so that replay stays deterministic. The paper’s prefill factor also counts the prefill already queued on the worker, L_i^{pf} ; the port as evaluated here omits that term, a gap that matters for the scope of the headline (Sections 7.1 and 9.11). After this study froze its results, the port in the policy-stack pull request (ai-dynamo/dynamo#15450) was fixed to add L_i^{pf} to the prefill factor, as the paper defines it. Every `lmetric` result in this paper is from the port before that fix.

Ramjet. The `ramjet` port follows Helix’s Ramjet load balancer [14]. Each worker scores its credited prefix affinity minus `alpha` (default 4) times its load units, and the highest score wins, then the deeper raw overlap, then a rotating tie-break. Affinity counts the cached prefix in 512-token blocks (`affinity_block_tokens`). With the default `relative` basis, which the port’s documentation says Ramjet used in its published comparison with Dynamo, a worker is credited the warmest candidate’s affinity capped at `max_affinity_blocks` (32), minus how far it trails that candidate, also capped; differences near the leader therefore survive the cap. A worker’s load units are $L_i^{\text{req}} + L_i^{\text{pf}}/8192$ (`load_unit_tokens`). Ramjet charges each in-flight request between 1 and 8 units according to its uncached prompt, but Dynamo exposes load per worker rather than per request, so the port approximates Ramjet’s phase-aware mode.

DualMap. In the `dualmap` port, following Yuan et al. [61], two rendezvous hashes of a prompt-prefix key pick two distinct candidates, so requests that share a prefix share candidates. A candidate’s backlog is $L_i^{\text{pf}} + \max(\text{ISL}_j - 16 o_i, 0)$, the prefill tokens it would carry after taking the request. The request goes to the candidate with more overlap unless that candidate’s backlog would exceed `pending_prefill_token_budget` (65,536 tokens); then, and whenever the two overlaps are equal, it goes to the candidate with the smaller backlog. The key hashes the first `hash_prefix_blocks` (4) blocks. A prefix is hot when its share of the last `window_requests` (1000) requests exceeds $2/|S|$, and a hot prefix’s key doubles in depth, up to seven times, which spreads its requests over more candidate pairs.

CHWBL. `chwb1` is consistent hashing with bounded loads [32], ported from KubeAI’s balancer with the defaults of SGLang’s `prefix_hash` gateway policy. The key is the prefix hash of the prompt’s first 16 blocks (`prefix_tokens` = 256), or of all its full blocks if the prompt is shorter. The worker with the highest rendezvous weight owns the key and takes the request while $L_i^{\text{req}} \leq 1.25 (1 + \sum_{i' \in S} L_{i'}^{\text{req}})/|S|$ (`load_factor` = 1.25); otherwise the request walks the other workers in hash order to the first one within that bound, and returns to the owner if none is. The policy reads no cache state, so it routes the same with or without KV events.

llm-d precise prefix. `llm-d-precise-prefix` ports the precise prefix-cache profile of llm-d’s inference scheduler [25]. It is the one policy here built from several scorers: a prefix scorer, a queue scorer and a KV-utilization scorer, weighted 2:1:1, with a picker that takes the highest total. At the shipped prefix-length weight of 0 it routes to

$$\arg \max_{i \in S} 2 \min \left(1, \frac{o_i}{\lfloor \text{ISL}_j / 16 \rfloor} \right) + \frac{\max_{i' \in S} L_{i'}^{\text{req}} - L_i^{\text{req}}}{\max_{i' \in S} L_{i'}^{\text{req}} - \min_{i' \in S} L_{i'}^{\text{req}}} + 1 - \min \left(1, \frac{L_i^{\text{dec}}}{\text{cap}_i} \right), \quad (11)$$

where the queue term is 1 when every candidate has the same L_i^{req} , and ties rotate. A nonzero `prefix_match_length_weight` blends the first term with the squared matched length relative to `prefix_match_length_scale_tokens` (8192), and per-class weights can follow the request’s policy class, which replay never sets.

llm-d optimized baseline. `llm-d-optimized-baseline` ports llm-d’s optimized-baseline profile, a prefix-cache affinity filter followed by a token-load scorer [24]. Workers whose overlap covers at least `affinity_threshold` (0.8) of the prompt’s complete blocks are sticky. Each worker’s TTFT is estimated from its prefill backlog: with `ttft_source: throughput` as $1000 L_i^{\text{pf}} / 15,928 \text{ ms}$, where $\text{peak_prefill_tokens_per_second} = 15,928$ is llm-d’s calibration for Qwen3-32B at TP2 on H100, and with `ttft_source: modeled` from the host’s PREFILL_TIME backlog. If the best sticky estimate trails the best non-sticky one by more than `max_ttft_penalty_ms` (18,000), stickiness breaks and every worker stays in play; otherwise only the sticky workers do, or all workers if none is sticky. Among those, the request goes to the worker with the lowest token load $L_i^{\text{pf}} + \max(\text{ISL}_j - 16 o_i, 0)$ (scored as one minus that load over `queue_threshold_tokens`, 4,194,304, capped at 1), and ties rotate. The port implements llm-d’s filter inside its picker, since the API’s filters cannot see the other candidates. The headline league uses the throughput source; the modeled source reads a performance-model estimate and belongs to the secondary AIS-informed league (Section 7.2).

Sticky sessions. Dynamo’s live router can pin a session to a worker, but offline replay never sets the router’s affinity target. This study therefore added `sticky-session`, which keeps a bounded least-recently-used map of up to 65,536 sessions, each mapped to the worker that served its last request. A request without a session, or the first of its session, takes the default cost function’s choice, built from the router’s configured default-cost knobs. In `hard` mode a later request goes to its session’s worker whenever that worker is eligible; otherwise it takes the default choice and the session rebinds. In `bounded` mode it also leaves the session’s worker when that worker’s active requests exceed `load_factor` (1.25) times the candidates’ mean. The policy mirrors the intent of the live router’s session affinity; it is not a port of a published method. On the flat traces (Mooncake and FAST25) open-loop requests carry no session and closed-loop requests carry a single-use one, so both modes make the default choice there, apart from tie-breaking.

ThunderAgent. `thunderagent`’s worker selection is a scorer of each worker’s prefill tokens plus its decode blocks in tokens and a picker that follows the affinity target its paired request classifier sets, falling back to the lowest cost [19]. Its program state lives in the classifier, a separate kind of plugin that runs before worker selection. Replay runs worker-selection policies but not request classifiers, so this policy was not evaluated (Section 7.2).

B.6.2 Sanity check

The decision to evaluate the ports as implemented (Section 7.1) called for only a brief sanity check, that each port’s decisions respond to its inputs in the documented direction. A setup audit confirmed that the parameters of `chwb1`, `dualmap`, `ramjet` and both llm-d ports change their routing deterministically, and that `lmetric` and two-tier replay identically across runs. In the pilot’s screen of the nine untuned heuristics on 12 train cells with replicates 0 and 1, every port ran without error, none routed against its documented direction in aggregate, and seven beat π_0 . `dualmap` and `chwb1` lost to it, with mean clipped log-ratios of -0.150 and -0.214 , which the pilot attributed to hash-pinned placement on traffic with few hot prefixes; round-robin scored -0.481 . These screen numbers are preliminary (pilot, train subset).

B.6.3 Where the ports differ

Besides `lmetric`’s, which Section 7.1 states, the known differences between the ports and the published methods are these.

- **dualmap**’s budget is a fixed default; the paper derives it as the prefill backlog a GPU can clear within the TTFT SLO [61], and our SLO has no TTFT term. The port uses rendezvous hashing in place of the paper’s consistent-hash rings, and it omits the paper’s migration of queued requests between the two candidates, because Dynamo places each request once.
- **ramjet** approximates Ramjet’s per-request load units from per-worker load, reads Dynamo’s KV-event index instead of Ramjet’s own approximate prefix index, and omits Ramjet’s lane that confines very long prompts to designated replicas.
- **llm-d-precise-prefix**’s queue score counts running as well as waiting requests, where llm-d reads the engine’s waiting queue.
- **llm-d-optimized-baseline**’s prefill rate is llm-d’s hardware calibration for this model, GPU and parallelism, while replay times prefill with AIS; the port also omits llm-d’s optional exploration probability.
- Two-tier reads Dynamo’s KV-event overlap instead of the SGLang router’s approximate cache history, and breaks ties by worker ID instead of the router’s worker-list order.
- **chwb1** replaces the hash ring with rendezvous hashing.

Section 7.1 notes the two of these that tuning absorbs.

C Major caveats

Building, calibrating and auditing the study surfaced a number of properties of the API, of the replay host and of the workloads that can change which policy wins. Table 21 lists the major ones with their evidence and status: *fixed* (a fix exists on the study’s branch or upstream), *follow-up* (recorded for an upstream change, worked around here), or *inherent* (a property of the setting that this study can only scope). Several of them shape the results in later sections, which refer to them by row number; Section 2.6 highlights those.

Table 21: Major caveats this study uncovered. Evidence names the campaign record; “status” says what was done.

#	Caveat	Evidence and effect	Status
1	Replay’s router state is perfectly fresh.	Replay applies KV events, prefill completions and request completions to the router synchronously, so policies always see exact overlap and load; a live router sees them late. Policies that lean on exact state could lose their advantage live.	Follow-up: a lag knob on a separate branch; M1-v2 stays first and ahead of ramjet at 10–200 ms on test and ahead at 1,000 ms on validation (Section 9.9), and the live ranking agrees with replay’s on six test cells (Section 11).
2	expected_output_tokens equals the true output length in replay.	The replay host fills the field from the trace’s output length, a value no live router has. A policy reading it would see the future.	Inherent to replay; no learned feature reads it, and two leakage audits found no decision that depends on it (Section 5.1).

#	Caveat	Evidence and effect	Status
3	Prefix hashes are unique per request when the host does not assume KV reuse.	With <code>router_assume_kv_reuse=false</code> the tracking hashes are random, so prefix-keyed policies (<code>chwb1</code> , <code>dualmap</code> , <code>lmetric</code> 's classes, <code>learned-choice</code> 's <code>hash_home</code>) lose their keys and decode-block accounting changes; turning off <code>router_track_prefill_tokens</code> zeroes the active-prefill features.	Inherent to the API contract, documented in the policy stack; the learned policy must run with the training-time host knobs (Section 5.1).
4	Synthesized closed-loop session IDs reach the router.	For a flat trace in closed loop, replay gives every request a single-use session ID and passes it to the router; the same trace in open loop, or a live client, sends none. A session-keyed feature then behaved differently by load mode (2 against 8 workers per prefix key).	Follow-up: a fix with a regression test is in AISimulate pull request #378, and Dynamo needs a pin bump after its release; the learned feature was re-keyed on the prompt prefix.
5	One public trace is a relabeled copy of another.	The FAST'25 tool-agent trace matches Mooncake on all 23,608 output lengths, maps one-to-one onto its hash blocks and scales its timestamps by 0.9825. It would have put one workload in two families and possibly two splits.	Fixed: excluded from every split (Section 4.4).
6	The default router's tie-breaks are unseeded.	Identical replays of one cell gave goodput 2.8934 and 2.8290 under the catalog default, so re-runs could not be paired.	Fixed on the study's branch (a <code>seed</code> parameter); follow-up for the policy stack.
7	Open-loop agent traffic has a load cliff.	With Poisson play arrivals, between 0.002 and 0.0025 plays per second per worker the number of plays in flight grows without bound at every N , so no stationary open-loop load reaches the heaviest calibration band.	Inherent: AgentX runs only in agentic lanes (Appendix D.2.1).
8	The slowdown clause has a knife edge at slowdown 1, and drain-window scoring dilutes differences.	A request that ran alone has $E2E = E_0$ exactly; float noise and an off-by-one decode context made it fail a strict comparison in a policy-dependent way (round-robin's ratio 0.57 instead of 0.88 on one cell). Scoring closed-loop windows through the end-of-run drain diluted policy differences by 23–41%, once flipping a sign.	Fixed: E_0 aligned with replay's decode context plus a 10^{-6} relative tolerance; windows end at full occupancy (Section 3.3).
9	Logged arrival timestamps are quantized.	Mooncake's arrivals sit on a 3,053 ms grid, with up to 47 requests per instant, and replay delivers each burst at once at any speedup. At $N = 2$ the default's 85th-percentile slowdown was 8.32 with bursts and 1.46 with arrivals spread inside their slot.	Follow-up: the harness spreads arrivals per replicate (Section 4.3).
10	No AgentX play fits a 32K context.	Every one of the 82 plays that fits the 131,072-token context peaks at 44,048 tokens or more, so the deployment runs at 128K with YaRN, where AIS's accuracy is least validated.	Inherent; disclosed (Section 12.1).
11	The per-request temperature override is ignored by the default policy.	The plugin default picker reads only its own temperature; a documentation example still shows the override.	Intentional upstream; documentation follow-up.

#	Caveat	Evidence and effect	Status
12	The policy file could change between hashing and use.	A policy file edited during a replay benchmark run could be reported under a hash that does not match the policy that ran.	Fixed in the policy-stack pull request ai-dynamo/dynamo#15450: the benchmark hashes the file once and rejects a change before reporting.
13	AISim prices a mixed prefill pass at the batch mean.	Adding a short fresh prompt to a long-context chunk makes the pass cheaper than the chunk alone (0.73–0.82× in four probes), a simulator artifact a search could exploit.	Follow-up (not patched, it would change every cached result); the exploit check is negative for the headline policy (Section 10).
14	Catalog policies saw no device-tier overlap in replay on Dynamo’s main branch.	Replay left the device-tier overlap at zero, which reduces every cache-aware port to load-only routing.	Fixed in the policy stack this study builds on, with a regression test.
15	A request-count objective can be gamed.	The first tuned learned model kept one worker whose requests were all bad (Section 10).	Remedied by a sign constraint on load coefficients; the remaining concentration is reported on every learned row.

D Simulator, workload and calibration details

This appendix gives the details behind Section 4, in the order of that section: the replay environment, the workloads, the calibration and the freeze of the test set.

D.1 Replay

Policies in replay. Worker selection in replay goes through the same Rust policy catalog as in the live router, and a policy is configured by the same policy YAML. Replay rejects unknown policy types and unknown parameters by name, and the harness writes every policy YAML in one canonical form. Running catalog policies inside offline replay is a feature of the branch this study builds on; on Dynamo’s main branch, replay rejects custom policies.

Engine facts. A single 1,024-token request replayed alone has a TTFT of 55.49 ms, equal to the AIS estimator’s prediction to within 1 ns, and an 8,192-token request 475.97 ms. Table 22 compares AIS timing with the mock engine’s built-in polynomial model. The fallback model underestimates long prompts badly (7.2 times at 120,000 tokens), so a replay without AIS would mislead about exactly the long-context traffic this study cares about.

Prefill cost per token grows with the context already in the KV cache: an 8,192-token chunk costs 59.5 ms per thousand tokens with no prefix and 249.6 ms per thousand after a 114,688-token prefix. A decode step takes 15.1 ms for one 2,048-token sequence and 26.5–29.2 ms for about 262,000 tokens of context in the batch, whether they are 128 sequences of 2,048 tokens, 32 of 8,192 or 8 of 32,768. One request at the 131,072-token context limit would occupy 43% of a worker’s KV cache,

Table 22: Time to first token of one request replayed alone on an idle worker, with AIS timing and with the mock engine’s built-in polynomial model, which replay falls back to when the bindings lack AIS support. Every replay in this paper uses AIS timing.

Prompt length (tokens)	1,024	8,192	32,768	65,536	120,000
TTFT, AIS (ms)	55.49	475.97	2,550.45	6,838.33	17,815.54
TTFT, polynomial fallback (ms)	32.49	169.14	676.55	1,353.10	2,476.95

and the mean AgentX prompt (66,313 tokens) 22%, so four or five AgentX contexts fill a worker. Replay rejects a request whose prompt reaches the context limit and silently truncates the output of one whose prompt plus output exceeds it; the trace transforms therefore drop or clamp such rows explicitly (Section 4.4).

Determinism across hosts. Within one process and across fresh processes a seeded replay reproduces every per-request row; request UUIDs only reorder the output (Appendix M.5). Bit-exact results also need the same operating-system image, because replay calls the host’s math library. Each distinct remote CPU node image (operating system and C library) was checked against the workstation before use: on the first check, 28 of 28 remote records and all 186,548 per-request rows matched a fresh local run. A node image added later passed its own check with the same counts (Appendix M.6).

Where noise comes from. Because a replay is deterministic, running it twice measures nothing; the setup audit rejected the original plan to estimate noise that way (Table 31). What is arbitrary in a recorded workload is the order of requests logged at the same instant. The Mooncake trace has 1,180 distinct timestamps in its hour, a grid of about 3,053 ms, and up to 47 requests share one; the generated sessions use a 1 s grid. Replay delivers each burst at once and places it in file order.

The replicate protocol. Replicate k of a cell re-draws only that arbitrary part of the trace, and the same replicate is used for every policy (common random numbers):

- **Arrival spread** (`crn-spread-v1`; Mooncake, FAST25 conversation and sessions). Each session’s first arrival moves to a uniform random point inside its logging slot (3,053 ms, 3,000 ms and 1,000 ms for the three traces); sessions are re-sorted by the new times and later turns keep their completion-relative delays.
- **Order permutation** (`crn-order-v1`). FAST25 synthetic, whose timestamps are nearly unique, permutes the sessions that tie; AgentX permutes the order of its recycled play copies, which re-deals them to lanes (Appendix D.2.1).
- **Policy seed** $k + 1$ for every policy that takes a seed.

The replicate seed is a hash of the protocol, the source trace’s SHA-256 and k . Cells that share a trace (other N , other load level) therefore share their replicates, and every replicate, including $k = 0$, is a fresh draw, so the recorded file order has no special status. Spreading arrivals is also a modeling choice, discussed in Section 4.3.

Replicate noise. Replicates move goodput by amounts comparable to the effects of interest. On three uncalibrated 2,000-request Mooncake cells, goodput varied across 8 replicates with a coefficient of variation of 1.2–3.9% for every policy tested. On the calibrated cells, the standard deviation of

round robin’s per-replicate goodput ratio against π_0 , pooled by family, N and load mode, ranges from 0.004 to 0.045. Segment heterogeneity, not replicate noise, limits what the study can resolve (Section 8.2).

Cost. A 2,000-request Mooncake replay takes about 1.5 s of one CPU core in a fresh process, nearly flat in N from 2 to 32, and the full 23,608-request trace 7.6–8.8 s. AgentX traffic costs 2.9–4.9 ms per request, so an AgentX cell at $N = 32$ takes 2–4 minutes. Results are cached under a key built from the policy, the cell’s content, the replicate, the harness version and the replay build, so a rebuilt binding or an edited cell can never return a stale result (Appendix M.5).

D.2 Workloads

Mooncake slices. An earlier design used overlapping 12-minute Mooncake slices, in which each window’s warm-up was the previous window’s measurement; an audit found that half of test window w3’s scored rows were also replayed in train cells, and the slices were redesigned before calibration. No row now appears in two slices.

FAST25. The FAST25 conversation trace shares Mooncake’s (timestamp, prompt length, output length) skeleton on 91% of rows but has a different prefix structure (reuse 0.38 against 0.63, one shared first block). The synthetic trace has nearly unique timestamps; it is cut into two windows of 8.5 minutes, each with a 3-minute warm-up.

The excluded tool-agent trace. A further trace from that release, the tool-agent trace, is excluded from every split. It is a relabeled copy of the Mooncake trace: its output lengths match on all 23,608 rows, its hash blocks map one-to-one onto Mooncake’s over 409,356 aligned blocks with no conflict, and its timestamps are Mooncake’s scaled by 0.9825. Including it would have put the same workload in two families, and possibly in two splits.

The session generator. In replay’s built-in synthetic-session source every request has the same prompt and output length, every inter-turn delay is the same constant, and a later turn does not extend the earlier turns’ context. The study’s generator writes its sessions as a Mooncake session trace. Sessions start as a Poisson process at one per second, floored to a 1 s grid. Each draws one of 16 system prompts with Zipf popularity (exponent 1.1; lognormal length, median 2,000 tokens). Turns per session are geometric with mean 5 (at most 24); user messages are lognormal (median 600 tokens for the first, 250 later) and outputs lognormal (median 250, at most 2,000), and a session ends before its context would exceed 49,152 tokens. Each later turn is released an exponential think time (mean 8 s) after the previous turn completes. For the calibrated cells each seed was regenerated with a 48,000 s duration; the generator is sequential, so the 720 s trace is a byte prefix of the long one, which adds independent sessions from the same seed rather than copies.

AgentX source. The AgentX plays are the public dataset `semianalysisai/cc-traces-weka-062126` at revision `23f152f6` [46], 393 plays and 98,827 requests, of which the context rule of Section 4.4 keeps 82 plays and 3,132 requests.

AgentX idle gaps. Recorded gaps include users stepping away for hours: 140 of 2,666 gaps (in 56 of 82 plays) exceed 300 s, the longest 355,532 s. Every AgentX trace caps idle gaps at 300 s with a monotone time warp that keeps every busy period, request duration and ordering; uncapped, the 82 plays would simulate for 399,658 s instead of 11,942 s, mostly idle.

D.2.1 Recycling AgentX plays at any worker count

Replaying the plays natively cannot hold a steady AgentX load on many workers. In timestamp mode every play’s root starts at time zero; in lanes mode the plays are dealt to lanes once and never reused, so 82 plays cannot keep 16 or 32 workers busy, and the first design had no AgentX cell above $N = 8$. We therefore convert each play once into AISim’s Agentic Mooncake v2 trace format and recycle copies of it.

- **Lowering.** The conversion is AISim’s own: a small Rust tool calls the public Weka importer of the `aisimulate-core` version the bindings link, so the rows are the importer’s output, not a reimplement.
- **Copy isolation.** Each copy of a play gets fresh play, session and request identifiers and its own consecutive range of hash identifiers. Replay interns hash identifiers globally across a file, so without the new range two copies of one play would share KV prefixes; with it, copies never do. Dependency edges, delays and lengths are copied unchanged.
- **Draws.** Each cell draws its copies with replacement from its own segment’s plays, so segments remain independent workloads.
- **Lanes.** Every AgentX cell runs closed: a fixed number of lanes per worker, each lane running its dealt copies back to back, with many more copies than lanes so that every lane stays busy through the measurement window. The lowering also supports open-loop Poisson play arrivals, but open-loop AgentX has a cliff between 0.002 and 0.0025 plays per second per worker, past which the number of plays in flight keeps growing; no stationary open-loop load reaches the L3 band, so no cell uses it.

The conversion preserves replay semantics exactly where it can be compared with native replay. For all 82 plays and each of the six play transforms the cells use, the native graph and the converted file compile to the same canonical graph digest. Replaying six plays that cover subagents, joins and concurrent spawns (alone, as one corpus, and as a 12-copy recycled corpus, at $N = 2$ and 4, under the seeded default and round robin) gave identical per-request records in 84 of 84 cases for every transform. The order of copies within a file is a free choice: the same recycled corpus in a different copy order changed mean end-to-end latency by -7.4% to $+4.7\%$ for the untransformed plays, and by up to -15.6% and $+18.0\%$ under the output-length transforms, in a synchronized-start worst case at $N = 2$. That is the same kind of variation as a replicate, and the replicate protocol re-draws it (Section 4.3).

D.2.2 Load-mode details

Replay’s speedup divides inter-turn delays as well as arrival times. In the first session cells this meant that think time shrank with load and with N : at matched per-worker load the mean think time was 8 s at $N = 2$ and about 0.3 s at $N = 32$. Session cells now pre-multiply each delay by the speedup, so a user’s think time is the same at every load and every N ; only the session arrival rate scales. Release is causal in both multi-turn families: a session’s next turn waits for the previous turn to complete, and an AgentX request for the requests it depends on [41]. Open-loop arrivals of flat traces do not react to latency, a known bias of trace-driven evaluation [4]. AgentX plays average 38 requests each, well past the point at which a partly-open system behaves as a closed one [45], which is one more reason lanes are its natural mode.

D.2.3 Transforms

Transforms vary the workload along axes a deployment might drift on, so that the test set can ask whether a policy extrapolates. They are deterministic and seeded, applied in a fixed order, and each derived trace is stored under the SHA-256 of its bytes with a record of its specification, sources and realized statistics.

- **Prompt length, shared and unique parts separately.** A block that occurs in two or more requests is *shared*; it is expanded by the same seeded per-hash draw wherever it occurs, so hash identity stays consistent. Each request’s *unique* suffix is scaled on its own.
- **Output length.** Each OSL is multiplied, rounded and kept at least 1.
- **Prefix roots.** The shared root segment of the requests that start with the same block (a system prompt) is split into several disjoint hash-space copies (two to four here), and each request family below it goes to one copy; sharing within a family is unchanged, but there are proportionally more distinct system prompts.
- **Think time.** Session delays are scaled; AgentX plays are dilated in time about their first request, which scales every recorded gap and keeps every ordering.
- **Context cap.** Rows whose prompt reaches 131,072 tokens are dropped and outputs are clamped so that prompt plus output fits (AgentX plays, which cannot lose a request without breaking their graph, clamp outputs only).

D.3 Calibration

Load levels. Each level is interpolated linearly in log-load between the two grid points that bracket it. The bands are targets for the mean over train segments, not for each cell: on individual train and validation cells, π_0 ’s realized good fraction ranges from 0.423 to 0.965.

The knee and the SLO fixed point. The SLO of Equation (2) has two bounds per family, set at $N = 8$ in the family’s primary mode:

- the *knee* marks where π_0 starts to saturate: in open loop, it is the load at which the mean slowdown of the last third of in-window arrivals exceeds the first third’s by more than 15%; in lanes, the load at which completions per second reach 0.95 of the grid’s maximum (sessions use a different rule, below);
- $s_{\max}$ is chosen so that π_0 ’s good fraction is 0.65 at 0.9 times the knee, which makes that load L3;
- $I_{\max}$ is 1.2 times the 95th percentile of π_0 ’s per-request mean ITL at L2.

L2 depends on $(I_{\max}, s_{\max})$ and $I_{\max}$ depends on L2, so the three are iterated to a fixed point; for Mooncake, $I_{\max}$ went from 260.31 to 253.78 to 252.12 ms.

Headroom below the knee. The 10% headroom below the knee matters. A first version anchored L3 at the knee itself, which gave Mooncake $s_{\max} = 5.69$ and left two train cells near the open-loop cliff with good fractions of 0.243 and 0.018; with the headroom they are 0.548 and 0.817.

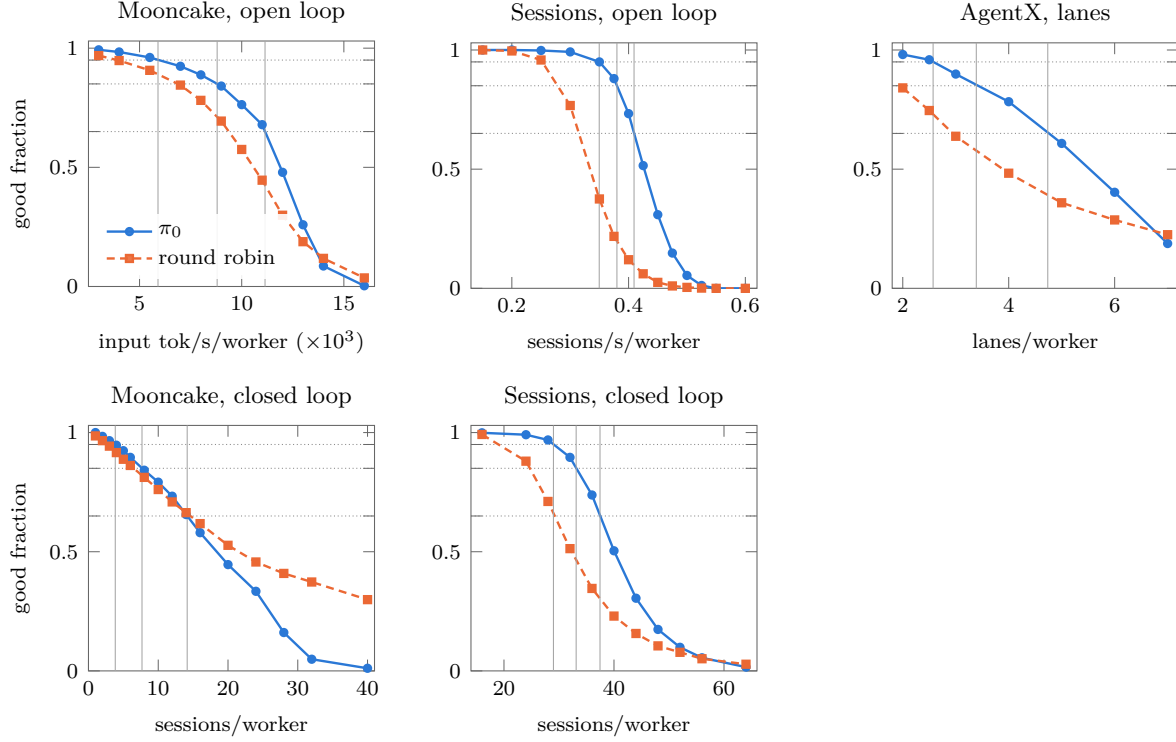


Figure 5: Calibration load curves at $N = 8$ on train segments: the fraction of in-window requests that are good (Equation (2)) under the family’s final SLO, for π_0 (blue, solid) and round robin (orange, dashed), averaged over segments and two replicates each (6 runs per point for Mooncake and AgentX, 8 for sessions). Dotted horizontal lines mark 0.95, 0.85 and 0.65; vertical lines mark the calibrated L1, L2 and L3, where π_0 ’s curve crosses them. Top row: each family’s primary mode, which also anchors its SLO.

Load curves. Figure 5 shows what the levels mean. In the primary modes the bands are steep: on sessions, π_0 ’s good fraction falls from 0.95 to 0.65 between 0.350 and 0.409 sessions per second per worker, and round robin, which ignores where a conversation’s history is cached, is already at 0.198 at L2. On Mooncake’s closed loop the two curves stay within 0.04 of each other up to L3 and cross just below it.

Warm-up and measurement windows. Each cell’s measurement window $\mathcal{T}_c$ (Section 3.3) starts after a warm-up that lets caches and queues fill:

- **Open loop, flat traces:** the slice’s 4-minute warm-up and 6-minute window (FAST25 synthetic: 3 minutes and 5.5), divided by the cell’s speedup; scored by arrival time.
- **Closed loop:** sessions that arrive during the warm-up are excluded by identity, and the window ends at full occupancy, the last instant all sessions are busy; for generated sessions, the first twice-the-concurrency sessions are warm-up and the next ten times the concurrency are measured.
- **AgentX lanes:** the warm-up is the 90th percentile of the recorded span of the cell’s plays (1,825s to 13,191s of simulated time across cells), and each lane gets the fewest copies among 8, 10, 12, 16 and 24 for which its least-loaded lane still covers the warm-up plus two mean play spans in every replicate $k < 16$.

- **Open loop, generated sessions:** a 540s window after a warm-up long enough for the population of open sessions to settle, described next.

The session warm-up fix. The first calibration opened the window of open-loop session cells after a fixed 180s warm-up. An independent re-run of that calibration found that the session population was still growing then: by a policy-free lifetime model, at 180s it had reached 0.78 of its plateau at the base think time and 0.54 at four times the think time. The windows therefore mixed a lightly loaded ramp with saturation. With a 900s warm-up, one train cell labeled L3 had a π_0 good fraction of 0.435 instead of 0.814, and the light ramp had also biased the sessions knee and so the sessions SLO. The fix sets the warm-up from session lifetimes. A session’s envelope lifetime is its summed think time (scaled by the cell’s think multiplier) plus three times its summed uncontended latency E_0 ; the warm-up is twice the 99th percentile of that lifetime over the train sessions, rounded up to a minute and at least 180s. That gives 960 to 2,160s, growing with the think multiplier. One lifetime (600s at the base think time) was tried first and failed the acceptance check. The check replays the same in-window sessions with twice the warm-up history: across the 12 open-loop session cells that calibration replays (train and validation cells, plus calibration-only cells on train segments at other N), π_0 ’s good fraction changed by -0.0006 on average (standard error 0.0019, no cell beyond three standard errors) and round robin’s goodput ratio to π_0 by $+0.0074$, against $+0.061$ and $+0.217$ for the original 180s warm-up on the same sessions. Per cell, doubling the warm-up history moved π_0 ’s good fraction by up to 0.353 under the original 180s warm-up (9 of 12 cells beyond 3 standard errors) and by at most 0.014 under the adopted warm-up of two session lifetimes (no cell beyond 3 standard errors). With the population settled, the within-window drift test no longer detected the knee: it placed the sessions knee at 0.510 sessions per second per worker, although at 0.475 and 0.5 doubling the history still moved π_0 ’s good fraction by 15–17%. The sessions knee is therefore the load at which doubling the warm-up history lowers π_0 ’s good fraction by more than 5%, solved jointly with the SLO fixed point (0.4548 at $N = 8$). The sessions SLO moved from $I_{\max} = 40.4165$ ms, $s_{\max} = 1.90627$ to the values in Table 5. Only session cells changed (8 train, 5 validation and 15 test cells); the other families’ SLOs, levels and cells are byte-identical to the first calibration. A second independent re-run then passed.

Calibration compute. Calibration took 13,532 replays (about 3.3 hours on one 24-core host), and the session fix another 9,792. Table 34 lists it with the rest of the study’s compute.

D.4 Freeze history

The freeze record of the frozen cell set holds the SHA-256 of the test cell file (7b998b8e..., 60 cells), of each of the 60 test cells’ contents, of the 39 trace files they replay (2.13 GB) and the 7 sources those were derived from, of the engine configuration and of the split manifest, together with the replay build’s identifier. The rule is that nobody evaluates a test cell before the single test pass, and that pass verifies every hash before it runs. The test set was frozen twice. The first freeze (b8648ac5...) was replaced after the session warm-up fix (Appendix D.3); that was allowed because none of the 35,735 result records then on disk touched a test cell, trace or split. At the end of the pilot, the pilot had produced no record that touches the test set, and the test file still matched the freeze. The single test pass verified all 149 freeze hashes before running.

```

worker_selection:
  aggregated: candidate
  instances:
    - name: candidate
      type: learned-choice
      parameters:
        feature_set: v1
        theta: [-1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] # = default router
        temperature: 0.0
        tie_break: one_draw
        seed: 1 # replay sets k + 1; omit in production
        # M2 adds a named context term:
        # context:
        #   sources: [<source 1>, <source 2>]
        #   p: [[8 floats], [8 floats]]

```

Figure 6: A learned-choice policy YAML at the parity anchor $\theta = -\mathbf{e}_0$ (restart s1). The same file is a replay input and a live router configuration; unknown keys and wrong vector lengths fail at startup. The selected M1-v2 file has 23 coefficients (Section 9.8).

E Policy implementation details

This appendix collects the implementation details of `learned-choice` that Section 5 summarizes. Together with that section it specifies the policy completely enough to reimplement it; both follow the policy’s normative specification, `FEATURES.md`.

E.1 Integration, inputs and parameters

A provider deserializes and validates the parameters once at startup and returns a factory that builds one policy instance per routing partition and worker role; each instance owns its random generator and its session map.

The picker declares the plugin inputs `CACHE` and `LOAD`. Per request it reads the prompt length ISL_j , the request block count b_j , the block size, the prefix block hashes (feature set `v2` only) and the session identifier, if any. Per candidate $i \in S$ it reads the router’s cache estimate (overlap blocks o_i and estimated cached tokens h_i), the router’s own load tracking (active prefill tokens L_i^{pf} , decode cost blocks L_i^{dec} including this request’s new blocks, active requests L_i^{req}) and the configured KV capacity cap_i in blocks. Host pins and eligibility always win: a pinned request goes to its pinned worker, and only eligible workers are candidates.

The coefficients are ordinary parameters in the policy YAML (Figure 6). Parsing is strict: unknown keys at any level, a vector of the wrong length, a non-finite coefficient, a negative or non-finite temperature, a context with both pooled and named forms, a context direction without either, an unknown or repeated context source, and `max_sessions: 0` all fail at startup with a message naming the problem. The unit tests check 21 invalid and 5 valid specifications through the same registry path the Python bindings use, and an independent audit of the implementation checked 67 invalid and 15 valid ones.

E.2 Cost per decision

The picker fills an $|S| \times d$ feature table, folds the context into one effective weight vector (rd operations), takes one dot product per candidate and makes one pass for the maximum. Before that

it sorts the candidates by worker ID, which fixes the visiting order (Section 5.3). A decision therefore costs $O(Nd + rd)$ arithmetic plus an $O(N \log N)$ sort; feature set `v2` adds three more sorts and N rendezvous hashes. Feature 0 is the default scorer’s output, which the default router computes anyway. Apart from scratch buffers that are reused across decisions, the only state is the session map: at most `max_sessions` entries (default 65,536), updated in logarithmic time. This study did not measure the per-decision latency of the Rust policy against `dynamo-default-cost-fn`; the bound above is from the code.

E.3 Leakage audits

Two independent audits checked that the policy reads neither the request’s expected output length nor any AIS estimate (Section 5.1). The first replayed the first 2,000 requests of a Mooncake trace at $N = 4$ and, from a probe arrival burst onward, either multiplied every output length by four or deleted the requests. None of the 965 decisions made up to and including the probe burst changed, for three `learned-choice` configurations (`v1` with a named context, `v1` with a pooled context at $\tau = 0.3$, and `v2`) as well as for the default and sticky-session policies. The same perturbation started 26 bursts earlier, as a positive control, changed 210 to 286 of those decisions. The second audit reimplemented the feature map from the specification and matched the policy’s choice in all of 18,000 random-coefficient trials across the three context forms, detected 30 of 31 deliberately mis-specified feature maps, and found that 16,000 pairs of decisions that differed only in excluded inputs never differed in outcome (positive controls changed 272 to 744).

E.4 Porting notes

Three properties matter when the YAML leaves replay; Section 12.6 turns them into the settings a deployment must match. First, like the default policy, `learned-choice` asks the host to treat an eligible session-affinity target as exclusive. Replay never sets such a target, but a live router with session affinity enabled pins every later turn of a session to its target, so the decision never reaches the picker and the learned session coefficient θ_6 is replaced by hard stickiness. Shipping the learned behavior requires host session affinity to be off, or accepting hard stickiness in its place. Second, the router settings that track prefill tokens and assume KV reuse change what features 0, 3, 4 and 7 mean (and the `v2` prefix key), so the live router must keep the values used in training. Third, without a `seed` each policy instance draws fresh entropy, so router replicas do not break ties in lockstep. How replay’s information differs from a live router’s (fresher cache state, one cache tier) is discussed in Section 4.1.

E.5 Feature details

- **Cache.** Features 1 and 2 come from the router’s accounting estimate. In replay the effective overlap equals the device-tier overlap and cached tokens are whole blocks. Feature 1 is not clamped; it stays at most 1 whenever the estimate does not exceed the prompt.
- **Load.** Features 3 to 5 come from the router’s own request tracking. Feature 4 divides by the capacity a worker advertises, or by 65,536 blocks when it advertises none; replay advertises the engine’s 18,863 blocks, so the fallback is unused here.
- **Session.** Feature 6 needs state. Replay never sets the router’s own affinity target, so the policy keeps a bounded LRU map from session identifier to the worker it last chose for that session, pinned choices included. It is 0 for every candidate when the request has no session or the session is new or evicted.

Algorithm 1 One learned-choice decision. Before the call, the host has fixed the eligible set S and written each candidate’s default cost c_i^{def} at default weights. The draw discipline is `one_draw`; under `reservoir` there is no up-front draw, a tie draws once per tied candidate during the scan, a sample draws one uniform, and a pin draws nothing, as in `dynamo-default-cost-fn`.

Require: parameters θ , sources and directions $p_1, \dots, p_r$, temperature τ ; per-instance random generator and session map

Require: request j ; candidates $S \neq \emptyset$ with their cache, load and capacity signals

- 1: $draw \leftarrow$ next 64-bit value of the generator ▷ exactly one per decision
- 2: sort S by worker ID ▷ all later loops and sums use this order
- 3: **if** the host pins j to a worker **then**
- 4: $i^* \leftarrow$ the pinned worker
- 5: **else**
- 6: $prev \leftarrow$ the worker the session map holds for j ’s session, or none
- 7: compute x_i for every $i \in S$ (Table 6); $x_{i,6} = 1$ iff $i = prev$
- 8: $w_S \leftarrow \theta + \sum_{\ell} z_{S,\ell} p_{\ell}$
- 9: $u_i \leftarrow w_S^{\top} x_i$ for every $i \in S$; **fail** if any is not finite
- 10: **if** $\tau = 0$ **then**
- 11: $tied \leftarrow \{i \in S : u_i = \max_{i'} u_{i'}\}$, in sorted order
- 12: $i^* \leftarrow tied[\lfloor draw \cdot |tied|/2^{64} \rfloor]$
- 13: **else**
- 14: $target \leftarrow \lfloor draw/2^{11} \rfloor 2^{-53} \cdot \sum_i e^{(u_i - \max_{i'} u_{i'})/\tau}$
- 15: $i^* \leftarrow$ first i whose running sum of $e^{(u_i - \max_{i'} u_{i'})/\tau}$ exceeds $target$
- 16: **end if**
- 17: **end if**
- 18: **if** j has a session **then**
- 19: session map $\leftarrow (j$ ’s session $\mapsto i^*)$; evict the least recently used entry beyond `max_sessions`
- 20: **end if**
- 21: **return** i^*

- **Scales.** The features are dimensionless, but they are not on a common scale. On candidate tables reconstructed from the default router’s train replays in the pilot, the within-set standard deviation ranged from 0.060 (`active_requests_s`) to 6.6 (`default_logit_scaled`); the tables are approximate, because replay does not log candidate tables. Table 24 gives every feature’s value.

The home indicator’s key. The v2 home indicator (feature 11) keys on the prompt prefix only. An earlier version keyed on the session identifier when one existed, but replay gives every request of a closed-loop flat trace a synthetic single-use session identifier that a live client would not send, so the feature meant different things in open and closed loop on the same trace; the implementation audit caught this. Session continuity is feature 6’s job. On Mooncake the prefix key takes only four distinct values, with about 85% of requests on two of them, so a large positive θ_{11} concentrates load; the evaluation audits worker shares (Section 8.8).

E.6 The decision algorithm and its random draws

Algorithm 1 gives one decision in full.

Candidates are visited in worker-ID order, never in the host’s unspecified row order, and set

Table 23: Named context sources $z_{S,\ell}$. Each is unchanged when the candidate set is duplicated (Section 5.5).

Source	Value
<code>mean_kv_load_frac</code>	$(\bar{x}_S)_4$, mean over S of $x_{i,4}$
<code>mean_active_prefill_tokens_k</code>	$(\bar{x}_S)_3$
<code>mean_active_requests_s</code>	$(\bar{x}_S)_5$
<code>max_overlap_frac</code>	$\max_{i \in S} x_{i,1}$
<code>isl_k</code>	$\text{ISL}_j / T_{\text{ch}}$
<code>is_first_turn</code>	1 if the policy remembers no previous worker for j 's session (new, evicted or none), else 0

means are summed in that order, so a seeded policy decides identically across processes. Each policy instance owns one random generator. Under the default discipline, `one_draw`, every decision consumes exactly one 64-bit draw, whether it ends in a tie, a sample, a unique best or a pin. Two policies with the same seed therefore stay on the same random stream even after one early decision differs, which keeps common random numbers aligned across the CMA-ES candidates of a generation (Section 6.3); a unit test checks that two policies differing in their first decision break 40 later ties identically under `one_draw` and not under `reservoir`. The `reservoir` discipline reproduces the seeded default picker draw for draw and exists for parity checks (Section 5.6). In replay, the harness sets the seed to $k + 1$ for replicate k (Table 19).

E.7 Context sources and their form

Named rather than pooled. The policy also accepts a *pooled* form, $z_{S,\ell} = q_\ell^\top \bar{x}_S$ with a free q_ℓ , but this study uses the *named* form of Table 23, for two reasons. The pooled form has a gauge freedom ($p_\ell \rightarrow c p_\ell$, $q_\ell \rightarrow q_\ell/c$, including the sign), a flat direction in which CMA-ES spends budget for nothing; low-rank choice models have the same non-uniqueness [20]. The named form is linear in the free p_ℓ and has none. And mean-pooling a one-hot feature leaks the set size: the candidate mean of `session_affinity` is $1/N$ or 0. The named sources are chosen to avoid that (Section 5.5). Two redundancies remain and are handled in the search space rather than in the policy: when `isl_k` is a source, θ_7 duplicates the (`isl_k`, feature 3) entry of the context term and is frozen at 0; and on traffic without sessions `is_first_turn` is always 1, so its direction duplicates θ . A unit test checks both context forms against the literal bilinear formula on 100 random tables.

E.8 Set-size invariance: counterexamples and test

Under the duplication test of Section 5.5, means and maxima are invariant, but several natural set statistics are not. Sums and counts grow with N ; a mean of a one-hot feature is $1/N$; a z-score is ± 1 at $N = 2$ whatever the gap; and a leave-one-out rank fraction of 1 at $N = 2$ becomes $2/3$ after duplication. For that reason `v2`'s below-fraction counts strictly smaller loads over $|S|$, which keeps its value ($1/2$ in the same example). A unit test duplicates 50 random candidate tables under random `v2` coefficients and checks that features, sources and the chosen worker are unchanged. Two features are excluded from the property because they are relations to particular workers: `hash_home` (its coefficient is zero in the test) and `session_affinity`, since a copy of a session's previous worker did not serve the session (the test uses requests without a session).

E.9 Parity tests

Three unit tests check the parity of Section 5.6 through the same YAML path a deployment uses. On 2,000 random candidate tables, spread over the four input situations the default scorer distinguishes (complete inputs, no per-tier cache matches, no load reports, prefill not tracked), **learned-choice** with **v1** and with **v2** picks the default’s unique minimum on every table that has one (at least 1,500 are required). On 500 tables built to tie (more than 250 do), the **reservoir** discipline with the same seed picks exactly what the seeded default picks. And on 200 tables, feature 0 stays at default weights when the host is configured with other score weights. The one known exception is two costs that differ by less than the rounding error of the division by b_j .

In replay, on 11 cells with 8 replicates each, **learned-choice** at $-\mathbf{e}_0$ with **reservoir** ties produced per-request rows identical to **dynamo-default-cost-fn** on all 88 (cell, replicate) pairs. With the **one_draw** discipline used in training, the rows differ on all 88, because tied decisions are broken by a different random draw, but goodput does not differ beyond replicate noise. The largest deviation of a cell’s mean goodput ratio from 1 was 0.0088, below three times the pooled replicate standard deviation of the ratio (0.0102), and over 7 segments the mean deviation was 0.00055, below twice its standard error (0.00117). In this check the session and AgentX cells sat near a good fraction of 1.0 under the provisional SLO used at integration, so the Mooncake cells carry the goodput comparison. In an earlier smoke replay of 1,000 Mooncake requests at $N = 4$, the two disciplines gave the same goodput with 733 of 1,000 assignments different, and the per-worker request counts were a permutation of each other: ties among idle workers at the start relabel the workers.

E.10 The nested-logit option

A third rung, M3, is a nested logit over groups of candidates, such as the data-parallel ranks of one engine or the workers on one node. A nested logit lets unobserved utility be correlated within a nest, so IIA holds within each nest but not across nests, and the choice factors into a choice of nest, driven by the nest’s inclusive value, times a logit within the nest [51]. Two observations limit its value here. Every worker in this deployment is one engine with data-parallel size 1, so there are no nontrivial nests. And the nest parameters only shape the distribution of the unobserved utility; in the deterministic limit the choice is the argmax of the same systematic utilities, so at $\tau = 0$ they would not change a decision. This corollary is ours, not Train’s. Where nests exist, the useful part of M3 at $\tau = 0$ is nest-level features, such as the aggregate load of a node or a data-parallel group, added to x_i . M3 was not run.

E.11 Sticky-session policies

Session affinity is the simplest alternative to learning a session coefficient, so this study added **sticky-session** (Section 2.4), built from the same parts as **learned-choice**: the canonical-order chooser with both draw disciplines and the bounded LRU session map. A pinned request rebinds its session to the pinned worker. Without sessions, **sticky-session** with **reservoir** ties picks exactly what the seeded default picks, and the implementation audit’s independent model matched it on 20,000 decisions. In this study both modes are tuned baselines (Section 7), tuning the default cost’s knobs as well.

learned-choice can express hard stickiness: with $\theta = -\mathbf{e}_0 + 10^6 \mathbf{e}_6$ a session’s later requests stay on its previous worker whenever that worker is a candidate, and otherwise feature 6 is 0 for every candidate and the default cost decides. This matches **hard** mode with default weights. In a build smoke test, both kept every session on one worker on three session workloads with 20, 60 and

10 sessions, where the seeded default split 14, 54 and 1 of them across workers. The **bounded** rule is a threshold on relative load, which a linear utility can only approximate.

F Tuning details

This appendix holds the tuning details that Section 6 summarizes: how the tuning harness keeps scores paired, every search space and how its step size was set, the diagnostic that is logged but steers nothing, one tuning run in pseudocode, the initial points and the other learned arms, how the sign constraint is implemented, the drift check that preceded M2, and the compute behind the runs.

F.1 Objective bookkeeping

Before scoring, the tuning harness checks that every returned record carries its task’s cell, replicate and policy hash, and that candidate and reference cover the same (cell, replicate) set; a mismatch stops the run rather than producing an unpaired score. After its first replay, π_0 ’s record comes from the result cache.

A candidate whose replay fails on any cell receives the generation’s lowest finite objective minus one, so CMA-ES ranks it last; a failed reference replay stops the generation, which is retried on the next call because errors are not cached. In the pilot no candidate failed in 4 runs of 480 evaluations (*preliminary (pilot, train subset)*). One generation replays 16 candidates and π_0 on $34 \text{ cells} \times 2 \text{ replicates}$, 1,156 replays, plus 204 for the rank-stability diagnostic (Appendix F.3); a single candidate evaluation on the full train split costs about 700.9 replay-seconds, as measured in the pilot.

F.2 Search spaces

Internal coordinates. Each policy’s `space.yaml` lists its free parameters with bounds and a linear or log scale, its pinned parameters, and CMA-ES options. CMA-ES searches internal coordinates ϕ : each bounded parameter maps to $[0, 1]$, linearly or in log space, and integer parameters are rounded after decoding. `pycma`’s boundary transform keeps every sample inside the box, so every candidate decodes to a valid policy file. Every parameter of the main tuning is bounded.

M1’s bounds. A feature’s scale ratio $\text{sd}(x_{\cdot,0})/\text{sd}(x_{\cdot,f})$ is the coefficient at which feature f varies across a candidate set as much as the anchor does; at its bound of about four times that ratio, a feature’s term spreads four times as widely as the anchor’s (Table 24). The standard deviations are measured within candidate sets, as the geometric mean over the pilot’s 12 train cells of each cell’s mean, on candidate tables reconstructed from π_0 ’s per-request rows, because replay does not log candidate tables. The reconstruction is approximate (router load is rebuilt from arrivals, first tokens and completions, and overlap is exact only for the chosen worker and for earlier turns of the same session), so the ratios set scales, not policy inputs. The initial placeholder bounds of ± 8 could not have let `kv_load_frac` or `active_requests_s` matter. Because each bound is proportional to the feature’s scale ratio, one internal unit moves every coefficient by the same multiple of its ratio, so the initial search distribution is isotropic in units of decision-relevant spread. This is the per-variable scaling Hansen [12] recommends when initial ranges differ by orders of magnitude, and the analogue of the state normalization without which Mania et al. [27] could not train a linear policy on their hardest task.

Table 24: M1’s search space and its three initial points. Within-set sd: geometric mean over the pilot’s 12 train cells of the mean within-candidate-set standard deviation, on reconstructed tables. Ratio: $\text{sd}(x_{\cdot,0})/\text{sd}(x_{\cdot,f})$. The search box is $|\theta_f| \leq \text{bound}$. Start s1 is $\theta = -\mathbf{e}_0$ (all free coefficients 0); s2 and s3 are described in Section 6.4 and Appendix F.5.

f	Feature	Within-set sd	Ratio	Bound	s2	s3
0	default_logit_scaled	6.607	1	pinned	−1	−1
1	overlap_frac	0.2585	25.6	100	63.9	76.8
2	new_prefill_tokens_k	0.3742	17.7	70	4.18	0
3	active_prefill_tokens_k	1.239	5.33	21	1.28	0
4	kv_load_frac	0.08173	80.8	320	12.5	0
5	active_requests_s	0.06031	110	440	−138	0
6	session_affinity	0.3111	21.2	85	−0.324	42.4
7	isl_x_prefill_load	2.053	3.22	13	−0.196	0

F.2.1 Baseline spaces

Each baseline searches the numeric parameters its catalog entry exposes, over ranges set in the parameter’s own units, log-scaled where a parameter spans orders of magnitude, starting from its shipped defaults (Table 25); restarts 1, 2 and 3 use CMA-ES seeds 1, 2 and 3. Four kinds of parameter are pinned rather than searched.

- **Rescalings.** A parameter that only rescales others adds a direction along which decisions do not change, and CMA-ES’s step size drifts along such directions [12]. Three are pinned. `ramjet`’s affinity block size stays at 512 tokens: multiplying `affinity_block_tokens` by some factor while dividing `max_affinity_blocks` and `alpha` by the same factor leaves its decisions unchanged up to integer rounding. `llm-d-precise-prefix`’s prefix weight stays at 2: scaling all three of its weights never changes its choice. And `llm-d-optimized-baseline`’s peak prefill rate stays at 15,928 tokens/s: with the throughput source, its gate compares a token difference divided by `peak_prefill_tokens_per_second` with `max_ttft_penalty_ms`, so their product is the only quantity that policy identifies.
- **Categorical choices.** CMA-ES searches continuous coordinates, so these structural parameters keep their shipped values: `lmetric`’s hot-spot detector stays on, `ramjet`’s affinity basis stays `relative` and the optimized baseline’s TTFT source stays `throughput`.
- **Capacity.** The sticky-session map keeps its 65,536-session bound.
- **Inputs replay does not supply.** `llm-d-precise-prefix`’s per-class weights stay empty, because replay sets no policy class.

Of the baselines, only `sticky-session` reads the default cost’s router-level knobs, because its first-turn and fallback choice is the default cost function; both sticky variants therefore also search the five M0 knobs over M0’s ranges. Sticky-session and M0 at the same knob values gave identical results on a session-free Mooncake cell, both at the start point and at the upper corner of the box. No other baseline reads a tunable router-configuration knob, and `router_queue_threshold` keeps its default for every policy; the queue-threshold ablation tunes it separately (Section 5.7).

M0’s space is the pilot’s, with the prefill-load scale β_{pf} widened to $[0.05, 500]$ on a log scale because the pilot’s tuned values (17 to 25) sat near the top of the old range $[0.05, 50]$. The decode request weight β_{req} starts at 0.5, the bottom of its log range $[0.5, 4096]$, which stands in for the shipped value 0.

Table 35 gives each tuned baseline’s selected restart and checkpoint with its validation and test scores; the selected **ramjet**, for instance, came from restart 3 at generation 25, with **alpha** 0.883, **max_affinity_blocks** 68 and **load_unit_tokens** 4552 (Figure 2).

F.2.2 Initial step size

Parameter units are not comparable across policies, so σ_0 is set in decision space. For a step size σ , we draw internal points from $\mathcal{N}(\phi_0, \sigma^2 I)$ clipped to the box, decode them, and measure the share of routing decisions that differ from the policy’s own decisions at its start ϕ_0 , as the total-variation distance between the two decision distributions. The decisions are one-step counterfactuals on the candidate tables reconstructed from the pilot’s default-router replays of its 12 train cells, with 8 draws per σ and cell; stateful policies run their own state machine over each cell in arrival order. σ_0 is the σ , interpolated on a grid from 0.03 to 0.35, at which this flip rate reaches 17.5%, so every policy starts with a comparable amount of exploration measured in decisions. The literature review’s suggested band of 10–30% is itself a hypothesis. For M1, $\sigma_0 = 0.004$ internal units, a standard deviation of 0.032 scale ratios per coefficient, flips about 20% of decisions relative to $\theta = -\mathbf{e}_0$. Five baseline spaces cannot reach the target by $\sigma = 0.3$, the step-size cap below, and start at 0.3: two-tier (0.083 of decisions change), **lmetric** (0.0006), **ramjet** (0.133), **dualmap** (0.018) and the optimized baseline (0.034). Around their shipped defaults their decisions barely respond to their parameters: **lmetric**’s flip rate at $\sigma = 0.3$ is 0.06%. For **chwb1**’s and **dualmap**’s hashed prefix depths, any change re-draws the whole prefix-to-worker map, so the objective is flat between block boundaries and jumps across them (flip rates of 0.69 and 0.18 already at $\sigma = 0.03$); those two coordinates are held at their defaults during the measurement and share their space’s σ_0 .

F.2.3 Step-size cap

Every baseline space caps **pycma**’s per-coordinate standard deviation (**maxstd**) at 0.3 internal units. The pilot recommended the cap after M0’s first run, whose global step size grew from 0.18 to 10.8 internal units after its validation score had plateaued, along nearly flat directions in the decode request weight and the temperature (*preliminary (pilot, train subset)*). M1’s space, whose step size stayed small in the pilot, sets no explicit cap. Two details qualify the measure. **pycma** 4.5.0 already caps each bounded coordinate at one third of its range by default, so on these $[0, 1]$ coordinates the explicit cap tightens the default from $1/3$ to 0.3, and the default applies to M1. The cap also limits the per-coordinate spread of the samples, not the global step size σ , which can still drift.

F.3 Rank-stability diagnostic

After each update, the two best candidates are re-scored on the same cells with the next generation’s replicates $\mathcal{K}_{t+1}$, and the change in their order is logged. The re-scored share, $2/\lambda = 0.125$, equals UH-CMA-ES’s default rate $\max(0.1, 2/\lambda)$ [13], but unlike UH-CMA-ES nothing reacts to the measurement: neither the step size nor the number of evaluations per candidate changes. It runs identically for every policy, does not count against B , and is skipped in a generation whose wall-clock chunk ends during it. In the pilot, the top two candidates swapped in 7 to 11 of 30 generations per run (*preliminary (pilot, train subset)*).

F.4 One tuning run

Algorithm 2 summarizes one restart of the tuning harness.

Table 25: Tuning spaces. Each searched parameter is listed with its range, “log” if it is searched on a log scale, and its starting value, which is the shipped default except where marked. d : free parameters. Flips: the share of decisions that one CMA-ES step of size σ_0 changes, measured from the policy’s start on candidate tables reconstructed from the pilot’s default-router replays of 12 train cells (target 0.175). Every baseline space caps the per-coordinate standard deviation at 0.3. M0’s symbols are the router knobs `overlap_score_credit` (β_{ov}), `overlap_score_credit_decay` (β_{decay}), `prefill_load_scale` (β_{pf}) and `decode_active_request_weight` (β_{req}). M1’s row is for comparison; M1 has three starts (Table 24).

Policy	Searched: range (scale), start	Pinned	d	σ_0	Flips
default cost (M0)	β_{ov} [0, 4], 1; β_{decay} [0, 4], 0; β_{pf} [0.05, 500] log, 1; β_{req} [0.5, 4096] log, 0.5 ^a ; <code>router_temperature</code> [0, 1], 0	—	5	0.11	0.169
two-tier	<code>cache_threshold</code> [0, 1], 0.5; <code>balance_abs_threshold</code> [1, 1024] log, 32; <code>balance_rel_threshold</code> [1, 8] log, 1.1	—	3	0.3	0.083
LMetric	<code>class_prefix_blocks</code> [1, 64] log, 4; <code>window_requests</code> [50, 50,000] log, 1000	hot-spot detector on	2	0.3	0.0006
Ramjet	<code>alpha</code> [0.1, 160] log, 4; <code>max_affinity_blocks</code> [1, 1024] log, 32; <code>load_unit_tokens</code> [256, 262,144] log, 8192	affinity block 512 tokens ^b ; basis <code>relative</code>	3	0.3	0.133
DualMap	<code>hash_prefix_blocks</code> [1, 64] log, 4 ^c ; <code>pending_prefill_token_budget</code> [2048, 2,097,152] log, 65,536; <code>window_requests</code> [50, 50,000] log, 1000	—	3	0.3	0.018 ^c
CHWBL	<code>prefix_tokens</code> [16, 16,384] log, 256 ^c ; <code>load_factor</code> [1, 8] log, 1.25	—	2	0.16	0.173 ^c
llm-d precise prefix	<code>weights.queue</code> [0.01, 100] log, 1; <code>weights.kv_cache_utilization</code> [0.01, 100] log, 1; <code>prefix_match_length_weight</code> [0, 1], 0; <code>prefix_match_length_scale_tokens</code> [512, 131,072] log, 8192	<code>weights.prefix</code> 2 ^b ; per-class weights empty	4	0.16	0.177
llm-d optimized baseline	<code>affinity_threshold</code> [0, 1], 0.8; <code>max_ttft_penalty_ms</code> [100, 3,240,000] log, 18,000; <code>queue_threshold_tokens</code> [16,384, 1,073,741,824] log, 4,194,304	peak prefill rate 15,928 tokens/s ^b ; TTFT source <code>throughput</code>	3	0.3	0.034
sticky, hard	the five default-cost knobs, with M0’s ranges and starts	mode <code>hard</code> ; 65,536 sessions	5	0.17	0.179
sticky, bounded	<code>load_factor</code> [1, 8] log, 1.25; the five default-cost knobs, as M0	mode <code>bounded</code> ; 65,536 sessions	6	0.067	0.175
learned (M1)	$\theta_1, \dots, \theta_7$, bounds in Table 24	$\theta_0 = -1, \tau = 0$	7	0.004	about 0.20 ^d

^a The shipped value of β_{req} is 0, which a log scale cannot reach; 0.5, at most half a block per active request, stands in for it. ^b Pinned because it only rescales the other parameters. ^c Hashed key depth, held at its default when σ_0 was set: any change of it redraws the prefix-to-worker map (0.69 of decisions change for `chwbl` and 0.18 for `dualmap` at a step of 0.03). ^d Measured in the pilot, from $\theta = -\mathbf{e}_0$.

Algorithm 2 Tuning one policy (one restart of the tuning harness). Every policy, learned or baseline, runs this with the same B , λ , K , cells, validation schedule and selection rule.

Require: space (bounds, pins, σ_0 , cap), start ϕ_0 , seed, cells $\mathcal{C}_{\text{train}}$ and $\mathcal{C}_{\text{val}}$

- 1: initialize CMA-ES at ϕ_0 with step size σ_0 and population λ
- 2: $best \leftarrow \perp$; $selected \leftarrow \perp$
- 3: **for** $t = 0, 1, \dots, B/\lambda - 1$ **do**
- 4: $\mathcal{K}_t \leftarrow \{(tK + i) \bmod K_{\text{pool}} : i < K\}$
- 5: sample candidates $\phi_1, \dots, \phi_\lambda$; checkpoint $\triangleright$ ask
- 6: replay every candidate and π_0 on $\mathcal{C}_{\text{train}} \times \mathcal{K}_t$; check CRN coverage
- 7: compute J per candidate (Equation (8)); a failed one gets the worst finite $J - 1$
- 8: update CMA-ES with fitness $-J$ $\triangleright$ tell
- 9: $best \leftarrow$ the candidate with the highest J seen so far
- 10: re-score the top 2 on $\mathcal{C}_{\text{train}} \times \mathcal{K}_{t+1}$; log rank changes $\triangleright$ diagnostic only
- 11: **if** $(t + 1) \bmod 5 = 0$ **then**
- 12: score $best$ and the CMA-ES mean on $\mathcal{C}_{\text{val}} \times \{0, 1, 2\}$
- 13: $selected \leftarrow$ the validated candidate with the highest score so far
- 14: **end if**
- 15: checkpoint (CMA-ES state and random state)
- 16: **end for**
- 17: **return** $selected$, and $best$ for reference

F.5 Restarts and initial points

Restart 1 of every policy runs in the first wave, restart 2 in the second and restart 3 in the third, so a compute cut at any point removes the same restart from every policy.

The behavior clone (s2). Start s2 clones `llm-d-precise-prefix`, the best of the untuned heuristics evaluated on validation in the pilot. It is the conditional-logit maximum-likelihood fit to that heuristic’s choices: with $\Pr[i \mid S] \propto \exp(\theta^\top x_i)$ over reconstructed candidate tables, it maximizes the mean over cells of the mean log-probability of the worker the heuristic chose, with θ_0 pinned at -1 and $\theta_1, \dots, \theta_7$ held inside M1’s box. The problem is convex and bounded and is solved by L-BFGS-B. The data are the heuristic’s own replays at shipped defaults on all 34 train cells, replicate 0: 144,048 decisions in a stride sample of each cell. The fit chooses the heuristic’s worker on 0.875 of these decisions (0.866 and 0.883 in a two-fold split by cell), against 0.512 for the default rule $\theta = -\mathbf{e}_0$, and no coefficient sits at its bound.

The cache-heavy point (s3). Start s3 sets $\theta_1 = 3 \times 25.6 = 76.8$ and $\theta_6 = 2 \times 21.2 = 42.4$, so that across a candidate set the overlap term spreads three times and the session-affinity term twice as widely as the anchor, with all other free coefficients 0, so load enters only through the anchor. It targets the agent workloads on which the pilot’s M1 trailed the cache-aware heuristic (by -0.019 in mean clipped log-ratio on AgentX, *preliminary (pilot, validation split)*). It was preferred to a fit to the second-best heuristic family because `lmetric`’s product of uncached tokens and load is not representable with `v1` features, and a fit to `ramjet` would be another linear cache-minus-load score close to s2.

How the starts differ. On one-step decisions, s2 and s3 differ from the default rule on 46% and 42% of requests and from each other on 11%. The clone’s agreement should be read with the

reconstruction’s bias in mind: overlap is exact only for the chosen worker, which favors overlap-heavy rules. With a hash-aware reconstruction the fit agrees on 0.785 of decisions and its overlap coefficient is 40 instead of 64, and the hand-set point s3 agrees with the heuristic on 0.872, almost as often as the fit. Agreement is lowest on AgentX (0.797, against 0.914 on Mooncake and 0.865 on sessions).

F.6 Other learned arms

Each arm of Table 8 is trained like M1, with $B = 400$, three restarts, the same cells and the same selection rule. Their spaces and starting points are as follows.

- **M2 rank 2 and M1 continued.** M2 starts at the selected M1 coefficients with $p = 0$ and context step sizes 0.3 times θ ’s; its two named sources, `isl_k` and `mean_active_prefill_tokens_k`, were fixed before any M2 evaluation, and θ_7 is pinned at 0 because the (`isl_k`, feature 3) context entry duplicates it. It is compared with M1 continued from its selected point for the same added 400 evaluations.
- **M1-noaff.** M1’s three starts with θ_6 pinned at 0; a further check, without retraining, zeroes θ_6 in the selected M1.
- **M1-v2.** Coefficients 0 to 7 keep M1’s bounds, so M1-v2 nests M1; coefficients 8 to 22 are bounded at four times their scale ratio, measured on candidate tables reconstructed from the default router’s train replays. $\sigma_0 = 0.0028$ flips 19.6% of decisions at the s1 start, the rate M1’s space has at its own σ_0 . The starts are s1, $\theta = -\mathbf{e}_0$; s2, the behavior clone of `llm-d-precise-prefix` refit on v2 features (with `hash_home` pinned at 0 in the fit, because its offline reconstruction is approximate); and s3, `lmetric`’s representable point $\theta = -\mathbf{e}_0 - 20.7(\mathbf{e}_{\log_ptok} + \mathbf{e}_{\log_bs})$, 90% of the bound on those coordinates, which agrees with the pure LMetric rule on 0.924 of reconstructed decisions.
- **M1 with a queue threshold** (the queue-threshold ablation). M1 continued’s space plus `router_queue_threshold` on a log scale from 0.25 to 256, starting at 32, a value that replays identically to the unset knob on every train cell; its baseline counterpart tunes the same knob on top of the best baseline’s selected configuration.

The secondary AIS-informed league is trained the same way on its own build (Appendix J).

The second planned ablation, adding simulator-only signals such as engine waiting counts and KV usage, was not run. It was first in the pre-registered order of cuts and was skipped: it needed new plugin inputs, replay plumbing and a separate build for an arm that never ships, and free compute existed later, so the reason was implementation cost, not compute.

F.7 Implementing the sign constraint

- **A clamp, not a box bound.** The restarts s1 and s3 start with the constrained coefficients of Section 6.5 at 0, which on a box bound of 0 would sit on the edge of pycma’s boundary transform, where its slope is zero and the first generation’s steps on those coordinates shrink by about two orders of magnitude. The space therefore keeps the original box and clamps the decoded value, $x = \min(x, 0)$. CMA-ES searches the identical internal space, start, step size and seed as the unconstrained run, and generation 0 asked the same internal points; only the decoded policies differ.
- **Starts.** s1 and s3 already satisfy the constraint. The behavior clone s2 did not (its θ_3 and θ_4 were positive, so decode load attracted above about 24K tokens), so it was projected onto the constraint; its agreement with the cloned heuristic stayed at 0.879.

- **Later arms.** M1-v2 clamps every load-ordered feature (3, 4, 5, 7, 8, 10 and 12 to 21), M2 also clamps the load columns of its context directions so that every effective load weight stays non-positive, and the AIS-informed league clamps its load and backlog features.

F.8 Drift check before M2

A context term can help only if the best coefficients change with the regime, and a larger model that nests a smaller one can still lose to it on a fixed optimizer budget; Tomlinson and Benson note that LCL, although it nests the conditional logit, can fail to beat it within a training budget of 500 epochs [50]. The study therefore measured drift before relying on context.

Procedure. Before M2, M1 is tuned at a quarter of the budget on each of four train subsets: L1 cells only (6 cells), L3 only (8), $N = 4$ only (18) and $N = 8$ only (16). The quarter budget is 96 evaluations, six whole generations, because 100 is not a multiple of 16. Each run validates every 3 generations on the matching validation cells and selects on $k = 0, 1, 2$. Each selected model is then scored on every regime’s validation cells with fresh replicates $k = 3, \dots, 10$. The cross-play loss of the model tuned on regime a , evaluated on regime b , is the mean clipped log-ratio against π_0 of b ’s own model on b ’s validation cells minus that of a ’s model; the pairs are L1 and L3, and $N = 4$ and $N = 8$. Cross-play losses beyond the minimum detectable effect argue for context, and the drifting statistic is the natural source. M2 was to be deprioritized if all four losses lay within the MDE of 0.038. M2 at rank 2 was later made mandatory, so the check no longer decides whether M2 trains; it is reported as evidence for or against context terms.

Outcome. The check came out inconclusive. The one cross-play loss above the MDE, 0.087 for the model tuned on L1 cells evaluated on L3 cells, rests on 3 cells and is 1.3 standard errors from zero, and a single model tuned on all cells beat every regime specialist on the specialist’s own cells; the specialists, at a quarter of the budget, had moved only a third as far from their start as the full M1. It is therefore read as an underpowered check with no evidence of drift, and it was not used to choose M2’s sources. Those were fixed before any M2 evaluation: `isl_k`, because the documented M1 failure axis is prompt length (Section 10), and `mean_active_prefill_tokens_k`, the candidate set’s prefill pressure.

F.9 Compute and numerics

Replays run on CPUs: locally through a pool of 20 slots, and on a pool of shared CPU nodes, one tuning run per node; the first wave started on 10 remote nodes and the workstation. A generation took 225 to 333s early in the main tuning, and about 300s later, on a 96-core EPYC 9654P node, and 610 to 690s on a 64-core EPYC 7702P node; the three sign-constrained M1 re-runs took 4 h 24 min to 4 h 31 min each on the latter. The main tuning ran 66 restarts of 400 evaluations in total (M1 and every tuned baseline, the later learned arms, the ablation and the AIS-informed league), and each run whose ingest the campaign record logs stored between 30,199 and 30,792 result records. The single test pass added 5,220 records on the tuning build and 900 on the AIS build, and the robustness pass 23,940 (Section 9.9). Replay-seconds for the main tuning were not totaled.

Runs move between machines, so a run must continue bit-exactly on any host. Two things depend on the host: replay results depend on the operating system’s math library, and CMA-ES’s own linear algebra depends on numpy’s and OpenBLAS’s CPU dispatch. Appendix M.6 gives the parity check of every node image against the workstation (6,312 remote records with every field

matching, 48,901,384 per-request rows byte for byte) and the three environment variables that pin the dispatch. With default settings, an EPYC 9654P node diverged from the workstation in the last bits from the first generation of a 60-generation tuning run on a synthetic objective without replays; with the pins, whose values equal the workstation’s defaults, it matched in all 60 generations, and two short real tuning runs on that node reproduced the workstation’s history (apart from wall-clock fields), selection and policy file exactly.

The harness writes its checkpoint twice per generation, after sampling and after the update, including the CMA-ES object and numpy’s random state, so a resumed run continues exactly as an uninterrupted one would; an audit killed a run three times and found its history, selection, CMA-ES state and continuation identical (Appendix M.5). A run refuses to resume if its space, cells, reference, objective or seeds have changed.

If compute ran short, the last resort in the pre-registered order of cuts was to score each generation on a rotating 12-cell subset of the train cells, for every policy of a wave alike. On the 16 candidates of one pilot generation, subset and full-train objectives had a Spearman correlation of 0.947 (*preliminary (pilot, train subset)*).

G The pilot (preliminary)

The pilot asked whether tuning moves any policy away from the default router by more than replicate noise, and how large the learned model’s margin over the best heuristics might be, before the study committed to the main tuning. Its numbers come from the validation split and from a smaller tuning setup than the main tuning, so they are *preliminary (pilot, validation split)* and were superseded by the results of the main tuning and the test pass (Section 9). They are kept because they shaped the main tuning: the budget, the step-size cap, the multi-start initialization, the extended feature set and the concentration audit all trace back to them.

The setup was smaller than the main tuning’s in four ways.

- M0 (the default cost, tuned) and M1 (**learned-choice** on feature set **v1**) were tuned on 12 of the 34 train cells, chosen to cover every family, both train worker counts and every load mode, with 480 evaluations (30 generations of 16) per run.
- Each model was tuned twice with different CMA-ES seeds from the *same* start (M1 from $\theta = -\mathbf{e}_0$, M0 from the shipped defaults). Below they are called seed 1 and seed 2; they are not the main tuning’s restarts 1 to 3, which differ in their start.
- The configuration of each model was selected on validation replicates $k = 0, 1, 2$ over both seeds and their checkpoints: seed 1 at generation 25 for M0, seed 2 at generation 30 for M1.
- The heuristics were not tuned. A screen of all nine catalog heuristics at shipped defaults on the 12 train cells ($k = 0, 1$) ranked **llm-d-precise-prefix** (+0.169), **ramjet** (+0.166) and **lmetric** (+0.165) first by mean clipped log-ratio; the last two are tied, so all three went to validation. **dualmap** (−0.150) and **chwb1** (−0.214) lost to the default on that subset.

All policies were then scored on the 14 validation cells with 8 fresh replicates ($k = 3, \dots, 10$) that no selection step had used, paired with π_0 on the same replicate. The pilot ran 56,552 fresh replays with 0 errors, and an independent judge, working from the raw records without importing the harness, reproduced every comparison against π_0 and against the heuristics in Tables 26 and 27 to four decimals.

“Beyond noise” below is the study’s two-clause rule (Section 8.2): on some cell the replicate-mean goodput ratio to the reference departs from 1 by more than three pooled replicate standard

Table 26: Pilot, validation split, every policy against π_0 (*preliminary (pilot, validation split)*). Mean Δ : mean over cells of the replicate-mean clipped log-ratio, overall and per family (Mooncake 5 cells, sessions 5, AgentX 4). Ratio: mean over cells of the replicate-mean goodput ratio $G(\pi)/G(\pi_0)$. Better: cells whose mean ratio exceeds 1. Seg. mean (SE): mean over the 6 validation segments of the per-segment mean of (ratio $- 1$), with its standard error. Over cap: cells on which, in at least one replicate, one worker received more than $\min(2/N, 1/N + 0.25)$ of the requests.

Policy	Mean Δ				Ratio	Better	Seg. mean (SE)	Over cap
	all	Mooncake	sessions	AgentX				
M1, seed 2 (selected)	0.165	0.221	0.205	0.045	1.191	13/14	0.147 (0.044)	5
M1, seed 1	0.162	0.220	0.200	0.043	1.187	13/14	0.142 (0.044)	4
llm-d-precise-prefix	0.135	0.124	0.203	0.064	1.153	14/14	0.135 (0.034)	1
ramjet	0.135	0.154	0.194	0.038	1.152	14/14	0.122 (0.038)	0
lmetric	0.133	0.116	0.208	0.060	1.150	14/14	0.132 (0.036)	0
M0, seed 2	0.129	0.129	0.194	0.047	1.145	14/14	0.124 (0.035)	0
M0, seed 1 (selected)	0.128	0.128	0.198	0.041	1.144	14/14	0.121 (0.037)	0
round-robin	-0.577	-0.273	-0.892	-0.562	0.571	0/14	-0.473 (0.066)	0

deviations, and over at least three independent segments the mean of the per-segment mean of (ratio $- 1$) exceeds twice its standard error in absolute value. The rule is two-sided, so a policy worse than the reference also passes it.

Everything beat the default router; round-robin collapsed. Every tuned and heuristic policy in Table 26 differed from π_0 beyond noise, in the positive direction, overall and separately in each load mode; round-robin differed in the negative direction. The three cache-aware heuristics at shipped defaults already reached a mean per-cell goodput ratio over π_0 of 1.150–1.153, and M1 reached 1.191. M0 was the weakest of the tuned and heuristic policies: tuning the default cost’s five knobs on 12 cells gained less than switching to an untuned cache-aware heuristic. The bar M1 had to clear in the main tuning was therefore set by those heuristics, after they got the same tuning budget, not by the default.

M1 against the strongest heuristic: not beyond noise. Table 27 pairs the selected M1 and M0 directly with each heuristic. Against llm-d-precise-prefix, the validation-best heuristic, M1’s mean clipped log-ratio was +0.030 (mean ratio 1.032, 10 of 14 cells better). The pre-registered minimum detectable effect (MDE) is $\max(1\%, 2 \text{ SE})$, where SE is the standard error of the paired segment mean on validation, in units of (ratio $- 1$). At the pilot that SE was 0.019, so the MDE is 0.038. M1’s segment mean was only +0.009: the Mooncake window that carried the gain is 5 of the 14 cells but 1 of the 6 segments. Under the study’s noise rule M1 did not differ from this heuristic, overall or in any single load mode. The same held against ramjet and lmetric, with one exception: M1’s lead over ramjet in closed loop (+0.038, SE 0.013, over 3 segments) passed the rule.

Where the margin came from. M1’s lead over llm-d-precise-prefix came from one of the six validation segments. Per segment, the mean of (ratio $- 1$) was +0.102 on the Mooncake window w1 (5 cells), +0.003 and +0.001 on the session seeds s4 and s5, and -0.014 , -0.025 and -0.012 on the three AgentX play subsets V1–V3. On validation, then, M1 gained where requests share a few hot prefixes across users, and lost slightly, but on every segment, on long agent contexts. That AgentX deficit shaped the main tuning: M1’s second and third restarts started from a behavior clone of llm-d-precise-prefix and from a cache-heavy point (Section 6.4), and the 23-feature set

Table 27: Pilot, validation split, head to head (*preliminary (pilot, validation split)*): the selected M1 and M0 against each untuned heuristic, paired on the same replicates $k = 3, \dots, 10$. Columns as in Table 26; positive values favor the tuned model. Noise: whether the pair differs beyond noise under the rule defined above. MDE = 0.038 in the units of the segment mean.

	Mean Δ				Ratio	Better	Seg. mean (SE)	Noise
Against	all	Mooncake	sessions	AgentX				
<i>M1 (seed 2, selected)</i>								
llm-d-precise-prefix	0.030	0.097	0.002	−0.019	1.032	10/14	0.009 (0.019)	no
ramjet	0.030	0.067	0.011	0.007	1.031	12/14	0.021 (0.011)	closed ^a
lmetric	0.032	0.105	−0.003	−0.015	1.034	6/14	0.012 (0.020)	no
<i>M0 (seed 1, selected)</i>								
llm-d-precise-prefix	−0.007	0.004	−0.005	−0.023	0.993	3/14	−0.012 (0.006)	no
ramjet	−0.007	−0.026	0.004	0.003	0.993	7/14	−0.001 (0.005)	no
lmetric	−0.005	0.012	−0.010	−0.019	0.995	4/14	−0.010 (0.005)	lanes ^a

^a Not beyond noise overall; beyond noise in this load mode only.

Table 28: What the pilot M1 learned (*preliminary (pilot, validation split)*). $s_f = \text{sd}(x_0)/\text{sd}(x_f)$, with within-set standard deviations measured on candidate tables reconstructed from π_0 's train replays; θ_f/s_f is the utility change per one within-set standard deviation of feature f , in units of the anchor's ($\theta_0 = -1$). The search bound was $|\theta_f| \leq 4s_f$.

f	Feature	s_f	θ_f , seed 2	θ_f/s_f , seed 2	θ_f/s_f , seed 1
0	default_logit_scaled	1	-1 (pinned)	-1	-1
1	overlap_frac	25.6	13.60	0.532	0.383
2	new_prefill_tokens_k	17.7	-8.71	-0.493	-0.143
3	active_prefill_tokens_k	5.33	-0.546	-0.102	-0.158
4	kv_load_frac	80.8	-0.434	-0.005	0.015
5	active_requests_s	110	-49.2	-0.449	-0.407
6	session_affinity	21.2	0.995	0.047	0.018
7	isl_x_prefill_load	3.22	0.041	0.013	0.021

v2 became a mandatory arm, the one that won.

The lead came with concentration. Over all validation records, the selected pilot M1 exceeded the share cap on 5 Mooncake cells (27 of 112 cell-replicate records), with a largest share of 0.646 in one record; llm-d-precise-prefix exceeded it on one cell. This flag became the mid-tuning gaming audit (Section 10).

What the pilot M1 learned. Relative to the default cost they start from, both pilot M1 seeds moved in three directions (Table 28): more weight on the overlap fraction, a penalty on new prefill tokens (strongly in seed 2 only), and a penalty on the number of active requests, a quantity the default cost ignores at its shipped weight $\beta_{\text{req}} = 0$. Both tuned M0 configurations moved the same way within their own family: more overlap credit ($\beta_{\text{ov}} = 1.62$ and 1.65 , shipped 1), a larger prefill-load scale ($\beta_{\text{pf}} = 24.8$ and 16.9 , shipped 1) and a nonzero active-request weight ($\beta_{\text{req}} = 854$ and $1,152$, shipped 0), so that each active request costs as much as 34 or 68 prefill blocks. Two model classes tuned independently thus both added a request-count load term, the direction the main tuning's M1-v2 took further with its prefill-by-requests interaction (Section 9.8).

How the search behaved. M1’s validation score (the CMA-ES mean, on $k = 0, 1, 2$) reached 0.163–0.165 within 160–240 evaluations and then stayed within 0.005 of each run’s maximum; the two seeds ended 0.0004 apart. M0 plateaued within 80–160 evaluations on train, but in one seed the CMA-ES step size then grew from 0.18 to 10.8 internal units and the generation mean fell to -0.23 , a drift along near-flat directions of the default cost’s knobs. The main tuning therefore capped the per-coordinate step size at 0.3 internal units and set the budget at 400 evaluations per run, the latest validation plateau among the four pilot runs (Sections 6.2 and 6.3). On 16 candidates, the objective on the 12-cell subset ranked candidates like the objective on all 34 train cells (Spearman 0.947); the main tuning nevertheless tuned on all 34.

H Evaluation protocol details

This appendix gives the measurements and rules behind the summary in Section 8, in the order the body uses them.

H.1 Splits and hold-out axes

Table 29 gives the three splits and Table 30 the test cells by worker count and primary hold-out axis.

The 30 cells at unseen worker counts also lie on held-out segments: 14 on held-out Mooncake windows, 8 on held-out session seeds and 8 on held-out AgentX plays. The 10 transform cells use values outside the train ranges: prompt stretches of 2.5 (three cells), output-length multipliers of 0.25, 2.5 and 4, think-time multipliers of 0.25 (one cell) and 4 (two cells), and four times as many prefix roots. Training sees only $N \in \{4, 8\}$, so $N = 2, 16$ and 32 are extrapolation and $N = 6$ is interpolation. Validation also has 4 cells at $N = 6$, so every configuration was selected with that worker count in view, and the 7 test cells at $N = 6$ are labeled *selection-exposed* [7].

Table 29: Splits. A segment is a disjoint piece of a source trace (Table 4); all cells of one segment, whatever their N , load or transform, are one workload for statistics. [†]One train cell pools the plays of AgentX segments A1 and A2.

Split	Cells	Segments	Worker counts N	Open / closed / lanes
Train	34	10 [†]	4, 8	18 / 8 / 8
Validation	14	6	4, 6, 8	6 / 4 / 4
Test	60	12	2, 4, 6, 8, 16, 32	31 / 15 / 14

Table 30: Test cells by worker count and primary hold-out axis, and the number of test segments at each worker count. Window: held-out Mooncake time window; Family: FAST25; Plays: held-out AgentX plays; Seeds: held-out session seeds. Every cell at $N \in \{2, 6, 16, 32\}$ is also on a held-out segment.

N	Worker count	Transform	Window	Family	Plays	Seeds	Cells	Segments
2	8						8	6
4		4	3	2	2	2	13	7
6	7						7	6
8		6	3	4	2	2	17	8
16	7						7	6
32	8						8	6
All	30	10	6	6	4	4	60	12

H.2 Noise-floor measurements

Changing only the default router’s tie-break seed moved its goodput by 1.8% on an 8-worker setup cell, which is why every policy runs seeded (Section 8.2). Over 8 replicates per cell at calibrated loads, π_0 ’s windowed goodput had a coefficient of variation between 0.37% and 6.5% across the 48 train and validation cells (median 1.4%). The paired ratio of round-robin to π_0 on the same replicate had a pooled standard deviation of 0.027 on train and 0.026 on validation.

In the pilot’s comparison of M1 with `llm-d-precise-prefix`, the standard error of the mean over 6 validation segments (0.019, against a pooled replicate standard deviation of the paired ratio of 0.015) was, by the pilot’s own reading, dominated by differences between segments. The mean lead of +0.030 came almost entirely from one Mooncake segment (+0.102), and M1 trailed on all three AgentX segments (−0.012 to −0.025); these numbers are *preliminary (pilot, validation split)*.

At the end of the pilot, a two-clause noise rule decided whether a policy differs from π_0 : it does if on some validation cell its mean paired ratio differs from 1 by more than three pooled replicate standard deviations, *and* the mean of its per-segment deltas exceeds two standard errors over at least three segments. The headline test (Section 8.4) replaces this rule for the final comparison.

H.3 Test-set freeze and completeness audit

The test split was frozen at calibration. The freeze record holds the SHA-256 of the test cell file, of each of the 60 cells’ contents, of the 39 test traces and their 7 sources, of the engine configuration and of the split manifest, together with the bindings build and harness version; all of them (149 of 149) were verified before the single test pass (Section 4.6 and Appendix M). A completeness audit later found that the test cells were touched once: the 30,060 test-cell records in the cache are exactly the expected set, all written after the list of finalists was frozen. The one exception predates calibration: five default-router-only validation replays of the AgentX trace conversion ran on test play pools, at pre-calibration loads, before the test set was frozen; they compared no policies. Every learned variant scored on validation (rung, rank, feature set, restart) is counted and reported: 314 variants, of which 5 arms competed for the headline (Section 9.1).

H.4 Per-worker-count significance under Holm’s correction

Assuming each per-count test is the same exact test as the headline, Holm’s correction across the six worker counts leaves little room. Its first threshold is $0.05/6 \approx 0.0083$, below the smallest p attainable with 6 segments ($1/64 \approx 0.016$). Each unseen worker count, $N = 2, 6, 16$ and 32 , has 6 test segments, so none of them can produce a significant per-count result, whatever the data; only $N = 4$ (7 segments, all favoring one policy) and $N = 8$ (8 segments), both train worker counts, can. This is our own arithmetic, not part of the pre-registration.

H.5 Constants of the alternative SLO definitions

The constants of the four alternative definitions of a good request (Section 8.7) were computed per family from 160 train records of π_0 at L2 and written down before any test record was scored. For Mooncake, for example, the TTFT bound’s intercept is π_0 ’s median TTFT there, 146.7 ms.

H.6 Validation of the lag patch

The lag patch lives in its own branch and build, so the tuning build and its result cache stay untouched (Appendix M). On 8 train cells, 4 policies and 2 replicates (64 runs), $\delta = 0$ on the

lag build reproduced the tuning build in every non-environment record field and in all 404,704 per-request rows. At $\delta = 50$ and 200 ms routing changed in all 64 runs, and goodput relative to $\delta = 0$ ranged from 0.972 to 1.023 and from 0.956 to 1.020; these single-replicate ratios show that the knob is live, not how lag affects the ranking. The robustness runs repeat the lag-0 identity check (Appendix I).

H.7 Herding guards

A deterministic argmax over stale state can herd requests onto one worker. Two variants guard against it: **learned-choice** at $\tau > 0$, which samples instead of taking the argmax (at a fixed temperature, the probability left to non-best workers grows with N), and a variant restricted to the request’s hash-home workers (feature **hash_home** of **v2**). Both were registered before M1-v2 won, so they ran as variants of M1 (**v1**), at two temperatures, on the lag build. Appendix I reports them.

H.8 Guard metrics

Every evaluation record carries guard metrics that the audits read: the largest worker’s share of requests against the cap $\min(2/N, 1/N + 0.25)$, the largest worker’s prefill tokens over the mean, the share of multi-turn sessions split across workers, the share of requests slower than three times the slowdown bound, and the largest worker’s share within each prompt-length bucket, which exposes length segregation. Black-box search is known to exploit gaps in its objective [22]; an evolution strategy tuning a router on 95th-percentile TTFT alone sent all traffic to one replica [42]; and binary goodput invites sacrificing requests that would miss their SLO anyway [55]. The gaming audit showed that the original guards, computed over all rows, could not see a worker whose in-window requests were all bad, so the mid-tuning fix added window-basis guards: the share cap over exactly the scored requests, each worker’s good fraction, the good fraction of long prompts against π_0 , and the normalized mutual information between worker and prompt-length quartile (Section 10).

H.9 Audit rounds

Each stage of the study ends at a checkpoint that independent auditors, each through one named lens, review before the next stage starts (Section 8.8 summarizes the outcome). A static audit reads code and artifacts; a dynamic audit also re-runs part of the pipeline in a private result root and recomputes results with its own code, without importing the study’s harness. Findings are graded blocker, major or minor; a fixer answers each major finding, either fixing it or rebutting it with evidence, and the lens is audited again.

Table 31 lists every audit round. Through the pilot’s review there were 17 audit reports and 6 fix reports. All 10 major findings were judged valid and none was rebutted. Nine were fixed and the fix verified by a later round; the tenth, a second public trace that turned out to be a relabeled copy of the Mooncake trace, was resolved by excluding that family. The pilot’s review was itself checked the same way: an independent judge recomputed every pilot number from raw records with its own scripts, reproduced each to four decimals, and approved continuing to the main tuning. Midway through tuning, three dynamic audits looked for overfitting, objective gaming and simulator exploitation; their five major findings were all fixed. After the single test pass, a completeness critic and three adversarial refuters reviewed the headline before any report was written; their two major findings were fixed by adding the missing secondary metrics and by narrowing the headline’s wording to the scope the evidence supports.

Table 31: Audit rounds by checkpoint and lens, with each major finding and its resolution.

Checkpoint, lens	Rounds	Major findings → resolution
Setup, AIS and policy in effect	r0 pass	none
Setup, determinism, cost, context	r0, r1 fail	identical re-runs cannot measure noise → CRN workload replicates; the FAST25 tool-agent trace is a relabeled Mooncake copy → family excluded
Build, goodput and normalization	r0–r2 fail, r3 pass	reports paired records across builds and cell contents → pairing on content, build and protocol; closed-loop and lane windows scored the drain → window ends at full occupancy; the slowdown clause was a coin flip at scale 1 → E_0 aligned with replay’s decode context, plus a 10^{-6} relative tolerance
Build, feature leakage	r0 fail, r1 pass	v2’s <code>hash_home</code> keyed on replay-synthesized session identifiers → keyed on the prompt prefix only
Build, harness robustness and splits	r0 fail, r1 pass	Mooncake windows overlapped across splits → six disjoint slices; the remote bundle lacked a dependency → shipped and checked at build time
AgentX lowering, isolation and arrivals	r0, r1 pass	none
AgentX lowering, parity and fidelity	r0 fail, r1 pass	think-time and output-length transforms could not be lowered → supported
Calibration, independent re-run	r0 fail, r1 pass	open-loop session windows opened during the session-population ramp → warm-up of two session-lifetime envelopes, sessions re-calibrated, test re-frozen
Pilot review, independent recompute	continue	none; all pilot numbers reproduced
Mid-tuning, overfitting and selection	fail, fixed	the drift check was recorded as decisive although underpowered → recorded as inconclusive, not used for M2’s sources
Mid-tuning, gaming and concentration	fail, fixed	M1 games the objective (a 0%-good worker of long requests) → sign-constrained re-runs (Section 6.5); the remaining size segregation → re-audited and reported on every learned row
Mid-tuning, simulator exploitation	fail, fixed	the long-prompt sacrifice is stable under perturbation → same re-runs; timing robustness scored against the nominal E_0 → consistent E'_0 scoring
Final, completeness	fail, fixed	TTFT by prompt length and the planned secondary metrics were missing → computed from the test records, no replay (Section 10)
Final, refute the headline (reproduction)	pass	none; the headline reproduced bit for bit
Final, refute the headline (fairness)	fail, fixed	faithful LMetric, outside the baseline pool, carries most of the margin → headline worded as beating the branch’s ported heuristics (Section 9.11)
Final, refute the headline (robustness, simulator)	pass	none

I Additional results

This appendix holds the supporting figures and tables of Sections 9 and 10. Like everything outside Section 11, every number comes from AIS-timed offline replay; intervals are 95% segment-bootstrap intervals, as in Section 9. Figure 7 plots the per-segment statistics of Table 9, and Figure 8 compares each test policy’s validation score with its test score (Section 9.3). Table 32 gives every rung-to-rung comparison that Section 9.7 reads, and Figure 9 shows each arm on the selection replicates, the fresh validation replicates and test. Figure 10 plots the standardized coefficients of Table 12 (Section 9.8), and Figure 11 the prompt-length trade of Table 16 (Section 10.4).

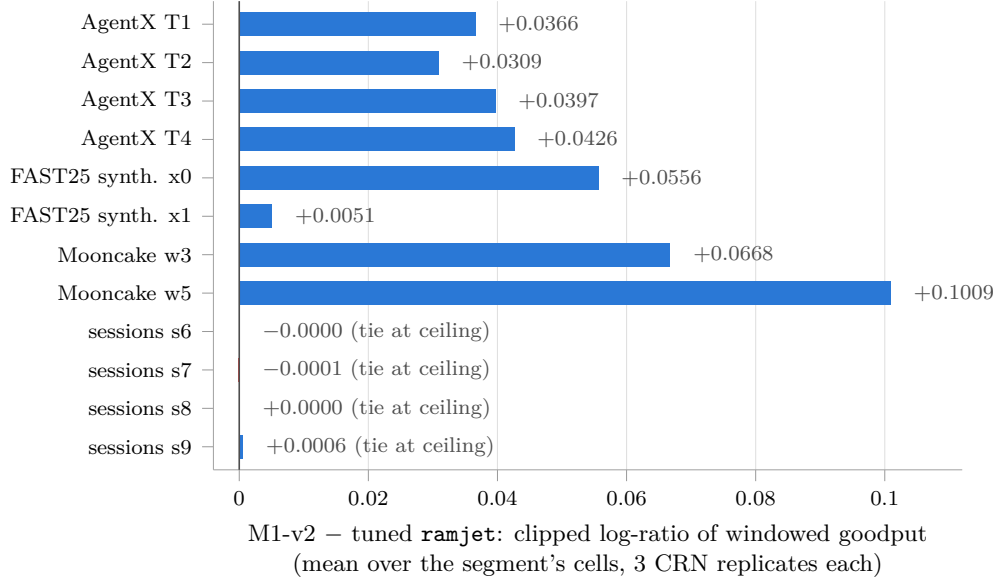


Figure 7: The headline per test segment, labelled by family and the segment names of Table 9. The four session segments tie at ceiling; every other segment favors M1-v2.

Table 32: Rung-to-rung comparisons on test (descriptive).

Comparison ($A - B$)	Segment mean [95% CI]	Ahead	p ($A > B$)	Cell mean
M1 – M0 (default cost fn, tuned)	+0.0467 [+0.0248, +0.0700]	11/12	0.0007	+0.0639
M2 rank 2 – M1	+0.0025 [−0.0008, +0.0070]	6/12	0.2349	+0.0033
M1 continued (secondary) – M1	+0.0071 [−0.0024, +0.0195]	6/12	0.3386	+0.0070
M2 rank 2 – M1 continued (secondary)	−0.0045 [−0.0134, +0.0029]	8/12	0.5452	−0.0037
M1-v2 (headline learned arm) – M1	+0.0130 [+0.0039, +0.0244]	11/12	0.0034	+0.0137
M1-noaff – M1	+0.0084 [−0.0012, +0.0211]	7/12	0.2119	+0.0081
M1 + queue threshold (ablation) – M1 continued (secondary)	+0.0015 [−0.0002, +0.0035]	8/12	0.0549	+0.0008
ramjet + queue threshold (ablation) – ramjet (val-best baseline)	−0.0016 [−0.0050, +0.0010]	4/12	0.8833	−0.0017
M1 – M1 unconstrained (flagged for gaming)	+0.0039 [−0.0065, +0.0144]	8/12	0.1902	+0.0053
M1-ais – M0-ais	+0.0275 [+0.0104, +0.0486]	10/12	0.0017	+0.0497
M2-ais – M1-ais	+0.0030 [+0.0010, +0.0053]	9/12	0.0046	+0.0027
M2-ais – M1-v2 (headline learned arm)	−0.0029 [−0.0052, −0.0006]	3/12	0.9788	−0.0040
M0-ais – M0 (default cost fn, tuned)	+0.0262 [+0.0101, +0.0445]	10/12	0.0017	+0.0211

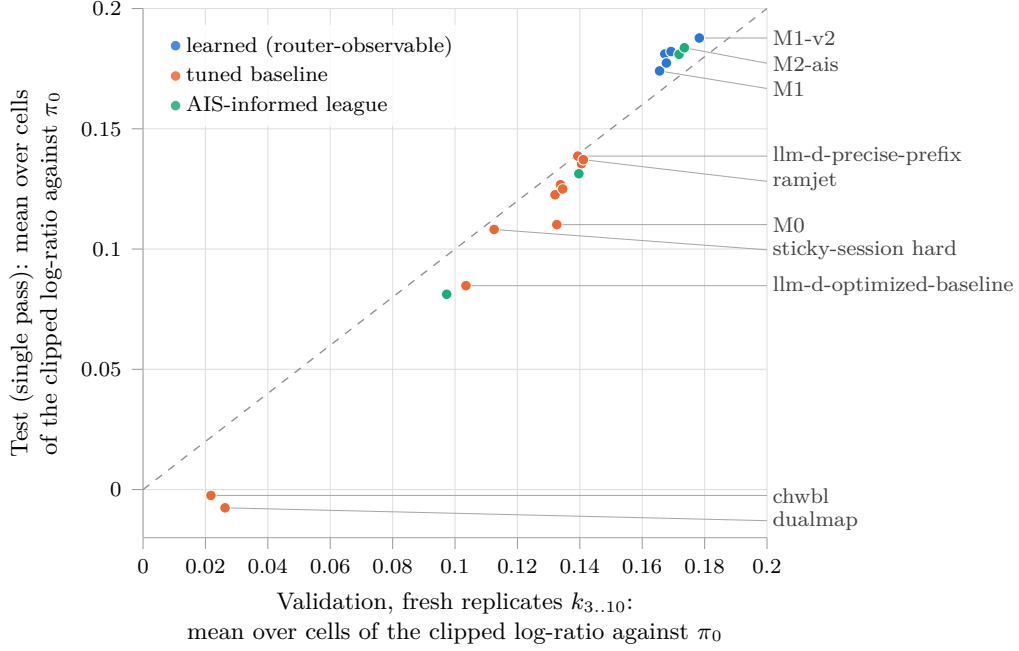


Figure 8: Validation (fresh replicates) against test, mean over cells of the clipped log-ratio against π_0 . The two compare different cells, so the dashed line is a reference, not an expectation.

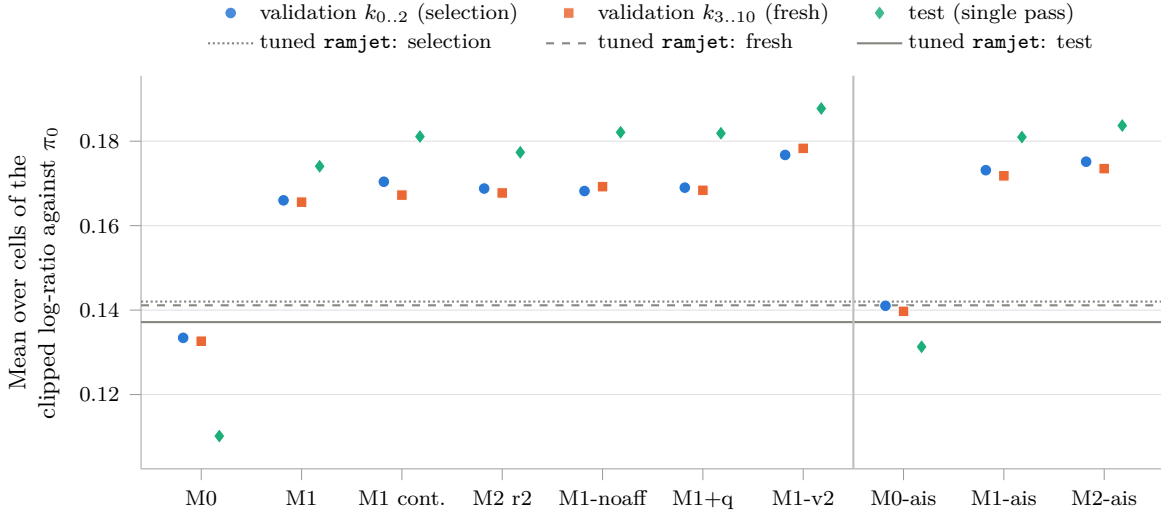


Figure 9: The model ladder on the selection replicates, the fresh validation replicates and test. Left of the vertical rule: router-observable arms; right of it: the AIS-informed league. “M1 cont.” is M1 continued, “M2 r2” is M2 at rank 2 and “M1+q” is M1 with the queue threshold.

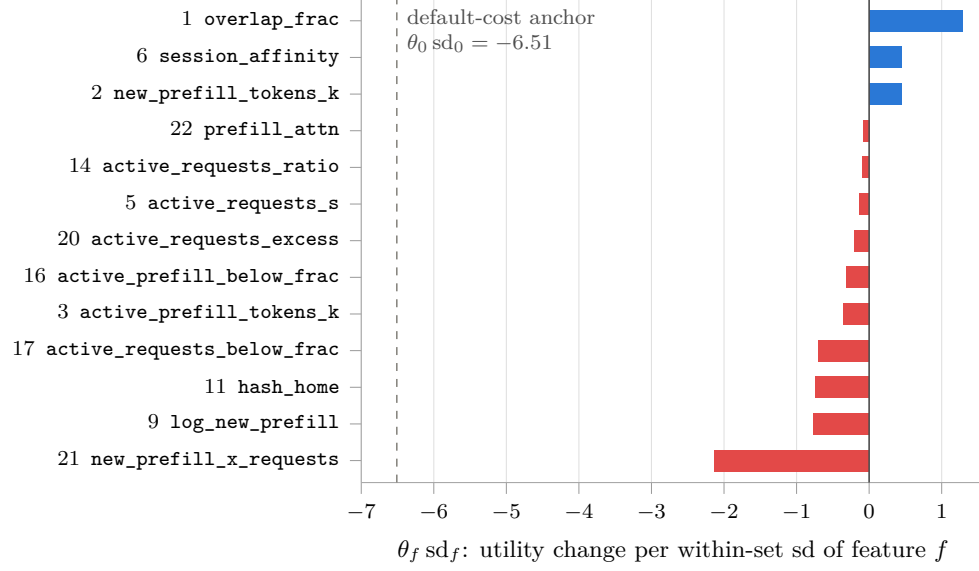


Figure 10: M1-v2’s nonzero coefficients, standardized, labelled with the feature index f and name of Table 12. Positive attracts a worker, negative repels it; the default-cost anchor is at -6.51 .

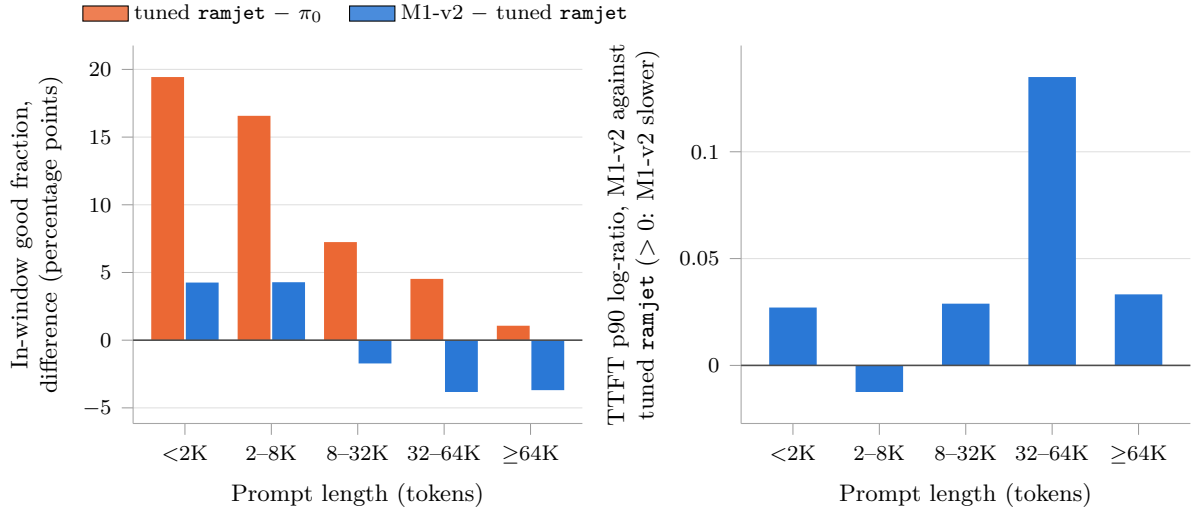


Figure 11: Where M1-v2’s gain comes from (segment means on test). Left: in-window good fraction by prompt length in percentage points, **ramjet** minus π_0 (orange) and M1-v2 minus **ramjet** (blue). Right: TTFT p90 log-ratio of M1-v2 against **ramjet**.

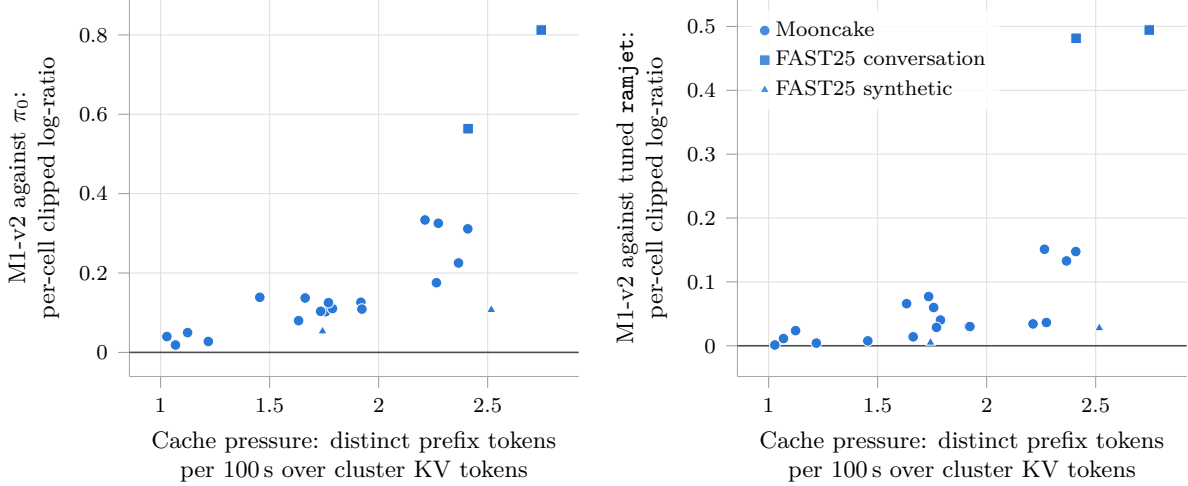


Figure 12: Per-cell gain against cache pressure on the 22 open-loop Mooncake and FAST25 test cells. Cache pressure is distinct prefix tokens per 100s over the cluster’s KV capacity. The trend is descriptive: pressure is confounded with load level, and cells of one segment are not independent. Left: against π_0 ; right: against tuned `ramjet`; marker shape: family.

I.1 Cache pressure

On the 22 open-loop Mooncake and FAST25 test cells, M1-v2’s gain rises with cache pressure: Spearman ρ 0.79 against π_0 and 0.71 against `ramjet` (Figure 12). Pressure ranges from 1.0 to 2.7, so no test cell has little cache pressure, a design gap calibration recorded (Section 4.5).

I.2 Robustness details

Figure 13 plots Table 13.

Lag-0 identity. The lag build reproduces the tuning build at lag 0 per request: 180 of 180 test records for π_0 and M1, and 448 of 448 validation records for M1-v2, `ramjet`, `llm-d-precise-prefix` and π_0 (Section 8.7).

Simulator-model counterfactuals. On validation only, an audit-only build of the simulator changes how AIS prices mixed prefill and decode passes. Against +0.0290 on the unmodified simulator, M1-v2’s lead over `ramjet` is +0.0274 when mixed prefill passes are priced attention-equivalent, +0.0291 when priced as a per-request sum, +0.0268 with an extra 0.1 ms of decode cost per running sequence, and +0.0269 when decode inside a prefill pass costs a quarter. These bracket fidelity errors that act on M1-v2’s concentration mechanism; they are not measurements of reality, and FAST25, a test-only family, is not covered.

I.3 Herding guards

Two guards against herding were registered before M1-v2 won, so they ran as variants of M1 (Appendix H; Table 33). A positive temperature does not help: $\tau = 0.747$ is within noise of M1, and $\tau = 2.75$ costs 0.028, more at $N = 16$ and 32 than at 2, as expected when the softmax mass on non-best workers grows with N . Restricting candidates to the prefix’s `hash_home` pair is harmful at

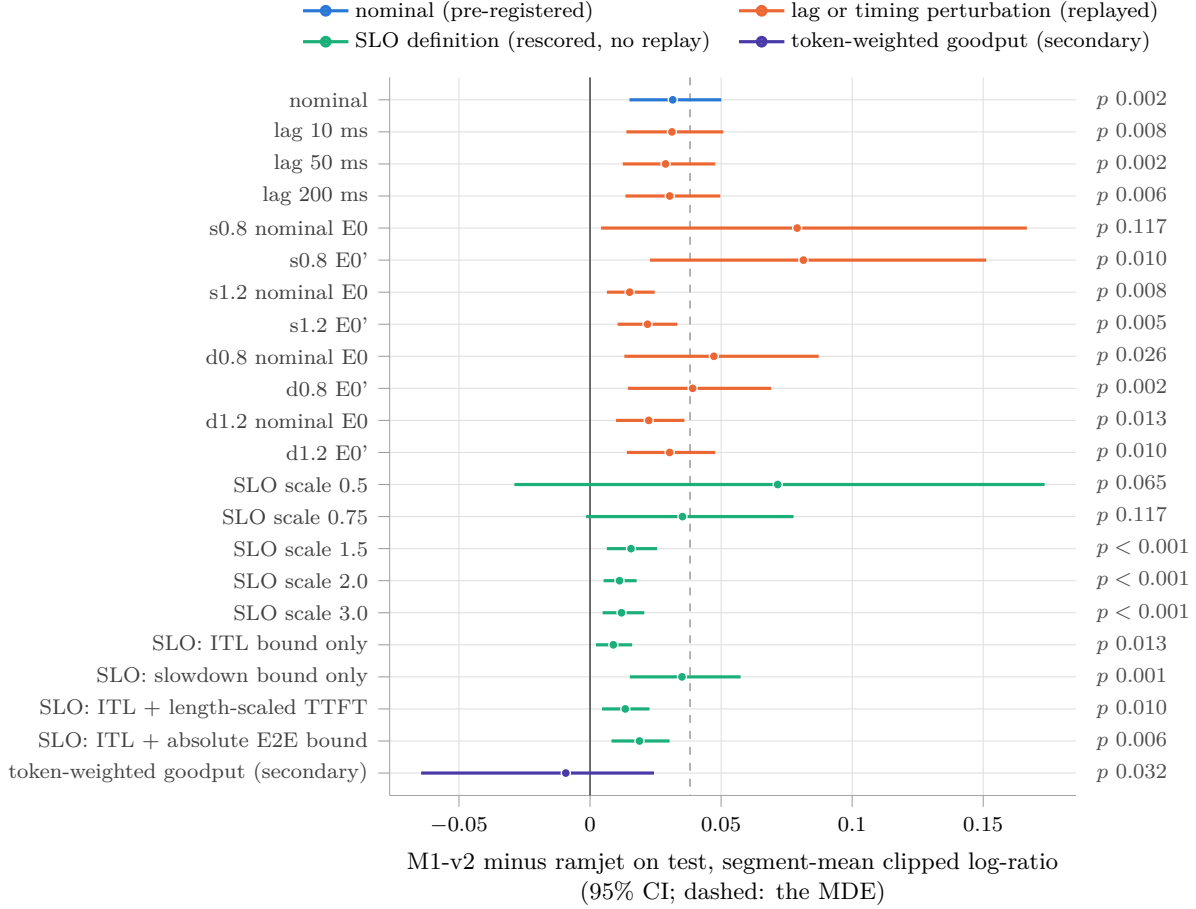


Figure 13: The headline under perturbations and alternative SLOs (dashed: the MDE). The four “SLO:” rows are the alternative SLO definitions, and the last row is the token-weighted secondary metric.

Table 33: Herding guards, as variants of M1 on the lag build at lag 0: segment-mean clipped log-ratio against M1 with 95% intervals, over all test cells and at the unseen worker counts.

Variant of M1	vs M1, all cells	$N = 2$	$N = 16$	$N = 32$
$\tau = 0.747$	-0.0044 [-0.0103, +0.0008]	-0.0114 [-0.0328, +0.0006]	-0.0016 [-0.0049, +0.0019]	-0.0044 [-0.0077, -0.0014]
$\tau = 2.75$	-0.0277 [-0.0352, -0.0202]	-0.0162 [-0.0227, -0.0092]	-0.0391 [-0.0557, -0.0199]	-0.0383 [-0.0566, -0.0184]
hash_home pair only	-0.2626 [-0.4025, -0.1394]	+0.0000 [+0.0000, +0.0000]	-0.5716 [-0.7895, -0.3406]	-0.6844 [-0.9727, -0.3910]

large N (-0.57 at 16 and -0.68 at 32); at $N = 2$ every worker is home, so the variant is identical there. No herding failure of the argmax appeared under the tested lags in replay.

I.4 Cost

Table 34 collects the compute. Replay-seconds sum the wall time of single-threaded replays, so they measure work, not elapsed time.

Table 34: Compute by stage. Every row is measured and comes from the stage facts, except where it says otherwise.

Stage	Value	Source
Calibration, first pass	13,532 replays, 122,436 replay-s, about 3.3 h on the workstation	<code>facts/calibration.json</code>
Calibration fix	9,792 replays, 112,561 replay-s, 0 errors	<code>facts/STATE.md</code>
Pilot	56,552 fresh replays, 0 errors	<code>facts/pilot.json</code>
One evaluation, all 34 train cells, $K = 2$	700.9 replay-s; 37.0s wall on the 20-slot workstation pool (pilot)	<code>facts/pilot.json</code>
Tuning	66 restarts of 400 evaluations; about 30,000 result records per run; generations of about 300s (EPYC 9654P) to 690s (EPYC 7702P); replay-seconds not totaled	<code>facts/STATE.md</code>
Test pass	5,220 + 900 records, 0 errors	<code>facts/test_results.json</code>
Robustness	23,940 records, 0 errors	<code>facts/robustness.json</code>
Per-decision policy latency	not measured	—

J The AIS-informed league

In the main study AIS times the engines only. The router-side AIS estimator, which some router modes use to model prefill load, is off, so the policies under study see the same router state a live router without a performance model would see (Section 3.1). A separate, secondary league, the AIS-informed league, admits policies that may read AIS-derived estimates at runtime (Section 7.2). It runs on its own build, with the router-side estimator on. Because these policies read estimates from the model that also times the simulation, the league is a secondary comparison and never enters the headline: its arms cannot become π_θ , and they compete in a leaderboard of their own.

J.1 Features

The league’s learned arms use a third feature set, `v3`, built on a separate branch and build: the eight `v1` features plus five per-worker estimates from AIS, in seconds. They are this request’s predicted prefill time for its uncached tokens on the worker (`est_prefill_ms_s`); the worker’s remaining modeled prefill backlog, which counts only the unfinished part of in-progress prefills (`prefill_backlog_ms_s`); their sum, an estimated TTFT (`est_ttft_ms_s`); the decode-step time once the request joins (`est_decode_step_ms_s`); and the per-step increase the request imposes on the requests already on the worker (`itl_externality_ms_s`). The backlog is the host’s `PREFILL_TIME` input, which a live router fills when it runs the AIS prefill-load model. The request’s own prefill time needed a new research input that only offline replay fills; live, a closed form fitted to AIS offline stands in for it, at 6.6% mean error on a grid up to 128K tokens. The decode terms use a closed form in the batch size and the router’s decode blocks, which undercount context shared by co-resident requests (Section 5.2 defines the shared features).

J.2 Baselines

The league’s baselines are M0 run with the router’s AIS prefill-load model (`router_prefill_load_model: ais`) and the optimized baseline with `ttft_source: modeled`, which estimates each worker’s TTFT from that model’s prefill backlog and counts a worker without an estimate as slowest. Both are tuned with the same budget and normalized by the same π_0 without AIS. Running the default cost with the AIS load model is not neutral: it changed the default router’s own routing on

12 of 12 probe pairs, so M0-ais shows that effect explicitly.

J.3 Training and comparison

The league has four arms: M1-ais and M2-ais on feature set v3, M0 with the AIS prefill-load model, and `llm-d-optimized-baseline` with modeled TTFT. Each is trained like the main arms (Section 6), with three restarts at the same budget, on the league’s own build. The comparison, registered before the test pass as a secondary (Section 8.6), sets the league’s arms against the best AIS-informed baseline and against the best baseline overall. Every arm is normalized by the same π_0 , which does not use AIS, so the two leagues share one scale and we can state how much the AIS estimates add.

J.4 Results

The league’s arms appear in Tables 11 and 32 and in the leaderboard (Figure 3).

- **Against its own baseline and the overall best.** M2-ais, the league’s selected learned arm, beats M0-ais, the better AIS-informed baseline on validation, by +0.0305 (p 0.0012), and tuned `ramjet` by +0.0286 (p 0.0005).
- **AIS features add nothing at the learned level.** M2-ais minus M1-v2 is -0.0029 , ahead on 3 of 12 segments.
- **The AIS load model helps the default cost function.** M0-ais beats M0 by +0.026 and roughly matches `ramjet` (-0.002).
- **No robustness evidence.** The league has no lag robustness, because the lag build predates feature set v3, and no timing robustness, which was never registered; under the reading rule of Section 8.7 even its direction is unverified under perturbation.
- **Not portable as is.** M1-ais and M2-ais read a request’s own modeled prefill time, a research input only offline replay fills; live they would need the closed-form estimate (6.6% mean error).

K Full result tables

This appendix holds the full result tables that Section 9 shows in part. Table 35 gives every test policy. For selected policies, Tables 36 and 37 break the result down by worker count, Tables 38 and 39 by family, Table 40 by load mode and Table 41 by hold-out stratum. Table 42 gives the coefficients of the v1 learned arms, Tables 43 and 44 the secondary metrics of Section 10, and Figure 14 the performance profiles. Intervals are 95% segment-bootstrap intervals ($\dagger$: within-segment cell bootstrap for strata with fewer than three segments); only the headline row of Table 35 is a test.

Table 35: Every test policy against π_0 and against the validation-selected best baseline (tuned `ramjet`), 60 cells in 12 segments, segment means with 95% segment-bootstrap intervals. Selected config: restart, generation and candidate (best-so-far sample or CMA-ES mean) chosen on validation replicates $k_{0..2}$. Val $k_{3..10}$: mean clipped log-ratio against π_0 on fresh validation replicates, the cross-policy selection score; policies at shipped defaults have none. The one-sided p is a test only in the headline row; elsewhere it is descriptive.

Policy	Selected config	Val $k_{3..10}$	Test vs π_0 [95% CI]	Test vs <code>ramjet</code> [95% CI]	Ahead	p
M1-v2 (headline learned arm)	restart 1, gen. 20, best-so-far sample	0.1783	+0.1758 [+0.1270, +0.2218]	+0.0316 [+0.0150, +0.0501]	10/12	0.0017
M2-ais	restart 3, gen. 10, CMA-ES mean	0.1735	+0.1729 [+0.1239, +0.2191]	+0.0286 [+0.0138, +0.0458]	11/12	0.0005

Policy	Selected config	Val $k_{3..10}$	Test vs π_0 [95% CI]	Test vs ramjet [95% CI]	Ahead	p
M1-noaff	restart 1, gen. 25, best-so-far sample	0.1692	+0.1712 [+0.1218, +0.2178]	+0.0270 [+0.0109, +0.0443]	8/12	0.0134
M1 + queue threshold (ablation)	restart 1, gen. 25, CMA-ES mean	0.1684	+0.1714 [+0.1222, +0.2179]	+0.0272 [+0.0115, +0.0444]	8/12	0.0171
M1 continued (secondary)	restart 3, gen. 25, best-so-far sample	0.1672	+0.1699 [+0.1211, +0.2158]	+0.0256 [+0.0090, +0.0437]	8/12	0.0212
M1-ais	restart 1, gen. 25, best-so-far sample	0.1718	+0.1699 [+0.1208, +0.2163]	+0.0256 [+0.0113, +0.0423]	11/12	0.0012
M2 rank 2	restart 3, gen. 25, CMA-ES mean	0.1677	+0.1653 [+0.1176, +0.2107]	+0.0211 [+0.0043, +0.0406]	7/12	0.0549
M1	restart 1, gen. 20, CMA-ES mean	0.1656	+0.1628 [+0.1156, +0.2072]	+0.0186 [+0.0013, +0.0378]	7/12	0.0549
M1 unconstrained (flagged for gaming)	restart 2, gen. 15, CMA-ES mean	n/a	+0.1590 [+0.1114, +0.2047]	+0.0147 [+0.0032, +0.0284]	7/12	0.0386
llm-d-precise-prefix	restart 2, gen. 25, CMA-ES mean	0.1393	+0.1464 [+0.0986, +0.1939]	+0.0022 [-0.0089, +0.0139]	4/12	0.4548
ramjet (val-best baseline)	restart 3, gen. 25, best-so-far sample	0.1411	+0.1442 [+0.0983, +0.1894]	(reference)		
ramjet + queue threshold (ablation)	restart 3, gen. 5, best-so-far sample	0.1405	+0.1426 [+0.0970, +0.1875]	-0.0016 [-0.0050, +0.0010]	4/12	0.8833
M0-ais	restart 2, gen. 20, best-so-far sample	0.1397	+0.1423 [+0.0967, +0.1874]	-0.0019 [-0.0086, +0.0048]	6/12	0.6890
llm-d-precise-prefix@defaults	shipped parameters	n/a	+0.1442 [+0.0952, +0.1932]	-0.0000 [-0.0137, +0.0139]	4/12	0.5452
sticky-session bounded	restart 2, gen. 25, CMA-ES mean	0.1338	+0.1357 [+0.0893, +0.1818]	-0.0086 [-0.0157, -0.0024]	3/12	0.9866
lmetric@defaults	shipped parameters	n/a	+0.1392 [+0.0889, +0.1901]	-0.0050 [-0.0174, +0.0068]	8/12	0.5750
two-tier	restart 1, gen. 15, CMA-ES mean	0.1345	+0.1374 [+0.0879, +0.1866]	-0.0068 [-0.0174, +0.0029]	6/12	0.7407
ramjet@defaults	shipped parameters	n/a	+0.1312 [+0.0926, +0.1706]	-0.0130 [-0.0227, -0.0036]	3/12	0.9788
two-tier@defaults	shipped parameters	n/a	+0.1371 [+0.0867, +0.1875]	-0.0071 [-0.0184, +0.0036]	6/12	0.7881
lmetric	restart 2, gen. 10, CMA-ES mean	0.1321	+0.1356 [+0.0860, +0.1857]	-0.0086 [-0.0192, +0.0011]	6/12	0.8303
M0 (default cost fn, tuned)	restart 3, gen. 20, CMA-ES mean	0.1326	+0.1162 [+0.0777, +0.1562]	-0.0281 [-0.0439, -0.0137]	1/12	0.9993
sticky-session hard	restart 1, gen. 25, CMA-ES mean	0.1125	+0.1127 [+0.0682, +0.1609]	-0.0316 [-0.0603, -0.0073]	4/12	0.9539
llm-d-optimized-baseline (throughput)	restart 1, gen. 5, CMA-ES mean	0.1035	+0.0844 [+0.0377, +0.1341]	-0.0598 [-0.0946, -0.0277]	0/12	1.0000
llm-d-optimized-baseline@defaults (throughput)	shipped parameters	n/a	+0.0809 [+0.0325, +0.1314]	-0.0633 [-0.0979, -0.0321]	0/12	1.0000
llm-d-optimized-baseline (modeled)	restart 2, gen. 15, CMA-ES mean	0.0973	+0.0724 [+0.0241, +0.1232]	-0.0718 [-0.1057, -0.0392]	0/12	1.0000
sticky-session bounded@defaults	shipped parameters	n/a	+0.0982 [+0.0475, +0.1514]	-0.0460 [-0.0760, -0.0201]	1/12	0.9983
sticky-session hard@defaults	shipped parameters	n/a	+0.0806 [+0.0305, +0.1368]	-0.0637 [-0.0970, -0.0311]	2/12	0.9976
chwb1	restart 1, gen. 5, CMA-ES mean	0.0218	-0.0299 [-0.0711, +0.0132]	-0.1741 [-0.2378, -0.1115]	0/12	1.0000
dualmap	restart 3, gen. 10, CMA-ES mean	0.0263	-0.0044 [-0.0366, +0.0297]	-0.1486 [-0.1794, -0.1148]	0/12	1.0000
dualmap@defaults	shipped parameters	n/a	-0.0817 [-0.1200, -0.0428]	-0.2257 [-0.2779, -0.1769]	0/12	1.0000
chwb1@defaults	shipped parameters	n/a	-0.1866 [-0.2557, -0.1258]	-0.3254 [-0.4185, -0.2435]	0/12	1.0000
round-robin	shipped parameters	n/a	-0.5631 [-0.7147, -0.4031]	-0.6657 [-0.8234, -0.5019]	0/12	1.0000

Table 43: Secondary metrics against ramjet on test, part 1 (latency) (descriptive, computed after the test pass from its records, no replay). Each entry is the segment-mean log-ratio, with the number of segments on which the policy’s value is higher. Lower is better for latencies, higher for throughput and prefix reuse.

Policy	TTFT p50	TTFT p90	TTFT p99	TTFT p90, 32–64K	E2E p90 (in window)
M1-v2 (headline learned arm)	+0.006 (9/12)	-0.002 (6/12)	-0.082 (4/12)	+0.169 (8/8)	-0.048 (3/12)
M2-ais	+0.018 (7/12)	-0.018 (3/12)	-0.105 (4/12)	+0.111 (6/8)	-0.054 (0/12)
M1-noaff	+0.023 (9/12)	+0.036 (5/12)	-0.057 (5/12)	+0.123 (5/8)	-0.043 (4/12)
M1 + queue threshold (ablation)	+0.025 (9/12)	+0.028 (6/12)	-0.069 (4/12)	+0.104 (6/8)	-0.042 (4/12)
M1 continued (secondary)	+0.025 (9/12)	+0.060 (5/12)	-0.047 (4/12)	+0.147 (7/8)	-0.037 (4/12)
M1-ais	+0.012 (7/12)	-0.019 (2/12)	-0.108 (4/12)	+0.103 (5/8)	-0.047 (1/12)
M2 rank 2	+0.028 (9/12)	+0.054 (7/12)	+0.013 (5/12)	+0.139 (5/8)	-0.034 (5/12)
M1	+0.020 (9/12)	+0.046 (7/12)	+0.032 (5/12)	+0.105 (5/8)	-0.032 (5/12)
M1 unconstrained (flagged for gaming)	+0.015 (9/12)	+0.053 (8/12)	+0.065 (7/12)	+0.207 (5/8)	-0.031 (4/12)
llm-d-precise-prefix	+0.125 (12/12)	+0.144 (9/12)	+0.002 (7/12)	+0.075 (7/8)	-0.005 (6/12)
ramjet + queue threshold (ablation)	+0.006 (9/12)	+0.015 (8/12)	+0.010 (9/12)	+0.004 (5/8)	+0.001 (8/12)

Policy	TTFT p50	TTFT p90	TTFT p99	TTFT p90, 32–64K	E2E p90 (in window)
M0-ais	+0.019 (9/12)	−0.006 (5/12)	−0.091 (1/12)	−0.010 (2/8)	+0.007 (10/12)
llm-d-precise-prefix@defaults	+0.106 (11/12)	+0.119 (8/12)	−0.008 (7/12)	+0.039 (7/8)	−0.006 (6/12)
sticky-session bounded	−0.007 (4/12)	−0.031 (2/12)	−0.062 (2/12)	+0.024 (5/8)	+0.010 (7/12)
lmetric@defaults	+0.115 (8/12)	+0.101 (6/12)	+0.046 (7/12)	+0.079 (8/8)	−0.005 (4/12)
two-tier	+0.103 (8/12)	+0.139 (12/12)	+0.119 (9/12)	+0.108 (8/8)	−0.000 (5/12)
ramjet@defaults	+0.070 (8/12)	+0.114 (10/12)	−0.041 (6/12)	+0.011 (6/8)	+0.021 (9/12)
two-tier@defaults	+0.125 (8/12)	+0.118 (8/12)	+0.103 (10/12)	+0.066 (8/8)	−0.002 (5/12)
lmetric	+0.125 (8/12)	+0.134 (7/12)	+0.058 (8/12)	+0.130 (8/8)	−0.004 (4/12)
M0 (default cost fn, tuned)	+0.090 (10/12)	+0.047 (10/12)	−0.071 (4/12)	+0.036 (7/8)	+0.038 (12/12)
sticky-session hard	+0.014 (7/12)	+0.078 (6/12)	+0.182 (7/12)	+0.079 (6/8)	+0.040 (7/12)
llm-d-optimized-baseline (throughput)	+0.011 (5/12)	+0.027 (5/12)	+0.036 (5/12)	+0.079 (6/8)	+0.066 (12/12)
llm-d-optimized-baseline@defaults (throughput)	+0.023 (6/12)	+0.071 (7/12)	+0.097 (8/12)	+0.087 (7/8)	+0.069 (12/12)
llm-d-optimized-baseline (modeled)	−0.015 (5/12)	+0.075 (6/12)	+0.193 (10/12)	+0.231 (8/8)	+0.099 (12/12)
sticky-session bounded@defaults	+0.198 (12/12)	+0.207 (12/12)	−0.015 (5/12)	+0.147 (7/8)	+0.032 (10/12)
sticky-session hard@defaults	+0.134 (8/12)	+0.109 (12/12)	+0.138 (5/12)	+0.096 (7/8)	+0.064 (9/12)
chwhl	+0.618 (12/12)	+0.714 (12/12)	+0.386 (11/12)	+0.327 (8/8)	+0.136 (12/12)
dualmap	+0.143 (12/12)	+0.230 (12/12)	+0.163 (9/12)	+0.205 (8/8)	+0.158 (12/12)
dualmap@defaults	+0.291 (12/12)	+0.483 (12/12)	+0.562 (12/12)	+0.356 (8/8)	+0.241 (12/12)
chwhl@defaults	+0.664 (12/12)	+0.858 (12/12)	+0.510 (11/12)	+0.430 (8/8)	+0.243 (12/12)
round-robin	+1.705 (12/12)	+1.340 (12/12)	+1.037 (12/12)	+0.897 (8/8)	+0.512 (12/12)
default@defaults (reference)	+0.579 (12/12)	+0.451 (12/12)	+0.034 (5/12)	+0.180 (6/8)	+0.116 (12/12)

Table 44: Secondary metrics against ramjet on test, part 2 (decode, throughput, reuse) (descriptive, computed after the test pass from its records, no replay). Each entry is the segment-mean log-ratio, with the number of segments on which the policy’s value is higher. Lower is better for latencies, higher for throughput and prefix reuse.

Policy	Mean-ITL p90	Per-token ITL p99	Output throughput	Prefix reuse	AgentX trajectory
M1-v2 (headline learned arm)	+0.006 (3/12)	−0.077 (4/12)	+0.006 (7/12)	−0.024 (1/12)	−0.012 (0/4)
M2-ais	+0.020 (1/12)	−0.107 (0/12)	+0.007 (10/12)	−0.013 (6/12)	−0.010 (0/4)
M1-noaff	+0.017 (5/12)	−0.090 (4/12)	+0.005 (7/12)	−0.015 (6/12)	−0.011 (0/4)
M1 + queue threshold (ablation)	+0.012 (5/12)	−0.084 (4/12)	+0.006 (7/12)	−0.014 (7/12)	−0.011 (0/4)
M1 continued (secondary)	+0.028 (5/12)	−0.070 (4/12)	+0.004 (7/12)	−0.018 (7/12)	−0.010 (0/4)
M1-ais	+0.029 (1/12)	−0.127 (0/12)	+0.007 (10/12)	−0.013 (8/12)	−0.009 (0/4)
M2 rank 2	+0.011 (5/12)	−0.082 (5/12)	+0.004 (5/12)	−0.012 (5/12)	−0.004 (1/4)
M1	+0.023 (6/12)	−0.080 (5/12)	+0.006 (7/12)	−0.011 (5/12)	−0.002 (1/4)
M1 unconstrained (flagged for gaming)	+0.035 (5/12)	−0.091 (5/12)	−0.006 (4/12)	−0.010 (7/12)	−0.006 (0/4)
llm-d-precise-prefix	−0.010 (5/12)	+0.027 (4/12)	−0.000 (4/12)	−0.008 (3/12)	−0.012 (0/4)
ramjet + queue threshold (ablation)	−0.005 (5/12)	+0.010 (7/12)	−0.001 (4/12)	+0.001 (4/12)	−0.000 (2/4)
M0-ais	+0.020 (9/12)	+0.034 (10/12)	−0.001 (4/12)	−0.031 (0/12)	−0.003 (0/4)
llm-d-precise-prefix@defaults	−0.010 (5/12)	+0.007 (5/12)	+0.001 (7/12)	−0.005 (6/12)	−0.012 (0/4)
sticky-session bounded	+0.026 (7/12)	+0.003 (4/12)	−0.002 (5/12)	−0.052 (4/12)	+0.000 (3/4)
lmetric@defaults	−0.001 (2/12)	−0.034 (0/12)	−0.000 (5/12)	+0.002 (5/12)	−0.006 (0/4)
two-tier	+0.001 (4/12)	−0.024 (0/12)	−0.001 (4/12)	−0.013 (2/12)	−0.004 (0/4)
ramjet@defaults	+0.027 (9/12)	+0.091 (8/12)	−0.005 (2/12)	−0.049 (0/12)	+0.004 (3/4)
two-tier@defaults	−0.002 (3/12)	−0.028 (1/12)	−0.001 (5/12)	−0.002 (6/12)	−0.003 (2/4)
lmetric	−0.003 (2/12)	−0.037 (0/12)	−0.001 (4/12)	−0.004 (5/12)	−0.003 (1/4)
M0 (default cost fn, tuned)	+0.066 (11/12)	+0.103 (10/12)	−0.006 (1/12)	−0.133 (0/12)	+0.009 (3/4)
sticky-session hard	+0.041 (7/12)	−0.005 (5/12)	−0.013 (3/12)	−0.029 (4/12)	+0.036 (4/4)
llm-d-optimized-baseline (throughput)	+0.115 (12/12)	+0.083 (11/12)	−0.010 (1/12)	−0.023 (4/12)	+0.026 (4/4)
llm-d-optimized-baseline@defaults (throughput)	+0.123 (12/12)	+0.082 (10/12)	−0.011 (0/12)	−0.011 (4/12)	+0.029 (4/4)
llm-d-optimized-baseline (modeled)	+0.217 (12/12)	+0.078 (9/12)	−0.015 (1/12)	−0.055 (4/12)	+0.037 (4/4)
sticky-session bounded@defaults	+0.151 (9/12)	+0.043 (8/12)	−0.006 (1/12)	−0.139 (0/12)	+0.001 (2/4)
sticky-session hard@defaults	+0.185 (11/12)	+0.032 (5/12)	−0.017 (2/12)	−0.117 (4/12)	+0.039 (4/4)
chwhl	+0.330 (11/12)	+0.297 (10/12)	−0.030 (0/12)	−0.252 (0/12)	+0.052 (4/4)
dualmap	+0.309 (12/12)	+0.135 (11/12)	−0.028 (0/12)	−0.050 (0/12)	+0.044 (4/4)
dualmap@defaults	+0.405 (12/12)	+0.260 (12/12)	−0.043 (0/12)	−0.033 (0/12)	+0.049 (4/4)
chwhl@defaults	+0.458 (11/12)	+0.473 (10/12)	−0.046 (0/12)	−0.255 (0/12)	+0.056 (4/4)
round-robin	+0.759 (12/12)	+1.045 (12/12)	−0.115 (0/12)	−0.960 (0/12)	+0.248 (4/4)
default@defaults (reference)	+0.277 (11/12)	+0.387 (10/12)	−0.025 (1/12)	−0.262 (0/12)	+0.030 (3/4)

Table 36: By worker count, against tuned **ramjet** (selected rows). Segment means with 95% intervals. Cells (segments) per N : 2: 8 (6); 4: 13 (7); 6: 7 (6), selection-exposed; 8: 17 (8); 16: 7 (6); 32: 8 (6). Table 37 gives the same against π_0 .

Policy	$N = 2$	$N = 4$	$N = 6$
M1-v2 (headline learned arm)	+0.038 [+0.014, +0.066]	+0.055 [+0.011, +0.109]	+0.030 [-0.000, +0.065]
M2-ais	+0.024 [+0.002, +0.054]	+0.053 [+0.014, +0.101]	+0.029 [+0.004, +0.060]
M1	+0.032 [+0.007, +0.058]	+0.042 [-0.002, +0.096]	+0.011 [-0.023, +0.047]
M1 unconstrained (flagged for gaming)	+0.015 [+0.000, +0.036]	+0.042 [+0.006, +0.089]	+0.014 [-0.005, +0.037]
llm-d-precise-prefix	+0.005 [-0.018, +0.028]	+0.004 [-0.013, +0.021]	-0.001 [-0.018, +0.019]
lmetric	+0.003 [-0.008, +0.013]	-0.009 [-0.029, +0.010]	-0.014 [-0.039, +0.013]
M0 (default cost fn, tuned)	-0.012 [-0.024, -0.002]	-0.050 [-0.130, -0.003]	-0.030 [-0.067, -0.006]
round-robin	-0.392 [-0.552, -0.213]	-0.480 [-0.645, -0.332]	-0.786 [-1.043, -0.500]
default@defaults (reference)	-0.067 [-0.141, -0.021]	-0.098 [-0.182, -0.044]	-0.137 [-0.198, -0.077]
Policy	$N = 8$	$N = 16$	$N = 32$
M1-v2 (headline learned arm)	+0.043 [+0.014, +0.079]	+0.029 [+0.010, +0.048]	+0.024 [+0.008, +0.042]
M2-ais	+0.039 [+0.012, +0.073]	+0.028 [+0.011, +0.045]	+0.026 [+0.009, +0.043]
M1	+0.033 [+0.006, +0.066]	+0.006 [-0.004, +0.022]	+0.011 [+0.000, +0.023]
M1 unconstrained (flagged for gaming)	+0.024 [+0.003, +0.053]	+0.007 [+0.001, +0.012]	+0.013 [+0.003, +0.023]
llm-d-precise-prefix	+0.001 [-0.010, +0.013]	+0.004 [-0.018, +0.026]	+0.003 [-0.013, +0.019]
lmetric	-0.016 [-0.037, -0.000]	-0.010 [-0.024, +0.004]	-0.012 [-0.023, -0.001]
M0 (default cost fn, tuned)	-0.022 [-0.044, -0.006]	-0.044 [-0.095, -0.010]	-0.029 [-0.052, -0.008]
round-robin	-0.584 [-0.804, -0.386]	-0.817 [-1.058, -0.530]	-0.813 [-1.080, -0.494]
default@defaults (reference)	-0.130 [-0.191, -0.075]	-0.238 [-0.357, -0.124]	-0.241 [-0.338, -0.137]

Table 37: By worker count, against π_0 (selected rows).

Policy	$N = 2$	$N = 4$	$N = 6$
M1-v2 (headline learned arm)	+0.105 [+0.060, +0.166]	+0.152 [+0.079, +0.233]	+0.167 [+0.104, +0.222]
M2-ais	+0.091 [+0.041, +0.158]	+0.151 [+0.076, +0.237]	+0.166 [+0.101, +0.223]
M1	+0.099 [+0.056, +0.161]	+0.139 [+0.065, +0.221]	+0.149 [+0.093, +0.202]
M1 unconstrained (flagged for gaming)	+0.083 [+0.038, +0.151]	+0.139 [+0.066, +0.229]	+0.151 [+0.091, +0.207]
llm-d-precise-prefix	+0.072 [+0.031, +0.128]	+0.102 [+0.041, +0.198]	+0.136 [+0.075, +0.198]
ramjet (val-best baseline)	+0.067 [+0.021, +0.141]	+0.098 [+0.044, +0.182]	+0.137 [+0.077, +0.198]
lmetric	+0.070 [+0.018, +0.148]	+0.089 [+0.031, +0.184]	+0.124 [+0.067, +0.181]
M0 (default cost fn, tuned)	+0.055 [+0.014, +0.118]	+0.047 [+0.031, +0.063]	+0.107 [+0.068, +0.150]
round-robin	-0.325 [-0.455, -0.185]	-0.388 [-0.534, -0.263]	-0.684 [-0.964, -0.397]
Policy	$N = 8$	$N = 16$	$N = 32$
M1-v2 (headline learned arm)	+0.173 [+0.106, +0.241]	+0.267 [+0.140, +0.399]	+0.265 [+0.146, +0.379]
M2-ais	+0.169 [+0.102, +0.237]	+0.266 [+0.141, +0.397]	+0.267 [+0.147, +0.381]
M1	+0.163 [+0.094, +0.232]	+0.245 [+0.133, +0.354]	+0.252 [+0.143, +0.356]
M1 unconstrained (flagged for gaming)	+0.154 [+0.090, +0.219]	+0.245 [+0.130, +0.362]	+0.254 [+0.142, +0.362]
llm-d-precise-prefix	+0.131 [+0.079, +0.188]	+0.242 [+0.121, +0.379]	+0.244 [+0.129, +0.356]
ramjet (val-best baseline)	+0.130 [+0.075, +0.191]	+0.238 [+0.124, +0.357]	+0.241 [+0.137, +0.338]
lmetric	+0.114 [+0.057, +0.180]	+0.228 [+0.116, +0.344]	+0.229 [+0.129, +0.327]
M0 (default cost fn, tuned)	+0.108 [+0.058, +0.167]	+0.195 [+0.113, +0.266]	+0.212 [+0.129, +0.292]
round-robin	-0.477 [-0.690, -0.277]	-0.675 [-0.950, -0.376]	-0.640 [-0.940, -0.322]

Table 38: By family, against tuned **ramjet** (selected rows). Each family has 2 to 4 segments, so these are descriptive; † marks the within-segment cell bootstrap. Table 39 gives the same against π_0 .

Policy	Mooncake	FAST25 conv.	FAST25 synth.
M1-v2 (headline learned arm)	+0.050 [+0.037, +0.064] [†]	+0.387 [+0.286, +0.488] [†]	+0.030 [+0.016, +0.044] [†]
M2-ais	+0.046 [+0.034, +0.059] [†]	+0.362 [+0.270, +0.453] [†]	+0.029 [+0.016, +0.042] [†]
M1	+0.041 [+0.029, +0.053] [†]	+0.365 [+0.263, +0.467] [†]	+0.033 [+0.022, +0.043] [†]
M1 unconstrained (flagged for gaming)	+0.028 [+0.017, +0.040] [†]	+0.343 [+0.245, +0.441] [†]	+0.002 [−0.007, +0.012] [†]
llm-d-precise-prefix	−0.014 [−0.019, −0.008] [†]	+0.095 [+0.088, +0.102] [†]	−0.021 [−0.030, −0.012] [†]
lmetric	−0.027 [−0.031, −0.022] [†]	−0.097 [−0.098, −0.095] [†]	−0.034 [−0.046, −0.022] [†]
M0 (default cost fn, tuned)	−0.010 [−0.014, −0.005] [†]	−0.151 [−0.155, −0.147] [†]	−0.036 [−0.038, −0.034] [†]
round-robin	−0.272 [−0.324, −0.223] [†]	−0.430 [−0.432, −0.427] [†]	−0.360 [−0.373, −0.347] [†]
default@defaults (reference)	−0.104 [−0.132, −0.077] [†]	−0.200 [−0.200, −0.200] [†]	−0.089 [−0.115, −0.064] [†]
Policy	Sessions	AgentX	
M1-v2 (headline learned arm)	+0.000 [−0.000, +0.000]	+0.037 [+0.033, +0.041]	
M2-ais	+0.002 [+0.000, +0.004]	+0.031 [+0.025, +0.034]	
M1	−0.001 [−0.002, −0.000]	+0.004 [−0.019, +0.025]	
M1 unconstrained (flagged for gaming)	−0.002 [−0.003, −0.000]	+0.015 [+0.010, +0.020]	
llm-d-precise-prefix	−0.010 [−0.015, −0.004]	+0.029 [+0.025, +0.034]	
lmetric	+0.003 [+0.001, +0.005]	+0.004 [−0.004, +0.012]	
M0 (default cost fn, tuned)	−0.012 [−0.019, −0.005]	−0.044 [−0.079, −0.010]	
round-robin	−0.935 [−1.040, −0.829]	−0.740 [−0.865, −0.562]	
default@defaults (reference)	−0.178 [−0.264, −0.088]	−0.155 [−0.219, −0.063]	

Table 39: By family, against π_0 (selected rows).

Policy	Mooncake	FAST25 conv.	FAST25 synth.
M1-v2 (headline learned arm)	+0.154 [+0.126, +0.181] [†]	+0.587 [+0.486, +0.688] [†]	+0.120 [+0.080, +0.159] [†]
M2-ais	+0.150 [+0.122, +0.178] [†]	+0.562 [+0.471, +0.653] [†]	+0.118 [+0.079, +0.157] [†]
M1	+0.144 [+0.116, +0.172] [†]	+0.565 [+0.463, +0.667] [†]	+0.122 [+0.086, +0.158] [†]
M1 unconstrained (flagged for gaming)	+0.132 [+0.104, +0.159] [†]	+0.543 [+0.445, +0.641] [†]	+0.092 [+0.057, +0.127] [†]
llm-d-precise-prefix	+0.090 [+0.065, +0.116] [†]	+0.296 [+0.289, +0.302] [†]	+0.069 [+0.034, +0.103] [†]
ramjet (val-best baseline)	+0.104 [+0.077, +0.132] [†]	+0.200 [+0.200, +0.200] [†]	+0.089 [+0.064, +0.115] [†]
lmetric	+0.077 [+0.051, +0.104] [†]	+0.103 [+0.102, +0.105] [†]	+0.055 [+0.017, +0.093] [†]
M0 (default cost fn, tuned)	+0.094 [+0.070, +0.120] [†]	+0.049 [+0.045, +0.053] [†]	+0.053 [+0.026, +0.081] [†]
round-robin	−0.168 [−0.208, −0.133] [†]	−0.229 [−0.232, −0.227] [†]	−0.270 [−0.283, −0.258] [†]
Policy	Sessions	AgentX	
M1-v2 (headline learned arm)	+0.178 [+0.089, +0.264]	+0.192 [+0.096, +0.257]	
M2-ais	+0.179 [+0.089, +0.265]	+0.186 [+0.089, +0.252]	
M1	+0.176 [+0.088, +0.261]	+0.159 [+0.065, +0.221]	
M1 unconstrained (flagged for gaming)	+0.176 [+0.088, +0.260]	+0.170 [+0.074, +0.233]	
llm-d-precise-prefix	+0.168 [+0.084, +0.251]	+0.184 [+0.089, +0.249]	
ramjet (val-best baseline)	+0.178 [+0.088, +0.264]	+0.155 [+0.063, +0.219]	
lmetric	+0.180 [+0.090, +0.266]	+0.159 [+0.067, +0.224]	
M0 (default cost fn, tuned)	+0.165 [+0.082, +0.248]	+0.111 [+0.051, +0.144]	
round-robin	−0.811 [−0.904, −0.687]	−0.656 [−0.742, −0.528]	

Table 40: By load mode, against tuned **ramjet** (top) and against π_0 (bottom) (selected rows).

Policy	Open loop (31 cells, 8 seg.)	Closed loop (15, 6)	Agentic lanes (14, 4)
M1-v2 (headline learned arm)	+0.028 [+0.003, +0.061]	+0.035 [+0.011, +0.063]	+0.037 [+0.033, +0.041]
M2-ais	+0.026 [+0.003, +0.055]	+0.037 [+0.013, +0.064]	+0.031 [+0.025, +0.034]
M1	+0.027 [+0.005, +0.056]	+0.030 [+0.006, +0.059]	+0.004 [-0.019, +0.025]
M1 unconstrained (flagged for gaming)	+0.016 [-0.003, +0.042]	+0.017 [+0.001, +0.033]	+0.015 [+0.010, +0.020]
llm-d-precise-prefix	-0.008 [-0.019, -0.000]	-0.014 [-0.021, -0.008]	+0.029 [+0.025, +0.034]
lmetric	-0.021 [-0.037, -0.006]	-0.006 [-0.016, +0.004]	+0.004 [-0.004, +0.012]
M0 (default cost fn, tuned)	-0.016 [-0.032, -0.002]	-0.024 [-0.034, -0.014]	-0.044 [-0.079, -0.010]
round-robin	-0.597 [-0.844, -0.372]	-0.631 [-0.867, -0.400]	-0.740 [-0.865, -0.562]
default@defaults (reference)	-0.098 [-0.162, -0.051]	-0.205 [-0.258, -0.154]	-0.155 [-0.219, -0.063]

Policy	Open loop (31 cells, 8 seg.)	Closed loop (15, 6)	Agentic lanes (14, 4)
M1-v2 (headline learned arm)	+0.126 [+0.065, +0.199]	+0.241 [+0.208, +0.273]	+0.192 [+0.096, +0.257]
M2-ais	+0.124 [+0.064, +0.196]	+0.243 [+0.211, +0.274]	+0.186 [+0.089, +0.252]
M1	+0.125 [+0.065, +0.195]	+0.236 [+0.203, +0.269]	+0.159 [+0.065, +0.221]
M1 unconstrained (flagged for gaming)	+0.114 [+0.056, +0.186]	+0.222 [+0.187, +0.259]	+0.170 [+0.074, +0.233]
llm-d-precise-prefix	+0.090 [+0.040, +0.157]	+0.191 [+0.145, +0.236]	+0.184 [+0.089, +0.249]
ramjet (val-best baseline)	+0.098 [+0.051, +0.162]	+0.205 [+0.154, +0.258]	+0.155 [+0.063, +0.219]
lmetric	+0.078 [+0.030, +0.146]	+0.200 [+0.140, +0.259]	+0.159 [+0.067, +0.224]
M0 (default cost fn, tuned)	+0.082 [+0.037, +0.148]	+0.182 [+0.135, +0.230]	+0.111 [+0.051, +0.144]
round-robin	-0.550 [-0.825, -0.302]	-0.426 [-0.623, -0.230]	-0.656 [-0.742, -0.528]

Table 41: Hold-out strata, against tuned **ramjet** (top) and against π_0 (bottom) (selected rows).

Policy	Unseen N 2, 16, 32 (23 cells)	Seen N 4, 8 (30)	Transforms (10)
M1-v2 (headline learned arm)	+0.034 [+0.016, +0.052]	+0.035 [+0.012, +0.061]	+0.025 [+0.007, +0.044]
M2-ais	+0.030 [+0.014, +0.046]	+0.033 [+0.013, +0.057]	+0.022 [+0.006, +0.039]
M1	+0.015 [-0.001, +0.032]	+0.022 [+0.000, +0.048]	+0.010 [+0.001, +0.018]
M1 unconstrained (flagged for gaming)	+0.012 [+0.003, +0.021]	+0.023 [+0.006, +0.043]	+0.011 [-0.000, +0.023]
llm-d-precise-prefix	+0.009 [-0.009, +0.028]	+0.004 [-0.006, +0.016]	-0.001 [-0.019, +0.018]
lmetric	-0.007 [-0.016, +0.001]	-0.007 [-0.021, +0.005]	-0.003 [-0.021, +0.015]
M0 (default cost fn, tuned)	-0.042 [-0.086, -0.012]	-0.037 [-0.085, -0.007]	-0.007 [-0.017, -0.001]
round-robin	-0.718 [-0.928, -0.470]	-0.597 [-0.753, -0.453]	-0.576 [-0.815, -0.367]
default@defaults (reference)	-0.212 [-0.317, -0.120]	-0.118 [-0.177, -0.066]	-0.091 [-0.166, -0.043]

Policy	Unseen N 2, 16, 32 (23 cells)	Seen N 4, 8 (30)	Transforms (10)
M1-v2 (headline learned arm)	+0.246 [+0.145, +0.362]	+0.152 [+0.097, +0.212]	+0.116 [+0.070, +0.183]
M2-ais	+0.241 [+0.141, +0.356]	+0.151 [+0.094, +0.213]	+0.113 [+0.065, +0.182]
M1	+0.226 [+0.137, +0.322]	+0.140 [+0.083, +0.200]	+0.101 [+0.053, +0.171]
M1 unconstrained (flagged for gaming)	+0.223 [+0.132, +0.326]	+0.140 [+0.085, +0.203]	+0.102 [+0.057, +0.171]
llm-d-precise-prefix	+0.221 [+0.119, +0.341]	+0.122 [+0.069, +0.185]	+0.090 [+0.049, +0.155]
ramjet (val-best baseline)	+0.212 [+0.120, +0.317]	+0.118 [+0.066, +0.177]	+0.091 [+0.043, +0.166]
lmetric	+0.205 [+0.111, +0.310]	+0.111 [+0.057, +0.175]	+0.088 [+0.038, +0.166]
M0 (default cost fn, tuned)	+0.170 [+0.107, +0.234]	+0.081 [+0.047, +0.126]	+0.084 [+0.041, +0.149]
round-robin	-0.605 [-0.840, -0.356]	-0.498 [-0.640, -0.364]	-0.498 [-0.732, -0.312]

Table 42: Coefficients of the v1 learned arms, θ_f (with $\theta_f \text{sd}_f$ in parentheses); $\theta_0 = -1$ throughout. M2 rank 2 also has a context term, which the table omits.

Arm	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6	θ_7
M1	+16.5 (+4.24)	-2.27 (-0.80)	+0 (+0.00)	+0 (+0.00)	-39.2 (-2.35)	+0.206 (+0.07)	+0 (+0.00)
M1-noaff	+18.9 (+4.84)	+0.996 (+0.35)	+0 (+0.00)	+0 (+0.00)	-44.2 (-2.65)	+0 (+0.00)	+0 (+0.00)
M1 + queue threshold (ablation)	+19.1 (+4.88)	+1.04 (+0.36)	+0 (+0.00)	+0 (+0.00)	-38.3 (-2.30)	+0.00905 (+0.00)	+0 (+0.00)
M2 rank 2	+16.2 (+4.16)	-0.954 (-0.34)	+0 (+0.00)	+0 (+0.00)	-34.6 (-2.07)	-0.493 (-0.16)	+0 (+0.00)
M1 continued (secondary)	+17.9 (+4.58)	+0.748 (+0.26)	+0 (+0.00)	+0 (+0.00)	-47.5 (-2.85)	-0.794 (-0.26)	+0 (+0.00)
M1 unconstrained (flagged for gaming)	+61.3 (+15.71)	+1.98 (+0.70)	-0.724 (-0.93)	-0.222 (-0.02)	-146 (-8.74)	-2.85 (-0.92)	+0.0878 (+0.19)

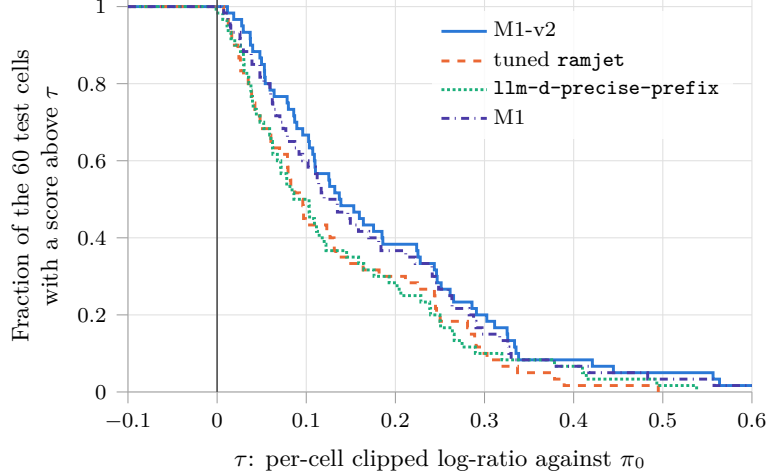


Figure 14: Performance profiles over the 60 test cells: the fraction of cells on which a policy’s clipped log-ratio against π_0 exceeds τ . Cells are not independent, so the profiles are descriptive.

L Live-run details

This appendix gives the details behind Section 11: the load-generator audit and the smoke test, the parts of the registered plan the main text summarizes, what ran, per-cell noise and discordances, the per-cell departures from simulation, the remaining registered secondaries, and the tables of paired deltas, goodput and engine descriptives.

L.1 Load-generator equivalence

The CPU-only audit of Section 11.1 covered every family and load mode the generator supports: Mooncake, FAST25 conversation and synthetic, and generated sessions, each in open and closed loop. For each it found the same request set and order, bit-identical open-loop arrivals including the replicate re-draws and speedup, exact prompt and output lengths with output length forced, the same prefix-sharing structure (a one-to-one map between the KV blocks the router and vLLM see and those of replay), the same session membership and think delays, the same closed-loop admission rule, and the same warm-up and scoring window. Replaying replay’s own per-request latencies through AIPerf and a stub server moved windowed goodput by at most 0.06%. Two non-equivalences remain. AgentX has no live load generator, because its recycled lanes need dependency, join and recycling semantics that neither AIPerf nor its prior fork implements, so the live check covers Mooncake, FAST25 and sessions only. And live latency is measured at the client, so it includes frontend, transport and streaming overhead that replay and the reference latency E_0 do not; this biases every policy slightly toward “not good” at the slowdown boundary.

L.2 The smoke test

One GPU smoke test ran π_0 on one Mooncake train cell, open loop at load L2 with four workers, on one 8×H100 node, and compared it with the cached replay of the same cell and replicate.

- **Fidelity of the workload.** All 4,002 requests arrived with exact prompts, prompt lengths and forced output lengths; AIPerf’s dispatch was within 2.5 ms of the schedule at the 99th percentile, with no drift.
- **What agreed.** Windowed goodput was 3.4454 live against 3.4535 in replay (ratio 0.9976; 2113 against 2118 good of 2376 in-window requests), the prefix-cache hit rate 0.360 against 0.359, and the end-to-end latency distributions differed by a Kolmogorov–Smirnov statistic of 0.022.
- **What differed, in offsetting directions.** Live TTFT was higher (p50 176 against 130 ms) and live decode faster (per-request ITL p50 22.9 against 25.6 ms). Idle single-request calibration on two nodes agreed within 2.2%; in it, decode steps were 7 to 10% faster than AIS’s, and prefill 5 to 10% slower above 4K tokens.

L.3 Further plan details

The registered plan of Section 11.2 also fixed the following.

- **Deployment.** The deployment is the smoke test’s. A second worker count was not planned: the two-node path is scripted but untested and would double the cost of every run.
- **Run order.** The order of the seven runs within a cell is randomized by a fixed seed.
- **Validity.** A run counts only if every request matched its intended prompt, prompt length and forced output length and the load generator exited cleanly.
- **Secondaries.** The token-weighted goodput, goodput under a live-measured E_0 , and TTFT, ITL, prefix hits and worker shares.
- **Simulated counterparts** are the frozen test pass’s own records for the same cells, policies and replicate; no test cell is replayed again. In those records M1-v2 minus **ramjet** is +0.069, +0.122, +0.141, +0.470, +0.011 and −0.0004 on the six cells in the order of Section 11.2.

L.4 What ran

All 42 planned cell runs (six cells, seven runs each) and all four applicable idle calibration runs are valid, so no cell is short. Two cell runs failed the validity rule at their first attempt: after every request had completed, AIPerf exited non-zero in its shutdown phase, because a slow export to a shared file system tripped its heartbeat watchdog. Each was re-run once under the registered rule, and the rerun is used. The first attempts are descriptive only: π_0 ’s second run on FAST25 conversation scored 0.886 good requests per second (0.893 in the rerun), and round-robin on Mooncake open L2 scored 3.085 (3.087). Jobs submitted after the first failure raised the watchdog’s missed-heartbeat threshold from 3 to 60; the setting does not touch the request schedule, prompts, lengths, headers, timing or scoring. Some comparisons span two jobs, and so two nodes: both noise pairs on FAST25 conversation and sessions, round-robin against π_0 and against **ramjet** on Mooncake open L2, and M0 and round-robin against **ramjet** on sessions. Every other comparison is within one job. Two jobs were scheduled through a different queue than planned, on the same node type and with the same inputs. Two properties of the workload carry over from replay unchanged:

Table 45: Live GPU validation: paired deltas, live (simulated at $k = 0$), as the clipped log-ratio of the headline. Top: against π_0 , the mean of its runs in the same job. Bottom: against tuned **ramjet**. One run per cell and policy; live noise $\sigma_{\text{live}} = 0.0090$ per run.

Policy	Mooncake open L2	Mooncake open L3	Mooncake closed L3	FAST25 conv. open L2	FAST25 synth. open L2	Sessions open L2	Mean live [range]	Mean sim
tuned ramjet	+0.023 (+0.037)	+0.086 (+0.102)	+0.147 (+0.135)	+0.171 (+0.056)	+0.096 (+0.084)	+0.016 (+0.086)	+0.090 [+0.016, +0.171]	+0.084
M1-v2	+0.100 (+0.106)	+0.276 (+0.224)	+0.280 (+0.276)	+0.833 (+0.526)	+0.131 (+0.095)	+0.016 (+0.086)	+0.273 [+0.016, +0.833]	+0.219
M1	+0.095 (+0.085)	+0.244 (+0.225)	+0.267 (+0.244)	+0.771 (+0.537)	+0.140 (+0.131)	+0.016 (+0.086)	+0.256 [+0.016, +0.771]	+0.218
M0	+0.025 (+0.029)	+0.075 (+0.059)	+0.130 (+0.140)	+0.100 (+0.043)	+0.012 (+0.038)	+0.011 (+0.084)	+0.059 [+0.011, +0.130]	+0.066
round-robin	-0.139 (-0.106)	-0.383 (-0.297)	-0.262 (-0.193)	-0.010 (-0.132)	-0.206 (-0.165)	-0.770 (-1.099)	-0.295 [-0.770, -0.010]	-0.332

Policy	Mooncake open L2	Mooncake open L3	Mooncake closed L3	FAST25 conv. open L2	FAST25 synth. open L2	Sessions open L2	Mean live [range]	Mean sim
M1-v2	+0.077 (+0.069)	+0.190 (+0.122)	+0.133 (+0.141)	+0.663 (+0.470)	+0.035 (+0.011)	+0.0002 (-0.0004)	+0.183 [+0.0002, +0.663]	+0.135
M1	+0.072 (+0.048)	+0.159 (+0.123)	+0.119 (+0.109)	+0.600 (+0.481)	+0.043 (+0.046)	-0.0002 (-0.001)	+0.166 [-0.0002, +0.600]	+0.134
M0	+0.001 (-0.008)	-0.011 (-0.043)	-0.017 (+0.005)	-0.070 (-0.014)	-0.084 (-0.046)	+0.001 (-0.002)	-0.030 [-0.084, +0.001]	-0.018
π_0	-0.023 (-0.037)	-0.086 (-0.102)	-0.147 (-0.135)	-0.171 (-0.056)	-0.096 (-0.084)	-0.016 (-0.086)	-0.090 [-0.171, -0.016]	-0.084
round-robin	-0.162 (-0.143)	-0.469 (-0.399)	-0.410 (-0.328)	-0.181 (-0.188)	-0.303 (-0.249)	-0.780 (-1.099)	-0.384 [-0.780, -0.162]	-0.401

Table 46: Kendall τ_K -b between simulated and live deltas against π_0 per cell, at the registered $k = 0$ and in the three registered sensitivities, and the discordant policy pairs. “Within noise”: the live difference is at most $2\sqrt{2}\sigma_{\text{live}}$.

	Mooncake open L2	Mooncake open L3	Mooncake closed L3	FAST25 conv. open L2	FAST25 synth. open L2	Sessions open L2
τ_K -b, sim $k = 0$ (registered)	0.87	0.87	0.87	0.87	1.00	0.87
τ_K -b, sim mean of $k = 0..2$	0.87	1.00	0.87	0.87	1.00	0.73
τ_K -b, sim lag 10 ms	0.87	1.00	1.00	1.00	0.87	0.97
τ_K -b, sim lag 50 ms	0.87	1.00	1.00	0.87	0.87	0.87
Discordant pairs ($k = 0$)	ramjet (tuned)/M0 (within noise)	M1-v2/M1	ramjet (tuned)/M0 (within noise)	M1-v2/M1	none	ramjet (tuned)/M1-v2 (within noise)

the closed-loop cell admits requests by concurrency, not by an arrival schedule, and in FAST25 synthetic seven groups of two requests share an arrival instant, whose order within the group the live load generator cannot enforce.

L.5 Live noise and ranking detail

The two runs of π_0 in each cell differ by a clipped log-ratio of -0.010 , -0.016 , $+0.023$, $+0.008$, -0.004 and -0.006 (cells in the order of Table 47); these are the r of $\sigma_{\text{live}} = \sqrt{r^2/2} = 0.0090$. On FAST25 conversation and sessions only one default ran in the same job as each policy, so there a delta against π_0 has the larger standard deviation, 0.013. The six live deltas against π_0 that do not exceed twice their standard deviation are round-robin on FAST25 conversation, M0 on FAST25 synthetic and the four tuned policies on sessions.

Table 45 gives every paired delta, live and simulated, and Table 46 the Kendall τ_K -b between them. Of the five per-cell discordances, three are within live noise ($2\sqrt{2}\sigma_{\text{live}}$); the other two put M1-v2 ahead of M1 live, on Mooncake open L3 ($+0.032$ live against -0.001 simulated) and on FAST25 conversation ($+0.062$ against -0.011).

L.6 Where live and simulation differ, per cell

Table 47 gives windowed goodput live and simulated, and Table 48 the live-minus-simulated difference of every paired delta, with the descriptive departure rule of Section 11.3 marked.

Table 47: Windowed goodput (good requests per second), live / simulated at $k = 0$. Live π_0 is the mean of its two runs.

Policy	Mooncake open L2	Mooncake open L3	Mooncake closed L3	FAST25 conv. open L2	FAST25 synth. open L2	Sessions open L2
π_0	3.547 / 3.544	3.582 / 3.733	3.948 / 4.034	0.896 / 1.249	1.894 / 1.962	8.249 / 7.691
tuned ramjet	3.631 / 3.678	3.902 / 4.135	4.575 / 4.617	1.068 / 1.321	2.086 / 2.134	8.357 / 8.385
M1-v2	3.921 / 3.939	4.720 / 4.671	5.225 / 5.318	2.072 / 2.115	2.159 / 2.157	8.359 / 8.381
M1	3.902 / 3.860	4.573 / 4.677	5.155 / 5.148	1.947 / 2.137	2.178 / 2.235	8.356 / 8.380
M0	3.636 / 3.649	3.859 / 3.961	4.497 / 4.641	0.995 / 1.303	1.917 / 2.037	8.365 / 8.367
round-robin	3.087 / 3.187	2.442 / 2.774	3.037 / 3.326	0.891 / 1.095	1.541 / 1.663	3.831 / 1.757

Table 48: Live minus simulated paired delta, for every delta of Table 45. *: a difference larger than twice the standard deviation of the live delta (a descriptive rule, not registered).

Comparison	Mooncake open L2	Mooncake open L3	Mooncake closed L3	FAST25 conv. open L2	FAST25 synth. open L2	Sessions open L2
tuned ramjet vs π_0	-0.014	-0.017	+0.012	+0.114*	+0.012	-0.070*
M1-v2 vs π_0	-0.005	+0.052*	+0.004	+0.307*	+0.036*	-0.070*
M1 vs π_0	+0.010	+0.019	+0.023*	+0.234*	+0.009	-0.070*
M0 vs π_0	-0.004	+0.015	-0.010	+0.058*	-0.026*	-0.073*
round-robin vs π_0	-0.033*	-0.086*	-0.069*	+0.121*	-0.041*	+0.329*
M1-v2 vs ramjet	+0.008	+0.068*	-0.008	+0.193*	+0.024	+0.001
M1 vs ramjet	+0.024	+0.035*	+0.011	+0.120*	-0.003	+0.0004
M0 vs ramjet	+0.009	+0.032*	-0.022	-0.056*	-0.038*	+0.003
π_0 vs ramjet	+0.014	+0.017	-0.012	-0.114*	-0.012	+0.070*
round-robin vs ramjet	-0.019	-0.070*	-0.082*	+0.007	-0.053*	+0.319*

- **FAST25 conversation.** π_0 , **ramjet**, M0 and round-robin reach 0.72 to 0.81 of their simulated goodput, M1 and M1-v2 0.91 to 0.98. Every gain over π_0 is larger live (M1-v2 +0.833 against +0.526; **ramjet** +0.171 against +0.056), which widens M1-v2’s lead over **ramjet** (+0.663 against +0.470). It also nearly erases round-robin’s deficit against π_0 (-0.010 live against -0.132, within live noise), because π_0 loses the most live. vLLM preempted 18 to 24 requests per run under π_0 , **ramjet** and M0 and 12 under round-robin, against 1 to 6 under M1 and M1-v2, and the non-learned policies’ median TTFT rose 1.43 to 1.76 times over simulation, the learned ones’ 1.08 to 1.13 times.
- **Sessions.** π_0 runs at 1.07 times its simulated goodput, the four tuned policies at 0.997 to 1.000. Live, 97.8 to 98.4% of π_0 ’s in-window requests are good, against 91.6% in simulation; the tuned policies serve 99.9 to 100% in both, at ceiling. So every tuned policy’s simulated gain over π_0 , +0.084 to +0.086, is only +0.011 to +0.016 live, inside twice the standard deviation of a live delta (0.026): on this cell the simulated gain over π_0 is not reproduced beyond live noise. The registration included this cell to check the default and round-robin gaps live; the default half of that check fails. Round-robin reaches 2.18 times its simulated goodput; it is still last by far, but its deficit against π_0 shrinks from -1.099 (the clip at $\ln 3$) to -0.770.
- **Mooncake and FAST25 synthetic.** Round-robin’s deficit against π_0 is larger live on all four cells, by 0.033 to 0.086. The eleven other departures on these cells are 0.023 to 0.082 in size (Table 48).

L.7 Other registered secondaries

- **Live reference latency.** Re-scoring every run against an E_0 fitted on the live idle runs (72 requests from four jobs) instead of AIS’s gives a pooled τ_K -b of 1.00, and M1-v2 minus **ramjet**

Table 49: Engine-level descriptives per policy: median over the six cells of the live / simulated ratio of TTFT and of per-request mean ITL, and the largest per-worker request share over the cells, live / simulated.

Policy	TTFT p50	TTFT p90	Mean-ITL p50	Mean-ITL p90	Max worker share, live / sim
π_0	1.39	1.12	0.99	1.02	0.28 / 0.29
tuned ramjet	1.37	1.09	0.97	1.02	0.33 / 0.34
M1-v2	1.14	1.09	0.90	0.99	0.82 / 0.81
M1	1.22	1.10	0.93	1.01	0.81 / 0.78
M0	1.35	1.09	0.99	1.05	0.27 / 0.28
round-robin	1.32	1.22	0.97	1.10	0.25 / 0.25

is positive on all five informative cells, mean +0.241. So the ranking and the sign do not hinge on the simulator’s E_0 . The per-node fit has only two complete cells, since only some jobs ran an idle calibration; there τ_K -b is 0.87 and the sign holds on both.

- **Engine timing** (Table 49). Live TTFT is longer than AIS’s (median ratios over the cells 1.14 to 1.39 at p50, 1.09 to 1.22 at p90), and the median request decodes faster (0.90 to 0.99; at p90 0.99 to 1.10), the offsetting pattern of the smoke test.
- **Prefix-cache reuse.** Each successful request’s usage record carries the server’s count of cached prompt tokens. The live hit rate computed from it (cached over prompt tokens) differs from replay’s reuse fraction by at most 0.015 in each of the 42 runs, and by at most 0.010 in 40, so routing reproduced replay’s cache reuse on hardware for every policy and cell. The registered descriptive used vLLM’s own hit counter instead, which is comparable in only 14 runs, all with little queueing; there it is within 0.009 of replay. By our reading of the vLLM 0.24.0 scheduler, the counter records a query each time a waiting request is retried, and preemption re-queries too, so queueing inflates it.
- **Load generation.** On the 35 open-loop runs, requests left at most 2.6 ms late at the 99th percentile (15.2 ms at most). In 16 runs, 18 requests failed in AIPerf because an output of 1 to 3 tokens carried no text; the scorer counts them not good, as registered.

M Reproducibility

The campaign’s durable record is the directory `notes/learned-routing/` on the Dynamo branch `rupei/learned-routing`, whose sanitized copy `rupei/learned-routing-public` is public. `READ ME.md` describes the campaign and its state, `REPRODUCE.md` rebuilds it step by step on a clean Linux x86_64 machine, and `campaign/` mirrors every small file the steps need: facts, cells, audits, run summaries, calibration scripts and the AgentX base manifests. Bulky artifacts (traces, workload replicates, the replay cache) stay in the campaign root and are regenerated from pinned public sources. Off the original host, git and the public sources are enough for the steps `REPRODUCE.md` covers. This appendix lists the identifiers; the procedure is in `REPRODUCE.md`.

M.1 Branches

The campaign’s code and record live on three branches of the Dynamo repository.

- `rupei/learned-routing`: the policies (`learned-choice` and `sticky-session` in the builtin catalog, and the `seed` parameter on `dynamo-default-cost-fn`), the harness (`lr-eval`, `lr-train`, `lr-report` and the remote CPU lane), the live lane, and the campaign notes

under `notes/learned-routing/`, including this paper. It starts from the head of the router-policy pull-request stack (#15450 and #15453), which already runs catalog policies in offline replay and passes the device-tier overlap to them.

- `rupei/learned-routing-lag`: the router-state lag knob for offline replay, built separately so that the tuning build and its result cache stay untouched.
- `rupei/learned-routing-ais`: feature set `v3` and the research input for a request’s modeled prefill time, also on its own build, for the AIS-informed league.

Every stage’s facts file records the worktree head it ran at. Only the first branch is published, as the sanitized copy `rupei/learned-routing-public`, in which internal commits appear as position labels; the lag and AIS branches are not, so the lag part of the robustness pass and the AIS-informed league can be checked against the mirrored facts but not yet rebuilt from public sources. `REPRODUCE.md` rebuilds the campaign to its calibrated, frozen cells. Its section 15 describes how the later stages ran (the pilot, the main tuning, the test pass, the robustness pass and the live runs): the commands and frozen inputs each used, the facts file each wrote, and which of their records the mirror holds. Their numbers are re-derived from the records by the analysis scripts that the final report’s reproduction section names; those scripts and the bulky records stay in the campaign root, and the mirrored facts carry every resulting number.

M.2 Environment

Python, Rust	CPython 3.12.3; rustc 1.96.1 (pinned by <code>rust-toolchain.toml</code>); maturin 1.15.0
Simulator	<code>aisimulate</code> 0.13.0.dev202609300000000061 (wheel) and the matching <code>aisimulate-core</code> crate
Dynamo	<code>ai-dynamo</code> and <code>ai-dynamo-runtime</code> 1.6.0, editable from the worktree
Bindings	built with feature <code>ais-forward-pass</code> (required for AIS engine timing); <code>_core.abi3.so</code> SHA-256 1d67c131..., build ID 6955b0ee...
Lag-check bindings	separate build, <code>.so</code> 90464b9b..., build ID a6c6b04e...
Harness	package <code>learned_routing</code> , harness version <code>lrh-4</code>
Python packages	<code>cma</code> 4.5.0, <code>numpy</code> 2.5.3, <code>scipy</code> 1.18.1, <code>pandas</code> 3.0.6, <code>matplotlib</code> 3.11.2
Host	Ubuntu 24.04, glibc 2.39, 24 cores; local replays share a pool of 20 slots

The build ID is a hash of the bindings’ shared object together with the simulator version, and it is part of every result-cache key. A rebuild almost always yields a new ID, because the shared object embeds build-tree paths, so cached results from another build are never reused by accident.

M.3 The rebuild procedure in brief

Table 50 condenses `REPRODUCE.md`. “Verified” there means the step was re-run while the notes were written, on the campaign host or in a fresh private root.

M.4 Frozen inputs

- Engine configuration `config/engine.json`, SHA-256 7b5ff164...: the single source of truth for model, hardware, backend, parallelism, context length and engine arguments.
- Cells `lr-cells-v5`: test file `cells/test.jsonl`, SHA-256 7b998b8e...; split manifest, SHA-256 0e84edf1.... The freeze record `facts/test_freeze.json` also hashes the content of every test cell and all 39 test traces (2,130,865,265 bytes).
- Source traces, pinned by SHA-256 in `traces/MANIFEST.json` (b2616d0d...).
- The headline test, pre-registered in `facts/HEADLINE_TEST.json` before any training.

Table 50: Sections of `REPRODUCE.md` and what each establishes.

§	Step	What it establishes, and how it was checked
1	Branch and SHAs	worktree at the snapshot commit; base is an ancestor
2–3	Environment, bindings	pinned versions; the KV-capacity estimate (18,863 blocks) and a one-request replay whose TTFT equals the AIS value prove AIS timing is in effect
4	Harness and tests	Rust policy tests (127 pass, 2 pre-existing ignored) and harness tests
5	Campaign root	<code>config/engine.json</code> copied from git and checked by SHA-256; every <code>DYN_ROUTER_*</code> variable unset, since replay workers inherit them and they change policies without changing the cache key
6	Traces	source traces regenerated byte-identically (106 of 106 files, verified)
7	Cells	frozen cells <code>lr-cells-v5</code> installed from git, not regenerated
8	AgentX lowering	six base manifests from git; lowering checked field for field by an independent re-implementation for all 82 plays and 6 transforms
9	Derived traces	every trace the frozen cells reference regenerated byte-identically (34/34 train, 14/14 validation, 60/60 test cells, verified); then a parity smoke against a mirrored reference (4 identical, 0 different)
10	Calibration	mirrored driver scripts; every load level follows from the mirrored load curves (171/171 levels and 108/108 cell loads, verified)
11	Frozen test set	SHA-256 of <code>cells/test.jsonl</code> and of every test trace
12	Replicates	the replicate protocols, policy seeds and the noise rule
13	Entry points	<code>lr-eval</code> , <code>lr-train</code> , <code>lr-report</code> ; the remote bundle flow and the cluster CPU lane
14	Gate and headline	the pilot’s review (its gate) and the pre-registered headline test
15	Later stages	listed as not yet run when the notes were written (see above)
16–17	Costs, mirrored files	measured costs; the small files that used to live only in the campaign root

- The single test pass first verified all 149 freeze hashes, and `facts/finalists.json` was frozen (SHA-256 `e0bae29d...`) before the first test record.

M.5 Determinism

Replay is deterministic: the same policy on the same cell and replicate reproduces every per-request row. The campaign relies on this in four places.

- **Seeds.** The shipped default router broke ties with an unseeded random number generator, so identical runs of one Mooncake cell gave different goodput (2.8934 against 2.8290 for the catalog policy). The branch adds a `seed` parameter. The harness passes seed $k + 1$ to the seeded policy types (`dynamo-default-cost-fn`, `learned-choice`, `sticky-session`); every other type takes no seed.
- **Replicates.** Variation comes from workload replicates shared by every policy: replicate k permutes arrivals logged at the same instant (protocol `crn-order-v1`), re-draws each session’s first arrival inside its logging slot (`crn-spread-v1`, for traces logged on a grid), or permutes recycled AgentX copies.
- **Parity with the default.** `learned-choice` at $\theta = -e_0$, with the tie-break that replays the default picker’s random stream, chose the same worker as the seeded `dynamo-default-cost-fn` on every request of 88 of 88 (cell, replicate) pairs, and again on 22 of 22 after the bindings were rebuilt. With the tie-break used in training (`one_draw`) individual ties resolve differently, but goodput did not differ beyond noise.
- **Re-runs and resumes.** An independent calibration audit re-ran replicates 0 and 1 of 11 cells for π_0 and round-robin and matched the cache on 44 of 44 records, every field and every per-request byte; new replicates run twice were identical on 44 of 44. A harness audit killed

`lr-train` three times by process ID and the resumed runs were bit-identical, including their continuation.

Results are cached under a SHA-256 of the canonical policy, the cell identifier and content, the replicate and its protocol, the harness version and the bindings build ID. One residual nondeterminism is harmless: replay orders its per-request output by random request identifiers, so summary means can differ in the last bits ($\sim 10^{-15}$ relative); the harness recomputes every metric from a sorted copy of the rows.

M.6 Cross-host parity

Replay calls the host’s math library, so bit-exact results require the same OS and C library image. Most tuning ran on a pool of shared CPU nodes, one run per node, from a self-contained bundle holding the exact local shared object, the harness, the traces and the cells. Before any node image was used, a parity smoke compared its records with the workstation’s.

- **Lane validation.** Ten comparisons on AMD EPYC 7702P and EPYC 9654P nodes (Ubuntu 24.04.5, glibc 2.39, CPython 3.12.3) were all identical: 6,312 remote records with no field differences, and 48,901,384 per-request rows byte-identical. They covered seven cells (Mooncake open and closed loop, sessions, lowered AgentX, including one AgentX cell at $N = 16$), six replicas of the calibration reference batch, and two short tuning runs.
- **Tuning numerics.** With default settings, `numpy` and OpenBLAS dispatch AVX-512 kernels on the EPYC 9654P, and the CMA-ES state differed from the workstation’s in the last bits from the first generation. The node scripts pin the dispatch (`OPENBLAS_CORETYPE=Haswell`, `OPENBLAS_NUM_THREADS=1`, `NPY_DISABLE_CPU_FEATURES`); with the pins, a 60-generation check matched the workstation on every generation on both CPU types, and the remote tuning smoke runs wrote byte-identical histories, best points and policy files.
- **Test pass.** Tasks that ran twice, once in a first wave and again after a timeout fix, were bit-identical on different nodes in every field and per-request byte.
- **Pilot and main tuning.** The 384 records of the pilot’s 16 first-generation candidates on the 12 subset cells were identical locally and on the cluster. A node image added during the main tuning passed its smoke (28 of 28 records, 186,548 rows) before it ran any tuning.
- **Lag build.** The separate lag-check build was validated on five node images before the robustness pass (the first: 192 of 192 records, 1,214,112 rows).

M.7 Entry points and stage facts

Three command-line tools carry every number: `lr-eval` evaluates policies on cells with replicates, through the slot pool and the cache; `lr-train` runs Algorithm 2; and `lr-report` builds paired tables with segment-bootstrap intervals and refuses to pool records from different builds or cell contents. Every long job is resumable and takes `--max-wall-seconds`. A policy is one JSON or YAML object, and the tuned result `best_policy.yaml` is itself a valid policy for `lr-eval`.

Each stage writes one facts file under `facts/` and one line in the append-only `facts/STATE.md`: setup, the three build stages and their fixes, AgentX lowering, noise, calibration and its fix, the test freeze, the pre-registered headline test, the cluster lane, the pilot, the gate, the preparation of the main tuning (`facts/phase2_prep.json`), the lag patch, the AIS features (`facts/ais_features.json`), the tuning waves (`facts/tierA.json`, `facts/tierBC.json`) and the mid-tuning fix (`facts/phase2_mid_fix.json`), the SLO-transfer constants (`facts/sla_transfer_spec.json`),

the frozen finalists (`facts/finalists.json`), the single test pass (`facts/test_results.json` with its `report_must_state` wording), the robustness pass (`facts/robustness.json`), the secondary metrics (`facts/secondary_metrics_test.json`), the in-class heuristic diagnostic (`facts/fairness_in_class_heuristic.json`), the live lane (`facts/live.json`, `facts/live_smoke.json`, `facts/live_loadgen_parity.json`), the live finalist runs (the registration `facts/live_plan.json` and the analysis `facts/live_results.json`) and this paper’s derived numbers (`facts/paper_draft.json`, `facts/paper_final.json`). Every audit report is under `audits/`, and the final report is `report/REPORT.md`; its live section is also `report/LIVE.md`. The live analysis is reproducible from the raw load-generator exports: `runs/live/finalists/scripts/live_results.py` verify re-scores every live run and requires identical scores, and `build` re-runs the registered analysis and writes the live facts, `report/LIVE.md` and the report’s table fragment, from which `make tables` copies the live tables of Section 11.

Footprint. On 2026-10-03 the campaign root held 95 GB of traces and 247 GB of run outputs, mostly workload replicates (183 GB) and the result cache (37 GB); all of it regenerates from the steps above. Compute per stage is in Table 34.

M.8 Building this document

The sources are in `notes/learned-routing/paper/`; `make` there runs `latexmk` with `pdflatex` and BibTeX and writes `build/main.pdf`. Most result tables are not typed by hand. The campaign’s final report builds its tables from records the study had already written (the single test pass, the robustness runs and the stages’ facts files), without any new replay, and `make tables`, given the campaign root, copies their cells unchanged into `generated/`, with the row and column labels this paper uses (Appendix N). `make figures` draws every results figure in `generated/` from the report’s data, with the same labels as the tables. Every number in the text carries a source comment (`% src:`) naming the campaign file it comes from. Items that were open while drafting carried a boxed PENDING tag; none remains.

N Names used in this paper and the campaign record

The study’s campaign record (Appendix M) identifies its registered decisions, literature lessons, stages and runs by short labels. The paper uses descriptive names instead. Table 51 maps each name to the record’s label, so a reader can find the registration, lesson or run behind a statement. Amendments are the dated decisions recorded in the record’s contract during the study; lessons are the ranked findings of the study’s literature review. Rows whose last column is empty are not referred to in the text and are listed so that the numbering is complete.

Table 51: Names used in this paper and the campaign record’s identifiers. Amendments (A_n) are in the record’s contract, lessons (LR- nn) in its literature review; a decimal marks an item of an amendment.

Name in this paper	Record identifier	Where
<i>Registered decisions (amendments to the study’s contract)</i>		
The replicate rule (paired workload replicates replace identical re-runs, which are deterministic) and the pilot’s two-clause noise rule	A1	Sections 4.3 and 8.2 and Appendix H

Name in this paper	Record identifier	Where
The comparison against the validation-selected best baseline	A2.1	Section 8
The service objective in this study (an ITL bound and an end-to-end slowdown bound against E_0 ; no TTFT bound)	A2.2	Section 3.3
Windowed goodput	A2.3	Section 3.3
Baselines evaluated as implemented, by design, with a brief sanity check	A2.5	Section 7 and Appendix B.6
AgentX replay through the study’s own lowering, with play recycling	A3	Appendix D.2.1
Duplicated trace copies	A4	Appendix D.2
Router-state lag as a robustness check, re-run on the test cells	A5.1	Section 8.7
Worker sets differ only in their count	A5.2	Section 3
How fixes found during the study reach the upstream repositories	A6	
Device-tier KV overlap passed to catalog policies in replay	A7	Table 21 and Appendix M
The repository mirror of the campaign record	A8	Appendix M
The remote CPU lane and its cross-host parity	A9, A10	Appendix M.6
Informed starts for M1’s restarts	A11.1	Section 6.4 and Appendix F.5
The concentration audit, and the sign constraint registered in advance as its remedy	A11.2	Sections 6.5 and 10
$N = 6$ in both validation and test, reported as selection-exposed	A11.3	Sections 8.1 and 9.5
Breadth-first scheduling of the main tuning on the remote lane; the lag patch in a separate build	A11.4, A11.5	Appendices I and M
M1 restricted to its default-router start (a secondary test finalist)	A12 (“M1-default-init”)	Sections 8.6 and 9.7
The live GPU check	A13	Section 11
The alternative SLO definitions	A14	Sections 8.7 and 9.9
M1-noaff (session coefficient pinned at 0)	A15	Section 5.7
The secondary AIS-informed league	A16	Appendix J
The extended-feature arm M1-v2	A17.1	Section 5.7
The headline-arm rule (the learned arm with the best score on fresh validation replicates, the rule that also picks the best baseline)	A17.2	Section 6.6
The order in which arms would have been cut had compute run short	A17.3	
The public-content rule (no internal infrastructure names in public material)	A18	
Token-weighted goodput (secondary)	A19	Section 10.5
The live finalists, registered before any finalist live job	A20	Section 11.2
The <code>lmetric</code> port fixed after the study froze its results	A21	Section 7.1; the footnote at each headline statement

Name in this paper	Record identifier	Where
<i>Literature lessons</i>		
Windowed goodput scored by a clipped mean log-ratio	LR-01	Section 3.3
A noise floor from replicates and workload segments, not from repeats	LR-02	Sections 4.3 and 8.2
Common random numbers within each CMA-ES generation	LR-03	Section 6.3
Paper-faithful baselines (not adopted: the baselines are evaluated as implemented)	LR-04	Section 7
Search spaces without flat directions; a suggested band of 10–30% of decisions changed by the initial step; named context sources	LR-05	Section 6.2 and Appendix F.2
Context and set-relative features that stay stable across N	LR-06	Section 5.2
The strongest heuristics inside M1’s class, and restarts started from them	LR-07	Sections 5.2 and 6.4
SLO re-scoring at several scales (one TTFT threshold per family fails across 1K–128K prompts)	LR-08	Sections 3.3 and 8.7
Sessions released causally; strata by load mode	LR-09	Sections 3.2 and 8.5
Selection on validation over restarts, the same for baselines	LR-10	Section 6.6
Segments as the unit of the pre-registered headline test; the intersection-union test; the probability-of-improvement criterion; performance profiles	LR-11	Section 8.4 and Figure 14
Calibration per worker count, and the cache-pressure analysis	LR-12	Section 4.5 and Figure 12
The worker-share cap and the other concentration flags	LR-13	Section 10 and Table 15
The reading rule for the robustness checks (staleness and timing)	LR-14	Section 8.7
The drift check before M2, warm starts, and nest-level features for M3	LR-15	Appendices E.10 and F.8
<i>Stages</i>		
The pilot	the pilot stage (facts/pilot.json)	Appendix G
The pilot’s review	the gate (facts/gate.json)	Appendices G and H.9
The main tuning	phase 2	Section 6
The first round of tuning (the ported baselines, M0, and M1 with its sign-constrained re-run)	tier A (facts/tierA.json)	Section 6.7
Later rounds: M1 continued, M1-noaff, M1-v2 and M2 rank 2; both arms of the queue-threshold ablation; the AIS-informed league	tier B; tier C; tier AIS (facts/tierBC.json)	Section 6.7
The first, unconstrained M1	the tier-A M1	Section 10
The audit and fix midway through tuning	the phase-2 midpoint (phase2-mid)	Section 10 and Appendix H.9

Name in this paper	Record identifier	Where
The single test pass	the Select+Test stage (“test stage”)	Section 8
The audit after the test pass (a completeness critic and three refuters)	the final audit (<code>audits/final/</code>)	Section 8.8 and Appendix H.9
The live GPU check	the live phase (A13, A20)	Section 11
<i>Tools, cell set and runs</i>		
The frozen cell set	<code>lr-cells-v5</code>	Section 4.6 and Appendix M
The tuning harness; the evaluation and report tools	<code>lr-train</code> ; <code>lr-eval</code> , <code>lr-report</code> (harness version <code>lrh-4</code>)	Section 6 and Appendix M
“restart r , gen. g , CMA-ES mean” or “..., best-so-far sample”	run key <code><run>-s<r> g<g></code> mean or <code>best_so_far</code> , kept in a <code>% run key:</code> comment on each row of the generated tables	Table 35
Run prefixes of the learned arms	<code>m1c</code> (M1), <code>m1</code> (M1 unconstrained), <code>m1cont</code> , <code>m1noaff</code> , <code>m1v2</code> , <code>m2r2</code> , <code>m0</code> (M0)	
Run prefixes of the ablation and the AIS-informed league	<code>ablam1</code> , <code>ablabase</code> ; <code>m0ais</code> , <code>m1ais</code> , <code>m2ais</code> , <code>llmdobm</code>	
<i>Row labels in tables and figures</i>		
M1	“M1 (= A12 M1-default-init)”	Table 11 and Figure 3
M1 unconstrained (flagged for gaming)	“M1 unconstrained (gaming-flagged)”, “(tier A, gaming-flagged)”	Tables 11 and 15
M1-noaff; M1-v2	“M1-noaff (A15)”; “M1-v2 (A17.1)”	Table 11
M1 or ramjet + queue threshold (ablation)	“(abl. a)”, “(ablation a)”	Table 11 and Figure 3
SLO: ITL bound only; SLO: slowdown bound only; SLO: ITL + length-scaled TTFT; SLO: ITL + absolute E2E bound	“A14 <code>itl_only</code> ”, “A14 <code>e2e_only</code> ”, “A14 <code>ttft_len_itl</code> ”, “A14 <code>abs_e2e_itl</code> ”	Table 13 and Figure 13
Token-weighted goodput (secondary)	“A19 good tokens (secondary)”	Table 13 and Figure 13
Worker with no good request	“dump worker”	Table 15
round-robin; <code>lmetric</code> ; sticky-session bounded@defaults, sticky-session hard@defaults; <code>llm-d-optimized-baseline@defaults</code> (throughput)	“round_robin”; “ <code>lmetric</code> (port)”; “sticky-bounded@defaults”, “sticky-hard@defaults”; “llm-d-optimized- baseline@throughput”	Figure 3 and Table 35
Test segments, for example Mooncake w3, AgentX T1, FAST25 synth. x0, sessions s6	<code>mooncake:w3</code> , <code>agentx:T1</code> , <code>fast25_synthetic:x0</code> , <code>sessions:s6</code>	Figure 7